从 C++到 AI 计算：第三册源码卷

LCQI 本地 CPU 推理引擎完整源码与阅读路径

CPU Performance Study

本卷把第三册贯穿项目 LCQI 的构建入口、模型资产、公共头文件、核心实现、工具脚本、benchmark、trace、报告生成器和 CTest 验收源码独立成册。原理卷负责解释机制，实践卷负责安排阶段任务；源码卷负责给出一份可完整审阅、可构建、可回归的代码证据。

前言

第三册原理卷讲 AI 推理系统的机器事实，实践卷把任务拆成阶段作业。本源码卷只处理一件事：把贯穿项目 books/cpu-volume-3/labs/linux_cpu_inference 以及相关工具脚本单独成册，让读者能从构建入口一路读到测试和报告，而不是在原理正文或实践任务后面翻一段过长的源码附录。

核心思想

> 源码卷不是“代码仓库的打印版”。每章先说明该组文件承担的工程责任，再给完整源码。阅读时要把函数签名、错误边界、测试断言和报告字段连起来看：一个推理引擎是否可信，不取决于某个单独函数看起来像模型，而取决于资产能校验、shape 能拒绝错误、kernel 能对照、trace 能定位、benchmark 能说明口径、CTest 能长期守住回归。

本卷收录的主线代码是 LCQI。它从 tiny MLP 教学闭环开始，逐步覆盖 tokenizer、safetensors manifest gate、int8 kernel、packed/AVX2 路径、reference decoder、KV cache trace、GPT-2 reference path、7B compute ledger、serving SLO probe、真实 small 开源模型 smoke、自研 GPT-2 smoke 和 benchmark compare。它不是生产级推理引擎，但每个能力都要求有源码、命令、测试或报告证据。

阅读顺序建议如下。第一章先看构建入口和模型资产，明确哪些目标一定构建、哪些对标依赖外部库。第二章读公共头文件，先把数据结构和函数合同固定下来。第三章读 tiny MLP、int8 kernel 和基础 CLI，建立最小推理闭环。第四章读 reference decoder、safetensors、tokenizer 和 GPT-2 代码，理解真实模型路径为什么比模板 demo 多出许多边界检查。第五章读工具脚本、benchmark、trace、ledger、serving probe 和测试，把代码和可复查证据连起来。

源码卷的 LaTeX 文件通过 lstinputlisting 直接读取第三册工程源码。也就是说，源码不在书中复制第二份；代码更新后重新构建本卷，PDF 会读取当前仓库里的真实文件。

缩写与读码约定

LCQI Local CPU Quantized Inference，本书第三册的本地 CPU 量化推理贯穿项目。

CLI Command Line Interface，命令行入口。源码里通常只负责参数解析和结果打印，不承载核心推理逻辑。

MLP Multi-Layer Perceptron，多层感知机。tiny MLP 用来建立可手算、可测试的最小推理闭环。

KV Cache Key/Value Cache，decoder-only 模型跨 token 复用 attention key/value 的状态缓存。

BPE Byte Pair Encoding，GPT-2 使用的 byte-level 子词编码方法。

GGUF GPT-Generated Unified Format，llama.cpp 常用的模型文件格式。

TTFT Time To First Token，服务请求从进入系统到输出第一个 token 的时间。

TPOT Time Per Output Token，生成阶段每个输出 token 的平均或分位耗时。

SLO Service Level Objective，服务等级目标。本书用它描述 TTFT、TPOT、拒绝率和取消回收等运行时约束。

PMU Performance Monitoring Unit，硬件性能计数器单元。没有 PMU 权限时，性能报告不能伪装成硬件级结论。

AVX2 Advanced Vector Extensions 2，x86-64 SIMD 指令集扩展。源码存在不等于当前机器可用，报告要看运行时可用性。

读码时先看头文件中的数据结构和入口，再看实现文件中的检查、循环和错误分支，最后回到测试文件确认哪些行为已经被锁成回归合同。遇到 benchmark 或 smoke 脚本时，要区分“同模型同精度对比”和“成熟引擎参考口径”，不能把不同模型、不同量化格式的速度直接写成质量排名。

目录

第 1 章

构建入口、模型资产与项目边界

1.1 源码卷覆盖范围

LCQI 的源码边界由三个层次组成。第一层是仓库构建入口，它决定 CMake 怎样把第三册的实验代码纳入构建和测试。第二层是 labs/linux_cpu_inference，它包含真正的库、CLI、benchmark、trace、ledger、serving probe 和测试目标。第三层是工具脚本，它们下载或复用外部模型资产，生成 smoke 和 benchmark 报告。

源码阅读任务

> 先读本章的 CMake 文件，再读 README。读者要能回答三个问题：一是 lcqi 静态库包含哪些实现文件；二是哪些可执行文件只依赖自研代码，哪些依赖可选外部库；三是真实 GPT-2 和 SmolLM2 smoke 为什么不进入默认 CTest。

1.2 根构建入口

第三册根目录的 CMake 文件只做项目级选择：是否构建第三册 labs，以及如何把 labs/linux_cpu_inference 接入。它应该保持薄层，避免把具体目标定义分散到多个地方。

Listing 1.1 cpu-volume-3/README.md

```
# 从 C++ 到 AI 计算：第三册

第三册用于承接 AI、AI 编译器、算子开发和本地大模型推理引擎。第二册只讨论硬件原理、多核、高性能计算、并发、锁、无锁数据结构、并行算法和分布式计算，不再直接进入 AI。

本册贯穿项目是一台能推理开源 7B 左右 decoder-only 大模型的本地引擎，并把 AI 编译器作为引擎内部主线：模型图导入、typed tensor IR、算子融合、内存规划、CPU/GPU lowering、kernel selection 和 runtime dispatch 都要能落回编译原理与底层机器账本。工程路线从 CPU 上可解释的 reference 和量化切片开始，逐步扩展到 tokenizer、模型加载、Transformer 层、KV cache、CPU SIMD/多线程后端、GPU 后端、正确性报告和性能对标。

正式书稿工程位于：

```text
source/latex/
```

构建命令：

```bash
make -C source/latex check
make -C source/latex pdf
```

```

生成文件：

```text
source/latex/main.pdf
```

当前保留的第一阶段切片：

```text
labs/linux_cpu_inference/
```

该目录目前包含最小 MLP/int8 权重教学切片、tiny reference decoder、GPT-2 reference path、int8 scalar baseline、packed layout、AVX2 SIMD 路径和 shape sweep benchmark，用来验证张量、线性层、量化权重、tokenizer、safetensors、GPT-2 forward、KV cache、oracle 和 benchmark 的基本边界。当前 x86-64 AVX2 结果见：

```text
results/lcqi-x86-avx2-benchmark-2026-06-25.csv
results/lcqi-x86-avx2-decode4096-2026-06-25.csv
```

自研 GPT-2 smoke 入口如下：

```bash
make cpu3-gpt2-smoke
make cpu3-gpt2-benchmark-compare
```

第一条命令下载 `openai-community/gpt2` 的标准 safetensors 资产到用户缓存目录，用 LCQI C++ reference path 生成 1 个 token，并把报告写入：

```text
results/lcqi-gpt2-smoke.txt
```

第二条命令用同一个 GPT-2 F32 模型对比 LCQI full-prefix 和 cached-KV 生成路径，并在本机存在 llama.cpp/SmolLM2 缓存时附带成熟引擎参考，报告写入：

```text
results/lcqi-gpt2-benchmark-compare.txt
```

它仍然不是生产级 AI Infra。生产级门禁见：

```text
reports/ai-infra-maturity-audit.md
```

第三册后续必须把 lab 推进到“小型 CPU 推理 runtime + 手写高性能算子 + 严格 benchmark 报告 + 真实系统对标”，否则只能证明系统学习，不能证明生产级能

```
    ↪ 力。
```

Listing 1.2　cpu-volume-3/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.31)
project(CPUVolume3 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

option(CPU_VOLUME_3_BUILD_LABS "Build CPU Volume 3 labs" ON)

if(CPU_VOLUME_3_BUILD_LABS)
    enable_testing()
    add_subdirectory(labs/linux_cpu_inference)
endif()
```

实践卷也提供一个薄 CMake 入口，它把同一份 LCQI 源码作为实践卷可测试对象接入。这样实践卷不会复制实现，只复用第三册主工程。

Listing 1.3　cpu-volume-3-practice/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.31)
project(CPUVolume3Practice LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

enable_testing()
add_subdirectory(../cpu-volume-3/labs/linux_cpu_inference linux_cpu_inference)
```

源码卷自身的 Makefile 负责构建 PDF，并通过 `test` 目标回到第三册真实 LCQI 工程做 CMake/CTest 验证。源码卷如果只排版、不测试，就不能宣称“完整源码可运行”。

Listing 1.4　cpu-volume-3-source/Makefile

```
LATEX_DIR := source/latex

.PHONY: all check pdf test clean

all: pdf

check:
    $(MAKE) -C $(LATEX_DIR) check

pdf:
    $(MAKE) -C $(LATEX_DIR) pdf
```

```
12
13 test:
14     cmake -S ../cpu-volume-3 -B ../cpu-volume-3/build/lcqi-debug -↩
       ↪ DCMAKE_BUILD_TYPE=Debug
15     cmake --build ../cpu-volume-3/build/lcqi-debug
16     ctest --test-dir ../cpu-volume-3/build/lcqi-debug --output-on-failure
17
18 clean:
19     $(MAKE) -C $(LATEX_DIR) clean
```

Listing 1.5 cpu-volume-3-source/source/latex/Makefile

```
1 LATEXMK ?= latexmk
2 ENGINE ?= -xelatex
3 MAIN ?= main.tex
4
5 .PHONY: all pdf clean check
6
7 all: pdf
8
9 pdf:
10     $(LATEXMK) $(ENGINE) -interaction=nonstopmode -halt-on-error $(MAIN)
11
12 clean:
13     $(LATEXMK) -C
14
15 check:
16     @python3 scripts/check_inputs.py
17     @command -v xelatex >/dev/null 2>&1 || { echo "missing xelatex"; exit 1; }
18     @command -v latexmk >/dev/null 2>&1 || { echo "missing latexmk"; exit 1; }
```

1.3 LCQI 目标定义

linux_cpu_inference/CMakeLists.txt 是源码卷的第一份核心文件。这里能看到 lcqi 库、lcqi_cli、lcqi_bench、lcqi_trace、lcqi_gpt2、lcqi_ledger、lcqi_serving_probe、可选 lcqi_onednn_bench 和 lcqi_tests 的完整关系。

Listing 1.6 linux_cpu_inference/CMakeLists.txt

```
1 find_package(Threads REQUIRED)
2
3 add_library(lcqi STATIC
4     src/f32_kernels.cpp
5     src/ggml_matvec.cpp
6     src/ggml_tensors.cpp
7     src/gguf.cpp
8     src/gpt2_reference.cpp
9     src/inference.cpp
10     src/int8_kernels.cpp
11     src/llama_gguf.cpp
12     src/model.cpp
13     src/q4_k.cpp
```

```
    src/q5_0_packed.cpp
    src/reference_decoder.cpp
    src/safetensors.cpp
    src/tokenizer.cpp
)

if(CMAKE_SYSTEM_PROCESSOR MATCHES "x86_64|AMD64|i[3-6]86")
    if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
        target_sources(lcqi PRIVATE src/f32_kernels_avx2.cpp)
        target_sources(lcqi PRIVATE src/ggml_matvec_avx2.cpp)
        target_sources(lcqi PRIVATE src/int8_kernels_avx2.cpp)
        target_sources(lcqi PRIVATE src/q4_k_avx2.cpp)
        target_sources(lcqi PRIVATE src/q5_0_packed_avx2.cpp)
        set_source_files_properties(src/f32_kernels_avx2.cpp PROPERTIES ↩
    ↪ COMPILE_OPTIONS "-mavx2;-mfma")
        set_source_files_properties(src/ggml_matvec_avx2.cpp PROPERTIES ↩
    ↪ COMPILE_OPTIONS "-mavx2")
        set_source_files_properties(src/int8_kernels_avx2.cpp PROPERTIES ↩
    ↪ COMPILE_OPTIONS "-mavx2;-mfma")
        set_source_files_properties(src/q4_k_avx2.cpp PROPERTIES ↩
    ↪ COMPILE_OPTIONS "-mavx2")
        set_source_files_properties(src/q5_0_packed_avx2.cpp PROPERTIES ↩
    ↪ COMPILE_OPTIONS "-mavx2")
        target_compile_definitions(lcqi PRIVATE LCQI_ENABLE_AVX2=1)
    endif()
endif()

if(NOT CMAKE_SYSTEM_NAME STREQUAL "Linux")
    message(WARNING "LCQI is written for the book's local Linux CPU target; ↩
    ↪ current system is ${CMAKE_SYSTEM_NAME}.")
endif()

target_include_directories(lcqi
    PUBLIC
        ${CMAKE_CURRENT_SOURCE_DIR}/include
)

target_compile_features(lcqi PUBLIC cxx_std_20)
target_link_libraries(lcqi PUBLIC Threads::Threads)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi PRIVATE -Wall -Wextra -Wpedantic)
endif()

add_executable(lcqi_cli
    src/main.cpp
)

target_link_libraries(lcqi_cli PRIVATE lcqi)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_cli PRIVATE -Wall -Wextra -Wpedantic)
```

```
endif()

add_executable(lcqi_bench
    src/bench.cpp
)

target_link_libraries(lcqi_bench PRIVATE lcqi)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_bench PRIVATE -Wall -Wextra -Wpedantic)
endif()

add_executable(lcqi_gguf_q4_bench
    src/gguf_q4_bench.cpp
)

target_link_libraries(lcqi_gguf_q4_bench PRIVATE lcqi)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_gguf_q4_bench PRIVATE -Wall -Wextra -Wpedantic)
endif()

add_executable(lcqi_trace
    src/trace.cpp
)

target_link_libraries(lcqi_trace PRIVATE lcqi)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_trace PRIVATE -Wall -Wextra -Wpedantic)
endif()

add_executable(lcqi_gpt2
    src/gpt2_cli.cpp
)

target_link_libraries(lcqi_gpt2 PRIVATE lcqi)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_gpt2 PRIVATE -Wall -Wextra -Wpedantic)
endif()

add_executable(lcqi_llama_gguf
    src/llama_cli.cpp
)

target_link_libraries(lcqi_llama_gguf PRIVATE lcqi)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_llama_gguf PRIVATE -Wall -Wextra -Wpedantic)
endif()

```

```
add_executable(lcqi_ledger
    src/ledger.cpp
)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_ledger PRIVATE -Wall -Wextra -Wpedantic)
endif()

add_executable(lcqi_serving_probe
    src/serving_probe.cpp
)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_serving_probe PRIVATE -Wall -Wextra -Wpedantic)
endif()

find_package(dnnl QUIET)
if(dnnl_FOUND)
    add_executable(lcqi_onednn_bench
        src/onednn_bench.cpp
    )
    target_link_libraries(lcqi_onednn_bench PRIVATE DNNL::dnnl)
    target_compile_features(lcqi_onednn_bench PRIVATE cxx_std_20)
    if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
        target_compile_options(lcqi_onednn_bench PRIVATE -Wall -Wextra -
    ↪ Wpedantic)
    endif()
endif()

add_executable(lcqi_tests
    tests/lcqi_tests.cpp
)

target_link_libraries(lcqi_tests PRIVATE lcqi)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_tests PRIVATE -Wall -Wextra -Wpedantic)
endif()

add_test(NAME lcqi_tests COMMAND $<TARGET_FILE:lcqi_tests>)
add_test(NAME lcqi_gpt2_tiny COMMAND $<TARGET_FILE:lcqi_gpt2>)
set_tests_properties(lcqi_gpt2_tiny PROPERTIES
    PASS_REGULAR_EXPRESSION "generated_ids 1 2 3 3 3"
)
add_test(NAME lcqi_trace COMMAND $<TARGET_FILE:lcqi_trace>)
set_tests_properties(lcqi_trace PROPERTIES
    PASS_REGULAR_EXPRESSION "tokenizer,0,-1,5,1,i32,contiguous,10,4,0;1;2;3;4"
)
add_test(NAME lcqi_ledger COMMAND $<TARGET_FILE:lcqi_ledger>)
set_tests_properties(lcqi_ledger PROPERTIES
    PASS_REGULAR_EXPRESSION "bandwidth_limit,int4_at_100_gbps,23.602,tokens/s"
)
```

```
add_test(NAME lcqi_serving_probe COMMAND $<TARGET_FILE:lcqi_serving_probe>)
set_tests_properties(lcqi_serving_probe PROPERTIES
    PASS_REGULAR_EXPRESSION "request,req_too_long,0,memory_budget_exceeded"
)
```

1.4 项目说明与模型资产

README 是复现实验的入口。它写清楚当前能力、后续边界、构建命令、tiny 推理命令、benchmark 字段、reference trace 字段、GPT-2 smoke、7B ledger、serving SLO probe 和 SmolLM2 small smoke。读源码前先读 README，可以避免把 tiny fixture、GPT-2 reference path 和外部 llama.cpp smoke 混成同一个能力等级。

Listing 1.7 linux_cpu_inference/README.md

```
# Linux CPU Quantized Inference

这是第三册贯穿项目的第一阶段切片：Linux 本地 CPU 量化线性层和最小推理流程。第三册的贯穿目标不是这个 MLP 本身，而是一台能推理开源 7B 左右 decoder-only 大模型的本地引擎，并逐步覆盖 tokenizer、模型加载、Transformer 层、KV cache、CPU/GPU 后端、正确性报告和性能对标。

目标平台是 x86-64 Linux 实验环境。完整性能报告应记录 CPU 型号、内核版本、编译器版本、是否能使用 PMU、NUMA 和线程亲和能力。若运行环境缺少 perf、NUMA、线程亲和性或硬件性能计数器权限，报告必须把相关结论降级为“未验证”，不能把源码路径存在当作实测性能证据。后续 GPU 后端会作为独立 backend 接入，不改变当前切片的语义合同。

当前版本支持：

- 教学文本模型格式 `LCQI_MODEL_V1`
- 最小 tokenizer 合同格式 `LCQI_TOKENIZER_V1`
- safetensors header manifest 解析：metadata、tensor name、dtype、shape、data offsets、offset 边界和 dtype/shape 字节数校验
- 两层 MLP 教学切片
- int8 权重加 per-layer scale
- float 输入和激活
- 量化线性层、ReLU、分类 argmax
- int8 linear scalar baseline、packed layout、AVX2 路径和 shape sweep benchmark
- GPT-2 F32 dot kernel: cached generation 热路径使用可替换 dot 边界，x86-64/GNU/Clang 构建接入 AVX2/FMA 实现，其他平台保留标量 fallback
- tiny reference decoder: RMSNorm、float linear、RoPE、GQA KV cache、decode attention、SwiGLU、lm head
- GPT-2 reference path: byte-level BPE tokenizer、`config.json`、F32/F16/BF16 safetensors 张量读取、HuggingFace GPT-2 name mapping、LayerNorm、绝对位置嵌入、packed QKV、causal attention、GELU、tied lm head、full-prefix greedy generation、KV cache greedy generation 和分阶段 benchmark
- reference trace CSV: 从 prompt/tokenizer 合同到 logits/sampler/token，逐 checkpoint 输出 shape、stride、dtype、layout、checksum、max_abs、values 摘要
- 7B ledger CSV: 输出典型 7B shape 的 FLOPs、KV 容量、权重/KV 字节和 bandwidth-only tokens/s 上限
- serving SLO probe CSV: 输出 admission、batch state、KV 预算、queue wait、TTFT
```

```
    ↪ 、TPOT、取消回收和拒绝率
23 - 自研 GPT-2 冒烟：加载 `openai-community/gpt2` 的 `config.json`、`vocab.json`↩
    ↪ 、`merges.txt`、`model.safetensors`，在 LCQI C++ reference path 上生成 1 ↩
    ↪ 个 token，并把文件 hash、prompt ids、generated ids 和文本输出写入报告
24 - 自研 GPT-2 benchmark 对比：同一个 GPT-2 F32 safetensors 模型分别跑 full-↩
    ↪ prefix 和 cached-KV，并在可用时附带 llama.cpp/SmolLM2 Q4_K_M 作为成熟引擎↩
    ↪ 参考
25 - 自研 GPT-2 优化 A/B：从指定 baseline commit 建临时 worktree，当前实现和 ↩
    ↪ baseline 都通过 `books/cpu-volume-3-practice` 的 Release CMake 入口构建，↩
    ↪ 再用同一个 GPT-2 F32 模型多轮测量 full-prefix 与 cached-KV
26 - 真实 small 开源模型冒烟：通过 llama.cpp 加载 SmolLM2-135M-Instruct 的 GGUF 量↩
    ↪ 化文件，在本地 CPU 上生成一段 assistant 文本，并把模型 hash、命令、输出和↩
    ↪ 性能行写入报告
27 - 自研 SmolLM2/GGUF reference decode：读取同一个 GGUF 内的 tokenizer tokens/↩
    ↪ merges 和 `F32/Q8_0/Q5_0/Q6_K/Q4_K` 混合权重，加载 30 层 SmolLM2，执行 ↩
    ↪ RMSNorm、normal RoPE、GQA KV cache、SwiGLU 和 tied lm head；默认把 shape ↩
    ↪ 兼容的 `Q4_K` linear tensor 留在 GGUF block bytes 中直接执行，其余格式走 ↩
    ↪ f32 fallback；`LCQI_LLAMA_GGML_DIRECT=1` 可打开 `Q5_0/Q6_K/Q8_0` 实验直算↩
    ↪ 路径用于分析
28 - CLI 推理和简单 benchmark
29 - CTest 正确性测试
30
31 后续扩展方向：
32
33 - 真实 7B 级模型 metadata、tokenizer 和量化权重导入
34 - SentencePiece tokenizer、safetensors 分片加载和更多模型方言 name mapping
35 - GPT-2 reference path 的 Transformers/PyTorch golden logits 对齐报告
36 - KV cache 分页、内存规划和可选量化
37 - CPU packed weight、SIMD、NUMA 和线程亲和优化
38 - GPU backend adapter 和 device kernel
39 - 与现有框架的 prefill/decode/内存/误差对标报告
40
41 构建：
42
43 ```bash
44 cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
45 cmake --build books/cpu-volume-3/build/lcqi-debug
46 ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
47 ```
48
49 运行：
50
51 ```bash
52 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_cli \
53   books/cpu-volume-3/labs/linux_cpu_inference/models/tiny_mlp_i8.txt \
54   1.0 2.0 -1.0 0.5 1000
55 ```
56
57 kernel benchmark：
58
59 ```bash
```

````
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_bench 200
```

输出 CSV 字段：

```text
target_arch,input_size,output_size,output_block,repeat,backend,available,average_us,max_abs_diff,checksum
```

当前 benchmark 按 backend 逐行输出：`scalar`、`packed_scalar`、`avx2`。x86-64 构建会编译 AVX2 路径；不支持 AVX2/FMA 的机器会把该 backend 标为 `available=0`，避免把源码存在误报成当前环境实测性能。

这还不是生产级高性能 kernel。当前 benchmark 建立 scalar baseline、packed layout、AVX2 基线正确性和 shape sweep 口径。要达到完整 AI Infra 证据，还必须继续补反汇编分析、AVX-512、perf counter、oneDNN/OpenBLAS/llama.cpp/ggml 对照、sanitizer、fuzz 和 CI。

safetensors manifest gate:

`lcqi_tests` 会运行时生成一个最小 safetensors fixture，验证 header 长度、`__metadata__`、tensor name、dtype、shape、data offsets、payload 边界和 dtype/shape 推导出的字节数。它还会生成 offset 超出 payload、dtype/shape 字节数不匹配的坏文件，确认加载器会拒绝。这个 gate 只覆盖模型资产目录表，不等于已经支持完整 LLaMA/Qwen 权重映射、分片合并或 mmap 执行；后续真实模型加载必须在这个 manifest 上继续接 name mapping、shape verifier、quant descriptor 和 first token trace。

reference decoder trace:

```bash
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_trace
```

输出 CSV 字段：

```text
name,token_position,layer_id,shape,stride,dtype,layout,checksum,max_abs,values
```

这条 trace 固定 prompt fixture `tok1 tok2 tok3`，tokenizer 输出完整 `tokens=[0,1,2,3,4]`，其中 `0/4` 是 BOS/EOS；decoder trace 会剥离 BOS/EOS 后进入 tiny decoder。这样报告能同时固定 tokenizer 合同和 decoder 输入合同，避免把二者混成一个不可解释的 token 数组。随后 trace 把 embedding、RMSNorm、Q/K/V、RoPE、KV cache slot、attention、SwiGLU、layer output、final norm、logits、argmax sampler 和 generated token 串成可回归检查的硬链路。`lcqi_tests` 会检查 tokenizer contract hash、关键 checkpoint 的 shape、stride、dtype、layout 和 logits golden，CTest 也会直接运行 `lcqi_trace`。

当前 tokenizer fixture 的关键输出：
````
````

```text
tokenizer,0,-1,5,1,i32,contiguous,10,4,0;1;2;3;4
tokenizer_contract,0,-1,scalar,scalar,u64,fnv1a,13452902845388333734,0,tiny-chat-template-v1
```

GPT-2 自研 reference smoke：

```bash
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_gpt2
make cpu3-gpt2-smoke
make cpu3-gpt2-benchmark-compare
make cpu3-gpt2-hotspot-profile
python3 books/cpu-volume-3/tools/run_gpt2_optimization_ab.py --rounds 3
python3 books/cpu-volume-3/tools/run_gpt2_hotspot_profile.py --token-counts 4,16 --rounds 3
```

第一条命令不需要下载模型，直接运行仓库内 tiny GPT-2 fixture，输出固定 prompt ids、logits、predicted token 和 greedy generated ids。`lcqi_tests` 会覆盖 tiny GPT-2 forward/generation、byte-level BPE 编码解码、F32/F16/BF16 safetensors 读取、HuggingFace 风格 tiny GPT-2 checkpoint 加载和坏 shape 拒绝。

第二条命令显式依赖网络，只下载 `openai-community/gpt2` 的 `config.json`、`vocab.json`、`merges.txt`、`model.safetensors` 到 `~/.cache/lcqi-gpt2-smoke/openai-community--gpt2`，模型文件不进入 Git。脚本会校验四个文件的 bytes 和 SHA256，构建 `lcqi_gpt2`，运行：

```text
Hello, my name is
```

当前验证报告写到：

```text
books/cpu-volume-3/results/lcqi-gpt2-smoke.txt
```

最近一次本机报告的关键行是：

```text
prompt_ids 15496 11 616 1438 318
generated_ids 15496 11 616 1438 318 1757
generated_text Hello, my name is John
validation=PASS
```

这说明 LCQI 自研 C++ reference path 已经能跑标准 GPT-2 safetensors 的 tokenizer、权重导入、forward 和 greedy generation。默认真实模型路径现在使用 `cached_kv`，也可以用 `--engine full` 复现旧的 full-prefix 重算路径：

```bash
books/cpu-volume-3/build/lcqi-release/labs/linux_cpu_inference/lcqi_gpt2 \
  --benchmark --engine cached --threads 0 \
  ~/.cache/lcqi-gpt2-smoke/openai-community--gpt2 \
  "Hello, my name is" 4
```

`--threads 0` 表示自动选择 cached decoder worker 数；`--threads 1` 强制串行；`--threads N` 固定 worker 数。当前 auto 上限是 16，避免在短 decode 上把线程调度开销放大到超过收益。

`make cpu3-gpt2-benchmark-compare` 会生成：

```text
books/cpu-volume-3/results/lcqi-gpt2-benchmark-compare.txt
```

`run_gpt2_optimization_ab.py` 会生成：

```text
books/cpu-volume-3/results/lcqi-gpt2-optimization-ab.txt
```

这个 A/B 报告专门回答“当前 LCQI 代码相对指定 baseline 改快了吗”。脚本默认 baseline 是 `4febf3f`，会临时创建 baseline worktree，分别构建 baseline 和当前工作区的 Release `lcqi_gpt2`，再多轮运行同一条 prompt。上一轮工作区复用优化的 2 轮 smoke 摘要保存在 `books/cpu-volume-3/reports/lcqi-gpt2-optimization-ab-summary.md`；本轮线程优化的正式 raw 报告保存在 `books/cpu-volume-3/reports/lcqi-gpt2-threaded-manual-ab-caw.txt`，汇总保存在 `books/cpu-volume-3/reports/lcqi-gpt2-threaded-optimization-summary.md`。以 `73f40c7` 作为 baseline，在 `caw` 上 5 轮中位数显示：4 token cached-KV 生成阶段从 `395.397 ms` 降到 `76.4141 ms`，16 token 从 `989.060 ms` 降到 `185.479 ms`，生成吞吐分别提升到 `52.3464 token/s` 和 `86.2633 token/s`。随后的 F32 dot kernel 优化以 `cb4d89f` 为 baseline，raw 报告保存在 `books/cpu-volume-3/reports/lcqi-gpt2-f32-dot-ab-caw.txt`，汇总保存在 `books/cpu-volume-3/reports/lcqi-gpt2-f32-dot-optimization-summary.md`。在同一台 `caw`、同一 GPT-2 F32 模型、同样 `--threads 0` 自动 16 workers 下，4 token 生成阶段中位数从 `74.4247 ms` 到 `62.3628 ms`，16 token 从 `181.257 ms` 到 `157.230 ms`；decode token/s 分别提升到 `130.073` 和 `127.405`。再往后，row-range kernel 和去 `std::function` 调度开销的 raw 报告保存在 `books/cpu-volume-3/reports/lcqi-gpt2-f32-rowrange-ab-caw.txt`，汇总保存在 `books/cpu-volume-3/reports/lcqi-gpt2-f32-rowrange-optimization-summary.md`。这一轮相对 `ab72ccb` 的端到端提升很小，16 token 生成中位数约 `153.354 ms` 到 `152.574 ms`；但直接比较 row-range 与 4-row AVX2 row-range 时，4 token 从 `62.0243 ms` 到 `60.3965 ms`，16 token 从 `154.087 ms` 到 `151.222 ms`。本轮 prefill-skip 优化的 raw 报告保存在 `books/cpu-volume-3/reports/lcqi-gpt2-prefill-skip-ab-caw.txt`，汇总保存在 `books/cpu-volume-3/reports/lcqi-gpt2-prefill-skip-optimization-summary.md`。它跳过 prompt 前缀 token 上不会被使用的 `lm_head` 预测：4 token 生成中位数从 `60.5154 ms` 到 `52.1949 ms`，16 token 从 `153.574 ms` 到 `147.424 ms`，生成文本和 token id 全部一致。端到端 GPT-2 数字受调度、温度和共享机器状态影响很大，因此正式结论必须看多轮中位数、min/max 和生成文本

```
    ↪ 一致性，不能只挑一次最快结果。
156
157 `run_gpt2_hotspot_profile.py` 专门回答“现在热点到底在哪”。它通过 `--profile-↩
    ↪ hotspots` 打开 cached decode 内部计时，字段包括 `hotspot_lm_head_pct`、`↩
    ↪ hotspot_mlp_fc_pct`、`hotspot_mlp_projection_pct`、`↩
    ↪ hotspot_qkv_projection_pct` 和 `hotspot_attention_pct`。`caw` 上的 3 轮 ↩
    ↪ Release profile 摘要保存在 `books/cpu-volume-3/reports/lcqi-gpt2-hotspot-↩
    ↪ profile-summary.md`，原始报告保存在 `books/cpu-volume-3/reports/lcqi-gpt2↩
    ↪ -hotspot-profile-caw.txt`。关键结论是：4 token 与 16 token 两个口径下，`↩
    ↪ lm_head` 中位数约 `30.2%`，MLP 两个线性投影合计约 `46%`，QKV 投影约 `17%`↩
    ↪ ，cached attention 本体只有 `0.06%` 到 `0.11%`。所以当前短上下文 GPT-2 的↩
    ↪ 慢点不是 KV attention，而是标量 F32 matvec 和 `50257 x 768` vocab 扫描。
158
159 当前优化点集中在 cached-KV generation：`Gpt2CachedGreedyDecoder` 把模型级 shape↩
    ↪  validation、KV cache、临时 workspace 和 worker pool 的生命周期提升到整段↩
    ↪ 生成；`Gpt2ForwardWorkspace` 复用 hidden、normed、Q/K/V、packed QKV、↩
    ↪ attention、MLP、scores 和 logits 缓冲区；生成路径只需要 greedy token 时走↩
    ↪  logits-free argmax，避免把完整 logits vector 作为 API 结果反复分配；↩
    ↪ prompt prefill 前缀通过 `advance_without_prediction()` 只更新 KV cache，↩
    ↪ 不跑 final norm 和 `lm_head`，因为这些预测结果不会被用作输出；QKV、↩
    ↪ attention output projection、MLP 两个 projection 和 tied `lm_head` 都在 ↩
    ↪ cached session 内按输出行并行；每个线程 chunk 再通过 `↩
    ↪ linear_f32_rows_unchecked` 或 `max_dot_f32_rows_unchecked` 进入 F32 row-↩
    ↪ range kernel，x86-64 可用时使用 AVX2/FMA，其他平台走标量 fallback。KV ↩
    ↪ cache 的 checked public API 仍保留，内部 cached attention 在一次边界校验↩
    ↪ 后使用私有 unchecked span 扫描热路径。它不改变 GPT-2 数学语义，只改变会话↩
    ↪ 生命周期、任务分片、无用工作剔除和热循环调度。
160
161 `make cpu3-gpt2-benchmark-compare` 仍用于和外部成熟引擎放在同一报告里看工程差↩
    ↪ 距。同机 llama.cpp/SmolLM2 Q4_K_M 是成熟引擎参照，不是同模型质量排名：↩
    ↪ LCQI 跑的是 GPT-2 F32 reference path，llama.cpp 跑的是 GGUF 量化 small ↩
    ↪ instruct 模型。它揭示的是工程差距：LCQI 已经消除了“每步重算前缀”的主要算↩
    ↪ 法错误，补上 cached F32 row parallelism，并把热路径接到 AVX2/FMA row-↩
    ↪ range kernel；但还缺 packed/quantized weight、GGUF 量化 block、mmap/lazy ↩
    ↪ load、采样器、分页 KV cache、NUMA/亲和和成熟调度。
162
163 它仍然不是生产级推理引擎：当前 GPT-2 路径还没有把 logits 和 Transformers/↩
    ↪ PyTorch 逐数值对齐成外部 golden 报告，也没有 GGUF 自研导入、分页 KV cache↩
    ↪ 、量化权重执行和高性能 SIMD/packed kernel。
164
165 7B ledger:
166
167 ```bash
168 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_ledger \
169   > books/cpu-volume-3/results/lcqi-sevenb-ledger-2026-06-24.csv
170 ```
171
172 输出 CSV 字段：
173
174 ```text
175 section,item,value,unit,notes
176 ```
```

这份账本固定 `L=32,H=4096,n_q=32,n_kv=8,head_dim=128,context=4096`，报告 decode FLOPs、KV 容量、fp16/int8/int4 每 token 字节和不同有效带宽下的 tokens/s 上限。CTest 会检查 `int4_at_100_gbps=23.602 tokens/s` 这一行，防止账本公式漂移。

serving SLO probe：

```bash
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_serving_probe \
  > books/cpu-volume-3/results/lcqi-serving-slo-probe-2026-06-24.csv
```

输出 CSV 字段：

```text
row_type,request_id,accepted,rejection_reason,prompt_tokens,reserved_tokens,kv_reserved_mib,queue_wait_ms,prefill_ms,ttft_ms,decode_step_p95_ms,tpot_ms,completion_tokens,finish_reason,cancel_reclaim_ms,metric_name,metric_value
```

probe 固定四个请求：短请求、RAG 请求、取消请求和超长请求。它演示 admission 先按 `prompt_tokens + max_new_tokens` 预留 KV，取消请求释放预算，超长请求返回 `memory_budget_exceeded`，并输出 `ttft_p95_ms`、`queue_wait_p95_ms`、`rejection_rate` 等服务化指标。

真实 small 开源模型 smoke：

```bash
make cpu3-smollm2-smoke
```

这条命令显式依赖网络和外部构建，因此不放进默认 CTest。脚本会把 llama.cpp 和模型文件缓存到 `~/.cache/lcqi-smollm2-small-smoke`，模型不进入 Git。固定模型是 `HuggingFaceTB/SmolLM2-135M-Instruct` 的 GGUF 量化版本，来源仓库为 `bartowski/SmolLM2-135M-Instruct-GGUF`，文件为 `SmolLM2-135M-Instruct-Q4_K_M.gguf`。脚本会校验文件大小 `105454432` 字节和 SHA256：

```text
2e8040ceae7815abe0dcb3540b9995eaa1fa0d2ca9e797d0a635ae4433c68c2d
```

脚本构建固定 commit 的 `llama-simple-chat`，用 `-ngl 0` 强制 CPU 路径，输入短 prompt，检查 assistant response 非空，并生成：

```text
books/cpu-volume-3/results/lcqi-smollm2-small-model-smoke.txt
```

这份报告只能证明 llama.cpp 成熟外部引擎上的真实 GGUF small 模型已经完成加载、

```
    ↪ tokenizer/chat template、prefill、decode 和文本输出。它不能证明 LCQI 自研↩
    ↪  C++ 引擎已经达到 llama.cpp 的性能，也不能替代 tiny fixture 的逐层 golden↩
    ↪  trace。

LCQI 自研 SmolLM2/GGUF reference decode：

```bash
books/cpu-volume-3/build/lcqi-release/labs/linux_cpu_inference/lcqi_llama_gguf ↩
    ↪ \
  ~/.cache/lcqi-smollm2-small-smoke/models/SmolLM2-135M-Instruct-Q4_K_M.gguf \
  --prompt "Hello" --max-new 2 --benchmark --decode-text
```

caw 上的最新报告写到：

```text
books/cpu-volume-3/results/lcqi-smollm2-gguf-reference-decode-caw.txt
```

关键行是：

```text
weight_execution gguf_mixed_q4_k_direct
generated_text ... <|im_start|>assistant
Hello!
benchmark_prompt_tokens 31
benchmark_generated_tokens 2
benchmark_decode_tokens_per_second 64.8586
benchmark_quantized_weight_bytes 103668480
benchmark_f32_weight_bytes 481436928
benchmark_direct_quantized_weight_bytes 7962624
benchmark_fallback_dequantized_weight_bytes 481436928
benchmark_q4_k_direct_tensors 16
benchmark_ggml_direct_tensors 0
benchmark_f32_fallback_tensors 256
benchmark_hotspot_w_down_ms 63.5371
benchmark_hotspot_q4_k_direct_calls 512
benchmark_hotspot_ggml_direct_calls 0
benchmark_hotspot_f32_fallback_calls 6208
```

这条路径证明 LCQI 自研代码已经能读取同一个 SmolLM2 GGUF、解析 tokenizer arrays↩
    ↪ 、处理混合量化权重并端到端生成文本。默认执行路径不再只是“加载时全部解成 ↩
    ↪ f32”：shape 兼容的 `Q4_K` linear tensor 会保留 GGUF block bytes，并在真实↩
    ↪ decoder 里通过 `matvec_q4_k_q8` 直接执行；`Q5_0/Q6_K/Q8_0/F32` tensor 仍↩
    ↪ 然 materialize 成 f32 fallback。当前只有约 `7.68%` 的加载权重字节进入默认↩
    ↪ direct Q4_K 路径，prefill 仍逐 token 执行；与 llama.cpp 的差距主要来自更↩
    ↪ 完整的低比特权重覆盖、prompt batching、packed kernel、线程调度、prefetch↩
    ↪ 、KV cache 和整图执行。

实验路径 `LCQI_LLAMA_GGML_DIRECT=1` 会尝试让 shape 兼容的 `Q5_0/Q6_K/Q8_0` ↩
    ↪ linear tensor 也保留 GGUF block bytes，并通过 `matvec_ggml_quantized_q8_0↩
```

```
      ↪ ` 执行。第一版单线程/整矩阵实验曾把 direct 量化字节覆盖率从 `0.076809` 提↩
      ↪ 高到 `0.708479`，但 prefill 从默认 `370.847 ms` 退化到 `1211.710 ms`，热↩
      ↪ 点 `benchmark_hotspot_ggml_direct_ms` 达到 `1207.13 ms`。当前版本已经给 ↩
      ↪ GGML direct 补上 row-range unchecked 入口，并接入同一个 `↩
      ↪ LlamaParallelWorkerPool`：caw 同输入 5 轮中位数显示，实验路径降到 prefill↩
      ↪  `242.149 ms`、decode step `8.98937 ms`、`↩
      ↪ benchmark_hotspot_ggml_direct_ms=228.300`。这说明 row-range 解决了“所有行↩
      ↪ 串行扫”的一大块问题，但它仍慢于默认 Q4_K+f32 fallback 的 `162.492 ms`、↩
      ↪ `6.41487 ms`，所以继续保留为 opt-in，不默认启用。
254
255 LCQI 与 llama.cpp 同输入对比：
256
257 ```bash
258 python3 books/cpu-volume-3/tools/run_smollm2_same_input_compare.py \
259   --cache-dir ~/.cache/lcqi-smollm2-same-input-compare \
260   --model-path ~/.cache/lcqi-smollm2-small-smoke/models/SmolLM2-135M-Instruct-↩
      ↪ Q4_K_M.gguf \
261   --build-dir books/cpu-volume-3/build/lcqi-release \
262   --rounds 5 --max-new 2
263 ```
264
265 这个脚本会构建固定 commit 的 llama.cpp，运行 `llama-tokenize`、`llama-simple` ↩
      ↪ 和 `llama-bench`，并确认 LCQI 和 llama.cpp 看到的 prompt token ids 完全一↩
      ↪ 致。caw 上的同输入报告保存在：
266
267 ```text
268 books/cpu-volume-3/reports/lcqi-smollm2-ggml-threaded-compare-caw.txt
269 ```
270
271 关键结论：同一个模型、同一个展开 chat prompt、同样 31 个 token id、同样 `max-↩
      ↪ new=2` 下，当前默认 LCQI 使用 `8` 个 worker，对大行数 f32 fallback、Q4_K ↩
      ↪ direct row-range 和实验 GGML direct row-range 都能做输出行切分。默认 Q4_K↩
      ↪  direct prefill 中位数从串行 `364.282 ms` 降到 `162.492 ms`，decode step ↩
      ↪ 从串行 `14.9794 ms` 降到 `6.41487 ms`；f32 fallback hotspot 从 `338.779 ↩
      ↪ ms` 降到 `148.132 ms`。相对同输入 llama.cpp，LCQI prefill 仍慢 `7.089529x↩
      ↪ `，decode step 仍慢 `2.250832x`。同一二进制里关闭 `LCQI_LLAMA_Q4_DIRECT↩
      ↪ =0` 后，LCQI prefill 为 `174.378 ms`，decode step 为 `6.78061 ms`，`↩
      ↪ w_down` 热点为 `36.4924 ms`；默认 Q4_K direct 后分别为 `162.492 ms`、↩
      ↪ `6.41487 ms`、`22.2228 ms`。所以这轮默认优化的 A/B 证据是：线程池带来 ↩
      ↪ prefill `2.241846x`、decode step `2.335106x`、f32 hotspot `2.287008x`；↩
      ↪ Q4_K direct 在并行后带来 prefill `1.073148x`、decode step `1.057014x`、`↩
      ↪ w_down` `1.642115x`，并减少 `56,623,104` 字节 f32 materialized 权重。实验↩
      ↪  GGML direct row-range 相比最初 `1211.710 ms` 已明显改善到 `242.149 ms`，↩
      ↪ 但仍慢于默认路径，下一步需要 packed multi-row layout、更完整 SIMD 覆盖和 ↩
      ↪ prefetch，而不是只扩大格式覆盖。
```

tiny MLP 模型文件是最短闭环的资产。它决定输入维度、hidden 维度、输出维度、权重量化值、scale 和 bias；测试中的 logits golden 依赖这份文件。

Listing 1.8 models/tiny_mlp_i8.txt

```
1 LCQI_MODEL_V1
2 input_size 4
```

```
hidden_size 3
output_size 2
hidden_scale 0.1
hidden_weights
5 -3 2 1
-4 6 1 -2
3 2 -5 4
hidden_bias
0.1 -0.2 0.0
output_scale 0.05
output_weights
4 -6 2
-3 5 8
output_bias
-0.5 0.4
```

tiny tokenizer fixture 固定 BOS、EOS、UNK、词表和模板 hash。tokenizer 合同如果漂移，后面的 decoder trace 和测试都会被污染。

Listing 1.9 models/tiny_tokenizer.txt

```
LCQI_TOKENIZER_V1
bos_id 0
eos_id 4
unk_id -1
chat_template_hash tiny-chat-template-v1
vocab_size 5
vocab
0 <bos>
1 tok1
2 tok2
3 tok3
4 <eos>
```

第 2 章

源码阅读路线：从入口到调用链

2.1 为什么要先讲调用链

源码卷如果只把 *.cpp 和 *.hpp 全部贴出来，读者仍然不知道从哪里下手。LCQI 的代码同时包含 tiny MLP、int8 kernel、tokenizer、safetensors、reference decoder、GPT-2、benchmark、trace、ledger、serving probe 和外部模型脚本；这些文件不是平铺关系，而是几条调用链。先讲调用链，再给完整源码，读者才能把每个文件放到正确位置。

核心思想

> 读 LCQI 的顺序不是“哪个文件长先读哪个”，而是“入口、合同、资产、执行、证据”。入口说明目标怎样构建，合同说明类型怎样连接，资产说明模型从哪里来，执行说明 token 或张量怎样流动，证据说明测试和报告守住了哪些边界。

2.2 一、全工程入口：哪些文件决定能不能复现

文件	负责什么	读源码时要确认什么
第三册README	第三册 AI 计算工程路线、当前 LCQI 能力、GPT-2 smoke 和结果报告位置。	能力声明必须能在 labs/linux_cpu_inference、tools 和 results 中找到证据。
第三册CMake	第三册根 CMake，负责把 linux_cpu_inference 接入。	根 CMake 应保持薄层；真正 target 定义必须在 lab 内部，避免入口文件承担实现细节。
实践卷CMake	实践卷复用同一份 LCQI lab。	实践卷不复制源码，只把同一工程接入自己的测试构建，防止两份代码漂移。
源码卷Makefile	源码卷的书籍构建入口。	它构建 PDF，也提供 test 目标去构建第三册 LCQI；源码卷不能只排版不验证。
LCQICMake	lcqi 库、CLI、benchmark、trace、GPT-2、ledger、serving probe、CTest 目标。	读这里能看到哪些文件进入库，哪些是可执行程序，哪些依赖可选外部库。
LCQIREADME	本地构建、tiny 推理、benchmark、trace、GPT-2、SmolLM2 smoke 的命令。	先按 README 跑命令，再读源码；否则不知道某个函数是否真的能从命令行触达。

上表里的短标签对应这些真实路径：cpu-volume-3/README.md、cpu-volume-3/CMakeLists.txt、cpu-volume-3-practice/CMakeLists.txt、cpu-volume-3-source/Makefile、linux_cpu_inference/CMakeLists.txt 和 linux_cpu_inference/README.md。

这些入口文件回答“代码在哪里”和“怎样运行”。读者打开源码卷时，先不要直接跳到 gpt2_reference.cpp。先确认 lcqi 静态库包含 model.cpp、inference.cpp、int8_kernels.cpp、reference_decoder.cpp、tokenizer.cpp、safetensors.cpp 和 gpt2_reference.cpp；再确认 lcqi_cli、lcqi_bench、lcqi_trace、lcqi_gpt2、lcqi_ledger、lcqi_serving_probe 和 lcqi_tests 分别从哪条路径进入。

2.3 二、Tiny MLP 调用链：最短闭环怎样跑通

Tiny MLP 是最小推理闭环，用来证明“模型资产、shape 检查、int8 linear、激活函数、logits、argmax、CLI 输出”可以完整连起来。它不是大模型能力本身，但它是后面所有复杂路径的可手算底座。

文件	关键类型或函数	这段代码的意思
models/tiny_mlp_i8.txt	输入维度、hidden 维度、输出维度、int8 权重、scale、bias。	这是模型资产。测试中的 golden logits 来自这里，资产改动必须带着测试改动。
include/lcqi/model.hpp	QuantizedLinearLayer、TinyMlpModel、load_model。	定义 tiny MLP 如何在内存中表示；只负责模型结构和加载入口，不做推理。
src/model.cpp	read_i8_vector、validate_layer、load_model。	解析文本模型，拒绝坏 key、坏维度、权重数量不匹配和非法 scale，把错误挡在执行前。
include/lcqi/inference.hpp	InferenceResult、linear_i8、relu_inplace、run_inference。	定义推理输出要包含 hidden、logits 和 predicted class，方便测试定位错误层。
src/inference.cpp	输入 shape 检查、两层 int8 linear、ReLU、argmax。	最短执行路径：模型加输入变成 logits。这里的代码应保持清楚，不能混入 CLI 或文件解析。
src/main.cpp	解析命令行输入并打印结果。	CLI 是薄层；它只接模型路径和输入，不复制 run_inference 逻辑。

这条链路按下面顺序读：main() 取得模型路径和输入向量，调用 load_model() 把 tiny_mlp_i8.txt 变成 TinyMlpModel，再调用 run_inference()。在 run_inference() 内部，第一层 linear_i8() 把 float 输入和 int8 权重相乘，乘以 scale 后加 bias；relu_inplace() 把负 hidden 截断；第二层 linear_i8() 得到 logits；最后 argmax() 给出 predicted class。

读这个路径时要抓住边界条件。输入长度不等于 input_size 要立即拒绝；权重数量不是 rows*columns 要在加载时拒绝；logits 为空不能取 argmax。这些错误如果拖到 SIMD kernel 或 benchmark 里才暴露，定位成本会高很多。

2.4 三、Int8 Kernel 调用链：reference、packed、AVX2 和 benchmark 怎样对照

文件	关键代码	这段代码的意思
include/lcqi/int8_kernels.hpp	PackedLinearI8、KernelBenchmarkCase、benchmark_linear_i8_case。	定义 kernel 对标口径：shape、repeat、backend、平均耗时、checksum、max diff。
src/int8_kernels.cpp	pack_linear_i8、linear_i8_packed、linear_i8_packed_with_workspace、deterministic fixture。	scalar 和 packed 路径是正确性基线，workspace 路径减少重复分配，fixture 让 benchmark 和测试可复现。

src/int8_kernels_avx2.cpp	linear_i8_packed_avx2_available、linear_i8_packed_avx2。	平台优化路径。必须先判断机器能力，再执行 AVX2/FMA；不可用时报告 unavailable，而不是假装测过。
src/bench.cpp	shape 列表、repeat 参数、backend 输出。	benchmark 程序只负责测 kernel 入口。性能结论必须和 max_abs_diff、CPU、编译器、构建类型一起报告。

kernel 文件要按“可信基线到优化路径”的顺序读。先看 deterministic layer 和 input 怎么生成，保证每次运行输入相同；再看 linear_i8_packed() 怎样把 int8 权重、scale、bias 和 float 输入算成输出；再看 workspace 路径怎样复用临时缓冲；最后看 AVX2 路径怎样在支持平台上替换内层循环。测试用 packed 与 scalar 的 max_abs_diff 对照，benchmark 再比较耗时。

2.5 四、Tokenizer、Safetensors 与 Reference Decoder：真实模型前的三道门

文件	关键类型或函数	这段代码的意思
models/tiny_tokenizer.txt	BOS、EOS、UNK、词表、模板 hash。	固定 tokenizer 合同，避免 prompt 到 token id 的规则漂移。
include/lcqi/tokenizer.hpp 与 src/tokenizer.cpp	TokenizerModel、load_tokenizer、encode_prompt、tokenizer_contract_hash。	把 prompt 切成稳定 token id；未知 token 是否允许，必须由 UNK 合同决定。
include/lcqi/safetensors.hpp 与 src/safetensors.cpp	SafeTensorManifest、SafeTensorFile、manifest parser、dtype 转换。	解析模型字节之前先验证 header、dtype、shape、offset 和 payload 边界，防止坏资产进入数学层。
include/lcqi/reference_decoder.hpp 与 src/reference_decoder.cpp	embedding、RMSNorm、RoPE、KV cache、attention、SwiGLU、trace checkpoint。	Tiny decoder-only reference path。它用 checkpoint 解释每一步张量状态，不只返回最终 token。
src/trace.cpp	trace CSV 输出。	把 reference decoder 的 checkpoint 打印成可复查表格，定位错误发生在哪个阶段。

这三道门分别解决三个问题。Tokenizer 解决“人写的 prompt 如何变成 token id”；Safetensors 解决“磁盘里的模型字节如何变成带 shape 的张量”；Reference Decoder 解决“token 和权重如何经过 decoder-only 层生成 logits”。任何一层含糊，后面的 GPT-2 路径都会变成看似能跑、实则无法定位的黑盒。

2.6 五、GPT-2 调用链：从 HuggingFace 资产到生成 token

GPT-2 路径是源码卷里最长的一条链，因为它要处理真实模型格式。读 gpt2_reference.cpp 时按模块读，不要从第一行一路滚到最后。

文件	关键代码	这段代码的意思
include/lcqi/gpt2_reference.hpp	Gpt2Config、Gpt2ReferenceModel、Gpt2KvCache、Gpt2Tokenizer、forward/generate API。	公开 GPT-2 reference path 的合同：配置、权重、KV cache、tokenizer、full-prefix 和 cached generation。
src/gpt2_reference.cpp	JSON parser、byte-level BPE、safetensors loader、LayerNorm、QKV、causal attention、GELU、lm head、KV cache。	正确性优先实现。它把 HuggingFace GPT-2 权重名、Conv1D 转置、tokenizer 和 forward 规则全部显式写出来。
src/gpt2_cli.cpp	tiny mode、真实模型目录模式、generation、benchmark 输出。	命令行入口。无参数跑 tiny fixture；传模型目录时加载真实 GPT-2 资产；benchmark 对比 full-prefix 和 cached-KV。
tools/run_gpt2_smoke.py	下载标准 GPT-2 资产，调用 lcqi_gpt2。	证明自研 safetensors、BPE、forward、KV cache 和 greedy generation 能跑真实开源模型。
tools/run_gpt2_benchmark_compare.py	运行 LCQI full-prefix、cached-KV，并可附带成熟引擎参考。	用同一模型口径比较算法差距和 kernel 差距，不把不同模型的速度混成一个结论。

GPT-2 代码按五段读。第一段是配置和权重加载：load_gpt2_config() 读 config.json，load_gpt2_reference_model() 从 safetensors 中取 embedding、每层 LayerNorm、QKV、MLP 和 final norm。HuggingFace GPT-2 的 Conv1D 权重形状和普通线性层方向不同，所以代码显式调用转置函数，不能跳过。

第二段是 tokenizer：load_gpt2_tokenizer() 读 vocab.json 和 merges.txt，gpt2_encode() 执行 byte-level BPE，gpt2_decode() 把 id 还原为文本。这里的重点是字节到 Unicode 映射、BPE merge rank 和 unknown token 边界。

第三段是 full-prefix forward：run_gpt2_forward() 每生成一个新 token 都用完整 token 前缀重新跑所有层。它慢，但容易验证。流程是 token embedding 加 position embedding，逐层执行 LayerNorm、packed QKV、causal attention、残差、MLP、残差，最后 final LayerNorm 和 tied lm head 得到 logits。

第四段是 cached-KV forward：Gpt2KvCache 保存每层每个 position 的 key/value，run_gpt2_forward_cached() 每次只处理最新 token，并让 attention 读取过去 cache。这里要特别看 validate_written()：读取未写入 cache slot 必须报错，否则生成错误会很难定位。

第五段是 greedy generation：gpt2_generate_greedy() 用 full-prefix 路径，gpt2_generate_greedy_cached() 用 KV cache 路径。两个函数都要检查 prompt 非空、max_new_tokens 非负、不会超过 max_positions，并在生成 EOS 时停止。

2.7 六、报告、服务探针、外部对标和测试

文件	关键代码	这段代码的意思
src/ledger.cpp	7B 参数量、KV cache、bandwidth-only 上限。	输出的是工程账本，不是实测速度；它帮助读者估算权重和 KV 内存压力。

src/serving_probe.cpp	短请求、RAG 请求、取消请求、超长请求，TTFT/TPOT 字段。	把服务层边界变成字段：哪些请求接受，哪些拒绝，取消后资源是否回收。
src/onednn_bench.cpp	oneDNN matmul benchmark。	可选外部库对标入口。没有 oneDNN 环境时不能声称已验证。
tools/run_smollm2_small_smoke.py	下载 SmolLM2-135M-Instruct GGUF，构建固定 commit 的 llama.cpp smoke。	这是成熟外部引擎上的真实 small 模型 smoke，用来建立外部参照，不代表 LCQI 已具备同等能力。
tools/run_lcqi_benchmark.sh	收集 LCQI 本地 benchmark。	在固定机器上重复运行同一组 shape，形成可复查 CSV 或文本报告。
tests/lcqi_tests.cpp	tiny MLP、tokenizer、safetensors、GPT-2 tiny、HF-style checkpoint、bad shape、decoder、trace、kernel。	这是源码卷最重要的证据。新增功能如果没有进入这里，就不能写成已经守住的能力。

读测试文件时，不要只看最后是否返回 0。它里面的 fixture 本身就是源码讲解的一部分：测试会手写 safetensors header，构造 F16/BF16/F32 payload，生成 tiny GPT-2 权重，验证 bad shape 会被拒绝，比较 packed int8 kernel 和 scalar 路径。这些测试告诉读者“代码真正承诺了什么”。如果一个能力只出现在 README 或 CLI 输出里，却没有进入测试或 smoke 报告，就只能算实验入口，不能算稳定能力。

源码阅读任务

> 读完本章后，再去读后面的完整源码。每看到一个文件，都把它归到五类之一：入口、合同、资产解析、执行路径、证据。归不进去的文件，就说明工程边界需要重新解释或重构。

第 3 章

公共合同：模型、Tokenizer、Kernel 与 Decoder

3.1 为什么先读头文件

头文件是系统对外承诺的最小合同。实现可以重写，kernel 可以替换，benchmark 可以补充平台路径，但头文件里暴露的数据结构、函数入口和返回字段决定了测试、CLI、工具脚本和章节讲解怎样调用这套工程。

核心思想

> 读 LCQI 时不要从最长的 gpt2_reference.cpp 开始。先读公共头文件，明确“模型如何表示”“推理返回什么”“trace 记录什么”“KV cache 怎样寻址”“tokenizer 的坏输入怎样处理”，再进入实现。这样读到长循环和 shape 检查时，知道它们在维护哪份合同。

3.2 Tiny MLP 与推理入口

model.hpp 定义 tiny MLP 的模型对象、权重、bias、scale 和加载入口。它的责任是把文本模型资产解释成 C++ 对象，不负责执行推理。

Listing 3.1 include/lcqi/model.hpp

```cpp
#pragma once

#include <cstdint>
#include <filesystem>
#include <string>
#include <vector>

namespace lcqi {

struct QuantizedLinearLayer {
    std::int32_t input_size = 0;
    std::int32_t output_size = 0;
    float scale = 1.0F;
    std::vector<std::int8_t> weights;
    std::vector<float> bias;
};

struct TinyMlpModel {
    std::int32_t input_size = 0;
    std::int32_t hidden_size = 0;
    std::int32_t output_size = 0;
    QuantizedLinearLayer hidden;
    QuantizedLinearLayer output;
};

TinyMlpModel load_model(const std::filesystem::path& path);
```

```
27
28 }  // namespace lcqi
```

inference.hpp 定义推理结果和 run_inference 入口。返回值包含 hidden、logits 和 predicted class，测试可以用这些字段定位是加载错、kernel 错，还是 argmax 错。

Listing 3.2 include/lcqi/inference.hpp

```cpp
#pragma once

#include <cstdint>
#include <span>
#include <vector>

#include <lcqi/model.hpp>

namespace lcqi {

struct InferenceResult {
    std::vector<float> logits;
    std::int32_t predicted_class = 0;
};

void linear_i8(const QuantizedLinearLayer& layer,
               std::span<const float> input,
               std::span<float> output);

void relu_inplace(std::span<float> values);

InferenceResult run_inference(const TinyMlpModel& model,
                              std::span<const float> input);

}  // namespace lcqi
```

3.3 Int8 Kernel、Tokenizer 与 Safetensors

int8 kernel 头文件定义 packed linear、deterministic fixture、scalar/workspace/AVX2 路径和误差比较。注意 AVX2 是运行时能力，不是源码存在就等于当前机器已验证。

Listing 3.3 include/lcqi/int8_kernels.hpp

```cpp
#pragma once

#include <cstdint>
#include <span>
#include <vector>

#include <lcqi/model.hpp>

namespace lcqi {
```

```
10
11 struct PackedLinearI8 {
12     std::int32_t input_size = 0;
13     std::int32_t output_size = 0;
14     std::int32_t output_block_size = 0;
15     float scale = 1.0F;
16     std::vector<std::int8_t> packed_weights;
17     std::vector<float> bias;
18 };
19
20 struct KernelBenchmarkCase {
21     std::int32_t input_size = 0;
22     std::int32_t output_size = 0;
23     std::int32_t output_block_size = 0;
24     std::int32_t repeat = 0;
25 };
26
27 struct KernelBackendBenchmark {
28     const char* name = "";
29     bool available = false;
30     double average_us = 0.0;
31     float max_abs_diff = 0.0F;
32     float checksum = 0.0F;
33 };
34
35 struct KernelBenchmarkResult {
36     KernelBenchmarkCase benchmark_case;
37     std::vector<KernelBackendBenchmark> backends;
38 };
39
40 PackedLinearI8 pack_linear_i8(const QuantizedLinearLayer& layer,
41                               std::int32_t output_block_size);
42
43 void linear_i8_packed(const PackedLinearI8& layer,
44                       std::span<const float> input,
45                       std::span<float> output);
46
47 void linear_i8_packed_with_workspace(const PackedLinearI8& layer,
48                                      std::span<const float> input,
49                                      std::span<float> output,
50                                      std::span<float> accumulator_workspace);
51
52 bool linear_i8_packed_avx2_available() noexcept;
53
54 void linear_i8_packed_avx2(const PackedLinearI8& layer,
55                            std::span<const float> input,
56                            std::span<float> output);
57
58 QuantizedLinearLayer make_deterministic_i8_layer(std::int32_t input_size,
59                                                  std::int32_t output_size,
60                                                  float scale);
61
```

```
std::vector<float> make_deterministic_input(std::int32_t input_size);

float max_abs_diff(std::span<const float> lhs, std::span<const float> rhs);

KernelBenchmarkResult benchmark_linear_i8_case(const KernelBenchmarkCase& ↩
    ↪ benchmark_case);

} // namespace lcqi
```

tokenizer 头文件固定 BOS/EOS/UNK、词表和 template hash。prompt 进入模型前，必须先变成稳定 token id；未知 token 行为也必须被写进合同。

Listing 3.4 include/lcqi/tokenizer.hpp

```
#pragma once

#include <cstdint>
#include <filesystem>
#include <string>
#include <string_view>
#include <vector>

namespace lcqi {

struct TokenEntry {
    std::string text;
    std::int32_t id = 0;
};

struct TokenizerModel {
    std::int32_t bos_id = -1;
    std::int32_t eos_id = -1;
    std::int32_t unk_id = -1;
    std::string chat_template_hash;
    std::vector<TokenEntry> vocab;
};

[[nodiscard]] TokenizerModel load_tokenizer(const std::filesystem::path& path);

[[nodiscard]] std::vector<std::int32_t> encode_prompt(
    const TokenizerModel& tokenizer,
    std::string_view prompt);

[[nodiscard]] std::uint64_t tokenizer_contract_hash(const TokenizerModel& ↩
    ↪ tokenizer);

} // namespace lcqi
```

safetensors 头文件固定 dtype、shape、offset 和 payload 边界。它负责模型字节怎样被解释，不应该和数学推理逻辑混在一起。

Listing 3.5 include/lcqi/safetensors.hpp

```cpp
#pragma once

#include <cstdint>
#include <filesystem>
#include <span>
#include <string>
#include <string_view>
#include <utility>
#include <vector>

namespace lcqi {

struct SafeTensorEntry {
    std::string name;
    std::string dtype;
    std::vector<std::int64_t> shape;
    std::uint64_t data_begin = 0;
    std::uint64_t data_end = 0;

    [[nodiscard]] std::uint64_t byte_size() const;
};

struct SafeTensorManifest {
    std::uint64_t header_size = 0;
    std::uint64_t data_start_offset = 0;
    std::vector<std::pair<std::string, std::string>> metadata;
    std::vector<SafeTensorEntry> tensors;

    [[nodiscard]] const SafeTensorEntry* find_tensor(std::string_view name) const;
};

struct SafeTensorFile {
    std::filesystem::path path;
    SafeTensorManifest manifest;
    std::vector<std::uint8_t> bytes;
};

[[nodiscard]] SafeTensorManifest load_safetensors_manifest(
    const std::filesystem::path& path);

[[nodiscard]] SafeTensorFile load_safetensors_file(const std::filesystem::path& path);

[[nodiscard]] std::vector<float> read_safetensor_f32_tensor(
    const SafeTensorFile& file,
    std::string_view name);

[[nodiscard]] std::span<const std::uint8_t> read_safetensor_tensor_bytes(
    const SafeTensorFile& file,
    std::string_view name);

```

```
} // namespace lcqi
```

3.4 Reference Decoder 与 GPT-2 合同

reference decoder 头文件定义张量、层配置、trace checkpoint、采样结果和 decode 函数。它是定位推理错误的主要合同，不只返回最终 token。

Listing 3.6 include/lcqi/reference_decoder.hpp

```cpp
#pragma once

#include <cstdint>
#include <span>
#include <string>
#include <vector>

namespace lcqi {

struct LinearWeightsF32 {
    std::int32_t input_size = 0;
    std::int32_t output_size = 0;
    std::vector<float> weights;
    std::vector<float> bias;
};

struct DecoderConfig {
    std::int32_t hidden_size = 0;
    std::int32_t query_heads = 0;
    std::int32_t kv_heads = 0;
    std::int32_t head_dim = 0;
    std::int32_t intermediate_size = 0;
    std::int32_t vocab_size = 0;
    std::int32_t max_context = 0;
    float rms_epsilon = 1.0e-5F;
    float rope_base = 10000.0F;
};

struct DecoderLayerWeightsF32 {
    std::vector<float> rms_attention_weight;
    LinearWeightsF32 wq;
    LinearWeightsF32 wk;
    LinearWeightsF32 wv;
    LinearWeightsF32 wo;
    std::vector<float> rms_mlp_weight;
    LinearWeightsF32 w_gate;
    LinearWeightsF32 w_up;
    LinearWeightsF32 w_down;
};

struct ReferenceDecoderModel {
    DecoderConfig config;
    std::vector<float> token_embedding;
```

```
    std::vector<DecoderLayerWeightsF32> layers;
    std::vector<float> final_norm_weight;
    LinearWeightsF32 lm_head;
};

struct ReferenceDecodeResult {
    std::vector<float> logits;
    std::int32_t predicted_token = 0;
};

struct ReferenceTensorCheckpoint {
    std::string name;
    std::int32_t token_position = 0;
    std::int32_t layer_id = 0;
    std::vector<std::int32_t> shape;
    std::vector<std::int32_t> stride;
    std::string dtype;
    std::string layout;
    float checksum = 0.0F;
    float max_abs = 0.0F;
    std::vector<float> values;
};

struct ReferenceDecodeTraceResult {
    ReferenceDecodeResult result;
    std::vector<ReferenceTensorCheckpoint> checkpoints;
};

class ReferenceKVCache {
public:
    ReferenceKVCache(const DecoderConfig& config, std::int32_t layer_count);

    void append(std::int32_t layer_id,
                std::int32_t model_position,
                std::span<const float> key,
                std::span<const float> value);

    [[nodiscard]] std::span<const float> key(std::int32_t layer_id,
                                             std::int32_t model_position,
                                             std::int32_t kv_head) const;

    [[nodiscard]] std::span<const float> value(std::int32_t layer_id,
                                               std::int32_t model_position,
                                               std::int32_t kv_head) const;

    [[nodiscard]] std::int32_t layer_count() const noexcept;
    [[nodiscard]] std::int32_t filled_tokens() const noexcept;

private:
    [[nodiscard]] std::size_t base_offset(std::int32_t layer_id,
                                          std::int32_t model_position,
                                          std::int32_t kv_head) const;
```

```
    void validate_address(std::int32_t layer_id,
                          std::int32_t model_position,
                          std::int32_t kv_head) const;

    DecoderConfig config_;
    std::int32_t layer_count_ = 0;
    std::int32_t filled_tokens_ = 0;
    std::vector<float> keys_;
    std::vector<float> values_;
    std::vector<std::uint8_t> written_;
};

void rms_norm(std::span<const float> input,
              std::span<const float> weight,
              float epsilon,
              std::span<float> output);

void linear_f32(const LinearWeightsF32& layer,
                std::span<const float> input,
                std::span<float> output);

void apply_rope(std::span<float> heads,
                std::int32_t head_count,
                std::int32_t head_dim,
                std::int32_t model_position,
                float rope_base);

void attention_decode(const DecoderConfig& config,
                      const ReferenceKVCache& cache,
                      std::int32_t layer_id,
                      std::int32_t model_position,
                      std::span<const float> query,
                      std::span<float> output);

ReferenceDecodeResult run_reference_decode(const ReferenceDecoderModel& model,
                                           std::span<const std::int32_t> ↩
    ↪ token_ids);

ReferenceDecodeTraceResult run_reference_decode_with_trace(
    const ReferenceDecoderModel& model,
    std::span<const std::int32_t> token_ids);

ReferenceDecoderModel make_tiny_reference_decoder_model();

}  // namespace lcqi
```

GPT-2 reference 头文件定义 GPT-2 配置、byte-level BPE tokenizer、HuggingFace safetensors 加载、forward、cached forward 和 greedy generation。它单独存在，是因为 GPT-2 使用 LayerNorm、绝对位置嵌入、packed QKV、GELU 和 tied lm head，不能直接塞进 LLaMA-like reference decoder。

Listing 3.7 include/lcqi/gpt2_reference.hpp

```hpp
#pragma once

#include <cstdint>
#include <filesystem>
#include <memory>
#include <span>
#include <string>
#include <string_view>
#include <unordered_map>
#include <vector>

namespace lcqi {

struct Gpt2Config {
    std::int32_t hidden_size = 0;
    std::int32_t head_count = 0;
    std::int32_t layer_count = 0;
    std::int32_t max_positions = 0;
    std::int32_t vocab_size = 0;
    std::int32_t intermediate_size = 0;
    std::int32_t bos_token_id = -1;
    std::int32_t eos_token_id = -1;
    float layer_norm_epsilon = 1.0e-5F;
    std::string activation_function = "gelu_new";
};

struct Gpt2LinearF32 {
    std::int32_t input_size = 0;
    std::int32_t output_size = 0;
    std::vector<float> weights;
    std::vector<float> bias;
};

struct Gpt2LayerWeightsF32 {
    std::vector<float> ln_1_weight;
    std::vector<float> ln_1_bias;
    Gpt2LinearF32 c_attn;
    Gpt2LinearF32 c_proj;
    std::vector<float> ln_2_weight;
    std::vector<float> ln_2_bias;
    Gpt2LinearF32 c_fc;
    Gpt2LinearF32 mlp_c_proj;
};

struct Gpt2ReferenceModel {
    Gpt2Config config;
    std::vector<float> token_embedding;
    std::vector<float> position_embedding;
    std::vector<Gpt2LayerWeightsF32> layers;
    std::vector<float> final_ln_weight;
    std::vector<float> final_ln_bias;
```

```
    std::vector<float> lm_head_weight;
    bool tie_lm_head_to_embedding = true;
};

struct Gpt2ForwardResult {
    std::vector<float> logits;
    std::int32_t predicted_token = 0;
};

struct Gpt2HotspotProfile {
    std::int64_t decoder_steps = 0;
    std::int64_t layer_steps = 0;
    double total_step_ms = 0.0;
    double embedding_ms = 0.0;
    double layer_norm_ms = 0.0;
    double final_norm_ms = 0.0;
    double qkv_projection_ms = 0.0;
    double kv_append_ms = 0.0;
    double attention_ms = 0.0;
    double attention_projection_ms = 0.0;
    double residual_add_ms = 0.0;
    double mlp_fc_ms = 0.0;
    double gelu_ms = 0.0;
    double mlp_projection_ms = 0.0;
    double lm_head_ms = 0.0;
    double logits_result_ms = 0.0;
};

struct Gpt2ExecutionOptions {
    std::int32_t worker_count = 0;
    std::int32_t parallel_min_rows = 512;
};

class Gpt2KvCache;

namespace detail {
class Gpt2ParallelWorkerPool;

void attend_cached_position(const Gpt2KvCache& cache,
                            std::int32_t layer_id,
                            std::int32_t model_position,
                            std::span<const float> query,
                            std::span<float> scores,
                            std::span<float> output);
}  // namespace detail

class Gpt2ForwardWorkspace {
public:
    explicit Gpt2ForwardWorkspace(const Gpt2Config& config);

    void reset_for_config(const Gpt2Config& config);

```

```
    [[nodiscard]] std::span<float> hidden() noexcept;
    [[nodiscard]] std::span<float> normed() noexcept;
    [[nodiscard]] std::span<float> query() noexcept;
    [[nodiscard]] std::span<float> key() noexcept;
    [[nodiscard]] std::span<float> value() noexcept;
    [[nodiscard]] std::span<float> attention() noexcept;
    [[nodiscard]] std::span<float> projected() noexcept;
    [[nodiscard]] std::span<float> qkv_packed() noexcept;
    [[nodiscard]] std::span<float> mlp_fc() noexcept;
    [[nodiscard]] std::span<float> mlp_out() noexcept;
    [[nodiscard]] std::span<float> logits() noexcept;
    [[nodiscard]] std::span<float> scores_prefix(std::int32_t count);

private:
    Gpt2Config config_;
    std::vector<float> hidden_;
    std::vector<float> normed_;
    std::vector<float> query_;
    std::vector<float> key_;
    std::vector<float> value_;
    std::vector<float> attention_;
    std::vector<float> projected_;
    std::vector<float> qkv_packed_;
    std::vector<float> mlp_fc_;
    std::vector<float> mlp_out_;
    std::vector<float> logits_;
    std::vector<float> scores_;
};

class Gpt2KvCache {
public:
    explicit Gpt2KvCache(const Gpt2Config& config);

    void append(std::int32_t layer_id,
                std::int32_t model_position,
                std::span<const float> key,
                std::span<const float> value);

    [[nodiscard]] std::span<const float> key(std::int32_t layer_id,
                                             std::int32_t model_position,
                                             std::int32_t head) const;

    [[nodiscard]] std::span<const float> value(std::int32_t layer_id,
                                               std::int32_t model_position,
                                               std::int32_t head) const;

    [[nodiscard]] std::int32_t filled_tokens() const noexcept;
    [[nodiscard]] std::size_t byte_size() const noexcept;

private:
    friend void detail::attend_cached_position(const Gpt2KvCache& cache,
                                               std::int32_t layer_id,
```

```
                                                std::int32_t model_position,
                                                std::span<const float> query,
                                                std::span<float> scores,
                                                std::span<float> output);

    [[nodiscard]] std::size_t base_offset(std::int32_t layer_id,
                                          std::int32_t model_position,
                                          std::int32_t head) const;
    [[nodiscard]] std::size_t base_offset_unchecked(std::int32_t layer_id,
                                                    std::int32_t model_position
    ↪ ,
                                                    std::int32_t head) const ↩
    ↪ noexcept;
    [[nodiscard]] std::span<const float> key_unchecked(std::int32_t layer_id,
                                                       std::int32_t ↩
    ↪ model_position,
                                                       std::int32_t head) const↩
    ↪  noexcept;
    [[nodiscard]] std::span<const float> value_unchecked(std::int32_t layer_id,
                                                         std::int32_t ↩
    ↪ model_position,
                                                         std::int32_t head) ↩
    ↪ const noexcept;
    void validate_address(std::int32_t layer_id,
                          std::int32_t model_position,
                          std::int32_t head) const;
    void validate_written(std::int32_t layer_id,
                          std::int32_t model_position) const;

    Gpt2Config config_;
    std::int32_t filled_tokens_ = 0;
    std::vector<float> keys_;
    std::vector<float> values_;
    std::vector<std::uint8_t> written_;
};

class Gpt2CachedGreedyDecoder {
public:
    explicit Gpt2CachedGreedyDecoder(const Gpt2ReferenceModel& model);
    Gpt2CachedGreedyDecoder(const Gpt2ReferenceModel& model,
                            Gpt2HotspotProfile* hotspot_profile);
    Gpt2CachedGreedyDecoder(const Gpt2ReferenceModel& model,
                            Gpt2HotspotProfile* hotspot_profile,
                            const Gpt2ExecutionOptions& execution_options);
    ~Gpt2CachedGreedyDecoder();

    Gpt2CachedGreedyDecoder(const Gpt2CachedGreedyDecoder&) = delete;
    Gpt2CachedGreedyDecoder& operator=(const Gpt2CachedGreedyDecoder&) = delete↩
    ↪ ;
    Gpt2CachedGreedyDecoder(Gpt2CachedGreedyDecoder&&) noexcept;
    Gpt2CachedGreedyDecoder& operator=(Gpt2CachedGreedyDecoder&&) noexcept;

```

```
    void advance_without_prediction(std::int32_t token_id);
    [[nodiscard]] std::int32_t step(std::int32_t token_id);
    [[nodiscard]] Gpt2ForwardResult step_with_logits(std::int32_t token_id);
    [[nodiscard]] const Gpt2KvCache& cache() const noexcept;
    [[nodiscard]] std::int32_t filled_tokens() const noexcept;
    [[nodiscard]] std::size_t kv_cache_bytes() const noexcept;
    [[nodiscard]] std::int32_t worker_count() const noexcept;

private:
    const Gpt2ReferenceModel* model_;
    Gpt2HotspotProfile* hotspot_profile_;
    Gpt2ExecutionOptions execution_options_;
    Gpt2KvCache cache_;
    Gpt2ForwardWorkspace workspace_;
    std::unique_ptr<detail::Gpt2ParallelWorkerPool> worker_pool_;
};

struct Gpt2Tokenizer {
    std::int32_t bos_token_id = -1;
    std::int32_t eos_token_id = -1;
    std::unordered_map<std::string, std::int32_t> vocab;
    std::vector<std::string> id_to_token;
    std::unordered_map<std::string, std::int32_t> bpe_ranks;
};

[[nodiscard]] Gpt2Tokenizer load_gpt2_tokenizer(
    const std::filesystem::path& vocab_json,
    const std::filesystem::path& merges_txt,
    std::int32_t bos_token_id,
    std::int32_t eos_token_id);

[[nodiscard]] std::vector<std::int32_t> gpt2_encode(
    const Gpt2Tokenizer& tokenizer,
    std::string_view text);

[[nodiscard]] std::string gpt2_decode(
    const Gpt2Tokenizer& tokenizer,
    std::span<const std::int32_t> token_ids);

[[nodiscard]] Gpt2Config load_gpt2_config(const std::filesystem::path& ↩
    ↪ config_json);

[[nodiscard]] Gpt2ReferenceModel load_gpt2_reference_model(
    const std::filesystem::path& config_json,
    const std::filesystem::path& safetensors_path);

[[nodiscard]] Gpt2ReferenceModel load_gpt2_from_directory(
    const std::filesystem::path& model_directory);

[[nodiscard]] Gpt2ForwardResult run_gpt2_forward(
    const Gpt2ReferenceModel& model,
    std::span<const std::int32_t> token_ids);
```

```
252
253 [[nodiscard]] Gpt2ForwardResult run_gpt2_forward_cached(
254     const Gpt2ReferenceModel& model,
255     Gpt2KvCache& cache,
256     std::int32_t token_id);
257
258 [[nodiscard]] std::vector<std::int32_t> gpt2_generate_greedy(
259     const Gpt2ReferenceModel& model,
260     std::span<const std::int32_t> prompt_token_ids,
261     std::int32_t max_new_tokens);
262
263 [[nodiscard]] std::vector<std::int32_t> gpt2_generate_greedy_cached(
264     const Gpt2ReferenceModel& model,
265     std::span<const std::int32_t> prompt_token_ids,
266     std::int32_t max_new_tokens);
267
268 [[nodiscard]] Gpt2ReferenceModel make_tiny_gpt2_reference_model();
269
270 }  // namespace lcqi
```

第 4 章

Tiny MLP、Int8 Kernel 与基础 CLI

4.1 最短闭环：从模型文件到 logits

tiny MLP 路径不是为了证明 LCQI 已经是完整大模型引擎，而是为了建立最短的、可手算的、可测试的推理闭环。读者要把模型加载、输入检查、int8 linear、ReLU、第二层 logits、argmax 和 CLI 输出连起来看。

4.2 模型加载与基础推理

model.cpp 负责解析 tiny 文本模型、权重、scale、bias 和 shape。坏格式、坏维度和坏数值都应该在这里被拒绝，不能拖到 kernel 内部才崩溃。

Listing 4.1 src/model.cpp

```cpp
#include <lcqi/model.hpp>

#include <fstream>
#include <limits>
#include <stdexcept>
#include <string>

namespace lcqi {
namespace {

constexpr std::int32_t LCQI_INT8_MIN_VALUE = -128;
constexpr std::int32_t LCQI_INT8_MAX_VALUE = 127;

std::string read_key(std::istream& input) {
    std::string key;
    if (!(input >> key)) {
        throw std::runtime_error("unexpected end of model file");
    }
    return key;
}

void expect_key(std::istream& input, const std::string& expected) {
    const std::string key = read_key(input);
    if (key != expected) {
        throw std::runtime_error("expected key '" + expected + "', got '" + key + "'");
    }
}

std::int32_t read_i32(std::istream& input, const std::string& key) {
    expect_key(input, key);
    std::int32_t value = 0;
    if (!(input >> value)) {
        throw std::runtime_error("invalid int32 value for key '" + key + "'");
```

```
    }
    return value;
}

float read_float(std::istream& input, const std::string& key) {
    expect_key(input, key);
    float value = 0.0F;
    if (!(input >> value)) {
        throw std::runtime_error("invalid float value for key '" + key + "'");
    }
    return value;
}

std::vector<std::int8_t> read_i8_vector(std::istream& input,
                                        const std::string& key,
                                        std::int32_t count) {
    expect_key(input, key);
    if (count < 0) {
        throw std::runtime_error("negative vector size for key '" + key + "'");
    }
    std::vector<std::int8_t> values(static_cast<std::size_t>(count));
    for (std::int32_t i = 0; i < count; ++i) {
        std::int32_t raw = 0;
        if (!(input >> raw)) {
            throw std::runtime_error("invalid int8 payload for key '" + key + "'");
        }
        if (raw < LCQI_INT8_MIN_VALUE || raw > LCQI_INT8_MAX_VALUE) {
            throw std::runtime_error("int8 payload out of range for key '" + key + "'");
        }
        values[static_cast<std::size_t>(i)] = static_cast<std::int8_t>(raw);
    }
    return values;
}

std::vector<float> read_float_vector(std::istream& input,
                                     const std::string& key,
                                     std::int32_t count) {
    expect_key(input, key);
    if (count < 0) {
        throw std::runtime_error("negative vector size for key '" + key + "'");
    }
    std::vector<float> values(static_cast<std::size_t>(count));
    for (std::int32_t i = 0; i < count; ++i) {
        if (!(input >> values[static_cast<std::size_t>(i)])) {
            throw std::runtime_error("invalid float payload for key '" + key + "'");
        }
    }
    return values;
}
```

```cpp

void validate_layer(const QuantizedLinearLayer& layer, const std::string& name) {
    if (layer.input_size <= 0 || layer.output_size <= 0) {
        throw std::runtime_error(name + " layer has invalid shape");
    }
    const std::size_t expected_weights =
        static_cast<std::size_t>(layer.input_size) *
        static_cast<std::size_t>(layer.output_size);
    if (layer.weights.size() != expected_weights) {
        throw std::runtime_error(name + " layer weight size mismatch");
    }
    if (layer.bias.size() != static_cast<std::size_t>(layer.output_size)) {
        throw std::runtime_error(name + " layer bias size mismatch");
    }
}

std::int32_t checked_element_count(std::int32_t rows,
                                   std::int32_t columns,
                                   const std::string& name) {
    if (rows <= 0 || columns <= 0) {
        throw std::runtime_error(name + " layer has invalid shape");
    }
    const std::int64_t count =
        static_cast<std::int64_t>(rows) * static_cast<std::int64_t>(columns);
    if (count > static_cast<std::int64_t>(std::numeric_limits<std::int32_t>::max())) {
        throw std::runtime_error(name + " layer element count is too large");
    }
    return static_cast<std::int32_t>(count);
}

}  // namespace

TinyMlpModel load_model(const std::filesystem::path& path) {
    std::ifstream input(path);
    if (!input) {
        throw std::runtime_error("cannot open model file: " + path.string());
    }

    expect_key(input, "LCQI_MODEL_V1");

    TinyMlpModel model;
    model.input_size = read_i32(input, "input_size");
    model.hidden_size = read_i32(input, "hidden_size");
    model.output_size = read_i32(input, "output_size");

    const std::int32_t hidden_weight_count =
        checked_element_count(model.input_size, model.hidden_size, "hidden");
    const std::int32_t output_weight_count =
        checked_element_count(model.hidden_size, model.output_size, "output");

```

```
    model.hidden.input_size = model.input_size;
    model.hidden.output_size = model.hidden_size;
    model.hidden.scale = read_float(input, "hidden_scale");
    model.hidden.weights = read_i8_vector(
        input,
        "hidden_weights",
        hidden_weight_count);
    model.hidden.bias = read_float_vector(input, "hidden_bias", model.hidden_size);

    model.output.input_size = model.hidden_size;
    model.output.output_size = model.output_size;
    model.output.scale = read_float(input, "output_scale");
    model.output.weights = read_i8_vector(
        input,
        "output_weights",
        output_weight_count);
    model.output.bias = read_float_vector(input, "output_bias", model.output_size);

    validate_layer(model.hidden, "hidden");
    validate_layer(model.output, "output");
    return model;
}

}  // namespace lcqi
```

inference.cpp 执行 int8 linear、ReLU、第二层 logits 和 predicted class。它是系统中最短的“模型资产到输出”路径。

Listing 4.2 src/inference.cpp

```
#include <lcqi/inference.hpp>

#include <algorithm>
#include <cmath>
#include <stdexcept>

namespace lcqi {
namespace {

void validate_input_size(std::span<const float> input, std::int32_t expected) {
    if (input.size() != static_cast<std::size_t>(expected)) {
        throw std::runtime_error("input size does not match model shape");
    }
}

std::int32_t argmax(std::span<const float> values) {
    if (values.empty()) {
        throw std::runtime_error("cannot take argmax of empty logits");
    }
    std::int32_t best_index = 0;
    float best_value = values[0];
```

```
    for (std::int32_t i = 1; i < static_cast<std::int32_t>(values.size()); ++i) {
        const float candidate = values[static_cast<std::size_t>(i)];
        if (candidate > best_value) {
            best_value = candidate;
            best_index = i;
        }
    }
    return best_index;
}

}  // namespace

void linear_i8(const QuantizedLinearLayer& layer,
               std::span<const float> input,
               std::span<float> output) {
    if (input.size() != static_cast<std::size_t>(layer.input_size)) {
        throw std::runtime_error("linear_i8 input size mismatch");
    }
    if (output.size() != static_cast<std::size_t>(layer.output_size)) {
        throw std::runtime_error("linear_i8 output size mismatch");
    }

    for (std::int32_t out = 0; out < layer.output_size; ++out) {
        const std::size_t row_base =
            static_cast<std::size_t>(out) * static_cast<std::size_t>(layer.input_size);
        float acc = layer.bias[static_cast<std::size_t>(out)];
        for (std::int32_t in = 0; in < layer.input_size; ++in) {
            const std::int8_t weight =
                layer.weights[row_base + static_cast<std::size_t>(in)];
            acc += static_cast<float>(weight) * layer.scale *
                   input[static_cast<std::size_t>(in)];
        }
        output[static_cast<std::size_t>(out)] = acc;
    }
}

void relu_inplace(std::span<float> values) {
    for (float& value : values) {
        value = std::max(0.0F, value);
    }
}

InferenceResult run_inference(const TinyMlpModel& model,
                              std::span<const float> input) {
    validate_input_size(input, model.input_size);

    std::vector<float> hidden(static_cast<std::size_t>(model.hidden_size), 0.0F);
    std::vector<float> logits(static_cast<std::size_t>(model.output_size), 0.0F);
```

```
70
71      linear_i8(model.hidden, input, hidden);
72      relu_inplace(hidden);
73      linear_i8(model.output, hidden, logits);
74
75      InferenceResult result;
76      result.predicted_class = argmax(logits);
77      result.logits = std::move(logits);
78      return result;
79  }
80
81  }  // namespace lcqi
```

main.cpp 把模型路径和输入接到 run_inference，并打印结果。CLI 应保持薄层，不复制模型解析和推理逻辑。

Listing 4.3 src/main.cpp

```
1  #include <lcqi/inference.hpp>
2  #include <lcqi/model.hpp>
3
4  #include <chrono>
5  #include <cstdint>
6  #include <cstdlib>
7  #include <filesystem>
8  #include <iostream>
9  #include <span>
10 #include <stdexcept>
11 #include <string>
12 #include <vector>
13
14 namespace {
15
16 constexpr std::int32_t LCQI_DEFAULT_REPEAT = 1000;
17 constexpr int LCQI_EXPECTED_ARGC_MIN = 2;
18 constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
19
20 std::vector<float> parse_input(int argc, char** argv, std::int32_t ↩
       ↪ expected_size) {
21     if (argc < LCQI_EXPECTED_ARGC_MIN + expected_size) {
22         throw std::runtime_error("usage: lcqi_cli <model.txt> <input floats...>↩
       ↪  [repeat]");
23     }
24     std::vector<float> input(static_cast<std::size_t>(expected_size));
25     for (std::int32_t i = 0; i < expected_size; ++i) {
26         input[static_cast<std::size_t>(i)] =
27             std::stof(argv[LCQI_EXPECTED_ARGC_MIN + i]);
28     }
29     return input;
30 }
31
32 std::int32_t parse_repeat(int argc, char** argv, std::int32_t input_size) {
33     const int repeat_index = LCQI_EXPECTED_ARGC_MIN + input_size;
```

```cpp
    if (argc <= repeat_index) {
        return LCQI_DEFAULT_REPEAT;
    }
    const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[repeat_index]));
    if (repeat <= 0) {
        throw std::runtime_error("repeat must be positive");
    }
    return repeat;
}

}  // namespace

int main(int argc, char** argv) {
    try {
        if (argc < LCQI_EXPECTED_ARGC_MIN) {
            throw std::runtime_error("usage: lcqi_cli <model.txt> <input floats ...> [repeat]");
        }

        const std::filesystem::path model_path = argv[1];
        const lcqi::TinyMlpModel model = lcqi::load_model(model_path);
        const std::vector<float> input = parse_input(argc, argv, model.input_size);
        const std::int32_t repeat = parse_repeat(argc, argv, model.input_size);

        const lcqi::InferenceResult result = lcqi::run_inference(model, input);
        std::cout << "predicted_class " << result.predicted_class << "\n";
        std::cout << "logits";
        for (const float value : result.logits) {
            std::cout << ' ' << value;
        }
        std::cout << "\n";

        const auto begin = std::chrono::steady_clock::now();
        std::int32_t checksum = 0;
        for (std::int32_t i = 0; i < repeat; ++i) {
            checksum += lcqi::run_inference(model, input).predicted_class;
        }
        const auto end = std::chrono::steady_clock::now();
        const std::chrono::duration<double> elapsed = end - begin;
        const double average_us =
            elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double>(repeat);
        std::cout << "repeat " << repeat << "\n";
        std::cout << "average_us " << average_us << "\n";
        std::cout << "checksum " << checksum << "\n";
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << "\n";
        return EXIT_FAILURE;
    }
```

```
}
```

4.3 Int8 Kernel 基础实现

int8_kernels.cpp 包含 scalar、packed、workspace、deterministic fixture 和误差比较。它是 AVX2 路径的 reference 对照，也是 benchmark 的正确性底座。

Listing 4.4 src/int8_kernels.cpp

```cpp
#include <lcqi/int8_kernels.hpp>

#include <lcqi/inference.hpp>

#include <algorithm>
#include <chrono>
#include <cmath>
#include <stdexcept>
#include <string>

namespace lcqi {
namespace {

constexpr std::int32_t LCQI_I8_PATTERN_MODULUS = 23;
constexpr std::int32_t LCQI_I8_PATTERN_CENTER = 11;
constexpr std::int32_t LCQI_INPUT_PATTERN_MODULUS = 17;
constexpr std::int32_t LCQI_INPUT_PATTERN_CENTER = 8;
constexpr float LCQI_INPUT_PATTERN_SCALE = 0.125F;
constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
constexpr std::int32_t LCQI_SIMD_OUTPUT_BLOCK = 8;
constexpr std::size_t LCQI_BENCHMARK_BACKEND_COUNT = 3;
constexpr const char* LCQI_BACKEND_SCALAR = "scalar";
constexpr const char* LCQI_BACKEND_PACKED_SCALAR = "packed_scalar";
constexpr const char* LCQI_BACKEND_AVX2 = "avx2";

void require_positive(std::int32_t value, const char* name) {
    if (value <= 0) {
        throw std::runtime_error(std::string(name) + " must be positive");
    }
}

std::size_t checked_size(std::int32_t value, const char* name) {
    if (value < 0) {
        throw std::runtime_error(std::string(name) + " cannot be negative");
    }
    return static_cast<std::size_t>(value);
}

void validate_quantized_layer(const QuantizedLinearLayer& layer) {
    require_positive(layer.input_size, "input_size");
    require_positive(layer.output_size, "output_size");
    const std::size_t expected_weights =
        checked_size(layer.input_size, "input_size") *
```

```
        checked_size(layer.output_size, "output_size");
    if (layer.weights.size() != expected_weights) {
        throw std::runtime_error("quantized layer weight size mismatch");
    }
    if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
        throw std::runtime_error("quantized layer bias size mismatch");
    }
}

std::int32_t round_up_to_block(std::int32_t value, std::int32_t block_size) {
    require_positive(value, "value");
    require_positive(block_size, "block_size");
    return ((value + block_size - 1) / block_size) * block_size;
}

float checksum(std::span<const float> values) {
    float sum = 0.0F;
    for (const float value : values) {
        sum += value;
    }
    return sum;
}

void add_unavailable_backend(KernelBenchmarkResult& result, const char* name) {
    result.backends.push_back(KernelBackendBenchmark{
        .name = name,
        .available = false,
        .average_us = 0.0,
        .max_abs_diff = 0.0F,
        .checksum = 0.0F,
    });
}

template <typename Callable>
double measure_average_us(std::int32_t repeat, Callable&& callable) {
    require_positive(repeat, "repeat");
    const auto begin = std::chrono::steady_clock::now();
    for (std::int32_t index = 0; index < repeat; ++index) {
        callable();
    }
    const auto end = std::chrono::steady_clock::now();
    const std::chrono::duration<double> elapsed = end - begin;
    return elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double>(repeat);
}

}  // namespace

PackedLinearI8 pack_linear_i8(const QuantizedLinearLayer& layer,
                              std::int32_t output_block_size) {
    validate_quantized_layer(layer);
    require_positive(output_block_size, "output_block_size");
```

```
95
96      PackedLinearI8 packed;
97      packed.input_size = layer.input_size;
98      packed.output_size = layer.output_size;
99      packed.output_block_size = output_block_size;
100     packed.scale = layer.scale;
101     packed.bias = layer.bias;
102
103     const std::int32_t padded_outputs =
104         round_up_to_block(layer.output_size, output_block_size);
105     packed.packed_weights.assign(
106         checked_size(padded_outputs, "padded_outputs") *
107             checked_size(layer.input_size, "input_size"),
108         0);
109
110     std::size_t packed_index = 0;
111     for (std::int32_t output_block = 0; output_block < padded_outputs;
112          output_block += output_block_size) {
113         for (std::int32_t input_index = 0; input_index < layer.input_size; ++↩
    ↪ input_index) {
114             for (std::int32_t offset = 0; offset < output_block_size; ++offset)↩
    ↪  {
115                 const std::int32_t output_index = output_block + offset;
116                 if (output_index < layer.output_size) {
117                     const std::size_t source =
118                         checked_size(output_index, "output_index") *
119                             checked_size(layer.input_size, "input_size") +
120                         checked_size(input_index, "input_index");
121                     packed.packed_weights[packed_index] = layer.weights[source↩
    ↪ ];
122                 }
123                 ++packed_index;
124             }
125         }
126     }
127     return packed;
128 }
129
130 void linear_i8_packed(const PackedLinearI8& layer,
131                       std::span<const float> input,
132                       std::span<float> output) {
133     std::vector<float> accumulators(
134         checked_size(layer.output_block_size, "output_block_size"),
135         0.0F);
136     linear_i8_packed_with_workspace(layer, input, output, accumulators);
137 }
138
139 void linear_i8_packed_with_workspace(const PackedLinearI8& layer,
140                                      std::span<const float> input,
141                                      std::span<float> output,
142                                      std::span<float> accumulator_workspace) {
143     require_positive(layer.input_size, "packed input_size");
```

```
144     require_positive(layer.output_size, "packed output_size");
145     require_positive(layer.output_block_size, "packed output_block_size");
146     if (input.size() != checked_size(layer.input_size, "input_size")) {
147         throw std::runtime_error("linear_i8_packed input size mismatch");
148     }
149     if (output.size() != checked_size(layer.output_size, "output_size")) {
150         throw std::runtime_error("linear_i8_packed output size mismatch");
151     }
152     if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
153         throw std::runtime_error("linear_i8_packed bias size mismatch");
154     }
155 
156     const std::int32_t padded_outputs =
157         round_up_to_block(layer.output_size, layer.output_block_size);
158     const std::size_t expected_packed_size =
159         checked_size(padded_outputs, "padded_outputs") *
160         checked_size(layer.input_size, "input_size");
161     if (layer.packed_weights.size() != expected_packed_size) {
162         throw std::runtime_error("linear_i8_packed packed weight size mismatch"↩
    ↪ );
163     }
164     if (accumulator_workspace.size() != checked_size(layer.output_block_size, "↩
    ↪ output_block_size")) {
165         throw std::runtime_error("linear_i8_packed accumulator workspace size ↩
    ↪ mismatch");
166     }
167 
168     std::size_t packed_index = 0;
169     for (std::int32_t output_block = 0; output_block < padded_outputs;
170          output_block += layer.output_block_size) {
171         std::fill(accumulator_workspace.begin(), accumulator_workspace.end(), ↩
    ↪ 0.0F);
172         for (std::int32_t offset = 0; offset < layer.output_block_size; ++↩
    ↪ offset) {
173             const std::int32_t output_index = output_block + offset;
174             if (output_index < layer.output_size) {
175                 accumulator_workspace[checked_size(offset, "offset")] =
176                     layer.bias[checked_size(output_index, "output_index")];
177             }
178         }
179 
180         for (std::int32_t input_index = 0; input_index < layer.input_size; ++↩
    ↪ input_index) {
181             const float input_value = input[checked_size(input_index, "↩
    ↪ input_index")];
182             for (std::int32_t offset = 0; offset < layer.output_block_size; ++↩
    ↪ offset) {
183                 const std::int32_t output_index = output_block + offset;
184                 const std::int8_t weight = layer.packed_weights[packed_index↩
    ↪ ++];
185                 if (output_index < layer.output_size) {
186                     accumulator_workspace[checked_size(offset, "offset")] +=
```

```
                        static_cast<float>(weight) * layer.scale * input_value;
                }
            }
        }

        for (std::int32_t offset = 0; offset < layer.output_block_size; ++offset) {
            const std::int32_t output_index = output_block + offset;
            if (output_index < layer.output_size) {
                output[checked_size(output_index, "output_index")] =
                    accumulator_workspace[checked_size(offset, "offset")];
            }
        }
    }
}

QuantizedLinearLayer make_deterministic_i8_layer(std::int32_t input_size,
                                                 std::int32_t output_size,
                                                 float scale) {
    require_positive(input_size, "input_size");
    require_positive(output_size, "output_size");
    QuantizedLinearLayer layer;
    layer.input_size = input_size;
    layer.output_size = output_size;
    layer.scale = scale;
    const std::size_t weight_count =
        checked_size(input_size, "input_size") * checked_size(output_size, "output_size");
    layer.weights.resize(weight_count);
    layer.bias.resize(checked_size(output_size, "output_size"));
    for (std::int32_t output_index = 0; output_index < output_size; ++output_index) {
        layer.bias[checked_size(output_index, "output_index")] =
            static_cast<float>((output_index % LCQI_INPUT_PATTERN_MODULUS) -
                               LCQI_INPUT_PATTERN_CENTER) *
            LCQI_INPUT_PATTERN_SCALE;
        for (std::int32_t input_index = 0; input_index < input_size; ++input_index) {
            const std::int32_t raw =
                ((output_index * input_size + input_index) % LCQI_I8_PATTERN_MODULUS) -
                LCQI_I8_PATTERN_CENTER;
            layer.weights[checked_size(output_index, "output_index") *
                              checked_size(input_size, "input_size") +
                          checked_size(input_index, "input_index")] =
                static_cast<std::int8_t>(raw);
        }
    }
    return layer;
}

std::vector<float> make_deterministic_input(std::int32_t input_size) {
```

```
234     require_positive(input_size, "input_size");
235     std::vector<float> input(checked_size(input_size, "input_size"));
236     for (std::int32_t input_index = 0; input_index < input_size; ++input_index)
        ↪  {
237         input[checked_size(input_index, "input_index")] =
238             static_cast<float>((input_index % LCQI_INPUT_PATTERN_MODULUS) -
239                                LCQI_INPUT_PATTERN_CENTER) *
240             LCQI_INPUT_PATTERN_SCALE;
241     }
242     return input;
243 }
244
245 float max_abs_diff(std::span<const float> lhs, std::span<const float> rhs) {
246     if (lhs.size() != rhs.size()) {
247         throw std::runtime_error("max_abs_diff size mismatch");
248     }
249     float diff = 0.0F;
250     for (std::size_t index = 0; index < lhs.size(); ++index) {
251         diff = std::max(diff, std::fabs(lhs[index] - rhs[index]));
252     }
253     return diff;
254 }
255
256 KernelBenchmarkResult benchmark_linear_i8_case(const KernelBenchmarkCase& ↩
    ↪ benchmark_case) {
257     require_positive(benchmark_case.input_size, "benchmark input_size");
258     require_positive(benchmark_case.output_size, "benchmark output_size");
259     require_positive(benchmark_case.output_block_size, "benchmark ↩
    ↪ output_block_size");
260     require_positive(benchmark_case.repeat, "benchmark repeat");
261
262     const QuantizedLinearLayer layer =
263         make_deterministic_i8_layer(benchmark_case.input_size,
264                                     benchmark_case.output_size,
265                                     0.015625F);
266     const PackedLinearI8 packed = pack_linear_i8(layer, benchmark_case.↩
    ↪ output_block_size);
267     const PackedLinearI8 packed_simd = pack_linear_i8(layer, ↩
    ↪ LCQI_SIMD_OUTPUT_BLOCK);
268     const std::vector<float> input = make_deterministic_input(benchmark_case.↩
    ↪ input_size);
269     std::vector<float> scalar_output(checked_size(benchmark_case.output_size, "↩
    ↪ output_size"),
270                                      0.0F);
271     std::vector<float> packed_output(scalar_output.size(), 0.0F);
272     std::vector<float> simd_output(scalar_output.size(), 0.0F);
273     std::vector<float> packed_workspace(
274         checked_size(benchmark_case.output_block_size, "output_block_size"),
275         0.0F);
276
277     linear_i8(layer, input, scalar_output);
278     linear_i8_packed_with_workspace(packed, input, packed_output, ↩
```

```
    ↪ packed_workspace);

    KernelBenchmarkResult result;
    result.benchmark_case = benchmark_case;
    result.backends.reserve(LCQI_BENCHMARK_BACKEND_COUNT);
    result.backends.push_back(KernelBackendBenchmark{
        .name = LCQI_BACKEND_SCALAR,
        .available = true,
        .average_us = measure_average_us(benchmark_case.repeat,
                                         [&]() { linear_i8(layer, input, ↩
    ↪ scalar_output); }),
        .max_abs_diff = 0.0F,
        .checksum = checksum(scalar_output),
    });
    result.backends.push_back(KernelBackendBenchmark{
        .name = LCQI_BACKEND_PACKED_SCALAR,
        .available = true,
        .average_us = measure_average_us(
            benchmark_case.repeat,
            [&]() { linear_i8_packed_with_workspace(packed,
                                                    input,
                                                    packed_output,
                                                    packed_workspace); }),
        .max_abs_diff = max_abs_diff(scalar_output, packed_output),
        .checksum = checksum(packed_output),
    });
    if (linear_i8_packed_avx2_available()) {
        linear_i8_packed_avx2(packed_simd, input, simd_output);
        result.backends.push_back(KernelBackendBenchmark{
            .name = LCQI_BACKEND_AVX2,
            .available = true,
            .average_us = measure_average_us(
                benchmark_case.repeat,
                [&]() { linear_i8_packed_avx2(packed_simd, input, simd_output);↩
    ↪  }),
            .max_abs_diff = max_abs_diff(scalar_output, simd_output),
            .checksum = checksum(simd_output),
        });
    } else {
        add_unavailable_backend(result, LCQI_BACKEND_AVX2);
    }
    return result;
}

}  // namespace lcqi

#if !defined(LCQI_ENABLE_AVX2)
namespace lcqi {

bool linear_i8_packed_avx2_available() noexcept {
    return false;
}
```

```
328
329 void linear_i8_packed_avx2(const PackedLinearI8&, std::span<const float>, std::↩
      ↪ span<float>) {
330     throw std::runtime_error("AVX2 packed kernel is not enabled in this build")↩
      ↪ ;
331 }
332
333 }  // namespace lcqi
334 #endif
```

int8_kernels_avx2.cpp 是平台优化路径。读者要先看可用性判断，再看向量化实现；如果机器不支持 AVX2/FMA，报告必须写 unavailable。

Listing 4.5 src/int8_kernels_avx2.cpp

```
1  #include <lcqi/int8_kernels.hpp>
2
3  #if defined(LCQI_ENABLE_AVX2)
4
5  #include <immintrin.h>
6
7  #include <algorithm>
8  #include <cstdint>
9  #include <cstring>
10 #include <stdexcept>
11 #include <string>
12
13 namespace lcqi {
14 namespace {
15
16 constexpr std::int32_t LCQI_AVX2_OUTPUT_BLOCK = 8;
17
18 void require_positive(std::int32_t value, const char* name) {
19     if (value <= 0) {
20         throw std::runtime_error(std::string(name) + " must be positive");
21     }
22 }
23
24 std::size_t checked_size(std::int32_t value, const char* name) {
25     if (value < 0) {
26         throw std::runtime_error(std::string(name) + " cannot be negative");
27     }
28     return static_cast<std::size_t>(value);
29 }
30
31 std::int32_t round_up_to_block(std::int32_t value, std::int32_t block_size) {
32     require_positive(value, "value");
33     require_positive(block_size, "block_size");
34     return ((value + block_size - 1) / block_size) * block_size;
35 }
36
37 void validate_avx2_contract(const PackedLinearI8& layer,
38                             std::span<const float> input,
```

```
                              std::span<float> output) {
    require_positive(layer.input_size, "packed input_size");
    require_positive(layer.output_size, "packed output_size");
    if (layer.output_block_size != LCQI_AVX2_OUTPUT_BLOCK) {
        throw std::runtime_error("AVX2 packed kernel requires output_block_size
    ↪  == 8");
    }
    if (input.size() != checked_size(layer.input_size, "input_size")) {
        throw std::runtime_error("AVX2 packed kernel input size mismatch");
    }
    if (output.size() != checked_size(layer.output_size, "output_size")) {
        throw std::runtime_error("AVX2 packed kernel output size mismatch");
    }
    if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
        throw std::runtime_error("AVX2 packed kernel bias size mismatch");
    }
    const std::int32_t padded_outputs =
        round_up_to_block(layer.output_size, layer.output_block_size);
    const std::size_t expected_packed_size =
        checked_size(padded_outputs, "padded_outputs") *
        checked_size(layer.input_size, "input_size");
    if (layer.packed_weights.size() != expected_packed_size) {
        throw std::runtime_error("AVX2 packed kernel packed weight size
    ↪ mismatch");
    }
}

}  // namespace

bool linear_i8_packed_avx2_available() noexcept {
#if defined(__GNUC__) || defined(__clang__)
    __builtin_cpu_init();
    return __builtin_cpu_supports("avx2") && __builtin_cpu_supports("fma");
#else
    return true;
#endif
}

void linear_i8_packed_avx2(const PackedLinearI8& layer,
                           std::span<const float> input,
                           std::span<float> output) {
    if (!linear_i8_packed_avx2_available()) {
        throw std::runtime_error("AVX2/FMA is not available on this CPU");
    }
    validate_avx2_contract(layer, input, output);

    const std::int32_t padded_outputs =
        round_up_to_block(layer.output_size, layer.output_block_size);
    std::size_t packed_index = 0;
    alignas(32) float block_output[LCQI_AVX2_OUTPUT_BLOCK] = {};

    for (std::int32_t output_block = 0; output_block < padded_outputs;
```

```
89          output_block += LCQI_AVX2_OUTPUT_BLOCK) {
90          __m256 accumulator = _mm256_setzero_ps();
91
92          for (std::int32_t offset = 0; offset < LCQI_AVX2_OUTPUT_BLOCK; ++offset
    ↪ ) {
93              const std::int32_t output_index = output_block + offset;
94              block_output[checked_size(offset, "offset")] =
95                  output_index < layer.output_size
96                      ? layer.bias[checked_size(output_index, "output_index")]
97                      : 0.0F;
98          }
99          accumulator = _mm256_load_ps(block_output);
100
101         for (std::int32_t input_index = 0; input_index < layer.input_size; ++
    ↪ input_index) {
102             std::uint64_t packed_bytes = 0;
103             std::memcpy(&packed_bytes,
104                         layer.packed_weights.data() + packed_index,
105                         sizeof(packed_bytes));
106             const __m128i packed_i8 =
107                 _mm_cvtsi64_si128(static_cast<long long>(packed_bytes));
108             packed_index += LCQI_AVX2_OUTPUT_BLOCK;
109             const __m256i weights_i32 = _mm256_cvtepi8_epi32(packed_i8);
110             const __m256 weights_f32 = _mm256_cvtepi32_ps(weights_i32);
111             const __m256 input_scaled =
112                 _mm256_set1_ps(input[checked_size(input_index, "input_index")] ↩
    ↪ * layer.scale);
113             accumulator = _mm256_fmadd_ps(weights_f32, input_scaled, ↩
    ↪ accumulator);
114         }
115
116         _mm256_store_ps(block_output, accumulator);
117         for (std::int32_t offset = 0; offset < LCQI_AVX2_OUTPUT_BLOCK; ++offset
    ↪ ) {
118             const std::int32_t output_index = output_block + offset;
119             if (output_index < layer.output_size) {
120                 output[checked_size(output_index, "output_index")] =
121                     block_output[checked_size(offset, "offset")];
122             }
123         }
124     }
125 }
126
127 }  // namespace lcqi
128
129 #endif
```

第 5 章

Reference Decoder、GPT-2 与模型导入

5.1 从 tiny MLP 走向 decoder-only 模型

Reference decoder 和 GPT-2 reference path 是 LCQI 从教学闭环走向真实模型导入的关键层。这里的代码更长，是因为真实模型不只有矩阵乘法：它还需要 tokenizer、位置、归一化、QKV 拆分、attention mask、KV cache、激活函数、lm head、权重命名映射、dtype 转换和坏资产拒绝。

5.2 Tiny Reference Decoder

reference_decoder.cpp 实现 embedding、RMSNorm、Q/K/V、RoPE、KV cache、attention、SwiGLU、final norm、logits 和 sampler，并在关键位置生成 trace checkpoint。读者应按 checkpoint 顺序读，而不是从最终 logits 倒推。

Listing 5.1 src/reference_decoder.cpp

```cpp
#include <lcqi/reference_decoder.hpp>

#include <algorithm>
#include <array>
#include <cmath>
#include <limits>
#include <stdexcept>
#include <string>

namespace lcqi {
namespace {

constexpr float LCQI_SOFTMAX_NEGATIVE_INFINITY = -std::numeric_limits<float>::infinity();
constexpr std::int32_t LCQI_ROPE_PAIR_WIDTH = 2;
constexpr const char* LCQI_TRACE_DTYPE_F32 = "f32";
constexpr const char* LCQI_TRACE_LAYOUT_CONTIGUOUS = "contiguous";
constexpr const char* LCQI_TRACE_LAYOUT_ROW_MAJOR = "row_major";
constexpr const char* LCQI_TRACE_LAYOUT_KV_CONTIGUOUS = "kv_contiguous";

void require_positive(std::int32_t value, const char* name) {
    if (value <= 0) {
        throw std::runtime_error(std::string(name) + " must be positive");
    }
}

void validate_config(const DecoderConfig& config) {
    require_positive(config.hidden_size, "hidden_size");
    require_positive(config.query_heads, "query_heads");
    require_positive(config.kv_heads, "kv_heads");
```

```
    require_positive(config.head_dim, "head_dim");
    require_positive(config.intermediate_size, "intermediate_size");
    require_positive(config.vocab_size, "vocab_size");
    require_positive(config.max_context, "max_context");
    if (config.query_heads % config.kv_heads != 0) {
        throw std::runtime_error("query_heads must be divisible by kv_heads");
    }
    if (config.hidden_size != config.query_heads * config.head_dim) {
        throw std::runtime_error("hidden_size must equal query_heads * head_dim");
    }
    if (config.head_dim % LCQI_ROPE_PAIR_WIDTH != 0) {
        throw std::runtime_error("head_dim must be even for RoPE");
    }
    if (config.rope_base <= 1.0F) {
        throw std::runtime_error("rope_base must be greater than 1");
    }
}

std::size_t checked_size(std::int32_t value, const char* name) {
    if (value < 0) {
        throw std::runtime_error(std::string(name) + " cannot be negative");
    }
    return static_cast<std::size_t>(value);
}

std::int32_t kv_width(const DecoderConfig& config) {
    return config.kv_heads * config.head_dim;
}

void validate_linear(const LinearWeightsF32& layer, const char* name) {
    require_positive(layer.input_size, "linear input_size");
    require_positive(layer.output_size, "linear output_size");
    const std::size_t expected_weights =
        checked_size(layer.input_size, name) * checked_size(layer.output_size, name);
    if (layer.weights.size() != expected_weights) {
        throw std::runtime_error(std::string(name) + " weight size mismatch");
    }
    if (!layer.bias.empty() &&
        layer.bias.size() != checked_size(layer.output_size, name)) {
        throw std::runtime_error(std::string(name) + " bias size mismatch");
    }
}

void validate_span_size(std::size_t actual, std::int32_t expected, const char* name) {
    if (actual != checked_size(expected, name)) {
        throw std::runtime_error(std::string(name) + " size mismatch");
    }
}

```

```cpp
std::int32_t argmax(std::span<const float> values) {
    if (values.empty()) {
        throw std::runtime_error("cannot take argmax of empty values");
    }
    std::int32_t best_index = 0;
    float best_value = values[0];
    for (std::int32_t index = 1; index < static_cast<std::int32_t>(values.size()); ++index) {
        const float candidate = values[static_cast<std::size_t>(index)];
        if (candidate > best_value) {
            best_value = candidate;
            best_index = index;
        }
    }
    return best_index;
}

float silu(float value) {
    return value / (1.0F + std::exp(-value));
}

std::span<const float> row_span(std::span<const float> values,
                                std::int32_t row,
                                std::int32_t row_width) {
    const std::size_t begin = checked_size(row, "row") * checked_size(row_width, "row_width");
    return values.subspan(begin, checked_size(row_width, "row_width"));
}

void add_inplace(std::span<float> target, std::span<const float> source) {
    if (target.size() != source.size()) {
        throw std::runtime_error("residual add size mismatch");
    }
    for (std::size_t index = 0; index < target.size(); ++index) {
        target[index] += source[index];
    }
}

void swiglu_mlp(const DecoderLayerWeightsF32& layer,
                std::span<const float> input,
                std::span<float> output) {
    std::vector<float> gate(checked_size(layer.w_gate.output_size, "gate"), 0.0F);
    std::vector<float> up(checked_size(layer.w_up.output_size, "up"), 0.0F);
    std::vector<float> hidden(gate.size(), 0.0F);
    linear_f32(layer.w_gate, input, gate);
    linear_f32(layer.w_up, input, up);
    if (gate.size() != up.size()) {
        throw std::runtime_error("SwiGLU gate/up size mismatch");
    }
    for (std::size_t index = 0; index < hidden.size(); ++index) {
        hidden[index] = silu(gate[index]) * up[index];
```

```
    }
    linear_f32(layer.w_down, hidden, output);
}

float checksum(std::span<const float> values) {
    float sum = 0.0F;
    for (const float value : values) {
        sum += value;
    }
    return sum;
}

float max_abs(std::span<const float> values) {
    float result = 0.0F;
    for (const float value : values) {
        result = std::max(result, std::fabs(value));
    }
    return result;
}

std::vector<std::int32_t> contiguous_stride(std::span<const std::int32_t> shape
    ↪ ) {
    std::vector<std::int32_t> stride(shape.size(), 1);
    std::int32_t running = 1;
    for (std::int32_t index = static_cast<std::int32_t>(shape.size()) - 1; ↩
    ↪ index >= 0; --index) {
        stride[checked_size(index, "stride index")] = running;
        running *= shape[checked_size(index, "shape index")];
    }
    return stride;
}

void add_checkpoint(std::vector<ReferenceTensorCheckpoint>* checkpoints,
                    const char* name,
                    std::int32_t token_position,
                    std::int32_t layer_id,
                    std::span<const std::int32_t> shape,
                    const char* layout,
                    std::span<const float> values) {
    if (checkpoints == nullptr) {
        return;
    }
    ReferenceTensorCheckpoint checkpoint;
    checkpoint.name = name;
    checkpoint.token_position = token_position;
    checkpoint.layer_id = layer_id;
    checkpoint.shape.assign(shape.begin(), shape.end());
    checkpoint.stride = contiguous_stride(shape);
    checkpoint.dtype = LCQI_TRACE_DTYPE_F32;
    checkpoint.layout = layout;
    checkpoint.checksum = checksum(values);
    checkpoint.max_abs = max_abs(values);
```

```
    checkpoint.values.assign(values.begin(), values.end());
    checkpoints->push_back(std::move(checkpoint));
}

void swiglu_mlp_with_trace(const DecoderLayerWeightsF32& layer,
                           std::span<const float> input,
                           std::span<float> output,
                           std::int32_t token_position,
                           std::int32_t layer_id,
                           std::vector<ReferenceTensorCheckpoint>* checkpoints) {
    std::vector<float> gate(checked_size(layer.w_gate.output_size, "gate"), 0.0F);
    std::vector<float> up(checked_size(layer.w_up.output_size, "up"), 0.0F);
    std::vector<float> hidden(gate.size(), 0.0F);
    linear_f32(layer.w_gate, input, gate);
    linear_f32(layer.w_up, input, up);
    if (gate.size() != up.size()) {
        throw std::runtime_error("SwiGLU gate/up size mismatch");
    }
    const std::array<std::int32_t, 1> intermediate_shape{layer.w_gate.output_size};
    add_checkpoint(checkpoints,
                   "mlp_gate",
                   token_position,
                   layer_id,
                   intermediate_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   gate);
    add_checkpoint(checkpoints,
                   "mlp_up",
                   token_position,
                   layer_id,
                   intermediate_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   up);
    for (std::size_t index = 0; index < hidden.size(); ++index) {
        hidden[index] = silu(gate[index]) * up[index];
    }
    add_checkpoint(checkpoints,
                   "mlp_hidden",
                   token_position,
                   layer_id,
                   intermediate_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   hidden);
    linear_f32(layer.w_down, hidden, output);
}

void forward_layer(const DecoderConfig& config,
                   const DecoderLayerWeightsF32& layer,
                   std::int32_t layer_id,
```

```
227                     std::int32_t model_position,
228                     ReferenceKVCache& cache,
229                     std::span<float> hidden,
230                     std::vector<ReferenceTensorCheckpoint>* checkpoints = ↩
    ↪ nullptr) {
231     validate_span_size(hidden.size(), config.hidden_size, "hidden");
232 
233     std::vector<float> normed(checked_size(config.hidden_size, "hidden_size"), ↩
    ↪ 0.0F);
234     std::vector<float> query(checked_size(config.hidden_size, "hidden_size"), ↩
    ↪ 0.0F);
235     std::vector<float> key(checked_size(kv_width(config), "kv_width"), 0.0F);
236     std::vector<float> value(checked_size(kv_width(config), "kv_width"), 0.0F);
237     std::vector<float> attention(checked_size(config.hidden_size, "hidden_size"↩
    ↪ ), 0.0F);
238     std::vector<float> attention_projected(checked_size(config.hidden_size, "↩
    ↪ hidden_size"), 0.0F);
239     std::vector<float> mlp(checked_size(config.hidden_size, "hidden_size"), 0.0↩
    ↪ F);
240 
241     const std::array<std::int32_t, 1> hidden_shape{config.hidden_size};
242     const std::array<std::int32_t, 2> query_shape{config.query_heads, config.↩
    ↪ head_dim};
243     const std::array<std::int32_t, 2> kv_shape{config.kv_heads, config.head_dim↩
    ↪ };
244     const std::array<std::int32_t, 4> kv_slot_shape{
245         1, 1, config.kv_heads, config.head_dim};
246 
247     rms_norm(hidden, layer.rms_attention_weight, config.rms_epsilon, normed);
248     add_checkpoint(checkpoints,
249                    "attn_norm",
250                    model_position,
251                    layer_id,
252                    hidden_shape,
253                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
254                    normed);
255     linear_f32(layer.wq, normed, query);
256     linear_f32(layer.wk, normed, key);
257     linear_f32(layer.wv, normed, value);
258     add_checkpoint(checkpoints,
259                    "q_before_rope",
260                    model_position,
261                    layer_id,
262                    query_shape,
263                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
264                    query);
265     add_checkpoint(checkpoints,
266                    "k_before_rope",
267                    model_position,
268                    layer_id,
269                    kv_shape,
270                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
```

```
                    key);
    add_checkpoint(checkpoints,
                    "v",
                    model_position,
                    layer_id,
                    kv_shape,
                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
                    value);
    apply_rope(query, config.query_heads, config.head_dim, model_position, config.rope_base);
    apply_rope(key, config.kv_heads, config.head_dim, model_position, config.rope_base);
    add_checkpoint(checkpoints,
                    "q_after_rope",
                    model_position,
                    layer_id,
                    query_shape,
                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
                    query);
    add_checkpoint(checkpoints,
                    "k_after_rope",
                    model_position,
                    layer_id,
                    kv_shape,
                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
                    key);

    cache.append(layer_id, model_position, key, value);
    add_checkpoint(checkpoints,
                    "kv_cache_key_slot",
                    model_position,
                    layer_id,
                    kv_slot_shape,
                    LCQI_TRACE_LAYOUT_KV_CONTIGUOUS,
                    key);
    attention_decode(config, cache, layer_id, model_position, query, attention);
    add_checkpoint(checkpoints,
                    "attention_out",
                    model_position,
                    layer_id,
                    hidden_shape,
                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
                    attention);
    linear_f32(layer.wo, attention, attention_projected);
    add_inplace(hidden, attention_projected);
    add_checkpoint(checkpoints,
                    "post_attention_residual",
                    model_position,
                    layer_id,
                    hidden_shape,
                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
```

```
                    hidden);

    rms_norm(hidden, layer.rms_mlp_weight, config.rms_epsilon, normed);
    add_checkpoint(checkpoints,
                   "mlp_norm",
                   model_position,
                   layer_id,
                   hidden_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   normed);
    if (checkpoints == nullptr) {
        swiglu_mlp(layer, normed, mlp);
    } else {
        swiglu_mlp_with_trace(layer, normed, mlp, model_position, layer_id, ↩
    ↪ checkpoints);
    }
    add_inplace(hidden, mlp);
    add_checkpoint(checkpoints,
                   "layer_out",
                   model_position,
                   layer_id,
                   hidden_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   hidden);
}

LinearWeightsF32 make_linear(std::int32_t input_size,
                             std::int32_t output_size,
                             std::vector<float> weights,
                             std::vector<float> bias = {}) {
    LinearWeightsF32 layer;
    layer.input_size = input_size;
    layer.output_size = output_size;
    layer.weights = std::move(weights);
    layer.bias = std::move(bias);
    validate_linear(layer, "tiny linear");
    return layer;
}

std::vector<float> zeros(std::int32_t count) {
    return std::vector<float>(checked_size(count, "zero count"), 0.0F);
}

}  // namespace

ReferenceDecodeTraceResult run_reference_decode_with_trace(
    const ReferenceDecoderModel& model,
    std::span<const std::int32_t> token_ids) {
    validate_config(model.config);
    if (token_ids.empty()) {
        throw std::runtime_error("reference decode needs at least one token");
    }
```

```
371     if (token_ids.size() > checked_size(model.config.max_context, "max_context"↩
    ↪ )) {
372         throw std::runtime_error("token_ids exceed max_context");
373     }
374     if (model.token_embedding.size() !=
375         checked_size(model.config.vocab_size, "vocab_size") *
376             checked_size(model.config.hidden_size, "hidden_size")) {
377         throw std::runtime_error("token embedding size mismatch");
378     }
379     if (model.final_norm_weight.size() != checked_size(model.config.hidden_size↩
    ↪ , "hidden_size")) {
380         throw std::runtime_error("final norm weight size mismatch");
381     }
382     validate_linear(model.lm_head, "lm_head");
383 
384     ReferenceDecodeTraceResult trace;
385     ReferenceKVCache cache(model.config, static_cast<std::int32_t>(model.layers↩
    ↪ .size()));
386     std::vector<float> hidden(checked_size(model.config.hidden_size, "↩
    ↪ hidden_size"), 0.0F);
387     const std::array<std::int32_t, 1> hidden_shape{model.config.hidden_size};
388     for (std::int32_t position = 0; position < static_cast<std::int32_t>(↩
    ↪ token_ids.size());
389          ++position) {
390         const std::int32_t token_id = token_ids[checked_size(position, "↩
    ↪ position")];
391         if (token_id < 0 || token_id >= model.config.vocab_size) {
392             throw std::runtime_error("token id out of vocabulary");
393         }
394         const std::span<const float> embedding =
395             row_span(model.token_embedding, token_id, model.config.hidden_size)↩
    ↪ ;
396         std::copy(embedding.begin(), embedding.end(), hidden.begin());
397         add_checkpoint(&trace.checkpoints,
398                        "embedding",
399                        position,
400                        -1,
401                        hidden_shape,
402                        LCQI_TRACE_LAYOUT_ROW_MAJOR,
403                        hidden);
404         for (std::int32_t layer_id = 0; layer_id < static_cast<std::int32_t>(↩
    ↪ model.layers.size());
405              ++layer_id) {
406             forward_layer(model.config,
407                           model.layers[checked_size(layer_id, "layer_id")],
408                           layer_id,
409                           position,
410                           cache,
411                           hidden,
412                           &trace.checkpoints);
413         }
414     }
```

```
415
416     std::vector<float> normed(checked_size(model.config.hidden_size, "
        ↪ hidden_size"), 0.0F);
417     std::vector<float> logits(checked_size(model.config.vocab_size, "vocab_size
        ↪ "), 0.0F);
418     const std::array<std::int32_t, 1> logits_shape{model.config.vocab_size};
419     const std::int32_t last_position = static_cast<std::int32_t>(token_ids.size
        ↪ ()) - 1;
420     rms_norm(hidden, model.final_norm_weight, model.config.rms_epsilon, normed)
        ↪ ;
421     add_checkpoint(&trace.checkpoints,
422                    "final_norm",
423                    last_position,
424                    -1,
425                    hidden_shape,
426                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
427                    normed);
428     linear_f32(model.lm_head, normed, logits);
429     add_checkpoint(&trace.checkpoints,
430                    "logits",
431                    last_position,
432                    -1,
433                    logits_shape,
434                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
435                    logits);
436
437     trace.result.predicted_token = argmax(logits);
438     trace.result.logits = std::move(logits);
439     return trace;
440 }
441
442 ReferenceKVCache::ReferenceKVCache(const DecoderConfig& config, std::int32_t
        ↪ layer_count)
443     : config_(config),
444       layer_count_(layer_count) {
445     validate_config(this->config_);
446     require_positive(this->layer_count_, "layer_count");
447     const std::size_t element_count =
448         checked_size(this->layer_count_, "layer_count") *
449         checked_size(this->config_.max_context, "max_context") *
450         checked_size(this->config_.kv_heads, "kv_heads") *
451         checked_size(this->config_.head_dim, "head_dim");
452     this->keys_.assign(element_count, 0.0F);
453     this->values_.assign(element_count, 0.0F);
454     this->written_.assign(
455         checked_size(this->layer_count_, "layer_count") *
456             checked_size(this->config_.max_context, "max_context"),
457         0);
458 }
459
460 void ReferenceKVCache::append(std::int32_t layer_id,
461                               std::int32_t model_position,
```

```
462                                 std::span<const float> key,
463                                 std::span<const float> value) {
464     validate_span_size(key.size(), kv_width(this->config_), "KV key");
465     validate_span_size(value.size(), kv_width(this->config_), "KV value");
466     this->validate_address(layer_id, model_position, 0);
467     for (std::int32_t kv_head = 0; kv_head < this->config_.kv_heads; ++kv_head)
    ↪  {
468         const std::size_t target = this->base_offset(layer_id, model_position,
    ↪ kv_head);
469         const std::size_t source =
470             checked_size(kv_head, "kv_head") * checked_size(this->config_.
    ↪ head_dim, "head_dim");
471         std::copy_n(key.begin() + static_cast<std::ptrdiff_t>(source),
472                     this->config_.head_dim,
473                     this->keys_.begin() + static_cast<std::ptrdiff_t>(target));
474         std::copy_n(value.begin() + static_cast<std::ptrdiff_t>(source),
475                     this->config_.head_dim,
476                     this->values_.begin() + static_cast<std::ptrdiff_t>(target)
    ↪ );
477     }
478     const std::size_t written_index =
479         checked_size(layer_id, "layer_id") * checked_size(this->config_.
    ↪ max_context, "max_context") +
480         checked_size(model_position, "model_position");
481     this->written_[written_index] = 1;
482     this->filled_tokens_ = std::max(this->filled_tokens_, model_position + 1);
483 }
484
485 std::span<const float> ReferenceKVCache::key(std::int32_t layer_id,
486                                              std::int32_t model_position,
487                                              std::int32_t kv_head) const {
488     this->validate_address(layer_id, model_position, kv_head);
489     const std::size_t offset = this->base_offset(layer_id, model_position,
    ↪ kv_head);
490     return std::span<const float>(this->keys_.data() + offset,
491                                   checked_size(this->config_.head_dim, "
    ↪ head_dim"));
492 }
493
494 std::span<const float> ReferenceKVCache::value(std::int32_t layer_id,
495                                                std::int32_t model_position,
496                                                std::int32_t kv_head) const {
497     this->validate_address(layer_id, model_position, kv_head);
498     const std::size_t offset = this->base_offset(layer_id, model_position,
    ↪ kv_head);
499     return std::span<const float>(this->values_.data() + offset,
500                                   checked_size(this->config_.head_dim, "
    ↪ head_dim"));
501 }
502
503 std::int32_t ReferenceKVCache::layer_count() const noexcept {
504     return this->layer_count_;
```

```
}

std::int32_t ReferenceKVCache::filled_tokens() const noexcept {
    return this->filled_tokens_;
}

std::size_t ReferenceKVCache::base_offset(std::int32_t layer_id,
                                          std::int32_t model_position,
                                          std::int32_t kv_head) const {
    return (((checked_size(layer_id, "layer_id") *
              checked_size(this->config_.max_context, "max_context")) +
             checked_size(model_position, "model_position")) *
                checked_size(this->config_.kv_heads, "kv_heads") +
            checked_size(kv_head, "kv_head")) *
           checked_size(this->config_.head_dim, "head_dim");
}

void ReferenceKVCache::validate_address(std::int32_t layer_id,
                                        std::int32_t model_position,
                                        std::int32_t kv_head) const {
    if (layer_id < 0 || layer_id >= this->layer_count_) {
        throw std::runtime_error("KV layer_id out of range");
    }
    if (model_position < 0 || model_position >= this->config_.max_context) {
        throw std::runtime_error("KV model_position out of range");
    }
    if (kv_head < 0 || kv_head >= this->config_.kv_heads) {
        throw std::runtime_error("KV head out of range");
    }
}

void rms_norm(std::span<const float> input,
              std::span<const float> weight,
              float epsilon,
              std::span<float> output) {
    if (input.size() != weight.size() || input.size() != output.size()) {
        throw std::runtime_error("RMSNorm size mismatch");
    }
    if (input.empty()) {
        throw std::runtime_error("RMSNorm input is empty");
    }
    float sum_square = 0.0F;
    for (const float value : input) {
        sum_square += value * value;
    }
    const float mean_square = sum_square / static_cast<float>(input.size());
    const float scale = 1.0F / std::sqrt(mean_square + epsilon);
    for (std::size_t index = 0; index < input.size(); ++index) {
        output[index] = input[index] * scale * weight[index];
    }
}

```

```
557 void linear_f32(const LinearWeightsF32& layer,
558                 std::span<const float> input,
559                 std::span<float> output) {
560     validate_linear(layer, "linear_f32");
561     validate_span_size(input.size(), layer.input_size, "linear input");
562     validate_span_size(output.size(), layer.output_size, "linear output");
563     for (std::int32_t out = 0; out < layer.output_size; ++out) {
564         const std::size_t row_base =
565             checked_size(out, "out") * checked_size(layer.input_size, "↩
    ↪ input_size");
566         float accumulator = layer.bias.empty() ? 0.0F : layer.bias[checked_size↩
    ↪ (out, "out")];
567         for (std::int32_t in = 0; in < layer.input_size; ++in) {
568             accumulator += layer.weights[row_base + checked_size(in, "in")] *
569                            input[checked_size(in, "in")];
570         }
571         output[checked_size(out, "out")] = accumulator;
572     }
573 }
574
575 void apply_rope(std::span<float> heads,
576                 std::int32_t head_count,
577                 std::int32_t head_dim,
578                 std::int32_t model_position,
579                 float rope_base) {
580     require_positive(head_count, "head_count");
581     require_positive(head_dim, "head_dim");
582     validate_span_size(heads.size(), head_count * head_dim, "RoPE heads");
583     if (head_dim % LCQI_ROPE_PAIR_WIDTH != 0) {
584         throw std::runtime_error("RoPE head_dim must be even");
585     }
586     if (model_position < 0) {
587         throw std::runtime_error("RoPE model_position cannot be negative");
588     }
589     if (rope_base <= 1.0F) {
590         throw std::runtime_error("RoPE base must be greater than 1");
591     }
592     for (std::int32_t head = 0; head < head_count; ++head) {
593         for (std::int32_t offset = 0; offset < head_dim; offset += ↩
    ↪ LCQI_ROPE_PAIR_WIDTH) {
594             const float exponent = static_cast<float>(offset) / static_cast<↩
    ↪ float>(head_dim);
595             const float theta = 1.0F / std::pow(rope_base, exponent);
596             const float angle = static_cast<float>(model_position) * theta;
597             const float cosine = std::cos(angle);
598             const float sine = std::sin(angle);
599             const std::size_t first =
600                 checked_size(head, "head") * checked_size(head_dim, "head_dim")↩
    ↪  +
601                 checked_size(offset, "offset");
602             const float x0 = heads[first];
603             const float x1 = heads[first + 1];
```

```
            heads[first] = x0 * cosine - x1 * sine;
            heads[first + 1] = x0 * sine + x1 * cosine;
        }
    }
}

void attention_decode(const DecoderConfig& config,
                      const ReferenceKVCache& cache,
                      std::int32_t layer_id,
                      std::int32_t model_position,
                      std::span<const float> query,
                      std::span<float> output) {
    validate_config(config);
    validate_span_size(query.size(), config.hidden_size, "attention query");
    validate_span_size(output.size(), config.hidden_size, "attention output");
    if (model_position < 0 || model_position >= cache.filled_tokens()) {
        throw std::runtime_error("attention model_position has not been written");
    }
    std::fill(output.begin(), output.end(), 0.0F);

    const std::int32_t group_size = config.query_heads / config.kv_heads;
    const float score_scale = 1.0F / std::sqrt(static_cast<float>(config.head_dim));
    std::vector<float> scores(checked_size(model_position + 1, "score count"),
                              LCQI_SOFTMAX_NEGATIVE_INFINITY);
    std::vector<float> probabilities(scores.size(), 0.0F);

    for (std::int32_t query_head = 0; query_head < config.query_heads; ++query_head) {
        const std::int32_t kv_head = query_head / group_size;
        const std::size_t query_base =
            checked_size(query_head, "query_head") * checked_size(config.head_dim, "head_dim");
        float max_score = LCQI_SOFTMAX_NEGATIVE_INFINITY;
        for (std::int32_t position = 0; position <= model_position; ++position) {
            const std::span<const float> key = cache.key(layer_id, position, kv_head);
            float dot = 0.0F;
            for (std::int32_t dim = 0; dim < config.head_dim; ++dim) {
                dot += query[query_base + checked_size(dim, "dim")] *
                       key[checked_size(dim, "dim")];
            }
            const float score = dot * score_scale;
            scores[checked_size(position, "position")] = score;
            max_score = std::max(max_score, score);
        }

        float probability_sum = 0.0F;
        for (std::int32_t position = 0; position <= model_position; ++position) {
```

```
649             const std::size_t index = checked_size(position, "position");
650             const float unnormalized = std::exp(scores[index] - max_score);
651             probabilities[index] = unnormalized;
652             probability_sum += unnormalized;
653         }
654         if (probability_sum <= 0.0F) {
655             throw std::runtime_error("attention probability sum is invalid");
656         }
657         for (std::int32_t position = 0; position <= model_position; ++position)
    ↪  {
658             const float probability =
659                 probabilities[checked_size(position, "position")] /
    ↪ probability_sum;
660             const std::span<const float> value = cache.value(layer_id, position
    ↪ , kv_head);
661             for (std::int32_t dim = 0; dim < config.head_dim; ++dim) {
662                 output[query_base + checked_size(dim, "dim")] +=
663                     probability * value[checked_size(dim, "dim")];
664             }
665         }
666     }
667 }
668
669 ReferenceDecodeResult run_reference_decode(const ReferenceDecoderModel& model,
670                                            std::span<const std::int32_t>
    ↪ token_ids) {
671     validate_config(model.config);
672     if (token_ids.empty()) {
673         throw std::runtime_error("reference decode needs at least one token");
674     }
675     if (token_ids.size() > checked_size(model.config.max_context, "max_context"
    ↪ )) {
676         throw std::runtime_error("token_ids exceed max_context");
677     }
678     if (model.token_embedding.size() !=
679         checked_size(model.config.vocab_size, "vocab_size") *
680             checked_size(model.config.hidden_size, "hidden_size")) {
681         throw std::runtime_error("token embedding size mismatch");
682     }
683     if (model.final_norm_weight.size() != checked_size(model.config.hidden_size
    ↪ , "hidden_size")) {
684         throw std::runtime_error("final norm weight size mismatch");
685     }
686     validate_linear(model.lm_head, "lm_head");
687
688     ReferenceKVCache cache(model.config, static_cast<std::int32_t>(model.layers
    ↪ .size()));
689     std::vector<float> hidden(checked_size(model.config.hidden_size, "
    ↪ hidden_size"), 0.0F);
690     for (std::int32_t position = 0; position < static_cast<std::int32_t>(
    ↪ token_ids.size());
691          ++position) {
```

```
692         const std::int32_t token_id = token_ids[checked_size(position, "position")];
693         if (token_id < 0 || token_id >= model.config.vocab_size) {
694             throw std::runtime_error("token id out of vocabulary");
695         }
696         const std::span<const float> embedding =
697             row_span(model.token_embedding, token_id, model.config.hidden_size);
698         std::copy(embedding.begin(), embedding.end(), hidden.begin());
699         for (std::int32_t layer_id = 0; layer_id < static_cast<std::int32_t>(model.layers.size());
700              ++layer_id) {
701             forward_layer(model.config,
702                           model.layers[checked_size(layer_id, "layer_id")],
703                           layer_id,
704                           position,
705                           cache,
706                           hidden);
707         }
708     }
709
710     std::vector<float> normed(checked_size(model.config.hidden_size, "hidden_size"), 0.0F);
711     std::vector<float> logits(checked_size(model.config.vocab_size, "vocab_size"), 0.0F);
712     rms_norm(hidden, model.final_norm_weight, model.config.rms_epsilon, normed);
713     linear_f32(model.lm_head, normed, logits);
714
715     ReferenceDecodeResult result;
716     result.predicted_token = argmax(logits);
717     result.logits = std::move(logits);
718     return result;
719 }
720
721 ReferenceDecoderModel make_tiny_reference_decoder_model() {
722     ReferenceDecoderModel model;
723     model.config.hidden_size = 4;
724     model.config.query_heads = 2;
725     model.config.kv_heads = 1;
726     model.config.head_dim = 2;
727     model.config.intermediate_size = 6;
728     model.config.vocab_size = 5;
729     model.config.max_context = 8;
730     model.config.rms_epsilon = 1.0e-5F;
731     model.config.rope_base = 10000.0F;
732
733     model.token_embedding = {
734         0.20F, -0.10F, 0.05F, 0.30F,
735         -0.40F, 0.10F, 0.25F, -0.20F,
736         0.15F, 0.35F, -0.30F, 0.05F,
737         0.50F, -0.25F, 0.10F, -0.15F,
```

```
        -0.05F, 0.20F, 0.40F, -0.35F,
    };

    DecoderLayerWeightsF32 layer;
    layer.rms_attention_weight = {1.0F, 0.9F, 1.1F, 1.0F};
    layer.wq = make_linear(4,
                           4,
                           {
                               0.20F, -0.10F, 0.05F, 0.30F,
                               -0.15F, 0.25F, 0.10F, -0.05F,
                               0.05F, 0.10F, -0.20F, 0.15F,
                               0.30F, -0.05F, 0.20F, 0.10F,
                           });
    layer.wk = make_linear(4,
                           2,
                           {
                               0.10F, 0.20F, -0.10F, 0.05F,
                               -0.05F, 0.15F, 0.25F, -0.20F,
                           });
    layer.wv = make_linear(4,
                           2,
                           {
                               0.25F, -0.05F, 0.10F, 0.15F,
                               -0.10F, 0.30F, 0.05F, 0.20F,
                           });
    layer.wo = make_linear(4,
                           4,
                           {
                               0.20F, 0.10F, -0.05F, 0.15F,
                               -0.10F, 0.25F, 0.20F, -0.05F,
                               0.05F, -0.20F, 0.30F, 0.10F,
                               0.15F, 0.05F, -0.10F, 0.25F,
                           });
    layer.rms_mlp_weight = {0.95F, 1.05F, 1.0F, 0.9F};
    layer.w_gate = make_linear(4,
                               6,
                               {
                                   0.20F, -0.10F, 0.05F, 0.10F,
                                   -0.05F, 0.15F, 0.20F, -0.10F,
                                   0.10F, 0.05F, -0.15F, 0.25F,
                                   -0.20F, 0.10F, 0.15F, 0.05F,
                                   0.05F, -0.25F, 0.10F, 0.20F,
                                   0.15F, 0.05F, 0.05F, -0.15F,
                               });
    layer.w_up = make_linear(4,
                             6,
                             {
                                 0.10F, 0.20F, -0.05F, 0.05F,
                                 -0.15F, 0.05F, 0.25F, 0.10F,
                                 0.20F, -0.10F, 0.05F, 0.15F,
                                 0.05F, 0.10F, -0.20F, 0.25F,
                                 -0.05F, 0.15F, 0.10F, -0.10F,
```

```
                                    0.25F, -0.05F, 0.05F, 0.10F,
                                });
    layer.w_down = make_linear(6,
                                4,
                                {
                                    0.10F, -0.05F, 0.15F, 0.20F, -0.10F, 0.05F,
                                    -0.20F, 0.10F, 0.05F, -0.05F, 0.15F, 0.20F,
                                    0.05F, 0.15F, -0.10F, 0.10F, 0.20F, -0.05F,
                                    0.20F, 0.05F, 0.10F, -0.15F, -0.05F, 0.10F,
                                });
    model.layers.push_back(std::move(layer));
    model.final_norm_weight = {1.0F, 0.95F, 1.05F, 1.0F};
    model.lm_head = make_linear(4,
                                5,
                                {
                                    0.20F, -0.10F, 0.05F, 0.15F,
                                    -0.05F, 0.25F, 0.10F, -0.20F,
                                    0.15F, 0.05F, -0.25F, 0.10F,
                                    -0.10F, 0.20F, 0.15F, 0.05F,
                                    0.05F, -0.15F, 0.20F, 0.25F,
                                },
                                zeros(5));
    return model;
}

}  // namespace lcqi
```

5.3 Tokenizer 与 Safetensors 实现

tokenizer.cpp 把 prompt 变成 token id，并固定 unknown token 行为。读者应检查 BOS/EOS 是否稳定插入，UNK 是否按合同处理。

Listing 5.2 src/tokenizer.cpp

```
#include <lcqi/tokenizer.hpp>

#include <cstddef>
#include <fstream>
#include <stdexcept>
#include <string>
#include <string_view>

namespace lcqi {
namespace {

constexpr std::uint64_t LCQI_FNV_OFFSET_BASIS = 14695981039346656037ULL;
constexpr std::uint64_t LCQI_FNV_PRIME = 1099511628211ULL;
constexpr std::int32_t LCQI_INVALID_TOKEN_ID = -1;

std::string read_key(std::istream& input) {
    std::string key;
    if (!(input >> key)) {
```

```
        throw std::runtime_error("unexpected end of tokenizer file");
    }
    return key;
}

void expect_key(std::istream& input, const std::string& expected) {
    const std::string key = read_key(input);
    if (key != expected) {
        throw std::runtime_error("expected tokenizer key '" + expected + "', got '" + key + "'");
    }
}

std::int32_t read_i32(std::istream& input, const std::string& key) {
    expect_key(input, key);
    std::int32_t value = 0;
    if (!(input >> value)) {
        throw std::runtime_error("invalid tokenizer int32 value for key '" + key + "'");
    }
    return value;
}

std::string read_string(std::istream& input, const std::string& key) {
    expect_key(input, key);
    std::string value;
    if (!(input >> value)) {
        throw std::runtime_error("invalid tokenizer string value for key '" + key + "'");
    }
    return value;
}

std::int32_t lookup_token_id(const TokenizerModel& tokenizer, std::string_view text) {
    for (const TokenEntry& entry : tokenizer.vocab) {
        if (entry.text == text) {
            return entry.id;
        }
    }
    return LCQI_INVALID_TOKEN_ID;
}

void validate_tokenizer(const TokenizerModel& tokenizer) {
    if (tokenizer.vocab.empty()) {
        throw std::runtime_error("tokenizer vocab is empty");
    }
    for (std::size_t i = 0; i < tokenizer.vocab.size(); ++i) {
        const TokenEntry& left = tokenizer.vocab[i];
        if (left.id < 0) {
            throw std::runtime_error("tokenizer token id must be non-negative");
```

```
        }
        for (std::size_t j = i + 1; j < tokenizer.vocab.size(); ++j) {
            const TokenEntry& right = tokenizer.vocab[j];
            if (left.id == right.id) {
                throw std::runtime_error("duplicate tokenizer token id");
            }
            if (left.text == right.text) {
                throw std::runtime_error("duplicate tokenizer token text");
            }
        }
    }
    if (tokenizer.bos_id != LCQI_INVALID_TOKEN_ID &&
        lookup_token_id(tokenizer, "<bos>") != tokenizer.bos_id) {
        throw std::runtime_error("tokenizer BOS id does not match vocab");
    }
    if (tokenizer.eos_id != LCQI_INVALID_TOKEN_ID &&
        lookup_token_id(tokenizer, "<eos>") != tokenizer.eos_id) {
        throw std::runtime_error("tokenizer EOS id does not match vocab");
    }
}

std::uint64_t hash_byte(std::uint64_t hash, unsigned char value) {
    hash ^= static_cast<std::uint64_t>(value);
    hash *= LCQI_FNV_PRIME;
    return hash;
}

std::uint64_t hash_string(std::uint64_t hash, std::string_view text) {
    for (const unsigned char value : text) {
        hash = hash_byte(hash, value);
    }
    return hash;
}

std::uint64_t hash_i32(std::uint64_t hash, std::int32_t value) {
    const std::uint32_t bits = static_cast<std::uint32_t>(value);
    for (std::int32_t shift = 0; shift < 32; shift += 8) {
        hash = hash_byte(hash, static_cast<unsigned char>((bits >> shift) & 0↩
    ↪ xFFU));
    }
    return hash;
}

}  // namespace

TokenizerModel load_tokenizer(const std::filesystem::path& path) {
    std::ifstream input(path);
    if (!input) {
        throw std::runtime_error("cannot open tokenizer file: " + path.string()↩
    ↪ );
    }

```

```
    expect_key(input, "LCQI_TOKENIZER_V1");

    TokenizerModel tokenizer;
    tokenizer.bos_id = read_i32(input, "bos_id");
    tokenizer.eos_id = read_i32(input, "eos_id");
    tokenizer.unk_id = read_i32(input, "unk_id");
    tokenizer.chat_template_hash = read_string(input, "chat_template_hash");
    const std::int32_t vocab_size = read_i32(input, "vocab_size");
    if (vocab_size <= 0) {
        throw std::runtime_error("tokenizer vocab_size must be positive");
    }

    tokenizer.vocab.reserve(static_cast<std::size_t>(vocab_size));
    expect_key(input, "vocab");
    for (std::int32_t i = 0; i < vocab_size; ++i) {
        TokenEntry entry;
        if (!(input >> entry.id >> entry.text)) {
            throw std::runtime_error("invalid tokenizer vocab entry");
        }
        tokenizer.vocab.push_back(entry);
    }

    validate_tokenizer(tokenizer);
    return tokenizer;
}

std::vector<std::int32_t> encode_prompt(const TokenizerModel& tokenizer,
                                        std::string_view prompt) {
    std::vector<std::int32_t> ids;
    if (tokenizer.bos_id != LCQI_INVALID_TOKEN_ID) {
        ids.push_back(tokenizer.bos_id);
    }

    std::size_t begin = 0;
    while (begin < prompt.size()) {
        while (begin < prompt.size() && prompt[begin] == ' ') {
            ++begin;
        }
        if (begin >= prompt.size()) {
            break;
        }
        std::size_t end = begin;
        while (end < prompt.size() && prompt[end] != ' ') {
            ++end;
        }
        const std::string_view token = prompt.substr(begin, end - begin);
        const std::int32_t token_id = lookup_token_id(tokenizer, token);
        if (token_id == LCQI_INVALID_TOKEN_ID) {
            if (tokenizer.unk_id == LCQI_INVALID_TOKEN_ID) {
                throw std::runtime_error("tokenizer encountered unknown token")↩
    ↪ ;
            }
```

```
            ids.push_back(tokenizer.unk_id);
        } else {
            ids.push_back(token_id);
        }
        begin = end;
    }

    if (tokenizer.eos_id != LCQI_INVALID_TOKEN_ID) {
        ids.push_back(tokenizer.eos_id);
    }
    return ids;
}

std::uint64_t tokenizer_contract_hash(const TokenizerModel& tokenizer) {
    std::uint64_t hash = LCQI_FNV_OFFSET_BASIS;
    hash = hash_string(hash, tokenizer.chat_template_hash);
    hash = hash_i32(hash, tokenizer.bos_id);
    hash = hash_i32(hash, tokenizer.eos_id);
    hash = hash_i32(hash, tokenizer.unk_id);
    for (const TokenEntry& entry : tokenizer.vocab) {
        hash = hash_string(hash, entry.text);
        hash = hash_i32(hash, entry.id);
    }
    return hash;
}

}  // namespace lcqi
```

safetensors.cpp 解析 header、metadata、dtype、shape 和 data offsets。坏 dtype、坏 shape、坏 offset 和 payload 太短都应该被拒绝。

Listing 5.3 src/safetensors.cpp

```
#include <lcqi/safetensors.hpp>

#include <algorithm>
#include <cctype>
#include <cstddef>
#include <cmath>
#include <cstring>
#include <fstream>
#include <limits>
#include <stdexcept>
#include <string>
#include <string_view>
#include <vector>

namespace lcqi {
namespace {

constexpr std::size_t LCQI_SAFETENSORS_PREFIX_BYTES = 8;
constexpr std::uint64_t LCQI_SAFETENSORS_MAX_HEADER_BYTES = 16U * 1024U * 1024U;
```

```
constexpr std::size_t LCQI_SAFETENSORS_OFFSET_COUNT = 2;
constexpr std::uint64_t LCQI_BYTE_BITS = 8;
constexpr std::uint64_t LCQI_DECIMAL_BASE = 10;
constexpr std::uint64_t LCQI_BYTE_MASK = 0xFFU;
constexpr unsigned char LCQI_JSON_CONTROL_CHAR_LIMIT = 0x20U;
constexpr std::uint64_t LCQI_DTYPE_BYTES_F64 = 8;
constexpr std::uint64_t LCQI_DTYPE_BYTES_F32 = 4;
constexpr std::uint64_t LCQI_DTYPE_BYTES_F16 = 2;
constexpr std::uint64_t LCQI_DTYPE_BYTES_I64 = 8;
constexpr std::uint64_t LCQI_DTYPE_BYTES_I32 = 4;
constexpr std::uint64_t LCQI_DTYPE_BYTES_I16 = 2;
constexpr std::uint64_t LCQI_DTYPE_BYTES_I8 = 1;
constexpr std::uint64_t LCQI_DTYPE_BYTES_U8 = 1;
constexpr std::uint32_t LCQI_F32_EXPONENT_MASK = 0x7F800000U;
constexpr std::uint32_t LCQI_F16_SIGN_MASK = 0x8000U;
constexpr std::uint32_t LCQI_F16_EXPONENT_MASK = 0x7C00U;
constexpr std::uint32_t LCQI_F16_MANTISSA_MASK = 0x03FFU;
constexpr std::uint32_t LCQI_F16_EXPONENT_BIAS = 15U;
constexpr std::uint32_t LCQI_F32_EXPONENT_BIAS = 127U;
constexpr std::uint32_t LCQI_F32_MANTISSA_BITS = 23U;
constexpr std::uint32_t LCQI_F16_MANTISSA_BITS = 10U;
constexpr std::uint32_t LCQI_F16_TO_F32_SIGN_SHIFT = 16U;

class HeaderCursor {
public:
    explicit HeaderCursor(std::string_view text) : text_(text) {}

    void expect(char expected) {
        this->skip_ws();
        if (this->eof() || this->text_[this->position_] != expected) {
            throw std::runtime_error("invalid safetensors header syntax");
        }
        ++this->position_;
    }

    [[nodiscard]] bool consume(char expected) {
        this->skip_ws();
        if (!this->eof() && this->text_[this->position_] == expected) {
            ++this->position_;
            return true;
        }
        return false;
    }

    [[nodiscard]] std::string parse_string() {
        this->skip_ws();
        this->expect('"');
        std::string result;
        while (!this->eof()) {
            const char ch = this->text_[this->position_++];
            if (ch == '"') {
                return result;
```

```
            }
            if (static_cast<unsigned char>(ch) < LCQI_JSON_CONTROL_CHAR_LIMIT) {
                throw std::runtime_error("control character in safetensors string");
            }
            if (ch == '\\') {
                result.push_back(this->parse_escape());
            } else {
                result.push_back(ch);
            }
        }
        throw std::runtime_error("unterminated safetensors string");
    }

    [[nodiscard]] std::uint64_t parse_u64() {
        this->skip_ws();
        if (this->eof() ||
            !std::isdigit(static_cast<unsigned char>(this->text_[this->position_]))) {
            throw std::runtime_error("expected unsigned integer in safetensors header");
        }
        std::uint64_t value = 0;
        while (!this->eof() &&
               std::isdigit(static_cast<unsigned char>(this->text_[this->position_]))) {
            const std::uint64_t digit =
                static_cast<std::uint64_t>(this->text_[this->position_] - '0');
            if (value >
                (std::numeric_limits<std::uint64_t>::max() - digit) / LCQI_DECIMAL_BASE) {
                throw std::runtime_error("safetensors integer overflow");
            }
            value = value * LCQI_DECIMAL_BASE + digit;
            ++this->position_;
        }
        return value;
    }

    [[nodiscard]] std::vector<std::int64_t> parse_i64_array() {
        std::vector<std::int64_t> values;
        this->expect('[');
        if (this->consume(']')) {
            return values;
        }
        while (true) {
            const std::uint64_t raw = this->parse_u64();
            if (raw > static_cast<std::uint64_t>(std::numeric_limits<std::int64_t>::max())) {
                throw std::runtime_error("safetensors shape value is too large");
```

```
            }
            values.push_back(static_cast<std::int64_t>(raw));
            if (this->consume(']')) {
                return values;
            }
            this->expect(',');
        }
    }

    [[nodiscard]] std::vector<std::uint64_t> parse_u64_array() {
        std::vector<std::uint64_t> values;
        this->expect('[');
        if (this->consume(']')) {
            return values;
        }
        while (true) {
            values.push_back(this->parse_u64());
            if (this->consume(']')) {
                return values;
            }
            this->expect(',');
        }
    }

    [[nodiscard]] bool eof_after_ws() {
        this->skip_ws();
        return this->eof();
    }

private:
    std::string_view text_;
    std::size_t position_ = 0;

    [[nodiscard]] bool eof() const {
        return this->position_ >= this->text_.size();
    }

    void skip_ws() {
        while (!this->eof() &&
               std::isspace(static_cast<unsigned char>(this->text_[this->position_]))) {
            ++this->position_;
        }
    }

    [[nodiscard]] char parse_escape() {
        if (this->eof()) {
            throw std::runtime_error("unterminated safetensors escape");
        }
        const char escaped = this->text_[this->position_++];
        switch (escaped) {
            case '"':
```

```
            case '\\':
            case '/':
                return escaped;
            case 'b':
                return '\b';
            case 'f':
                return '\f';
            case 'n':
                return '\n';
            case 'r':
                return '\r';
            case 't':
                return '\t';
            default:
                throw std::runtime_error("unsupported safetensors string escape");
        }
    }
};

std::uint64_t read_le_u64(std::span<const std::uint8_t> bytes) {
    if (bytes.size() < LCQI_SAFETENSORS_PREFIX_BYTES) {
        throw std::runtime_error("safetensors file is too small");
    }
    std::uint64_t value = 0;
    for (std::size_t i = 0; i < LCQI_SAFETENSORS_PREFIX_BYTES; ++i) {
        value |= static_cast<std::uint64_t>(bytes[i]) << (LCQI_BYTE_BITS * i);
    }
    return value;
}

std::string read_file_bytes(const std::filesystem::path& path) {
    std::ifstream input(path, std::ios::binary);
    if (!input) {
        throw std::runtime_error("cannot open safetensors file: " + path.string());
    }
    input.seekg(0, std::ios::end);
    const std::streamoff size = input.tellg();
    if (size < 0) {
        throw std::runtime_error("cannot determine safetensors file size");
    }
    input.seekg(0, std::ios::beg);
    std::string bytes(static_cast<std::size_t>(size), '\0');
    if (!bytes.empty() && !input.read(bytes.data(), static_cast<std::streamsize>(bytes.size()))) {
        throw std::runtime_error("cannot read safetensors file");
    }
    return bytes;
}

std::vector<std::uint8_t> read_file_u8(const std::filesystem::path& path) {
```

```
    std::ifstream input(path, std::ios::binary);
    if (!input) {
        throw std::runtime_error("cannot open safetensors file: " + path.string());
    }
    input.seekg(0, std::ios::end);
    const std::streamoff size = input.tellg();
    if (size < 0) {
        throw std::runtime_error("cannot determine safetensors file size");
    }
    input.seekg(0, std::ios::beg);
    std::vector<std::uint8_t> bytes(static_cast<std::size_t>(size), 0);
    if (!bytes.empty() &&
        !input.read(reinterpret_cast<char*>(bytes.data()),
                    static_cast<std::streamsize>(bytes.size()))) {
        throw std::runtime_error("cannot read safetensors file");
    }
    return bytes;
}

void parse_metadata_value(HeaderCursor& cursor,
                          SafeTensorManifest& manifest,
                          const std::string& key) {
    if (key == "__metadata__") {
        cursor.expect('{');
        if (cursor.consume('}')) {
            return;
        }
        while (true) {
            const std::string metadata_key = cursor.parse_string();
            cursor.expect(':');
            const std::string metadata_value = cursor.parse_string();
            manifest.metadata.emplace_back(metadata_key, metadata_value);
            if (cursor.consume('}')) {
                return;
            }
            cursor.expect(',');
        }
    }
    throw std::runtime_error("unexpected safetensors metadata value");
}

SafeTensorEntry parse_tensor_entry(HeaderCursor& cursor, const std::string& name) {
    SafeTensorEntry entry;
    entry.name = name;
    bool saw_dtype = false;
    bool saw_shape = false;
    bool saw_offsets = false;

    cursor.expect('{');
    if (cursor.consume('}')) {
```

```
266         throw std::runtime_error("empty safetensors tensor entry");
267     }
268     while (true) {
269         const std::string field = cursor.parse_string();
270         cursor.expect(':');
271         if (field == "dtype") {
272             entry.dtype = cursor.parse_string();
273             saw_dtype = true;
274         } else if (field == "shape") {
275             entry.shape = cursor.parse_i64_array();
276             saw_shape = true;
277         } else if (field == "data_offsets") {
278             const std::vector<std::uint64_t> offsets = cursor.parse_u64_array()
    ↪ ;
279             if (offsets.size() != LCQI_SAFETENSORS_OFFSET_COUNT) {
280                 throw std::runtime_error("safetensors data_offsets must contain
    ↪  two values");
281             }
282             entry.data_begin = offsets[0];
283             entry.data_end = offsets[1];
284             saw_offsets = true;
285         } else {
286             throw std::runtime_error("unsupported safetensors tensor field: " +
    ↪  field);
287         }
288         if (cursor.consume('}')) {
289             break;
290         }
291         cursor.expect(',');
292     }
293 
294     if (!saw_dtype || !saw_shape || !saw_offsets) {
295         throw std::runtime_error("safetensors tensor entry is missing required
    ↪ fields");
296     }
297     if (entry.data_end < entry.data_begin) {
298         throw std::runtime_error("safetensors tensor offset range is invalid");
299     }
300     return entry;
301 }
302 
303 void parse_header(std::string_view header, SafeTensorManifest& manifest) {
304     HeaderCursor cursor(header);
305     cursor.expect('{');
306     if (cursor.consume('}')) {
307         throw std::runtime_error("safetensors header is empty");
308     }
309 
310     while (true) {
311         const std::string key = cursor.parse_string();
312         cursor.expect(':');
313         if (key == "__metadata__") {
```

```
            parse_metadata_value(cursor, manifest, key);
        } else {
            manifest.tensors.push_back(parse_tensor_entry(cursor, key));
        }
        if (cursor.consume('}')) {
            break;
        }
        cursor.expect(',');
    }

    if (!cursor.eof_after_ws()) {
        throw std::runtime_error("trailing data after safetensors header");
    }
}

std::uint64_t expected_tensor_bytes(const SafeTensorEntry& entry);

void validate_manifest(const SafeTensorManifest& manifest,
                       std::uint64_t file_size) {
    const std::uint64_t payload_size = file_size - manifest.data_start_offset;
    for (std::size_t i = 0; i < manifest.tensors.size(); ++i) {
        const SafeTensorEntry& left = manifest.tensors[i];
        if (left.name.empty() || left.dtype.empty()) {
            throw std::runtime_error("safetensors tensor has empty name or dtype");
        }
        if (left.data_end > payload_size) {
            throw std::runtime_error("safetensors tensor data_offsets exceed payload size");
        }
        const std::uint64_t expected_bytes = expected_tensor_bytes(left);
        if (left.byte_size() != expected_bytes) {
            throw std::runtime_error("safetensors tensor byte size does not match dtype and shape");
        }
        for (const std::int64_t dim : left.shape) {
            if (dim < 0) {
                throw std::runtime_error("safetensors shape dimension is negative");
            }
        }
        for (std::size_t j = i + 1; j < manifest.tensors.size(); ++j) {
            const SafeTensorEntry& right = manifest.tensors[j];
            if (left.name == right.name) {
                throw std::runtime_error("duplicate safetensors tensor name");
            }
            const bool overlaps =
                left.data_begin < right.data_end && right.data_begin < left.data_end;
            if (overlaps) {
                throw std::runtime_error("overlapping safetensors tensor data range");
```

```
            }
        }
    }
}

std::uint64_t dtype_byte_size(std::string_view dtype) {
    if (dtype == "F64") {
        return LCQI_DTYPE_BYTES_F64;
    }
    if (dtype == "F32") {
        return LCQI_DTYPE_BYTES_F32;
    }
    if (dtype == "F16" || dtype == "BF16") {
        return LCQI_DTYPE_BYTES_F16;
    }
    if (dtype == "I64" || dtype == "U64") {
        return LCQI_DTYPE_BYTES_I64;
    }
    if (dtype == "I32" || dtype == "U32") {
        return LCQI_DTYPE_BYTES_I32;
    }
    if (dtype == "I16" || dtype == "U16") {
        return LCQI_DTYPE_BYTES_I16;
    }
    if (dtype == "I8") {
        return LCQI_DTYPE_BYTES_I8;
    }
    if (dtype == "U8" || dtype == "BOOL") {
        return LCQI_DTYPE_BYTES_U8;
    }
    throw std::runtime_error("unsupported safetensors dtype: " + std::string(dtype));
}

std::uint64_t expected_tensor_bytes(const SafeTensorEntry& entry) {
    std::uint64_t element_count = 1;
    for (const std::int64_t dim : entry.shape) {
        if (dim < 0) {
            throw std::runtime_error("safetensors shape dimension is negative");
        }
        const std::uint64_t extent = static_cast<std::uint64_t>(dim);
        if (extent != 0 &&
            element_count > std::numeric_limits<std::uint64_t>::max() / extent) {
            throw std::runtime_error("safetensors shape element count overflow");
        }
        element_count *= extent;
    }
    const std::uint64_t dtype_bytes = dtype_byte_size(entry.dtype);
    if (dtype_bytes != 0 &&
```

```
408         element_count > std::numeric_limits<std::uint64_t>::max() / dtype_bytes ) {
409         throw std::runtime_error("safetensors tensor byte size overflow");
410     }
411     return element_count * dtype_bytes;
412 }
413
414 SafeTensorManifest parse_manifest_from_bytes(std::span<const std::uint8_t> bytes) {
415     const std::uint64_t header_size = read_le_u64(bytes);
416     if (header_size == 0 || header_size > LCQI_SAFETENSORS_MAX_HEADER_BYTES) {
417         throw std::runtime_error("safetensors header size is invalid");
418     }
419     const std::uint64_t data_start_offset =
420         static_cast<std::uint64_t>(LCQI_SAFETENSORS_PREFIX_BYTES) + header_size;
421     if (data_start_offset > bytes.size()) {
422         throw std::runtime_error("safetensors header extends beyond file size");
423     }
424
425     SafeTensorManifest manifest;
426     manifest.header_size = header_size;
427     manifest.data_start_offset = data_start_offset;
428     const char* header_begin =
429         reinterpret_cast<const char*>(bytes.data() + LCQI_SAFETENSORS_PREFIX_BYTES);
430     const std::string_view header(header_begin, static_cast<std::size_t>(header_size));
431     parse_header(header, manifest);
432     validate_manifest(manifest, static_cast<std::uint64_t>(bytes.size()));
433     return manifest;
434 }
435
436 std::uint16_t read_le_u16(std::span<const std::uint8_t> bytes, std::size_t offset) {
437     return static_cast<std::uint16_t>(
438         static_cast<std::uint16_t>(bytes[offset]) |
439         (static_cast<std::uint16_t>(bytes[offset + 1]) << LCQI_BYTE_BITS));
440 }
441
442 std::uint32_t read_le_u32(std::span<const std::uint8_t> bytes, std::size_t offset) {
443     std::uint32_t value = 0;
444     for (std::size_t i = 0; i < LCQI_DTYPE_BYTES_F32; ++i) {
445         value |= static_cast<std::uint32_t>(bytes[offset + i]) << (LCQI_BYTE_BITS * i);
446     }
447     return value;
448 }
449
450 float bits_to_float(std::uint32_t bits) {
```

```
451     float value = 0.0F;
452     std::memcpy(&value, &bits, sizeof(value));
453     return value;
454 }
455
456 float f16_to_f32(std::uint16_t bits) {
457     const std::uint32_t sign = (static_cast<std::uint32_t>(bits) & ↩
    ↪ LCQI_F16_SIGN_MASK)
458                                << LCQI_F16_TO_F32_SIGN_SHIFT;
459     const std::uint32_t exponent = static_cast<std::uint32_t>(bits) & ↩
    ↪ LCQI_F16_EXPONENT_MASK;
460     const std::uint32_t mantissa = static_cast<std::uint32_t>(bits) & ↩
    ↪ LCQI_F16_MANTISSA_MASK;
461
462     if (exponent == 0U) {
463         if (mantissa == 0U) {
464             return bits_to_float(sign);
465         }
466         float value = static_cast<float>(mantissa) /
467                       static_cast<float>(1U << LCQI_F16_MANTISSA_BITS);
468         value = std::ldexp(value, 1 - static_cast<int>(LCQI_F16_EXPONENT_BIAS))↩
    ↪ ;
469         return (sign == 0U) ? value : -value;
470     }
471
472     if (exponent == LCQI_F16_EXPONENT_MASK) {
473         const std::uint32_t f32_mantissa =
474             mantissa << (LCQI_F32_MANTISSA_BITS - LCQI_F16_MANTISSA_BITS);
475         return bits_to_float(sign | LCQI_F32_EXPONENT_MASK | f32_mantissa);
476     }
477
478     const std::uint32_t f32_exponent =
479         ((exponent >> LCQI_F16_MANTISSA_BITS) - LCQI_F16_EXPONENT_BIAS +
480          LCQI_F32_EXPONENT_BIAS)
481         << LCQI_F32_MANTISSA_BITS;
482     const std::uint32_t f32_mantissa =
483         mantissa << (LCQI_F32_MANTISSA_BITS - LCQI_F16_MANTISSA_BITS);
484     return bits_to_float(sign | f32_exponent | f32_mantissa);
485 }
486
487 float bf16_to_f32(std::uint16_t bits) {
488     return bits_to_float(static_cast<std::uint32_t>(bits) << 16U);
489 }
490
491 std::span<const std::uint8_t> tensor_payload_span(const SafeTensorFile& file,
492                                                   const SafeTensorEntry& entry)↩
    ↪  {
493     const std::uint64_t begin = file.manifest.data_start_offset + entry.↩
    ↪ data_begin;
494     const std::uint64_t end = file.manifest.data_start_offset + entry.data_end;
495     if (begin > end || end > file.bytes.size()) {
496         throw std::runtime_error("safetensors tensor byte range is invalid");
```

```
    }
    return std::span<const std::uint8_t>(
        file.bytes.data() + static_cast<std::size_t>(begin),
        static_cast<std::size_t>(end - begin));
}

}  // namespace

std::uint64_t SafeTensorEntry::byte_size() const {
    return this->data_end - this->data_begin;
}

const SafeTensorEntry* SafeTensorManifest::find_tensor(std::string_view name) const {
    const auto found = std::find_if(
        this->tensors.begin(),
        this->tensors.end(),
        [name](const SafeTensorEntry& entry) {
            return entry.name == name;
        });
    if (found == this->tensors.end()) {
        return nullptr;
    }
    return &(*found);
}

SafeTensorManifest load_safetensors_manifest(const std::filesystem::path& path) {
    const std::string bytes = read_file_bytes(path);
    return parse_manifest_from_bytes(
        std::span<const std::uint8_t>(
            reinterpret_cast<const std::uint8_t*>(bytes.data()),
            bytes.size()));
}

SafeTensorFile load_safetensors_file(const std::filesystem::path& path) {
    SafeTensorFile file;
    file.path = path;
    file.bytes = read_file_u8(path);
    file.manifest = parse_manifest_from_bytes(file.bytes);
    return file;
}

std::span<const std::uint8_t> read_safetensor_tensor_bytes(
    const SafeTensorFile& file,
    std::string_view name) {
    const SafeTensorEntry* entry = file.manifest.find_tensor(name);
    if (entry == nullptr) {
        throw std::runtime_error("missing safetensors tensor: " + std::string(name));
    }
    return tensor_payload_span(file, *entry);
```

```
}

std::vector<float> read_safetensor_f32_tensor(const SafeTensorFile& file,
                                              std::string_view name) {
    const SafeTensorEntry* entry = file.manifest.find_tensor(name);
    if (entry == nullptr) {
        throw std::runtime_error("missing safetensors tensor: " + std::string(name));
    }
    const std::span<const std::uint8_t> bytes = tensor_payload_span(file, *entry);
    std::vector<float> values;
    if (entry->dtype == "F32") {
        values.resize(bytes.size() / LCQI_DTYPE_BYTES_F32);
        for (std::size_t index = 0; index < values.size(); ++index) {
            values[index] = bits_to_float(
                read_le_u32(bytes, index * LCQI_DTYPE_BYTES_F32));
        }
        return values;
    }
    if (entry->dtype == "F16") {
        values.resize(bytes.size() / LCQI_DTYPE_BYTES_F16);
        for (std::size_t index = 0; index < values.size(); ++index) {
            values[index] = f16_to_f32(
                read_le_u16(bytes, index * LCQI_DTYPE_BYTES_F16));
        }
        return values;
    }
    if (entry->dtype == "BF16") {
        values.resize(bytes.size() / LCQI_DTYPE_BYTES_F16);
        for (std::size_t index = 0; index < values.size(); ++index) {
            values[index] = bf16_to_f32(
                read_le_u16(bytes, index * LCQI_DTYPE_BYTES_F16));
        }
        return values;
    }
    throw std::runtime_error("safetensors tensor is not float-compatible: " + entry->name);
}

}  // namespace lcqi
```

5.4 GPT-2 Reference Path

gpt2_reference.cpp 包括 JSON 配置读取、byte-level BPE、safetensors F32/F16/BF16 张量读取、HuggingFace 权重名前缀解析、LayerNorm、packed QKV、causal attention、GELU、tied lm head、full-prefix greedy generation 和 cached-KV generation。它是正确性优先的 reference path，暂不追求吞吐。

Listing 5.4 src/gpt2_reference.cpp

```
#include <lcqi/gpt2_reference.hpp>
```

```cpp

#include <lcqi/f32_kernels.hpp>
#include <lcqi/safetensors.hpp>

#include <algorithm>
#include <array>
#include <atomic>
#include <chrono>
#include <cmath>
#include <cstddef>
#include <fstream>
#include <limits>
#include <optional>
#include <sstream>
#include <stdexcept>
#include <string>
#include <string_view>
#include <thread>
#include <unordered_set>
#include <utility>

namespace lcqi {
namespace {

constexpr std::int32_t LCQI_GPT2_DEFAULT_BOS_EOS = 50256;
constexpr std::int32_t LCQI_GPT2_ATTENTION_PROJECTIONS = 3;
constexpr std::int32_t LCQI_GPT2_PAIR_TOKEN_COUNT = 2;
constexpr std::int32_t LCQI_GPT2_BYTE_COUNT = 256;
constexpr std::int32_t LCQI_GPT2_UNICODE_PRIVATE_BASE = 256;
constexpr std::int32_t LCQI_GPT2_ASCII_EXCLAMATION = 33;
constexpr std::int32_t LCQI_GPT2_ASCII_TILDE = 126;
constexpr std::int32_t LCQI_GPT2_LATIN_INVERTED_EXCLAMATION = 161;
constexpr std::int32_t LCQI_GPT2_LATIN_NOT_SIGN = 172;
constexpr std::int32_t LCQI_GPT2_LATIN_REGISTERED = 174;
constexpr std::int32_t LCQI_GPT2_LATIN_Y_DIAERESIS = 255;
constexpr std::int32_t LCQI_GPT2_UTF8_CONTINUATION_MASK = 0x3F;
constexpr std::int32_t LCQI_GPT2_UTF8_CONTINUATION_BITS = 0x80;
constexpr std::int32_t LCQI_GPT2_UTF8_TWO_BYTE_HEAD = 0xC0;
constexpr std::int32_t LCQI_GPT2_UTF8_THREE_BYTE_HEAD = 0xE0;
constexpr std::int32_t LCQI_GPT2_UTF8_FOUR_BYTE_HEAD = 0xF0;
constexpr std::int32_t LCQI_GPT2_UTF8_ONE_BYTE_LIMIT = 0x80;
constexpr std::int32_t LCQI_GPT2_UTF8_TWO_BYTE_LIMIT = 0x800;
constexpr std::int32_t LCQI_GPT2_UTF8_THREE_BYTE_LIMIT = 0x10000;
constexpr unsigned char LCQI_GPT2_ASCII_APOSTROPHE = 0x27U;
constexpr float LCQI_GPT2_GELU_SQRT_2_OVER_PI = 0.7978845608028654F;
constexpr float LCQI_GPT2_GELU_CUBIC_COEFF = 0.044715F;
constexpr float LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY =
    -std::numeric_limits<float>::infinity();
constexpr std::int32_t LCQI_GPT2_MAX_AUTO_WORKERS = 8;
constexpr std::int32_t LCQI_GPT2_DEFAULT_PARALLEL_MIN_ROWS = 512;
constexpr std::int32_t LCQI_GPT2_PARALLEL_MIN_COLUMNS = 512;
constexpr std::int64_t LCQI_GPT2_PARALLEL_MIN_OPS = 1'000'000;
```

```
54 constexpr std::size_t LCQI_GPT2_PARALLEL_ROW_BLOCK_MULTIPLIER = 4;
55 constexpr std::size_t LCQI_GPT2_WORKER_SPIN_PAUSES_BEFORE_YIELD = 4096;
56
57 using Gpt2ProfileClock = std::chrono::steady_clock;
58
59 void pause_worker_spin(std::size_t& pause_count) noexcept {
60 #if defined(__GNUC__) && (defined(__x86_64__) || defined(__i386__))
61     __builtin_ia32_pause();
62 #endif
63     ++pause_count;
64     if (pause_count >= LCQI_GPT2_WORKER_SPIN_PAUSES_BEFORE_YIELD) {
65         pause_count = 0;
66         std::this_thread::yield();
67     }
68 }
69
70 }  // namespace
71
72 namespace detail {
73
74 class Gpt2ParallelWorkerPool {
75 public:
76     explicit Gpt2ParallelWorkerPool(std::int32_t requested_workers)
77         : worker_count_(Gpt2ParallelWorkerPool::normalize_worker_count(↩
    ↪ requested_workers)) {
78         if (this->worker_count_ <= 1) {
79             return;
80         }
81         const std::size_t background_count =
82             static_cast<std::size_t>(this->worker_count_ - 1);
83         this->threads_.reserve(background_count);
84         for (std::size_t index = 0; index < background_count; ++index) {
85             this->threads_.emplace_back([this, index]() { this->worker_loop(↩
    ↪ index); });
86         }
87     }
88
89     ~Gpt2ParallelWorkerPool() {
90         this->stopping_.store(true, std::memory_order_release);
91         this->generation_.fetch_add(1, std::memory_order_release);
92         for (std::thread& thread : this->threads_) {
93             if (thread.joinable()) {
94                 thread.join();
95             }
96         }
97     }
98
99     Gpt2ParallelWorkerPool(const Gpt2ParallelWorkerPool&) = delete;
100    Gpt2ParallelWorkerPool& operator=(const Gpt2ParallelWorkerPool&) = delete;
101
102    [[nodiscard]] std::int32_t worker_count() const noexcept {
103        return this->worker_count_;
```

```
    }

    template <typename Function>
    void parallel_for_rows(std::size_t begin,
                           std::size_t end,
                           std::size_t grain,
                           Function&& function) {
        if (end <= begin) {
            return;
        }
        const std::size_t row_count = end - begin;
        if (this->worker_count_ <= 1 || row_count <= grain) {
            function(begin, end);
            return;
        }

        const std::size_t task_count =
            std::min<std::size_t>(static_cast<std::size_t>(this->worker_count_),
                                  (row_count + grain - 1) / grain);
        if (task_count <= 1) {
            function(begin, end);
            return;
        }

        auto task_context = ParallelTaskContext<Function>{function};
        // Publish all task metadata before the generation bump; workers use the
        // generation acquire load as the hand-off point for this stack context.
        this->task_begin_.store(begin, std::memory_order_release);
        this->task_end_.store(end, std::memory_order_release);
        this->task_count_.store(task_count, std::memory_order_release);
        this->task_context_.store(&task_context, std::memory_order_release);
        this->task_invoke_.store(&ParallelTaskContext<Function>::invoke,
                                 std::memory_order_release);
        this->active_workers_.store(task_count - 1, std::memory_order_release);
        this->generation_.fetch_add(1, std::memory_order_release);

        const std::size_t main_worker = task_count - 1;
        const auto [main_begin, main_end] = this->chunk_bounds(begin, end, task_count, main_worker);
        function(main_begin, main_end);

        std::size_t pause_count = 0;
        while (this->active_workers_.load(std::memory_order_acquire) != 0) {
            pause_worker_spin(pause_count);
        }
        this->task_context_.store(nullptr, std::memory_order_release);
        this->task_invoke_.store(nullptr, std::memory_order_release);
    }

```

```
152 private:
153     template <typename Function>
154     struct ParallelTaskContext {
155         Function& function;
156 
157         static void invoke(void* context, std::size_t begin, std::size_t end) {
158             auto* typed_context = static_cast<ParallelTaskContext*>(context);
159             typed_context->function(begin, end);
160         }
161     };
162 
163     std::int32_t worker_count_ = 1;
164     std::vector<std::thread> threads_;
165     std::atomic<bool> stopping_ = false;
166     std::atomic<std::uint64_t> generation_ = 0;
167     std::atomic<std::size_t> task_begin_ = 0;
168     std::atomic<std::size_t> task_end_ = 0;
169     std::atomic<std::size_t> task_count_ = 0;
170     std::atomic<std::size_t> active_workers_ = 0;
171     std::atomic<void*> task_context_ = nullptr;
172     std::atomic<void (*)(void*, std::size_t, std::size_t)> task_invoke_ = ↩
    ↪ nullptr;
173 
174     [[nodiscard]] static std::int32_t normalize_worker_count(std::int32_t ↩
    ↪ requested_workers) {
175         if (requested_workers > 0) {
176             return requested_workers;
177         }
178         const unsigned int hardware_workers = std::thread::hardware_concurrency↩
    ↪ ();
179         if (hardware_workers == 0) {
180             return 1;
181         }
182         const std::int32_t capped =
183             std::min<std::int32_t>(static_cast<std::int32_t>(hardware_workers),
184                                    LCQI_GPT2_MAX_AUTO_WORKERS);
185         return std::max(capped, 1);
186     }
187 
188     [[nodiscard]] std::pair<std::size_t, std::size_t> chunk_bounds(
189         std::size_t begin,
190         std::size_t end,
191         std::size_t task_count,
192         std::size_t worker_index) const {
193         const std::size_t row_count = end - begin;
194         const std::size_t chunk_begin = begin + row_count * worker_index / ↩
    ↪ task_count;
195         const std::size_t chunk_end = begin + row_count * (worker_index + 1) / ↩
    ↪ task_count;
196         return {chunk_begin, chunk_end};
197     }
198 
```

```
199     void worker_loop(std::size_t worker_index) {
200         std::uint64_t observed_generation = 0;
201         std::size_t pause_count = 0;
202         while (true) {
203             const std::uint64_t current_generation =
204                 this->generation_.load(std::memory_order_acquire);
205             if (current_generation == observed_generation) {
206                 if (this->stopping_.load(std::memory_order_acquire)) {
207                     return;
208                 }
209                 pause_worker_spin(pause_count);
210                 continue;
211             }
212 
213             observed_generation = current_generation;
214             pause_count = 0;
215             if (this->stopping_.load(std::memory_order_acquire)) {
216                 return;
217             }
218 
219             const std::size_t task_count = this->task_count_.load(std::↩
    ↪ memory_order_acquire);
220             if (this->generation_.load(std::memory_order_acquire) != ↩
    ↪ current_generation) {
221                 continue;
222             }
223             const std::size_t background_task_count = task_count - 1;
224             if (worker_index >= background_task_count) {
225                 continue;
226             }
227             void* task_context = this->task_context_.load(std::↩
    ↪ memory_order_acquire);
228             void (*task_invoke)(void*, std::size_t, std::size_t) =
229                 this->task_invoke_.load(std::memory_order_acquire);
230             if (task_context == nullptr || task_invoke == nullptr) {
231                 continue;
232             }
233             const std::size_t task_begin = this->task_begin_.load(std::↩
    ↪ memory_order_acquire);
234             const std::size_t task_end = this->task_end_.load(std::↩
    ↪ memory_order_acquire);
235             if (this->generation_.load(std::memory_order_acquire) != ↩
    ↪ current_generation) {
236                 continue;
237             }
238             const auto [begin, end] =
239                 this->chunk_bounds(task_begin,
240                                    task_end,
241                                    task_count,
242                                    worker_index);
243             task_invoke(task_context, begin, end);
244 
```

```
            this->active_workers_.fetch_sub(1, std::memory_order_acq_rel);
        }
    }
};

}  // namespace detail

namespace {

class JsonCursor {
public:
    explicit JsonCursor(std::string_view text) : text_(text) {}

    void expect(char expected) {
        this->skip_ws();
        if (this->eof() || this->text_[this->position_] != expected) {
            throw std::runtime_error("invalid JSON syntax");
        }
        ++this->position_;
    }

    [[nodiscard]] bool consume(char expected) {
        this->skip_ws();
        if (!this->eof() && this->text_[this->position_] == expected) {
            ++this->position_;
            return true;
        }
        return false;
    }

    [[nodiscard]] std::string parse_string() {
        this->skip_ws();
        this->expect('"');
        std::string result;
        while (!this->eof()) {
            const char ch = this->text_[this->position_++];
            if (ch == '"') {
                return result;
            }
            if (ch == '\\') {
                result += this->parse_escape();
            } else {
                result.push_back(ch);
            }
        }
        throw std::runtime_error("unterminated JSON string");
    }

    [[nodiscard]] std::int64_t parse_i64() {
        this->skip_ws();
        bool negative = false;
        if (this->consume('-')) {
```

```
297             negative = true;
298         }
299         if (this->eof() || this->text_[this->position_] < '0' ||
300             this->text_[this->position_] > '9') {
301             throw std::runtime_error("expected JSON integer");
302         }
303         std::int64_t value = 0;
304         while (!this->eof() && this->text_[this->position_] >= '0' &&
305                this->text_[this->position_] <= '9') {
306             const std::int64_t digit = this->text_[this->position_] - '0';
307             if (value >
308                 (std::numeric_limits<std::int64_t>::max() - digit) / 10) {
309                 throw std::runtime_error("JSON integer overflow");
310             }
311             value = value * 10 + digit;
312             ++this->position_;
313         }
314         return negative ? -value : value;
315     }
316
317     [[nodiscard]] double parse_number() {
318         this->skip_ws();
319         const std::size_t begin = this->position_;
320         if (!this->eof() && this->text_[this->position_] == '-') {
321             ++this->position_;
322         }
323         while (!this->eof() && this->text_[this->position_] >= '0' &&
324                this->text_[this->position_] <= '9') {
325             ++this->position_;
326         }
327         if (!this->eof() && this->text_[this->position_] == '.') {
328             ++this->position_;
329             while (!this->eof() && this->text_[this->position_] >= '0' &&
330                    this->text_[this->position_] <= '9') {
331                 ++this->position_;
332             }
333         }
334         if (!this->eof() &&
335             (this->text_[this->position_] == 'e' || this->text_[this->position_↩
    ↪ ] == 'E')) {
336             ++this->position_;
337             if (!this->eof() &&
338                 (this->text_[this->position_] == '+' || this->text_[this->↩
    ↪ position_] == '-')) {
339                 ++this->position_;
340             }
341             while (!this->eof() && this->text_[this->position_] >= '0' &&
342                    this->text_[this->position_] <= '9') {
343                 ++this->position_;
344             }
345         }
346         if (begin == this->position_) {
```

```
            throw std::runtime_error("expected JSON number");
        }
        return std::stod(std::string(this->text_.substr(begin, this->position_ - begin)));
    }

    void skip_value() {
        this->skip_ws();
        if (this->eof()) {
            throw std::runtime_error("unexpected end of JSON");
        }
        const char ch = this->text_[this->position_];
        if (ch == '"') {
            static_cast<void>(this->parse_string());
            return;
        }
        if (ch == '{') {
            this->expect('{');
            if (this->consume('}')) {
                return;
            }
            while (true) {
                static_cast<void>(this->parse_string());
                this->expect(':');
                this->skip_value();
                if (this->consume('}')) {
                    return;
                }
                this->expect(',');
            }
        }
        if (ch == '[') {
            this->expect('[');
            if (this->consume(']')) {
                return;
            }
            while (true) {
                this->skip_value();
                if (this->consume(']')) {
                    return;
                }
                this->expect(',');
            }
        }
        if (ch == 't') {
            this->expect_literal("true");
            return;
        }
        if (ch == 'f') {
            this->expect_literal("false");
            return;
        }
```

```
        if (ch == 'n') {
            this->expect_literal("null");
            return;
        }
        static_cast<void>(this->parse_number());
    }

    [[nodiscard]] bool eof_after_ws() {
        this->skip_ws();
        return this->eof();
    }

private:
    std::string_view text_;
    std::size_t position_ = 0;

    [[nodiscard]] bool eof() const {
        return this->position_ >= this->text_.size();
    }

    void skip_ws() {
        while (!this->eof() &&
               (this->text_[this->position_] == ' ' ||
                this->text_[this->position_] == '\n' ||
                this->text_[this->position_] == '\r' ||
                this->text_[this->position_] == '\t')) {
            ++this->position_;
        }
    }

    void expect_literal(std::string_view literal) {
        for (const char expected : literal) {
            if (this->eof() || this->text_[this->position_] != expected) {
                throw std::runtime_error("invalid JSON literal");
            }
            ++this->position_;
        }
    }

    [[nodiscard]] std::string parse_escape() {
        if (this->eof()) {
            throw std::runtime_error("unterminated JSON escape");
        }
        const char escaped = this->text_[this->position_++];
        switch (escaped) {
            case '"':
            case '\\':
            case '/':
                return std::string(1, escaped);
            case 'b':
                return "\b";
            case 'f':
```

```
                return "\f";
            case 'n':
                return "\n";
            case 'r':
                return "\r";
            case 't':
                return "\t";
            case 'u':
                return this->parse_unicode_escape();
            default:
                throw std::runtime_error("unsupported JSON escape");
        }
    }

    [[nodiscard]] std::string parse_unicode_escape() {
        std::uint32_t value = 0;
        for (std::int32_t i = 0; i < 4; ++i) {
            if (this->eof()) {
                throw std::runtime_error("unterminated JSON unicode escape");
            }
            const char ch = this->text_[this->position_++];
            value <<= 4U;
            if (ch >= '0' && ch <= '9') {
                value += static_cast<std::uint32_t>(ch - '0');
            } else if (ch >= 'a' && ch <= 'f') {
                value += static_cast<std::uint32_t>(10 + ch - 'a');
            } else if (ch >= 'A' && ch <= 'F') {
                value += static_cast<std::uint32_t>(10 + ch - 'A');
            } else {
                throw std::runtime_error("invalid JSON unicode escape");
            }
        }
        std::string encoded;
        if (value < LCQI_GPT2_UTF8_ONE_BYTE_LIMIT) {
            encoded.push_back(static_cast<char>(value));
        } else if (value < LCQI_GPT2_UTF8_TWO_BYTE_LIMIT) {
            encoded.push_back(static_cast<char>(
                LCQI_GPT2_UTF8_TWO_BYTE_HEAD | (value >> 6U)));
            encoded.push_back(static_cast<char>(
                LCQI_GPT2_UTF8_CONTINUATION_BITS |
                (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
        } else {
            encoded.push_back(static_cast<char>(
                LCQI_GPT2_UTF8_THREE_BYTE_HEAD | (value >> 12U)));
            encoded.push_back(static_cast<char>(
                LCQI_GPT2_UTF8_CONTINUATION_BITS |
                ((value >> 6U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
            encoded.push_back(static_cast<char>(
                LCQI_GPT2_UTF8_CONTINUATION_BITS |
                (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
        }
        return encoded;
```

```
    }
};

std::string read_text_file(const std::filesystem::path& path) {
    std::ifstream input(path);
    if (!input) {
        throw std::runtime_error("cannot open text file: " + path.string());
    }
    std::ostringstream buffer;
    buffer << input.rdbuf();
    return buffer.str();
}

std::size_t checked_size(std::int64_t value, const char* name) {
    if (value < 0) {
        throw std::runtime_error(std::string(name) + " cannot be negative");
    }
    return static_cast<std::size_t>(value);
}

std::string encode_codepoint(std::uint32_t value) {
    std::string encoded;
    if (value < LCQI_GPT2_UTF8_ONE_BYTE_LIMIT) {
        encoded.push_back(static_cast<char>(value));
    } else if (value < LCQI_GPT2_UTF8_TWO_BYTE_LIMIT) {
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_TWO_BYTE_HEAD | (value >> 6U)));
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_CONTINUATION_BITS |
            (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
    } else if (value < LCQI_GPT2_UTF8_THREE_BYTE_LIMIT) {
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_THREE_BYTE_HEAD | (value >> 12U)));
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_CONTINUATION_BITS |
            ((value >> 6U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_CONTINUATION_BITS |
            (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
    } else {
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_FOUR_BYTE_HEAD | (value >> 18U)));
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_CONTINUATION_BITS |
            ((value >> 12U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_CONTINUATION_BITS |
            ((value >> 6U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_CONTINUATION_BITS |
            (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
    }
```

```
    return encoded;
}

void require_positive(std::int32_t value, const char* name) {
    if (value <= 0) {
        throw std::runtime_error(std::string(name) + " must be positive");
    }
}

void validate_config(const Gpt2Config& config) {
    require_positive(config.hidden_size, "hidden_size");
    require_positive(config.head_count, "head_count");
    require_positive(config.layer_count, "layer_count");
    require_positive(config.max_positions, "max_positions");
    require_positive(config.vocab_size, "vocab_size");
    require_positive(config.intermediate_size, "intermediate_size");
    if (config.hidden_size % config.head_count != 0) {
        throw std::runtime_error("hidden_size must be divisible by head_count");
    }
    if (config.layer_norm_epsilon <= 0.0F) {
        throw std::runtime_error("layer_norm_epsilon must be positive");
    }
}

std::int32_t head_dim(const Gpt2Config& config) {
    return config.hidden_size / config.head_count;
}

std::size_t matrix_size(std::int32_t rows, std::int32_t columns) {
    return checked_size(rows, "rows") * checked_size(columns, "columns");
}

Gpt2ProfileClock::time_point profile_start(const Gpt2HotspotProfile* profile) {
    if (profile == nullptr) {
        return Gpt2ProfileClock::time_point{};
    }
    return Gpt2ProfileClock::now();
}

void add_profile_ms(Gpt2HotspotProfile* profile,
                    double Gpt2HotspotProfile::*field,
                    Gpt2ProfileClock::time_point start) {
    if (profile == nullptr) {
        return;
    }
    (*profile).*field +=
        std::chrono::duration<double, std::milli>(Gpt2ProfileClock::now() - start).count();
}

void validate_vector_size(std::size_t actual, std::int32_t expected, const char
```

```
    ↪ * name) {
604     if (actual != checked_size(expected, name)) {
605         throw std::runtime_error(std::string(name) + " size mismatch");
606     }
607 }
608
609 void validate_linear(const Gpt2LinearF32& linear, const char* name) {
610     require_positive(linear.input_size, "linear input_size");
611     require_positive(linear.output_size, "linear output_size");
612     if (linear.weights.size() != matrix_size(linear.output_size, linear.↩
    ↪ input_size)) {
613         throw std::runtime_error(std::string(name) + " weight size mismatch");
614     }
615     if (!linear.bias.empty() &&
616         linear.bias.size() != checked_size(linear.output_size, "linear ↩
    ↪ output_size")) {
617         throw std::runtime_error(std::string(name) + " bias size mismatch");
618     }
619 }
620
621 void validate_model(const Gpt2ReferenceModel& model) {
622     validate_config(model.config);
623     if (model.layers.size() != checked_size(model.config.layer_count, "↩
    ↪ layer_count")) {
624         throw std::runtime_error("GPT-2 layer count mismatch");
625     }
626     if (model.token_embedding.size() !=
627         matrix_size(model.config.vocab_size, model.config.hidden_size)) {
628         throw std::runtime_error("GPT-2 token embedding size mismatch");
629     }
630     if (model.position_embedding.size() !=
631         matrix_size(model.config.max_positions, model.config.hidden_size)) {
632         throw std::runtime_error("GPT-2 position embedding size mismatch");
633     }
634     validate_vector_size(model.final_ln_weight.size(), model.config.hidden_size↩
    ↪ , "ln_f weight");
635     validate_vector_size(model.final_ln_bias.size(), model.config.hidden_size, ↩
    ↪ "ln_f bias");
636     if (model.tie_lm_head_to_embedding) {
637         if (!model.lm_head_weight.empty()) {
638             throw std::runtime_error("tied GPT-2 lm_head should not carry ↩
    ↪ separate weights");
639         }
640     } else if (model.lm_head_weight.size() !=
641                matrix_size(model.config.vocab_size, model.config.hidden_size)) ↩
    ↪ {
642         throw std::runtime_error("GPT-2 lm_head size mismatch");
643     }
644     for (const Gpt2LayerWeightsF32& layer : model.layers) {
645         validate_vector_size(layer.ln_1_weight.size(), model.config.hidden_size↩
    ↪ , "ln_1 weight");
646         validate_vector_size(layer.ln_1_bias.size(), model.config.hidden_size, ↩
```

```
        "ln_1 bias");
        validate_linear(layer.c_attn, "c_attn");
        validate_linear(layer.c_proj, "c_proj");
        validate_vector_size(layer.ln_2_weight.size(), model.config.hidden_size, "ln_2 weight");
        validate_vector_size(layer.ln_2_bias.size(), model.config.hidden_size, "ln_2 bias");
        validate_linear(layer.c_fc, "c_fc");
        validate_linear(layer.mlp_c_proj, "mlp_c_proj");
    }
}

std::span<const float> row_span(std::span<const float> values,
                                std::int32_t row,
                                std::int32_t row_width) {
    return values.subspan(checked_size(row, "row") * checked_size(row_width, "row_width"),
                          checked_size(row_width, "row_width"));
}

void add_inplace(std::span<float> target, std::span<const float> source) {
    if (target.size() != source.size()) {
        throw std::runtime_error("GPT-2 residual add size mismatch");
    }
    for (std::size_t index = 0; index < target.size(); ++index) {
        target[index] += source[index];
    }
}

void layer_norm(std::span<const float> input,
                std::span<const float> weight,
                std::span<const float> bias,
                float epsilon,
                std::span<float> output) {
    if (input.empty() || input.size() != weight.size() || input.size() != bias.size() ||
        input.size() != output.size()) {
        throw std::runtime_error("GPT-2 LayerNorm size mismatch");
    }
    float mean = 0.0F;
    for (const float value : input) {
        mean += value;
    }
    mean /= static_cast<float>(input.size());

    float variance = 0.0F;
    for (const float value : input) {
        const float centered = value - mean;
        variance += centered * centered;
    }
    variance /= static_cast<float>(input.size());
    const float scale = 1.0F / std::sqrt(variance + epsilon);
```

```
694
695     for (std::size_t index = 0; index < input.size(); ++index) {
696         output[index] = (input[index] - mean) * scale * weight[index] + bias[index];
697     }
698 }
699
700 void linear_f32(const Gpt2LinearF32& linear,
701                 std::span<const float> input,
702                 std::span<float> output) {
703     validate_linear(linear, "GPT-2 linear");
704     if (input.size() != checked_size(linear.input_size, "linear input") ||
705         output.size() != checked_size(linear.output_size, "linear output")) {
706         throw std::runtime_error("GPT-2 linear input/output size mismatch");
707     }
708     const std::size_t input_size = checked_size(linear.input_size, "linear input");
709     const std::size_t output_size = checked_size(linear.output_size, "linear output");
710     for (std::size_t out = 0; out < output_size; ++out) {
711         float sum = linear.bias.empty() ? 0.0F : linear.bias[out];
712         const std::size_t row_base = out * input_size;
713         for (std::size_t in = 0; in < input_size; ++in) {
714             sum += linear.weights[row_base + in] * input[in];
715         }
716         output[out] = sum;
717     }
718 }
719
720 void linear_f32_unchecked(const Gpt2LinearF32& linear,
721                           std::span<const float> input,
722                           std::span<float> output) {
723     const std::size_t input_size = static_cast<std::size_t>(linear.input_size);
724     const std::size_t output_size = static_cast<std::size_t>(linear.output_size);
725     const float* weights = linear.weights.data();
726     const float* bias = linear.bias.empty() ? nullptr : linear.bias.data();
727     const float* input_data = input.data();
728     linear_f32_rows_unchecked(weights, input_data, bias, input_size, 0, output_size, output.data());
729 }
730
731 [[nodiscard]] bool should_parallelize_rows(const Gpt2ExecutionOptions& options,
732                                            std::size_t rows,
733                                            std::size_t columns,
734                                            const detail::Gpt2ParallelWorkerPool* worker_pool) {
735     if (worker_pool == nullptr || worker_pool->worker_count() <= 1) {
736         return false;
737     }
738     const std::int32_t min_rows =
739         options.parallel_min_rows > 0 ? options.parallel_min_rows
```

```
                                        : LCQI_GPT2_DEFAULT_PARALLEL_MIN_ROWS;
    if (rows < checked_size(min_rows, "parallel_min_rows")) {
        return false;
    }
    if (options.worker_count > 1) {
        return true;
    }
    return columns >= checked_size(LCQI_GPT2_PARALLEL_MIN_COLUMNS, "parallel_min_columns") ||
           rows * columns >= static_cast<std::size_t>(LCQI_GPT2_PARALLEL_MIN_OPS);
}

[[nodiscard]] std::size_t parallel_row_grain(std::size_t rows,
                                             const detail::Gpt2ParallelWorkerPool& worker_pool) {
    const std::size_t workers = checked_size(worker_pool.worker_count(), "worker_count");
    const std::size_t target_tasks = workers * LCQI_GPT2_PARALLEL_ROW_BLOCK_MULTIPLIER;
    return std::max<std::size_t>(1, (rows + target_tasks - 1) / target_tasks);
}

[[nodiscard]] bool model_has_parallel_work(const Gpt2ReferenceModel& model,
                                           const Gpt2ExecutionOptions& options) {
    const std::size_t hidden_size = checked_size(model.config.hidden_size, "hidden_size");
    const std::size_t vocab_size = checked_size(model.config.vocab_size, "vocab_size");
    const std::size_t intermediate_size =
        checked_size(model.config.intermediate_size, "intermediate_size");
    const std::size_t qkv_rows =
        checked_size(model.config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS,
                     "qkv output rows");
    const std::int32_t min_rows =
        options.parallel_min_rows > 0 ? options.parallel_min_rows
                                      : LCQI_GPT2_DEFAULT_PARALLEL_MIN_ROWS;
    const auto large_enough = [min_rows](std::size_t rows, std::size_t columns) {
        if (rows < checked_size(min_rows, "parallel_min_rows")) {
            return false;
        }
        return columns >= checked_size(LCQI_GPT2_PARALLEL_MIN_COLUMNS,
                                       "parallel_min_columns") ||
               rows * columns >= static_cast<std::size_t>(LCQI_GPT2_PARALLEL_MIN_OPS);
    };
    if (options.worker_count > 1) {
        return vocab_size >= checked_size(min_rows, "parallel_min_rows") ||
               qkv_rows >= checked_size(min_rows, "parallel_min_rows") ||
```

```
781                 intermediate_size >= checked_size(min_rows, "parallel_min_rows") ||
782                 hidden_size >= checked_size(min_rows, "parallel_min_rows");
783     }
784     if (options.worker_count == 1) {
785         return false;
786     }
787     return large_enough(vocab_size, hidden_size) ||
788            large_enough(qkv_rows, hidden_size) ||
789            large_enough(hidden_size, hidden_size) ||
790            large_enough(intermediate_size, hidden_size) ||
791            large_enough(hidden_size, intermediate_size);
792 }
793
794 [[nodiscard]] std::unique_ptr<detail::Gpt2ParallelWorkerPool> make_worker_pool(
795     const Gpt2ReferenceModel& model,
796     const Gpt2ExecutionOptions& options) {
797     if (!model_has_parallel_work(model, options)) {
798         return nullptr;
799     }
800     return std::make_unique<detail::Gpt2ParallelWorkerPool>(options.worker_count);
801 }
802
803 void linear_f32_unchecked_parallel(const Gpt2LinearF32& linear,
804                                    std::span<const float> input,
805                                    std::span<float> output,
806                                    const Gpt2ExecutionOptions& options,
807                                    detail::Gpt2ParallelWorkerPool* worker_pool) {
808     const std::size_t input_size = static_cast<std::size_t>(linear.input_size);
809     const std::size_t output_size = static_cast<std::size_t>(linear.output_size);
810     if (!should_parallelize_rows(options, output_size, input_size, worker_pool)) {
811         linear_f32_unchecked(linear, input, output);
812         return;
813     }
814
815     const float* weights = linear.weights.data();
816     const float* bias = linear.bias.empty() ? nullptr : linear.bias.data();
817     const float* input_data = input.data();
818     float* output_data = output.data();
819     const auto rows_fn =
820         [input_size, weights, bias, input_data, output_data](std::size_t begin,
821                                                              std::size_t end) {
822             linear_f32_rows_unchecked(weights,
823                                       input_data,
824                                       bias,
825                                       input_size,
826                                       begin,
827                                       end,
```

```
                                        output_data);
        };
    worker_pool->parallel_for_rows(0, output_size, parallel_row_grain(output_size, *worker_pool), rows_fn);
}

float gelu_new(float value) {
    const float cubic = value * value * value;
    return 0.5F * value *
           (1.0F + std::tanh(LCQI_GPT2_GELU_SQRT_2_OVER_PI *
                             (value + LCQI_GPT2_GELU_CUBIC_COEFF * cubic)));
}

std::int32_t argmax(std::span<const float> values) {
    if (values.empty()) {
        throw std::runtime_error("cannot take argmax of empty logits");
    }
    std::int32_t best_index = 0;
    float best_value = values.front();
    for (std::int32_t index = 1; index < static_cast<std::int32_t>(values.size()); ++index) {
        const float value = values[checked_size(index, "argmax index")];
        if (value > best_value) {
            best_value = value;
            best_index = index;
        }
    }
    return best_index;
}

void project_qkv(const Gpt2Config& config,
                 const Gpt2LayerWeightsF32& layer,
                 std::span<const float> hidden,
                 std::span<float> q,
                 std::span<float> k,
                 std::span<float> v) {
    std::vector<float> packed(matrix_size(LCQI_GPT2_ATTENTION_PROJECTIONS,
                                          config.hidden_size),
                              0.0F);
    linear_f32(layer.c_attn, hidden, packed);
    const std::size_t width = checked_size(config.hidden_size, "hidden_size");
    std::copy_n(packed.begin(), config.hidden_size, q.begin());
    std::copy_n(packed.begin() + static_cast<std::ptrdiff_t>(width),
                config.hidden_size,
                k.begin());
    std::copy_n(packed.begin() + static_cast<std::ptrdiff_t>(width * 2),
                config.hidden_size,
                v.begin());
}

void project_qkv_reuse_workspace(const Gpt2Config& config,
                                 const Gpt2LayerWeightsF32& layer,
```

```
                                    std::span<const float> hidden,
                                    std::span<float> packed,
                                    std::span<float> q,
                                    std::span<float> k,
                                    std::span<float> v,
                                    const Gpt2ExecutionOptions& options,
                                    detail::Gpt2ParallelWorkerPool* worker_pool) {
    linear_f32_unchecked_parallel(layer.c_attn, hidden, packed, options, worker_pool);
    const std::size_t width = checked_size(config.hidden_size, "hidden_size");
    std::copy_n(packed.begin(), config.hidden_size, q.begin());
    std::copy_n(packed.begin() + static_cast<std::ptrdiff_t>(width),
                config.hidden_size,
                k.begin());
    std::copy_n(packed.begin() + static_cast<std::ptrdiff_t>(width * 2),
                config.hidden_size,
                v.begin());
}

void attention_full_sequence(const Gpt2Config& config,
                             std::span<const float> q_by_pos,
                             std::span<const float> k_by_pos,
                             std::span<const float> v_by_pos,
                             std::int32_t sequence_length,
                             std::span<float> output_by_pos) {
    const std::int32_t local_head_dim = head_dim(config);
    const float score_scale = 1.0F / std::sqrt(static_cast<float>(local_head_dim));
    std::fill(output_by_pos.begin(), output_by_pos.end(), 0.0F);
    std::vector<float> scores(checked_size(sequence_length, "sequence_length"),
                              LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY);

    for (std::int32_t position = 0; position < sequence_length; ++position) {
        for (std::int32_t head = 0; head < config.head_count; ++head) {
            float max_score = LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY;
            for (std::int32_t past = 0; past <= position; ++past) {
                float dot = 0.0F;
                for (std::int32_t dim = 0; dim < local_head_dim; ++dim) {
                    const std::size_t q_index =
                        matrix_size(position, config.hidden_size) +
                        matrix_size(head, local_head_dim) + checked_size(dim, "head dim");
                    const std::size_t k_index =
                        matrix_size(past, config.hidden_size) +
                        matrix_size(head, local_head_dim) + checked_size(dim, "head dim");
                    dot += q_by_pos[q_index] * k_by_pos[k_index];
                }
                const float score = dot * score_scale;
                scores[checked_size(past, "past")] = score;
                max_score = std::max(max_score, score);
            }
```

```
926
927             float denominator = 0.0F;
928             for (std::int32_t past = 0; past <= position; ++past) {
929                 const std::size_t index = checked_size(past, "past");
930                 scores[index] = std::exp(scores[index] - max_score);
931                 denominator += scores[index];
932             }
933             if (denominator <= 0.0F) {
934                 throw std::runtime_error("GPT-2 attention denominator is invalid");
935             }
936             for (std::int32_t past = 0; past <= position; ++past) {
937                 const float probability = scores[checked_size(past, "past")] / denominator;
938                 for (std::int32_t dim = 0; dim < local_head_dim; ++dim) {
939                     const std::size_t output_index =
940                         matrix_size(position, config.hidden_size) +
941                         matrix_size(head, local_head_dim) + checked_size(dim, "head dim");
942                     const std::size_t value_index =
943                         matrix_size(past, config.hidden_size) +
944                         matrix_size(head, local_head_dim) + checked_size(dim, "head dim");
945                     output_by_pos[output_index] += probability * v_by_pos[value_index];
946                 }
947             }
948         }
949     }
950 }
951
952 void forward_gpt2_layer(const Gpt2Config& config,
953                         const Gpt2LayerWeightsF32& layer,
954                         std::int32_t sequence_length,
955                         std::vector<float>& hidden_by_pos) {
956     std::vector<float> normed(checked_size(config.hidden_size, "hidden_size"), 0.0F);
957     std::vector<float> q_by_pos(matrix_size(sequence_length, config.hidden_size), 0.0F);
958     std::vector<float> k_by_pos(q_by_pos.size(), 0.0F);
959     std::vector<float> v_by_pos(q_by_pos.size(), 0.0F);
960     std::vector<float> attention_by_pos(q_by_pos.size(), 0.0F);
961     std::vector<float> projected(checked_size(config.hidden_size, "hidden_size"), 0.0F);
962     std::vector<float> mlp_fc(checked_size(config.intermediate_size, "intermediate_size"), 0.0F);
963     std::vector<float> mlp_out(checked_size(config.hidden_size, "hidden_size"), 0.0F);
964
965     for (std::int32_t position = 0; position < sequence_length; ++position) {
966         const std::span<float> hidden = std::span<float>(
967             hidden_by_pos.data() + matrix_size(position, config.hidden_size),
```

```
            checked_size(config.hidden_size, "hidden_size"));
        layer_norm(hidden,
                   layer.ln_1_weight,
                   layer.ln_1_bias,
                   config.layer_norm_epsilon,
                   normed);
        std::span<float> q = std::span<float>(
            q_by_pos.data() + matrix_size(position, config.hidden_size),
            checked_size(config.hidden_size, "hidden_size"));
        std::span<float> k = std::span<float>(
            k_by_pos.data() + matrix_size(position, config.hidden_size),
            checked_size(config.hidden_size, "hidden_size"));
        std::span<float> v = std::span<float>(
            v_by_pos.data() + matrix_size(position, config.hidden_size),
            checked_size(config.hidden_size, "hidden_size"));
        project_qkv(config, layer, normed, q, k, v);
    }

    attention_full_sequence(config, q_by_pos, k_by_pos, v_by_pos, ↩
    ↪ sequence_length, attention_by_pos);

    for (std::int32_t position = 0; position < sequence_length; ++position) {
        const std::span<float> hidden = std::span<float>(
            hidden_by_pos.data() + matrix_size(position, config.hidden_size),
            checked_size(config.hidden_size, "hidden_size"));
        const std::span<const float> attention = std::span<const float>(
            attention_by_pos.data() + matrix_size(position, config.hidden_size)↩
    ↪ ,
            checked_size(config.hidden_size, "hidden_size"));
        linear_f32(layer.c_proj, attention, projected);
        add_inplace(hidden, projected);

        layer_norm(hidden,
                   layer.ln_2_weight,
                   layer.ln_2_bias,
                   config.layer_norm_epsilon,
                   normed);
        linear_f32(layer.c_fc, normed, mlp_fc);
        for (float& value : mlp_fc) {
            value = gelu_new(value);
        }
        linear_f32(layer.mlp_c_proj, mlp_fc, mlp_out);
        add_inplace(hidden, mlp_out);
    }
}

void forward_gpt2_layer_cached(const Gpt2Config& config,
                               const Gpt2LayerWeightsF32& layer,
                               std::int32_t layer_id,
                               std::int32_t model_position,
                               Gpt2KvCache& cache,
                               Gpt2ForwardWorkspace& workspace,
```

```
                                std::span<float> hidden,
                                Gpt2HotspotProfile* hotspot_profile,
                                const Gpt2ExecutionOptions& options,
                                detail::Gpt2ParallelWorkerPool* worker_pool) {
    if (hotspot_profile != nullptr) {
        ++hotspot_profile->layer_steps;
    }
    std::span<float> normed = workspace.normed();
    std::span<float> query = workspace.query();
    std::span<float> key = workspace.key();
    std::span<float> value = workspace.value();
    std::span<float> attention = workspace.attention();
    std::span<float> projected = workspace.projected();
    std::span<float> qkv_packed = workspace.qkv_packed();
    std::span<float> mlp_fc = workspace.mlp_fc();
    std::span<float> mlp_out = workspace.mlp_out();
    std::span<float> scores = workspace.scores_prefix(model_position + 1);

    Gpt2ProfileClock::time_point start =
        profile_start(hotspot_profile);
    layer_norm(hidden,
               layer.ln_1_weight,
               layer.ln_1_bias,
               config.layer_norm_epsilon,
               normed);
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::layer_norm_ms, start);

    start = profile_start(hotspot_profile);
    project_qkv_reuse_workspace(config,
                                layer,
                                normed,
                                qkv_packed,
                                query,
                                key,
                                value,
                                options,
                                worker_pool);
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::qkv_projection_ms,
    ↪ start);

    start = profile_start(hotspot_profile);
    cache.append(layer_id, model_position, key, value);
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::kv_append_ms, start);

    start = profile_start(hotspot_profile);
    detail::attend_cached_position(cache,
                                   layer_id,
                                   model_position,
                                   query,
                                   scores,
                                   attention);
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::attention_ms, start);
```

```
1069
1070     start = profile_start(hotspot_profile);
1071     linear_f32_unchecked_parallel(layer.c_proj, attention, projected, options, ↩
         ↪ worker_pool);
1072     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::↩
         ↪ attention_projection_ms, start);
1073
1074     start = profile_start(hotspot_profile);
1075     add_inplace(hidden, projected);
1076     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::residual_add_ms, start↩
         ↪ );
1077
1078     start = profile_start(hotspot_profile);
1079     layer_norm(hidden,
1080                layer.ln_2_weight,
1081                layer.ln_2_bias,
1082                config.layer_norm_epsilon,
1083                normed);
1084     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::layer_norm_ms, start);
1085
1086     start = profile_start(hotspot_profile);
1087     linear_f32_unchecked_parallel(layer.c_fc, normed, mlp_fc, options, ↩
         ↪ worker_pool);
1088     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::mlp_fc_ms, start);
1089
1090     start = profile_start(hotspot_profile);
1091     for (float& value_ref : mlp_fc) {
1092         value_ref = gelu_new(value_ref);
1093     }
1094     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::gelu_ms, start);
1095
1096     start = profile_start(hotspot_profile);
1097     linear_f32_unchecked_parallel(layer.mlp_c_proj, mlp_fc, mlp_out, options, ↩
         ↪ worker_pool);
1098     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::mlp_projection_ms, ↩
         ↪ start);
1099
1100     start = profile_start(hotspot_profile);
1101     add_inplace(hidden, mlp_out);
1102     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::residual_add_ms, start↩
         ↪ );
1103 }
1104
1105 void compute_logits(const Gpt2ReferenceModel& model,
1106                     std::span<const float> hidden,
1107                     std::span<float> logits,
1108                     const Gpt2ExecutionOptions& options,
1109                     detail::Gpt2ParallelWorkerPool* worker_pool) {
1110     if (logits.size() != checked_size(model.config.vocab_size, "vocab_size")) {
1111         throw std::runtime_error("GPT-2 logits size mismatch");
1112     }
1113     const std::span<const float> weight =
```

```
        model.tie_lm_head_to_embedding ? std::span<const float>(model.token_embedding)
                                       : std::span<const float>(model.lm_head_weight);
    const std::size_t hidden_size = checked_size(model.config.hidden_size, "hidden_size");
    const std::size_t vocab_size = checked_size(model.config.vocab_size, "vocab_size");
    const float* weight_data = weight.data();
    const float* hidden_data = hidden.data();
    const auto rows_fn =
        [hidden_size, weight_data, hidden_data, logits_data = logits.data()](
            std::size_t begin,
            std::size_t end) {
            linear_f32_rows_unchecked(weight_data,
                                      hidden_data,
                                      nullptr,
                                      hidden_size,
                                      begin,
                                      end,
                                      logits_data);
        };
    if (should_parallelize_rows(options, vocab_size, hidden_size, worker_pool)) {
        worker_pool->parallel_for_rows(0,
                                       vocab_size,
                                       parallel_row_grain(vocab_size, *worker_pool),
                                       rows_fn);
    } else {
        rows_fn(0, vocab_size);
    }
}

void compute_logits(const Gpt2ReferenceModel& model,
                    std::span<const float> hidden,
                    std::span<float> logits) {
    Gpt2ExecutionOptions options;
    options.worker_count = 1;
    compute_logits(model, hidden, logits, options, nullptr);
}

struct Gpt2TokenScore {
    std::int32_t token = 0;
    float value = -std::numeric_limits<float>::infinity();
};

enum class Gpt2CachedStepMode {
    advance_only,
    predict_token,
    logits,
};
```

```
1160
1161 [[nodiscard]] Gpt2TokenScore compute_predicted_token_range(const float* ↩
     ↪ weight_data,
1162                                                            const float* ↩
     ↪ hidden_data,
1163                                                            std::size_t ↩
     ↪ hidden_size,
1164                                                            std::size_t begin,
1165                                                            std::size_t end) {
1166     Gpt2TokenScore best;
1167     const F32RowMax row_max =
1168         max_dot_f32_rows_unchecked(weight_data, hidden_data, hidden_size, begin↩
     ↪ , end);
1169     best.token = static_cast<std::int32_t>(row_max.row);
1170     best.value = row_max.value;
1171     return best;
1172 }
1173
1174 std::int32_t compute_predicted_token(const Gpt2ReferenceModel& model,
1175                                      std::span<const float> hidden,
1176                                      const Gpt2ExecutionOptions& options,
1177                                      detail::Gpt2ParallelWorkerPool* ↩
     ↪ worker_pool) {
1178     const std::span<const float> weight =
1179         model.tie_lm_head_to_embedding ? std::span<const float>(model.↩
     ↪ token_embedding)
1180                                        : std::span<const float>(model.↩
     ↪ lm_head_weight);
1181     const std::size_t hidden_size = checked_size(model.config.hidden_size, "↩
     ↪ hidden_size");
1182     const std::size_t vocab_size = checked_size(model.config.vocab_size, "↩
     ↪ vocab_size");
1183     const float* weight_data = weight.data();
1184     const float* hidden_data = hidden.data();
1185     if (!should_parallelize_rows(options, vocab_size, hidden_size, worker_pool)↩
     ↪ ) {
1186         return compute_predicted_token_range(weight_data, hidden_data, ↩
     ↪ hidden_size, 0, vocab_size)
1187                .token;
1188     }
1189
1190     const std::size_t worker_count = checked_size(worker_pool->worker_count(), ↩
     ↪ "worker_count");
1191     const std::size_t chunk_count = std::min(worker_count, vocab_size);
1192     std::vector<Gpt2TokenScore> partials(chunk_count);
1193     const auto rows_fn =
1194         [chunk_count, hidden_size, vocab_size, weight_data, hidden_data, &↩
     ↪ partials](
1195             std::size_t begin,
1196             std::size_t end) {
1197             for (std::size_t chunk = begin; chunk < end; ++chunk) {
1198                 const std::size_t token_begin = vocab_size * chunk / ↩
```

```
        ↪ chunk_count;
1199                const std::size_t token_end = vocab_size * (chunk + 1) / ↩
        ↪ chunk_count;
1200                partials[chunk] = compute_predicted_token_range(
1201                    weight_data,
1202                    hidden_data,
1203                    hidden_size,
1204                    token_begin,
1205                    token_end);
1206            }
1207        };
1208    worker_pool->parallel_for_rows(0, chunk_count, 1, rows_fn);
1209
1210    Gpt2TokenScore best = partials.front();
1211    for (std::size_t index = 1; index < partials.size(); ++index) {
1212        if (partials[index].value > best.value) {
1213            best = partials[index];
1214        }
1215    }
1216    return best.token;
1217 }
1218
1219 Gpt2ForwardResult run_gpt2_cached_step_unchecked(const Gpt2ReferenceModel& ↩
        ↪ model,
1220                                                  Gpt2KvCache& cache,
1221                                                  Gpt2ForwardWorkspace& ↩
        ↪ workspace,
1222                                                  std::int32_t token_id,
1223                                                  Gpt2CachedStepMode mode,
1224                                                  Gpt2HotspotProfile* ↩
        ↪ hotspot_profile,
1225                                                  const Gpt2ExecutionOptions& ↩
        ↪ options,
1226                                                  detail::Gpt2ParallelWorkerPool↩
        ↪ * worker_pool) {
1227    const Gpt2ProfileClock::time_point step_start = profile_start(↩
        ↪ hotspot_profile);
1228    if (hotspot_profile != nullptr) {
1229        ++hotspot_profile->decoder_steps;
1230    }
1231    if (token_id < 0 || token_id >= model.config.vocab_size) {
1232        throw std::runtime_error("GPT-2 token id out of range");
1233    }
1234    const std::int32_t model_position = cache.filled_tokens();
1235    if (model_position >= model.config.max_positions) {
1236        throw std::runtime_error("GPT-2 cached forward would exceed ↩
        ↪ max_positions");
1237    }
1238
1239    std::span<float> hidden = workspace.hidden();
1240    const std::span<const float> token =
1241        row_span(model.token_embedding, token_id, model.config.hidden_size);
```

```
    const std::span<const float> position_embedding =
        row_span(model.position_embedding, model_position, model.config.hidden_size);
    Gpt2ProfileClock::time_point start = profile_start(hotspot_profile);
    for (std::int32_t dim = 0; dim < model.config.hidden_size; ++dim) {
        hidden[checked_size(dim, "dim")] =
            token[checked_size(dim, "dim")] + position_embedding[checked_size(dim, "dim")];
    }
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::embedding_ms, start);

    for (std::int32_t layer_id = 0;
         layer_id < static_cast<std::int32_t>(model.layers.size());
         ++layer_id) {
        forward_gpt2_layer_cached(model.config,
                                  model.layers[checked_size(layer_id, "layer_id")],
                                  layer_id,
                                  model_position,
                                  cache,
                                  workspace,
                                  hidden,
                                  hotspot_profile,
                                  options,
                                  worker_pool);
    }

    Gpt2ForwardResult result;
    if (mode == Gpt2CachedStepMode::advance_only) {
        add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::total_step_ms, step_start);
        return result;
    }

    std::span<float> normed = workspace.normed();
    start = profile_start(hotspot_profile);
    layer_norm(hidden,
               model.final_ln_weight,
               model.final_ln_bias,
               model.config.layer_norm_epsilon,
               normed);
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::final_norm_ms, start);

    if (mode == Gpt2CachedStepMode::logits) {
        std::span<float> logits = workspace.logits();
        start = profile_start(hotspot_profile);
        compute_logits(model, normed, logits, options, worker_pool);
        add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::lm_head_ms, start);
        start = profile_start(hotspot_profile);
        result.logits.assign(logits.begin(), logits.end());
        result.predicted_token = argmax(result.logits);
```

```
        add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::logits_result_ms, start);
    } else {
        start = profile_start(hotspot_profile);
        result.predicted_token = compute_predicted_token(model, normed, options, worker_pool);
        add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::lm_head_ms, start);
    }
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::total_step_ms, step_start);
    return result;
}

std::vector<std::string> gpt2_bytes_to_unicode() {
    std::vector<std::int32_t> bytes;
    for (std::int32_t value = LCQI_GPT2_ASCII_EXCLAMATION;
         value <= LCQI_GPT2_ASCII_TILDE;
         ++value) {
        bytes.push_back(value);
    }
    for (std::int32_t value = LCQI_GPT2_LATIN_INVERTED_EXCLAMATION;
         value <= LCQI_GPT2_LATIN_NOT_SIGN;
         ++value) {
        bytes.push_back(value);
    }
    for (std::int32_t value = LCQI_GPT2_LATIN_REGISTERED;
         value <= LCQI_GPT2_LATIN_Y_DIAERESIS;
         ++value) {
        bytes.push_back(value);
    }

    std::unordered_set<std::int32_t> byte_set(bytes.begin(), bytes.end());
    std::vector<std::int32_t> chars = bytes;
    std::int32_t private_offset = 0;
    for (std::int32_t byte = 0; byte < LCQI_GPT2_BYTE_COUNT; ++byte) {
        if (!byte_set.contains(byte)) {
            bytes.push_back(byte);
            chars.push_back(LCQI_GPT2_UNICODE_PRIVATE_BASE + private_offset);
            ++private_offset;
        }
    }

    std::vector<std::string> mapping(LCQI_GPT2_BYTE_COUNT);
    for (std::size_t index = 0; index < bytes.size(); ++index) {
        mapping[checked_size(bytes[index], "byte unicode index")] =
            encode_codepoint(static_cast<std::uint32_t>(chars[index]));
    }
    return mapping;
}

std::unordered_map<std::string, unsigned char> gpt2_unicode_to_bytes(
```

```
    const std::vector<std::string>& byte_encoder) {
    std::unordered_map<std::string, unsigned char> result;
    for (std::size_t byte = 0; byte < byte_encoder.size(); ++byte) {
        result.emplace(byte_encoder[byte], static_cast<unsigned char>(byte));
    }
    return result;
}

std::string bytes_to_bpe_alphabet(std::string_view text) {
    static const std::vector<std::string> BYTE_ENCODER = gpt2_bytes_to_unicode();
    std::string result;
    for (const unsigned char byte : text) {
        result += BYTE_ENCODER[byte];
    }
    return result;
}

std::vector<std::string> utf8_symbols(std::string_view text) {
    std::vector<std::string> symbols;
    std::size_t offset = 0;
    while (offset < text.size()) {
        const unsigned char lead = static_cast<unsigned char>(text[offset]);
        std::size_t width = 1;
        if ((lead & 0xE0U) == LCQI_GPT2_UTF8_TWO_BYTE_HEAD) {
            width = 2;
        } else if ((lead & 0xF0U) == LCQI_GPT2_UTF8_THREE_BYTE_HEAD) {
            width = 3;
        } else if ((lead & 0xF8U) == LCQI_GPT2_UTF8_FOUR_BYTE_HEAD) {
            width = 4;
        }
        if (offset + width > text.size()) {
            throw std::runtime_error("invalid UTF-8/BPE symbol boundary");
        }
        symbols.emplace_back(text.substr(offset, width));
        offset += width;
    }
    return symbols;
}

std::string pair_key(std::string_view left, std::string_view right) {
    std::string key(left);
    key.push_back('\n');
    key += right;
    return key;
}

std::vector<std::string> bpe_apply(const Gpt2Tokenizer& tokenizer, std::string_view token) {
    std::vector<std::string> word = utf8_symbols(token);
    if (word.size() <= 1) {
        return word;
```

```
    }

    while (true) {
        std::optional<std::int32_t> best_rank;
        std::string best_left;
        std::string best_right;
        for (std::size_t index = 0; index + 1 < word.size(); ++index) {
            const auto rank = tokenizer.bpe_ranks.find(pair_key(word[index], word[index + 1]));
            if (rank != tokenizer.bpe_ranks.end() &&
                (!best_rank.has_value() || rank->second < *best_rank)) {
                best_rank = rank->second;
                best_left = word[index];
                best_right = word[index + 1];
            }
        }
        if (!best_rank.has_value()) {
            return word;
        }

        std::vector<std::string> merged;
        std::size_t index = 0;
        while (index < word.size()) {
            if (index + 1 < word.size() && word[index] == best_left &&
                word[index + 1] == best_right) {
                merged.push_back(best_left + best_right);
                index += LCQI_GPT2_PAIR_TOKEN_COUNT;
            } else {
                merged.push_back(word[index]);
                ++index;
            }
        }
        word = std::move(merged);
        if (word.size() == 1) {
            return word;
        }
    }
}

bool is_ascii_letter(unsigned char value) {
    return (value >= 'A' && value <= 'Z') || (value >= 'a' && value <= 'z');
}

bool is_ascii_digit(unsigned char value) {
    return value >= '0' && value <= '9';
}

bool is_ascii_space(unsigned char value) {
    return value == ' ' || value == '\t' || value == '\n' ||
           value == '\r' || value == '\f' || value == '\v';
}

```

```
bool is_gpt2_punctuation(unsigned char value) {
    return !is_ascii_letter(value) && !is_ascii_digit(value) &&
           !is_ascii_space(value);
}

std::size_t consume_utf8_codepoint(std::string_view text, std::size_t offset) {
    const unsigned char lead = static_cast<unsigned char>(text[offset]);
    std::size_t width = 1;
    if ((lead & 0xE0U) == LCQI_GPT2_UTF8_TWO_BYTE_HEAD) {
        width = 2;
    } else if ((lead & 0xF0U) == LCQI_GPT2_UTF8_THREE_BYTE_HEAD) {
        width = 3;
    } else if ((lead & 0xF8U) == LCQI_GPT2_UTF8_FOUR_BYTE_HEAD) {
        width = 4;
    }
    if (offset + width > text.size()) {
        throw std::runtime_error("invalid UTF-8 text for GPT-2 tokenizer");
    }
    return offset + width;
}

bool starts_with_contraction(std::string_view text,
                             std::size_t offset,
                             std::string_view contraction) {
    if (offset + contraction.size() > text.size()) {
        return false;
    }
    for (std::size_t index = 0; index < contraction.size(); ++index) {
        char left = text[offset + index];
        const char right = contraction[index];
        if (left >= 'A' && left <= 'Z') {
            left = static_cast<char>(left - 'A' + 'a');
        }
        if (left != right) {
            return false;
        }
    }
    return true;
}

std::vector<std::string> gpt2_pretokenize(std::string_view text) {
    std::vector<std::string> pieces;
    std::size_t offset = 0;
    const std::array<std::string_view, 7> contractions{
        "'s", "'t", "'re", "'ve", "'m", "'ll", "'d",
    };
    while (offset < text.size()) {
        bool emitted_contraction = false;
        for (const std::string_view contraction : contractions) {
            if (starts_with_contraction(text, offset, contraction)) {
                pieces.emplace_back(text.substr(offset, contraction.size()));
                offset += contraction.size();
```

```
                emitted_contraction = true;
                break;
            }
        }
        if (emitted_contraction) {
            continue;
        }

        const std::size_t begin = offset;
        bool has_prefix_space = false;
        if (text[offset] == ' ' && offset + 1 < text.size() &&
            !is_ascii_space(static_cast<unsigned char>(text[offset + 1]))) {
            has_prefix_space = true;
            ++offset;
        }

        if (offset >= text.size()) {
            pieces.emplace_back(text.substr(begin, offset - begin));
            continue;
        }

        const unsigned char ch = static_cast<unsigned char>(text[offset]);
        if (is_ascii_letter(ch)) {
            while (offset < text.size() &&
                   is_ascii_letter(static_cast<unsigned char>(text[offset]))) {
                ++offset;
            }
        } else if (is_ascii_digit(ch)) {
            while (offset < text.size() &&
                   is_ascii_digit(static_cast<unsigned char>(text[offset]))) {
                ++offset;
            }
        } else if (is_gpt2_punctuation(ch) && ch != LCQI_GPT2_ASCII_APOSTROPHE)↩
    ↪  {
            while (offset < text.size() &&
                   is_gpt2_punctuation(static_cast<unsigned char>(text[offset])↩
    ↪ ) &&
                   static_cast<unsigned char>(text[offset]) != ↩
    ↪ LCQI_GPT2_ASCII_APOSTROPHE) {
                offset = consume_utf8_codepoint(text, offset);
            }
        } else if (is_ascii_space(ch)) {
            while (offset < text.size() &&
                   is_ascii_space(static_cast<unsigned char>(text[offset]))) {
                ++offset;
            }
        } else {
            offset = consume_utf8_codepoint(text, offset);
        }

        if (has_prefix_space || offset > begin) {
            pieces.emplace_back(text.substr(begin, offset - begin));
```

```
        } else {
            ++offset;
        }
    }
    return pieces;
}

std::int32_t parse_i32_field(JsonCursor& cursor) {
    const std::int64_t value = cursor.parse_i64();
    if (value < std::numeric_limits<std::int32_t>::min() ||
        value > std::numeric_limits<std::int32_t>::max()) {
        throw std::runtime_error("JSON int32 field out of range");
    }
    return static_cast<std::int32_t>(value);
}

std::unordered_map<std::string, std::int32_t> parse_vocab_json(std::string_view
    ↪ text) {
    JsonCursor cursor(text);
    std::unordered_map<std::string, std::int32_t> vocab;
    cursor.expect('{');
    if (cursor.consume('}')) {
        return vocab;
    }
    while (true) {
        const std::string key = cursor.parse_string();
        cursor.expect(':');
        const std::int32_t id = parse_i32_field(cursor);
        vocab.emplace(key, id);
        if (cursor.consume('}')) {
            break;
        }
        cursor.expect(',');
    }
    if (!cursor.eof_after_ws()) {
        throw std::runtime_error("trailing data after vocab JSON");
    }
    return vocab;
}

std::vector<std::string> build_id_to_token(
    const std::unordered_map<std::string, std::int32_t>& vocab) {
    std::int32_t max_id = -1;
    for (const auto& [token, id] : vocab) {
        if (id < 0) {
            throw std::runtime_error("GPT-2 vocab id cannot be negative");
        }
        max_id = std::max(max_id, id);
    }
    std::vector<std::string> result(checked_size(max_id + 1, "max vocab id"));
    for (const auto& [token, id] : vocab) {
        std::string& slot = result[checked_size(id, "vocab id")];
```

```
        if (!slot.empty()) {
            throw std::runtime_error("duplicate GPT-2 vocab id");
        }
        slot = token;
    }
    return result;
}

std::unordered_map<std::string, std::int32_t> parse_merges(std::string_view text) {
    std::unordered_map<std::string, std::int32_t> ranks;
    std::istringstream input{std::string(text)};
    std::string line;
    std::int32_t rank = 0;
    while (std::getline(input, line)) {
        if (line.empty() || line[0] == '#') {
            continue;
        }
        std::istringstream line_stream(line);
        std::string left;
        std::string right;
        if (!(line_stream >> left >> right)) {
            throw std::runtime_error("invalid GPT-2 merges line");
        }
        ranks.emplace(pair_key(left, right), rank);
        ++rank;
    }
    return ranks;
}

void transpose_hf_conv1d(std::vector<float>& values,
                         std::int32_t input_size,
                         std::int32_t output_size) {
    if (values.size() != matrix_size(input_size, output_size)) {
        throw std::runtime_error("GPT-2 Conv1D tensor size mismatch");
    }
    std::vector<float> transposed(matrix_size(output_size, input_size), 0.0F);
    for (std::int32_t input = 0; input < input_size; ++input) {
        for (std::int32_t output = 0; output < output_size; ++output) {
            transposed[matrix_size(output, input_size) + checked_size(input, "input")] =
                values[matrix_size(input, output_size) + checked_size(output, "output")];
        }
    }
    values = std::move(transposed);
}

std::vector<float> require_tensor(const SafeTensorFile& file,
                                  std::string_view name,
                                  std::span<const std::int64_t> expected_shape) {
```

```
    const SafeTensorEntry* entry = file.manifest.find_tensor(name);
    if (entry == nullptr) {
        throw std::runtime_error("missing GPT-2 tensor: " + std::string(name));
    }
    if (entry->shape.size() != expected_shape.size()) {
        throw std::runtime_error("GPT-2 tensor rank mismatch: " + std::string(name));
    }
    for (std::size_t index = 0; index < expected_shape.size(); ++index) {
        if (entry->shape[index] != expected_shape[index]) {
            throw std::runtime_error("GPT-2 tensor shape mismatch: " + std::string(name));
        }
    }
    return read_safetensor_f32_tensor(file, name);
}

std::vector<float> require_first_tensor(const SafeTensorFile& file,
                                        std::span<const std::string_view> names,
                                        std::span<const std::int64_t> expected_shape) {
    for (const std::string_view name : names) {
        if (file.manifest.find_tensor(name) != nullptr) {
            return require_tensor(file, name, expected_shape);
        }
    }
    throw std::runtime_error("missing GPT-2 tensor: " + std::string(names.front()));
}

Gpt2LinearF32 make_linear(std::int32_t input_size,
                          std::int32_t output_size,
                          std::vector<float> weights,
                          std::vector<float> bias = {}) {
    Gpt2LinearF32 linear;
    linear.input_size = input_size;
    linear.output_size = output_size;
    linear.weights = std::move(weights);
    linear.bias = std::move(bias);
    validate_linear(linear, "make GPT-2 linear");
    return linear;
}

std::vector<float> zeros(std::int32_t count) {
    return std::vector<float>(checked_size(count, "zero count"), 0.0F);
}

std::vector<float> ones(std::int32_t count) {
    return std::vector<float>(checked_size(count, "one count"), 1.0F);
}

```

```cpp
std::string tensor_name(std::int32_t layer, std::string_view suffix) {
    return "transformer.h." + std::to_string(layer) + "." + std::string(suffix);
}

std::string bare_tensor_name(std::int32_t layer, std::string_view suffix) {
    return "h." + std::to_string(layer) + "." + std::string(suffix);
}

std::array<std::string_view, 2> embedding_names(std::string_view bare) {
    if (bare == "wte.weight") {
        return {"transformer.wte.weight", "wte.weight"};
    }
    if (bare == "wpe.weight") {
        return {"transformer.wpe.weight", "wpe.weight"};
    }
    if (bare == "ln_f.weight") {
        return {"transformer.ln_f.weight", "ln_f.weight"};
    }
    if (bare == "ln_f.bias") {
        return {"transformer.ln_f.bias", "ln_f.bias"};
    }
    throw std::runtime_error("unsupported GPT-2 embedding tensor alias");
}

std::array<std::string, 2> layer_names(std::int32_t layer, std::string_view suffix) {
    return {tensor_name(layer, suffix), bare_tensor_name(layer, suffix)};
}

std::vector<float> require_layer_tensor(const SafeTensorFile& file,
                                        std::int32_t layer,
                                        std::string_view suffix,
                                        std::span<const std::int64_t> expected_shape) {
    const std::array<std::string, 2> names = layer_names(layer, suffix);
    const std::array<std::string_view, 2> views{names[0], names[1]};
    return require_first_tensor(file, views, expected_shape);
}

std::filesystem::path find_gpt2_weight_file(const std::filesystem::path& model_directory) {
    const std::array<const char*, 2> candidates{
        "model.safetensors",
        "pytorch_model.safetensors",
    };
    for (const char* candidate : candidates) {
        const std::filesystem::path path = model_directory / candidate;
        if (std::filesystem::exists(path)) {
            return path;
        }
    }
```

```
    throw std::runtime_error("cannot find model.safetensors in GPT-2 directory");
}

}  // namespace

Gpt2Tokenizer load_gpt2_tokenizer(const std::filesystem::path& vocab_json,
                                  const std::filesystem::path& merges_txt,
                                  std::int32_t bos_token_id,
                                  std::int32_t eos_token_id) {
    Gpt2Tokenizer tokenizer;
    tokenizer.bos_token_id = bos_token_id;
    tokenizer.eos_token_id = eos_token_id;
    tokenizer.vocab = parse_vocab_json(read_text_file(vocab_json));
    tokenizer.id_to_token = build_id_to_token(tokenizer.vocab);
    tokenizer.bpe_ranks = parse_merges(read_text_file(merges_txt));
    if (tokenizer.vocab.empty() || tokenizer.bpe_ranks.empty()) {
        throw std::runtime_error("GPT-2 tokenizer assets are empty");
    }
    return tokenizer;
}

std::vector<std::int32_t> gpt2_encode(const Gpt2Tokenizer& tokenizer,
                                      std::string_view text) {
    std::vector<std::int32_t> ids;
    for (const std::string& piece : gpt2_pretokenize(text)) {
        const std::string byte_piece = bytes_to_bpe_alphabet(piece);
        const std::vector<std::string> bpe_tokens = bpe_apply(tokenizer, byte_piece);
        for (const std::string& token : bpe_tokens) {
            const auto found = tokenizer.vocab.find(token);
            if (found == tokenizer.vocab.end()) {
                throw std::runtime_error("GPT-2 BPE token is missing from vocab");
            }
            ids.push_back(found->second);
        }
    }
    return ids;
}

std::string gpt2_decode(const Gpt2Tokenizer& tokenizer,
                        std::span<const std::int32_t> token_ids) {
    static const std::vector<std::string> BYTE_ENCODER = gpt2_bytes_to_unicode();
    static const std::unordered_map<std::string, unsigned char> BYTE_DECODER =
        gpt2_unicode_to_bytes(BYTE_ENCODER);

    std::string token_text;
    for (const std::int32_t token_id : token_ids) {
        if (token_id < 0 || token_id >= static_cast<std::int32_t>(tokenizer.id_to_token.size())) {
```

```
            throw std::runtime_error("GPT-2 token id out of range");
        }
        token_text += tokenizer.id_to_token[checked_size(token_id, "token id")];
    }

    std::string result;
    const std::vector<std::string> symbols = utf8_symbols(token_text);
    for (const std::string& symbol : symbols) {
        const auto found = BYTE_DECODER.find(symbol);
        if (found == BYTE_DECODER.end()) {
            throw std::runtime_error("GPT-2 token cannot be decoded to byte");
        }
        result.push_back(static_cast<char>(found->second));
    }
    return result;
}

Gpt2Config load_gpt2_config(const std::filesystem::path& config_json) {
    const std::string config_text = read_text_file(config_json);
    JsonCursor cursor(config_text);
    Gpt2Config config;
    config.bos_token_id = LCQI_GPT2_DEFAULT_BOS_EOS;
    config.eos_token_id = LCQI_GPT2_DEFAULT_BOS_EOS;

    cursor.expect('{');
    if (cursor.consume('}')) {
        throw std::runtime_error("GPT-2 config is empty");
    }
    while (true) {
        const std::string key = cursor.parse_string();
        cursor.expect(':');
        if (key == "n_embd") {
            config.hidden_size = parse_i32_field(cursor);
        } else if (key == "n_head") {
            config.head_count = parse_i32_field(cursor);
        } else if (key == "n_layer") {
            config.layer_count = parse_i32_field(cursor);
        } else if (key == "n_positions" || key == "n_ctx") {
            const std::int32_t value = parse_i32_field(cursor);
            config.max_positions = std::max(config.max_positions, value);
        } else if (key == "vocab_size") {
            config.vocab_size = parse_i32_field(cursor);
        } else if (key == "n_inner") {
            config.intermediate_size = parse_i32_field(cursor);
        } else if (key == "layer_norm_epsilon") {
            config.layer_norm_epsilon = static_cast<float>(cursor.parse_number());
        } else if (key == "activation_function") {
            config.activation_function = cursor.parse_string();
        } else if (key == "bos_token_id") {
            config.bos_token_id = parse_i32_field(cursor);
```

```
        } else if (key == "eos_token_id") {
            config.eos_token_id = parse_i32_field(cursor);
        } else {
            cursor.skip_value();
        }
        if (cursor.consume('}')) {
            break;
        }
        cursor.expect(',');
    }
    if (!cursor.eof_after_ws()) {
        throw std::runtime_error("trailing data after GPT-2 config JSON");
    }
    if (config.intermediate_size == 0 && config.hidden_size > 0) {
        config.intermediate_size = config.hidden_size * 4;
    }
    validate_config(config);
    return config;
}

Gpt2ReferenceModel load_gpt2_reference_model(const std::filesystem::path& config_json,
                                             const std::filesystem::path& safetensors_path) {
    Gpt2ReferenceModel model;
    model.config = load_gpt2_config(config_json);
    const SafeTensorFile file = load_safetensors_file(safetensors_path);
    const Gpt2Config& config = model.config;
    const std::array<std::int64_t, 2> wte_shape{config.vocab_size, config.hidden_size};
    const std::array<std::int64_t, 2> wpe_shape{config.max_positions, config.hidden_size};
    const std::array<std::int64_t, 1> hidden_shape{config.hidden_size};

    const std::array<std::string_view, 2> wte_names = embedding_names("wte.weight");
    const std::array<std::string_view, 2> wpe_names = embedding_names("wpe.weight");
    model.token_embedding = require_first_tensor(file, wte_names, wte_shape);
    model.position_embedding = require_first_tensor(file, wpe_names, wpe_shape);
    model.layers.resize(checked_size(config.layer_count, "layer_count"));

    for (std::int32_t layer_id = 0; layer_id < config.layer_count; ++layer_id) {
        Gpt2LayerWeightsF32& layer = model.layers[checked_size(layer_id, "layer_id")];
        layer.ln_1_weight = require_layer_tensor(file, layer_id, "ln_1.weight", hidden_shape);
        layer.ln_1_bias = require_layer_tensor(file, layer_id, "ln_1.bias", hidden_shape);
        layer.ln_2_weight = require_layer_tensor(file, layer_id, "ln_2.weight",
```

```
    hidden_shape);
1871        layer.ln_2_bias = require_layer_tensor(file, layer_id, "ln_2.bias", hidden_shape);
1872
1873        const std::array<std::int64_t, 2> c_attn_shape{
1874            config.hidden_size,
1875            config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS};
1876        std::vector<float> c_attn_weight =
1877            require_layer_tensor(file, layer_id, "attn.c_attn.weight", c_attn_shape);
1878        transpose_hf_conv1d(c_attn_weight,
1879                            config.hidden_size,
1880                            config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS);
1881        const std::array<std::int64_t, 1> c_attn_bias_shape{
1882            config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS};
1883        layer.c_attn = make_linear(config.hidden_size,
1884                                   config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS,
1885                                   std::move(c_attn_weight),
1886                                   require_layer_tensor(file,
1887                                                        layer_id,
1888                                                        "attn.c_attn.bias",
1889                                                        c_attn_bias_shape));
1890
1891        const std::array<std::int64_t, 2> c_proj_shape{config.hidden_size, config.hidden_size};
1892        std::vector<float> c_proj_weight =
1893            require_layer_tensor(file, layer_id, "attn.c_proj.weight", c_proj_shape);
1894        transpose_hf_conv1d(c_proj_weight, config.hidden_size, config.hidden_size);
1895        layer.c_proj = make_linear(config.hidden_size,
1896                                   config.hidden_size,
1897                                   std::move(c_proj_weight),
1898                                   require_layer_tensor(file,
1899                                                        layer_id,
1900                                                        "attn.c_proj.bias",
1901                                                        hidden_shape));
1902
1903        const std::array<std::int64_t, 2> c_fc_shape{
1904            config.hidden_size,
1905            config.intermediate_size};
1906        std::vector<float> c_fc_weight =
1907            require_layer_tensor(file, layer_id, "mlp.c_fc.weight", c_fc_shape);
1908        transpose_hf_conv1d(c_fc_weight, config.hidden_size, config.intermediate_size);
1909        const std::array<std::int64_t, 1> intermediate_shape{config.intermediate_size};
1910        layer.c_fc = make_linear(config.hidden_size,
1911                                 config.intermediate_size,
```

```
                                    std::move(c_fc_weight),
                                    require_layer_tensor(file,
                                                         layer_id,
                                                         "mlp.c_fc.bias",
                                                         intermediate_shape));

        const std::array<std::int64_t, 2> mlp_proj_shape{
            config.intermediate_size,
            config.hidden_size};
        std::vector<float> mlp_proj_weight =
            require_layer_tensor(file, layer_id, "mlp.c_proj.weight", mlp_proj_shape);
        transpose_hf_conv1d(mlp_proj_weight, config.intermediate_size, config.hidden_size);
        layer.mlp_c_proj = make_linear(config.intermediate_size,
                                       config.hidden_size,
                                       std::move(mlp_proj_weight),
                                       require_layer_tensor(file,
                                                            layer_id,
                                                            "mlp.c_proj.bias",
                                                            hidden_shape));
    }

    const std::array<std::string_view, 2> ln_weight_names = embedding_names("ln_f.weight");
    const std::array<std::string_view, 2> ln_bias_names = embedding_names("ln_f.bias");
    model.final_ln_weight = require_first_tensor(file, ln_weight_names, hidden_shape);
    model.final_ln_bias = require_first_tensor(file, ln_bias_names, hidden_shape);
    if (file.manifest.find_tensor("lm_head.weight") != nullptr) {
        model.tie_lm_head_to_embedding = false;
        model.lm_head_weight = require_tensor(file, "lm_head.weight", wte_shape);
    } else {
        model.tie_lm_head_to_embedding = true;
    }
    validate_model(model);
    return model;
}

Gpt2ReferenceModel load_gpt2_from_directory(const std::filesystem::path& model_directory) {
    return load_gpt2_reference_model(model_directory / "config.json",
                                     find_gpt2_weight_file(model_directory));
}

Gpt2ForwardWorkspace::Gpt2ForwardWorkspace(const Gpt2Config& config) : config_(config) {
    this->reset_for_config(config);
}
```

```
1955
1956 void Gpt2ForwardWorkspace::reset_for_config(const Gpt2Config& config) {
1957     validate_config(config);
1958     this->config_ = config;
1959     const std::size_t hidden_size = checked_size(this->config_.hidden_size, "↩
     ↪ hidden_size");
1960     const std::size_t intermediate_size =
1961         checked_size(this->config_.intermediate_size, "intermediate_size");
1962     this->hidden_.assign(hidden_size, 0.0F);
1963     this->normed_.assign(hidden_size, 0.0F);
1964     this->query_.assign(hidden_size, 0.0F);
1965     this->key_.assign(hidden_size, 0.0F);
1966     this->value_.assign(hidden_size, 0.0F);
1967     this->attention_.assign(hidden_size, 0.0F);
1968     this->projected_.assign(hidden_size, 0.0F);
1969     this->qkv_packed_.assign(
1970         matrix_size(LCQI_GPT2_ATTENTION_PROJECTIONS, this->config_.hidden_size)↩
     ↪ ,
1971         0.0F);
1972     this->mlp_fc_.assign(intermediate_size, 0.0F);
1973     this->mlp_out_.assign(hidden_size, 0.0F);
1974     this->logits_.assign(checked_size(this->config_.vocab_size, "vocab_size"), ↩
     ↪ 0.0F);
1975     this->scores_.assign(checked_size(this->config_.max_positions, "↩
     ↪ max_positions"), 0.0F);
1976 }
1977
1978 std::span<float> Gpt2ForwardWorkspace::hidden() noexcept {
1979     return this->hidden_;
1980 }
1981
1982 std::span<float> Gpt2ForwardWorkspace::normed() noexcept {
1983     return this->normed_;
1984 }
1985
1986 std::span<float> Gpt2ForwardWorkspace::query() noexcept {
1987     return this->query_;
1988 }
1989
1990 std::span<float> Gpt2ForwardWorkspace::key() noexcept {
1991     return this->key_;
1992 }
1993
1994 std::span<float> Gpt2ForwardWorkspace::value() noexcept {
1995     return this->value_;
1996 }
1997
1998 std::span<float> Gpt2ForwardWorkspace::attention() noexcept {
1999     return this->attention_;
2000 }
2001
2002 std::span<float> Gpt2ForwardWorkspace::projected() noexcept {
```

```
2003     return this->projected_;
2004 }
2005
2006 std::span<float> Gpt2ForwardWorkspace::qkv_packed() noexcept {
2007     return this->qkv_packed_;
2008 }
2009
2010 std::span<float> Gpt2ForwardWorkspace::mlp_fc() noexcept {
2011     return this->mlp_fc_;
2012 }
2013
2014 std::span<float> Gpt2ForwardWorkspace::mlp_out() noexcept {
2015     return this->mlp_out_;
2016 }
2017
2018 std::span<float> Gpt2ForwardWorkspace::logits() noexcept {
2019     return this->logits_;
2020 }
2021
2022 std::span<float> Gpt2ForwardWorkspace::scores_prefix(std::int32_t count) {
2023     if (count < 0 || count > this->config_.max_positions) {
2024         throw std::runtime_error("GPT-2 attention scores count out of range");
2025     }
2026     return std::span<float>(this->scores_.data(), checked_size(count, "scores ↩
        ↪ count"));
2027 }
2028
2029 Gpt2KvCache::Gpt2KvCache(const Gpt2Config& config) : config_(config) {
2030     validate_config(this->config_);
2031     const std::size_t element_count =
2032         checked_size(this->config_.layer_count, "layer_count") *
2033         checked_size(this->config_.max_positions, "max_positions") *
2034         checked_size(this->config_.head_count, "head_count") *
2035         checked_size(head_dim(this->config_), "head_dim");
2036     this->keys_.assign(element_count, 0.0F);
2037     this->values_.assign(element_count, 0.0F);
2038     this->written_.assign(
2039         checked_size(this->config_.layer_count, "layer_count") *
2040             checked_size(this->config_.max_positions, "max_positions"),
2041         0);
2042 }
2043
2044 void Gpt2KvCache::append(std::int32_t layer_id,
2045                          std::int32_t model_position,
2046                          std::span<const float> key,
2047                          std::span<const float> value) {
2048     const std::size_t hidden_size = checked_size(this->config_.hidden_size, "↩
        ↪ hidden_size");
2049     if (key.size() != hidden_size || value.size() != hidden_size) {
2050         throw std::runtime_error("GPT-2 KV key/value size mismatch");
2051     }
2052     this->validate_address(layer_id, model_position, 0);
```

```
    const std::int32_t local_head_dim = head_dim(this->config_);
    for (std::int32_t head = 0; head < this->config_.head_count; ++head) {
        const std::size_t source =
            checked_size(head, "head") * checked_size(local_head_dim, "head_dim");
        const std::size_t target = this->base_offset(layer_id, model_position, head);
        std::copy_n(key.begin() + static_cast<std::ptrdiff_t>(source),
                    local_head_dim,
                    this->keys_.begin() + static_cast<std::ptrdiff_t>(target));
        std::copy_n(value.begin() + static_cast<std::ptrdiff_t>(source),
                    local_head_dim,
                    this->values_.begin() + static_cast<std::ptrdiff_t>(target));
    }
    const std::size_t written_index =
        checked_size(layer_id, "layer_id") *
            checked_size(this->config_.max_positions, "max_positions") +
        checked_size(model_position, "model_position");
    this->written_[written_index] = 1;
    this->filled_tokens_ = std::max(this->filled_tokens_, model_position + 1);
}

std::span<const float> Gpt2KvCache::key(std::int32_t layer_id,
                                        std::int32_t model_position,
                                        std::int32_t head) const {
    this->validate_address(layer_id, model_position, head);
    this->validate_written(layer_id, model_position);
    return this->key_unchecked(layer_id, model_position, head);
}

std::span<const float> Gpt2KvCache::value(std::int32_t layer_id,
                                          std::int32_t model_position,
                                          std::int32_t head) const {
    this->validate_address(layer_id, model_position, head);
    this->validate_written(layer_id, model_position);
    return this->value_unchecked(layer_id, model_position, head);
}

void detail::attend_cached_position(const Gpt2KvCache& cache,
                                    std::int32_t layer_id,
                                    std::int32_t model_position,
                                    std::span<const float> query,
                                    std::span<float> scores,
                                    std::span<float> output) {
    cache.validate_address(layer_id, model_position, 0);
    cache.validate_written(layer_id, model_position);
    const std::int32_t local_head_dim = head_dim(cache.config_);
    const std::size_t hidden_size = checked_size(cache.config_.hidden_size, "hidden_size");
    const std::size_t local_head_dim_size = checked_size(local_head_dim, "head_dim");
```

```
2100     const std::size_t model_position_size =
2101         checked_size(model_position, "model_position");
2102     if (query.size() != hidden_size || output.size() != hidden_size ||
2103         scores.size() < model_position_size + 1) {
2104         throw std::runtime_error("GPT-2 cached attention workspace size ↩
       ↪ mismatch");
2105     }
2106
2107     const float score_scale = 1.0F / std::sqrt(static_cast<float>(↩
       ↪ local_head_dim));
2108     std::fill(output.begin(), output.end(), 0.0F);
2109     for (std::int32_t head = 0; head < cache.config_.head_count; ++head) {
2110         const std::size_t head_base =
2111             checked_size(head, "head") * local_head_dim_size;
2112         float max_score = LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY;
2113         for (std::size_t past = 0; past <= model_position_size; ++past) {
2114             const std::int32_t past_i32 = static_cast<std::int32_t>(past);
2115             const std::span<const float> key =
2116                 cache.key_unchecked(layer_id, past_i32, head);
2117             float dot = 0.0F;
2118             for (std::size_t dim = 0; dim < local_head_dim_size; ++dim) {
2119                 dot += query[head_base + dim] * key[dim];
2120             }
2121             const float score = dot * score_scale;
2122             scores[past] = score;
2123             max_score = std::max(max_score, score);
2124         }
2125
2126         float denominator = 0.0F;
2127         for (std::size_t past = 0; past <= model_position_size; ++past) {
2128             scores[past] = std::exp(scores[past] - max_score);
2129             denominator += scores[past];
2130         }
2131         if (denominator <= 0.0F) {
2132             throw std::runtime_error("GPT-2 cached attention denominator is ↩
       ↪ invalid");
2133         }
2134
2135         for (std::size_t past = 0; past <= model_position_size; ++past) {
2136             const std::int32_t past_i32 = static_cast<std::int32_t>(past);
2137             const float probability = scores[past] / denominator;
2138             const std::span<const float> value =
2139                 cache.value_unchecked(layer_id, past_i32, head);
2140             for (std::size_t dim = 0; dim < local_head_dim_size; ++dim) {
2141                 output[head_base + dim] += probability * value[dim];
2142             }
2143         }
2144     }
2145 }
2146
2147 std::span<const float> Gpt2KvCache::key_unchecked(std::int32_t layer_id,
2148                                                   std::int32_t model_position,
```

```
                                                    std::int32_t head) const ↩
    ↪ noexcept {
    const std::size_t offset = this->base_offset_unchecked(layer_id, ↩
    ↪ model_position, head);
    return std::span<const float>(
        this->keys_.data() + offset,
        static_cast<std::size_t>(head_dim(this->config_)));
}

std::span<const float> Gpt2KvCache::value_unchecked(std::int32_t layer_id,
                                                    std::int32_t model_position↩
    ↪ ,
                                                    std::int32_t head) const ↩
    ↪ noexcept {
    const std::size_t offset = this->base_offset_unchecked(layer_id, ↩
    ↪ model_position, head);
    return std::span<const float>(
        this->values_.data() + offset,
        static_cast<std::size_t>(head_dim(this->config_)));
}

std::int32_t Gpt2KvCache::filled_tokens() const noexcept {
    return this->filled_tokens_;
}

std::size_t Gpt2KvCache::byte_size() const noexcept {
    return (this->keys_.size() + this->values_.size()) * sizeof(float) +
           this->written_.size() * sizeof(std::uint8_t);
}

std::size_t Gpt2KvCache::base_offset(std::int32_t layer_id,
                                     std::int32_t model_position,
                                     std::int32_t head) const {
    return (((checked_size(layer_id, "layer_id") *
              checked_size(this->config_.max_positions, "max_positions")) +
             checked_size(model_position, "model_position")) *
                checked_size(this->config_.head_count, "head_count") +
            checked_size(head, "head")) *
           checked_size(head_dim(this->config_), "head_dim");
}

std::size_t Gpt2KvCache::base_offset_unchecked(std::int32_t layer_id,
                                               std::int32_t model_position,
                                               std::int32_t head) const ↩
    ↪ noexcept {
    return (((static_cast<std::size_t>(layer_id) *
              static_cast<std::size_t>(this->config_.max_positions)) +
             static_cast<std::size_t>(model_position)) *
                static_cast<std::size_t>(this->config_.head_count) +
            static_cast<std::size_t>(head)) *
           static_cast<std::size_t>(head_dim(this->config_));
}
```

```
2195
2196 void Gpt2KvCache::validate_address(std::int32_t layer_id,
2197                                     std::int32_t model_position,
2198                                     std::int32_t head) const {
2199     if (layer_id < 0 || layer_id >= this->config_.layer_count) {
2200         throw std::runtime_error("GPT-2 KV layer_id out of range");
2201     }
2202     if (model_position < 0 || model_position >= this->config_.max_positions) {
2203         throw std::runtime_error("GPT-2 KV model_position out of range");
2204     }
2205     if (head < 0 || head >= this->config_.head_count) {
2206         throw std::runtime_error("GPT-2 KV head out of range");
2207     }
2208 }
2209
2210 void Gpt2KvCache::validate_written(std::int32_t layer_id,
2211                                     std::int32_t model_position) const {
2212     const std::size_t written_index =
2213         checked_size(layer_id, "layer_id") *
2214             checked_size(this->config_.max_positions, "max_positions") +
2215         checked_size(model_position, "model_position");
2216     if (written_index >= this->written_.size() || this->written_[written_index]↩
        ↪  == 0) {
2217         throw std::runtime_error("GPT-2 KV slot has not been written");
2218     }
2219 }
2220
2221 Gpt2CachedGreedyDecoder::Gpt2CachedGreedyDecoder(const Gpt2ReferenceModel& ↩
        ↪ model)
2222     : Gpt2CachedGreedyDecoder(model, nullptr) {}
2223
2224 Gpt2CachedGreedyDecoder::Gpt2CachedGreedyDecoder(
2225     const Gpt2ReferenceModel& model,
2226     Gpt2HotspotProfile* hotspot_profile)
2227     : Gpt2CachedGreedyDecoder(model, hotspot_profile, Gpt2ExecutionOptions{}) ↩
        ↪ {}
2228
2229 Gpt2CachedGreedyDecoder::Gpt2CachedGreedyDecoder(
2230     const Gpt2ReferenceModel& model,
2231     Gpt2HotspotProfile* hotspot_profile,
2232     const Gpt2ExecutionOptions& execution_options)
2233     : model_(&model),
2234       hotspot_profile_(hotspot_profile),
2235       execution_options_(execution_options),
2236       cache_(model.config),
2237       workspace_(model.config),
2238       worker_pool_(make_worker_pool(model, execution_options)) {
2239     validate_model(*this->model_);
2240 }
2241
2242 Gpt2CachedGreedyDecoder::~Gpt2CachedGreedyDecoder() = default;
2243
```

```
Gpt2CachedGreedyDecoder::Gpt2CachedGreedyDecoder(Gpt2CachedGreedyDecoder&&) noexcept =
    default;

Gpt2CachedGreedyDecoder& Gpt2CachedGreedyDecoder::operator=(
    Gpt2CachedGreedyDecoder&&) noexcept = default;

void Gpt2CachedGreedyDecoder::advance_without_prediction(std::int32_t token_id) {
    static_cast<void>(run_gpt2_cached_step_unchecked(*this->model_,
                                                     this->cache_,
                                                     this->workspace_,
                                                     token_id,
                                                     Gpt2CachedStepMode::advance_only,
                                                     this->hotspot_profile_,
                                                     this->execution_options_,
                                                     this->worker_pool_.get()));
}

std::int32_t Gpt2CachedGreedyDecoder::step(std::int32_t token_id) {
    return run_gpt2_cached_step_unchecked(*this->model_,
                                          this->cache_,
                                          this->workspace_,
                                          token_id,
                                          Gpt2CachedStepMode::predict_token,
                                          this->hotspot_profile_,
                                          this->execution_options_,
                                          this->worker_pool_.get())
        .predicted_token;
}

Gpt2ForwardResult Gpt2CachedGreedyDecoder::step_with_logits(std::int32_t token_id) {
    return run_gpt2_cached_step_unchecked(*this->model_,
                                          this->cache_,
                                          this->workspace_,
                                          token_id,
                                          Gpt2CachedStepMode::logits,
                                          this->hotspot_profile_,
                                          this->execution_options_,
                                          this->worker_pool_.get());
}

const Gpt2KvCache& Gpt2CachedGreedyDecoder::cache() const noexcept {
    return this->cache_;
}

std::int32_t Gpt2CachedGreedyDecoder::filled_tokens() const noexcept {
    return this->cache_.filled_tokens();
}

```

```cpp
std::size_t Gpt2CachedGreedyDecoder::kv_cache_bytes() const noexcept {
    return this->cache_.byte_size();
}

std::int32_t Gpt2CachedGreedyDecoder::worker_count() const noexcept {
    return this->worker_pool_ == nullptr ? 1 : this->worker_pool_->worker_count();
}

Gpt2ForwardResult run_gpt2_forward(const Gpt2ReferenceModel& model,
                                   std::span<const std::int32_t> token_ids) {
    validate_model(model);
    if (token_ids.empty()) {
        throw std::runtime_error("GPT-2 forward needs at least one token");
    }
    if (token_ids.size() > checked_size(model.config.max_positions, "max_positions")) {
        throw std::runtime_error("GPT-2 token_ids exceed max_positions");
    }

    const std::int32_t sequence_length = static_cast<std::int32_t>(token_ids.size());
    std::vector<float> hidden_by_pos(matrix_size(sequence_length, model.config.hidden_size), 0.0F);
    for (std::int32_t position = 0; position < sequence_length; ++position) {
        const std::int32_t token_id = token_ids[checked_size(position, "position")];
        if (token_id < 0 || token_id >= model.config.vocab_size) {
            throw std::runtime_error("GPT-2 token id out of range");
        }
        const std::span<const float> token =
            row_span(model.token_embedding, token_id, model.config.hidden_size);
        const std::span<const float> position_embedding =
            row_span(model.position_embedding, position, model.config.hidden_size);
        std::span<float> hidden = std::span<float>(
            hidden_by_pos.data() + matrix_size(position, model.config.hidden_size),
            checked_size(model.config.hidden_size, "hidden_size"));
        for (std::int32_t dim = 0; dim < model.config.hidden_size; ++dim) {
            hidden[checked_size(dim, "dim")] =
                token[checked_size(dim, "dim")] + position_embedding[checked_size(dim, "dim")];
        }
    }

    for (const Gpt2LayerWeightsF32& layer : model.layers) {
        forward_gpt2_layer(model.config, layer, sequence_length, hidden_by_pos);
    }

```

```
    std::vector<float> normed(checked_size(model.config.hidden_size, "hidden_size"), 0.0F);
    const std::span<const float> final_hidden = std::span<const float>(
        hidden_by_pos.data() + matrix_size(sequence_length - 1, model.config.hidden_size),
        checked_size(model.config.hidden_size, "hidden_size"));
    layer_norm(final_hidden,
               model.final_ln_weight,
               model.final_ln_bias,
               model.config.layer_norm_epsilon,
               normed);

    Gpt2ForwardResult result;
    result.logits.assign(checked_size(model.config.vocab_size, "vocab_size"), 0.0F);
    compute_logits(model, normed, result.logits);
    result.predicted_token = argmax(result.logits);
    return result;
}

Gpt2ForwardResult run_gpt2_forward_cached(const Gpt2ReferenceModel& model,
                                          Gpt2KvCache& cache,
                                          std::int32_t token_id) {
    validate_model(model);
    Gpt2ForwardWorkspace workspace(model.config);
    Gpt2ExecutionOptions options;
    options.worker_count = 1;
    return run_gpt2_cached_step_unchecked(
        model,
        cache,
        workspace,
        token_id,
        Gpt2CachedStepMode::logits,
        nullptr,
        options,
        nullptr);
}

std::vector<std::int32_t> gpt2_generate_greedy(const Gpt2ReferenceModel& model,
                                               std::span<const std::int32_t> prompt_token_ids,
                                               std::int32_t max_new_tokens) {
    if (max_new_tokens < 0) {
        throw std::runtime_error("max_new_tokens cannot be negative");
    }
    std::vector<std::int32_t> tokens(prompt_token_ids.begin(), prompt_token_ids.end());
    for (std::int32_t step = 0; step < max_new_tokens; ++step) {
        if (tokens.size() >= checked_size(model.config.max_positions, "max_positions")) {
            throw std::runtime_error("GPT-2 generation would exceed max_positions");
```

```
        }
        const Gpt2ForwardResult result = run_gpt2_forward(model, tokens);
        tokens.push_back(result.predicted_token);
        if (model.config.eos_token_id >= 0 && result.predicted_token == model.config.eos_token_id) {
            break;
        }
    }
    return tokens;
}

std::vector<std::int32_t> gpt2_generate_greedy_cached(
    const Gpt2ReferenceModel& model,
    std::span<const std::int32_t> prompt_token_ids,
    std::int32_t max_new_tokens) {
    if (prompt_token_ids.empty()) {
        throw std::runtime_error("GPT-2 generation needs at least one prompt token");
    }
    if (max_new_tokens < 0) {
        throw std::runtime_error("max_new_tokens cannot be negative");
    }
    if (prompt_token_ids.size() > checked_size(model.config.max_positions, "max_positions")) {
        throw std::runtime_error("GPT-2 prompt exceeds max_positions");
    }

    Gpt2CachedGreedyDecoder decoder(model);
    std::vector<std::int32_t> tokens(prompt_token_ids.begin(), prompt_token_ids.end());
    if (max_new_tokens == 0) {
        return tokens;
    }
    for (std::size_t index = 0; index + 1 < prompt_token_ids.size(); ++index) {
        decoder.advance_without_prediction(prompt_token_ids[index]);
    }
    std::int32_t predicted_token = decoder.step(prompt_token_ids.back());
    for (std::int32_t step = 0; step < max_new_tokens; ++step) {
        if (tokens.size() >= checked_size(model.config.max_positions, "max_positions")) {
            throw std::runtime_error("GPT-2 generation would exceed max_positions");
        }
        tokens.push_back(predicted_token);
        if (model.config.eos_token_id >= 0 && predicted_token == model.config.eos_token_id) {
            break;
        }
        if (step + 1 < max_new_tokens) {
            predicted_token = decoder.step(predicted_token);
        }
    }
```

```
    return tokens;
}

Gpt2ReferenceModel make_tiny_gpt2_reference_model() {
    Gpt2ReferenceModel model;
    model.config.hidden_size = 4;
    model.config.head_count = 2;
    model.config.layer_count = 1;
    model.config.max_positions = 8;
    model.config.vocab_size = 6;
    model.config.intermediate_size = 8;
    model.config.bos_token_id = 0;
    model.config.eos_token_id = 5;
    model.config.layer_norm_epsilon = 1.0e-5F;
    model.config.activation_function = "gelu_new";

    model.token_embedding = {
        0.20F, -0.10F, 0.00F, 0.10F,
        -0.10F, 0.30F, 0.20F, -0.20F,
        0.05F, 0.10F, -0.30F, 0.25F,
        0.40F, -0.20F, 0.15F, 0.05F,
        -0.30F, 0.05F, 0.35F, -0.15F,
        0.10F, 0.20F, -0.10F, 0.30F,
    };
    model.position_embedding = {
        0.01F, 0.02F, 0.03F, 0.04F,
        -0.02F, 0.01F, 0.00F, 0.03F,
        0.03F, -0.01F, 0.02F, 0.00F,
        0.00F, 0.02F, -0.02F, 0.01F,
        0.02F, 0.00F, 0.01F, -0.01F,
        -0.01F, 0.03F, 0.00F, 0.02F,
        0.01F, -0.02F, 0.03F, 0.00F,
        0.00F, 0.01F, -0.01F, 0.02F,
    };

    Gpt2LayerWeightsF32 layer;
    layer.ln_1_weight = ones(4);
    layer.ln_1_bias = zeros(4);
    layer.ln_2_weight = {0.9F, 1.1F, 1.0F, 0.95F};
    layer.ln_2_bias = {0.01F, -0.02F, 0.00F, 0.03F};
    layer.c_attn = make_linear(
        4,
        12,
        {
            0.10F, -0.20F, 0.05F, 0.15F,
            -0.05F, 0.12F, 0.20F, -0.10F,
            0.18F, 0.02F, -0.08F, 0.11F,
            -0.12F, 0.06F, 0.14F, 0.04F,
            0.07F, 0.16F, -0.09F, 0.03F,
            -0.15F, 0.05F, 0.13F, 0.10F,
            0.04F, -0.11F, 0.19F, -0.02F,
            0.09F, 0.08F, -0.06F, 0.17F,
```

```
            0.20F, -0.04F, 0.01F, 0.06F,
            -0.08F, 0.15F, 0.07F, -0.03F,
            0.05F, 0.10F, -0.14F, 0.12F,
            0.11F, -0.07F, 0.16F, 0.02F,
        },
        {
            0.01F, -0.02F, 0.00F, 0.03F,
            -0.01F, 0.02F, 0.01F, -0.02F,
            0.00F, 0.01F, -0.01F, 0.02F,
        });
    layer.c_proj = make_linear(
        4,
        4,
        {
            0.10F, 0.05F, -0.08F, 0.12F,
            -0.04F, 0.11F, 0.09F, -0.02F,
            0.07F, -0.10F, 0.13F, 0.05F,
            0.02F, 0.14F, -0.06F, 0.08F,
        },
        {0.01F, 0.00F, -0.01F, 0.02F});
    layer.c_fc = make_linear(
        4,
        8,
        {
            0.20F, -0.10F, 0.05F, 0.03F,
            -0.05F, 0.14F, 0.12F, -0.08F,
            0.09F, 0.07F, -0.11F, 0.16F,
            -0.12F, 0.05F, 0.18F, 0.02F,
            0.04F, -0.09F, 0.13F, 0.10F,
            0.11F, 0.03F, -0.07F, 0.15F,
            -0.02F, 0.17F, 0.06F, -0.04F,
            0.08F, -0.06F, 0.10F, 0.12F,
        },
        {0.01F, -0.02F, 0.00F, 0.03F, -0.01F, 0.02F, 0.01F, 0.00F});
    layer.mlp_c_proj = make_linear(
        8,
        4,
        {
            0.10F, -0.08F, 0.06F, 0.12F, -0.04F, 0.07F, 0.03F, 0.11F,
            -0.05F, 0.13F, 0.02F, -0.09F, 0.15F, 0.04F, -0.03F, 0.08F,
            0.07F, 0.01F, -0.12F, 0.10F, 0.06F, -0.05F, 0.14F, 0.02F,
            0.03F, 0.09F, 0.11F, -0.06F, 0.05F, 0.12F, -0.08F, 0.04F,
        },
        {0.00F, 0.01F, -0.01F, 0.02F});
    model.layers.push_back(std::move(layer));
    model.final_ln_weight = {1.0F, 0.95F, 1.05F, 1.0F};
    model.final_ln_bias = {0.0F, 0.01F, -0.01F, 0.02F};
    model.tie_lm_head_to_embedding = true;
    validate_model(model);
    return model;
}

```

```
2528 } // namespace lcqi
```

gpt2_cli.cpp 负责把模型目录、prompt、max_new_tokens 和 benchmark 选项接到 GPT-2 reference path。无参数时运行 tiny GPT-2 fixture；传入 HuggingFace 模型目录时加载真实资产。

Listing 5.5 src/gpt2_cli.cpp

```cpp
#include <lcqi/gpt2_reference.hpp>

#include <chrono>
#include <cstdlib>
#include <exception>
#include <filesystem>
#include <iostream>
#include <span>
#include <stdexcept>
#include <string>
#include <string_view>
#include <vector>

namespace {

constexpr int LCQI_GPT2_TINY_ARGC = 1;
constexpr int LCQI_GPT2_MIN_REAL_ARGC = 3;
constexpr std::int32_t LCQI_GPT2_DEFAULT_MAX_NEW_TOKENS = 8;
constexpr std::int32_t LCQI_GPT2_TINY_PROMPT_0 = 1;
constexpr std::int32_t LCQI_GPT2_TINY_PROMPT_1 = 2;
constexpr std::int32_t LCQI_GPT2_TINY_PROMPT_2 = 3;

enum class Gpt2EngineMode {
    full_prefix,
    cached_kv,
};

struct Gpt2CliOptions {
    Gpt2EngineMode engine = Gpt2EngineMode::cached_kv;
    bool benchmark = false;
    bool profile_hotspots = false;
    std::filesystem::path model_dir;
    std::string prompt;
    std::int32_t max_new_tokens = LCQI_GPT2_DEFAULT_MAX_NEW_TOKENS;
    std::int32_t worker_count = 0;
};

struct Gpt2BenchmarkTimings {
    double load_ms = 0.0;
    double tokenize_ms = 0.0;
    double prefill_ms = 0.0;
    double decode_ms = 0.0;
    double generate_ms = 0.0;
    double total_ms = 0.0;
    std::size_t prefill_steps = 0;
```

```
    std::size_t decode_steps = 0;
    std::size_t kv_cache_bytes = 0;
    std::int32_t worker_count = 1;
};

using Clock = std::chrono::steady_clock;

void print_ids(std::span<const std::int32_t> ids) {
    for (std::size_t index = 0; index < ids.size(); ++index) {
        if (index != 0) {
            std::cout << ' ';
        }
        std::cout << ids[index];
    }
}

[[nodiscard]] double elapsed_ms(Clock::time_point start, Clock::time_point end)
    {
    return std::chrono::duration<double, std::milli>(end - start).count();
}

[[nodiscard]] std::size_t checked_size(std::int64_t value, const char* name) {
    if (value < 0) {
        throw std::runtime_error(std::string(name) + " cannot be negative");
    }
    return static_cast<std::size_t>(value);
}

[[nodiscard]] const char* engine_name(Gpt2EngineMode engine) {
    switch (engine) {
        case Gpt2EngineMode::full_prefix:
            return "full_prefix";
        case Gpt2EngineMode::cached_kv:
            return "cached_kv";
    }
    return "unknown";
}

[[nodiscard]] Gpt2EngineMode parse_engine(std::string_view value) {
    if (value == "full" || value == "full_prefix") {
        return Gpt2EngineMode::full_prefix;
    }
    if (value == "cached" || value == "cached_kv") {
        return Gpt2EngineMode::cached_kv;
    }
    throw std::runtime_error("unknown --engine value, expected cached or full")
    ;
}

[[nodiscard]] std::int32_t parse_non_negative_i32(const std::string& text,
                                                  const char* name) {
    const std::int32_t value = static_cast<std::int32_t>(std::stoi(text));
```

```
    if (value < 0) {
        throw std::runtime_error(std::string(name) + " cannot be negative");
    }
    return value;
}

[[nodiscard]] Gpt2CliOptions parse_options(int argc, char** argv) {
    Gpt2CliOptions options;
    std::vector<std::string> positional;
    for (int index = 1; index < argc; ++index) {
        const std::string arg = argv[index];
        if (arg == "--benchmark") {
            options.benchmark = true;
        } else if (arg == "--profile-hotspots") {
            options.profile_hotspots = true;
            options.benchmark = true;
        } else if (arg == "--engine") {
            if (index + 1 >= argc) {
                throw std::runtime_error("--engine requires a value");
            }
            ++index;
            options.engine = parse_engine(argv[index]);
        } else if (arg.rfind("--engine=", 0) == 0) {
            options.engine = parse_engine(std::string_view(arg).substr(std::string("--engine=").size()));
        } else if (arg == "--threads") {
            if (index + 1 >= argc) {
                throw std::runtime_error("--threads requires a value");
            }
            ++index;
            options.worker_count = parse_non_negative_i32(argv[index], "--threads");
        } else if (arg.rfind("--threads=", 0) == 0) {
            options.worker_count =
                parse_non_negative_i32(arg.substr(std::string("--threads=").size()), "--threads");
        } else if (arg == "--full") {
            options.engine = Gpt2EngineMode::full_prefix;
        } else if (arg == "--cached") {
            options.engine = Gpt2EngineMode::cached_kv;
        } else if (!arg.empty() && arg.front() == '-') {
            throw std::runtime_error("unknown option: " + arg);
        } else {
            positional.push_back(arg);
        }
    }

    if (positional.size() < 2 || positional.size() > 3) {
        throw std::runtime_error(
            "usage: lcqi_gpt2 [--engine cached|full] [--threads N] [--benchmark] "
            "model_dir prompt [max_new_tokens]");
```

```
    }
    options.model_dir = positional[0];
    options.prompt = positional[1];
    if (positional.size() == 3) {
        options.max_new_tokens = parse_non_negative_i32(positional[2], "max_new_tokens");
    }
    return options;
}

[[nodiscard]] std::vector<std::int32_t> generate_full_prefix(
    const lcqi::Gpt2ReferenceModel& model,
    std::span<const std::int32_t> prompt_ids,
    std::int32_t max_new_tokens,
    Gpt2BenchmarkTimings& timings) {
    const Clock::time_point generate_start = Clock::now();
    std::vector<std::int32_t> tokens(prompt_ids.begin(), prompt_ids.end());
    for (std::int32_t step = 0; step < max_new_tokens; ++step) {
        if (tokens.size() >= checked_size(model.config.max_positions, "max_positions")) {
            throw std::runtime_error("GPT-2 generation would exceed max_positions");
        }
        const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(model, tokens);
        ++timings.decode_steps;
        tokens.push_back(result.predicted_token);
        if (model.config.eos_token_id >= 0 && result.predicted_token == model.config.eos_token_id) {
            break;
        }
    }
    const Clock::time_point generate_end = Clock::now();
    timings.generate_ms = elapsed_ms(generate_start, generate_end);
    timings.decode_ms = timings.generate_ms;
    return tokens;
}

[[nodiscard]] std::vector<std::int32_t> generate_cached(
    const lcqi::Gpt2ReferenceModel& model,
    std::span<const std::int32_t> prompt_ids,
    std::int32_t max_new_tokens,
    Gpt2BenchmarkTimings& timings,
    lcqi::Gpt2HotspotProfile* hotspot_profile,
    std::int32_t worker_count) {
    if (prompt_ids.empty()) {
        throw std::runtime_error("GPT-2 generation needs at least one prompt token");
    }
    lcqi::Gpt2ExecutionOptions execution_options;
    execution_options.worker_count = worker_count;
    lcqi::Gpt2CachedGreedyDecoder decoder(model, hotspot_profile,
```

```
    ↪ execution_options);
    timings.kv_cache_bytes = decoder.kv_cache_bytes();
    timings.worker_count = decoder.worker_count();
    std::vector<std::int32_t> tokens(prompt_ids.begin(), prompt_ids.end());
    if (max_new_tokens == 0) {
        return tokens;
    }

    const Clock::time_point generate_start = Clock::now();
    const Clock::time_point prefill_start = Clock::now();
    for (std::size_t index = 0; index + 1 < prompt_ids.size(); ++index) {
        decoder.advance_without_prediction(prompt_ids[index]);
        ++timings.prefill_steps;
    }
    std::int32_t predicted_token = decoder.step(prompt_ids.back());
    ++timings.prefill_steps;
    const Clock::time_point prefill_end = Clock::now();
    timings.prefill_ms = elapsed_ms(prefill_start, prefill_end);

    const Clock::time_point decode_start = Clock::now();
    for (std::int32_t step = 0; step < max_new_tokens; ++step) {
        if (tokens.size() >= checked_size(model.config.max_positions, "↩
    ↪ max_positions")) {
            throw std::runtime_error("GPT-2 generation would exceed ↩
    ↪ max_positions");
        }
        tokens.push_back(predicted_token);
        if (model.config.eos_token_id >= 0 && predicted_token == model.config.↩
    ↪ eos_token_id) {
            break;
        }
        if (step + 1 < max_new_tokens) {
            predicted_token = decoder.step(predicted_token);
            ++timings.decode_steps;
        }
    }
    const Clock::time_point decode_end = Clock::now();
    const Clock::time_point generate_end = Clock::now();
    timings.decode_ms = elapsed_ms(decode_start, decode_end);
    timings.generate_ms = elapsed_ms(generate_start, generate_end);
    return tokens;
}

void print_benchmark(const Gpt2BenchmarkTimings& timings,
                     std::size_t prompt_tokens,
                     std::size_t generated_tokens) {
    const double new_tokens = static_cast<double>(generated_tokens);
    const double generated_tokens_per_second =
        timings.generate_ms > 0.0 ? new_tokens * 1000.0 / timings.generate_ms :↩
    ↪  0.0;
    const double decode_tokens_per_second =
        timings.decode_ms > 0.0
```

```
237             ? static_cast<double>(timings.decode_steps) * 1000.0 / timings.↩
    ↪ decode_ms
238             : 0.0;
239
240     std::cout << "benchmark_load_ms " << timings.load_ms << "\n";
241     std::cout << "benchmark_tokenize_ms " << timings.tokenize_ms << "\n";
242     std::cout << "benchmark_prefill_ms " << timings.prefill_ms << "\n";
243     std::cout << "benchmark_decode_ms " << timings.decode_ms << "\n";
244     std::cout << "benchmark_generate_ms " << timings.generate_ms << "\n";
245     std::cout << "benchmark_total_ms " << timings.total_ms << "\n";
246     std::cout << "benchmark_prompt_tokens " << prompt_tokens << "\n";
247     std::cout << "benchmark_generated_tokens " << generated_tokens << "\n";
248     std::cout << "benchmark_prefill_steps " << timings.prefill_steps << "\n";
249     std::cout << "benchmark_decode_steps " << timings.decode_steps << "\n";
250     std::cout << "benchmark_worker_count " << timings.worker_count << "\n";
251     std::cout << "benchmark_generate_tokens_per_second "
252               << generated_tokens_per_second << "\n";
253     std::cout << "benchmark_decode_tokens_per_second "
254               << decode_tokens_per_second << "\n";
255     std::cout << "benchmark_kv_cache_bytes " << timings.kv_cache_bytes << "\n";
256 }
257
258 void print_hotspot_profile(const lcqi::Gpt2HotspotProfile& profile) {
259     const double total = profile.total_step_ms > 0.0 ? profile.total_step_ms : ↩
    ↪ 1.0;
260     const auto print_ms = [total](std::string_view name, double value) {
261         std::cout << "hotspot_" << name << "_ms " << value << "\n";
262         std::cout << "hotspot_" << name << "_pct " << (value * 100.0 / total) ↩
    ↪ << "\n";
263     };
264     std::cout << "hotspot_decoder_steps " << profile.decoder_steps << "\n";
265     std::cout << "hotspot_layer_steps " << profile.layer_steps << "\n";
266     print_ms("total_step", profile.total_step_ms);
267     print_ms("embedding", profile.embedding_ms);
268     print_ms("layer_norm", profile.layer_norm_ms);
269     print_ms("final_norm", profile.final_norm_ms);
270     print_ms("qkv_projection", profile.qkv_projection_ms);
271     print_ms("kv_append", profile.kv_append_ms);
272     print_ms("attention", profile.attention_ms);
273     print_ms("attention_projection", profile.attention_projection_ms);
274     print_ms("residual_add", profile.residual_add_ms);
275     print_ms("mlp_fc", profile.mlp_fc_ms);
276     print_ms("gelu", profile.gelu_ms);
277     print_ms("mlp_projection", profile.mlp_projection_ms);
278     print_ms("lm_head", profile.lm_head_ms);
279     print_ms("logits_result", profile.logits_result_ms);
280 }
281
282 int run_tiny_mode() {
283     const lcqi::Gpt2ReferenceModel model = lcqi::make_tiny_gpt2_reference_model↩
    ↪ ();
284     const std::vector<std::int32_t> prompt{
```

```
285         LCQI_GPT2_TINY_PROMPT_0,
286         LCQI_GPT2_TINY_PROMPT_1,
287         LCQI_GPT2_TINY_PROMPT_2,
288     };
289     const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(model, prompt↩
    ↪ );
290     const std::vector<std::int32_t> generated =
291         lcqi::gpt2_generate_greedy(model, prompt, 2);
292     std::cout << "mode tiny\n";
293     std::cout << "prompt_ids ";
294     print_ids(prompt);
295     std::cout << "\n";
296     std::cout << "predicted_token " << result.predicted_token << "\n";
297     std::cout << "logits";
298     for (const float value : result.logits) {
299         std::cout << ' ' << value;
300     }
301     std::cout << "\n";
302     std::cout << "generated_ids ";
303     print_ids(generated);
304     std::cout << "\n";
305     return EXIT_SUCCESS;
306 }
307
308 int run_real_model_mode(int argc, char** argv) {
309     if (argc < LCQI_GPT2_MIN_REAL_ARGC) {
310         throw std::runtime_error(
311             "usage: lcqi_gpt2 [--engine cached|full] [--threads N] [--benchmark↩
    ↪ ] "
312             "model_dir prompt [max_new_tokens]");
313     }
314     const Clock::time_point total_start = Clock::now();
315     const Gpt2CliOptions options = parse_options(argc, argv);
316
317     Gpt2BenchmarkTimings timings;
318     const Clock::time_point load_start = Clock::now();
319     const lcqi::Gpt2ReferenceModel model = lcqi::load_gpt2_from_directory(↩
    ↪ options.model_dir);
320     const lcqi::Gpt2Tokenizer tokenizer = lcqi::load_gpt2_tokenizer(
321         options.model_dir / "vocab.json",
322         options.model_dir / "merges.txt",
323         model.config.bos_token_id,
324         model.config.eos_token_id);
325     timings.load_ms = elapsed_ms(load_start, Clock::now());
326
327     const Clock::time_point tokenize_start = Clock::now();
328     const std::vector<std::int32_t> prompt_ids = lcqi::gpt2_encode(tokenizer, ↩
    ↪ options.prompt);
329     timings.tokenize_ms = elapsed_ms(tokenize_start, Clock::now());
330
331     std::vector<std::int32_t> generated;
332     lcqi::Gpt2HotspotProfile hotspot_profile;
```

```
333     if (options.engine == Gpt2EngineMode::full_prefix) {
334         if (options.profile_hotspots) {
335             throw std::runtime_error("--profile-hotspots currently supports cached engine only");
336         }
337         generated = generate_full_prefix(model, prompt_ids, options.max_new_tokens, timings);
338     } else {
339         generated = generate_cached(model,
340                                     prompt_ids,
341                                     options.max_new_tokens,
342                                     timings,
343                                     options.profile_hotspots ? &hotspot_profile : nullptr,
344                                     options.worker_count);
345     }
346     timings.total_ms = elapsed_ms(total_start, Clock::now());
347 
348     std::cout << "mode gpt2_directory\n";
349     std::cout << "engine " << engine_name(options.engine) << "\n";
350     std::cout << "model_dir " << options.model_dir.string() << "\n";
351     std::cout << "prompt_ids ";
352     print_ids(prompt_ids);
353     std::cout << "\n";
354     std::cout << "generated_ids ";
355     print_ids(generated);
356     std::cout << "\n";
357     std::cout << "generated_text " << lcqi::gpt2_decode(tokenizer, generated) << "\n";
358     if (options.benchmark) {
359         const std::size_t generated_tokens = generated.size() - prompt_ids.size();
360         print_benchmark(timings, prompt_ids.size(), generated_tokens);
361     }
362     if (options.profile_hotspots) {
363         print_hotspot_profile(hotspot_profile);
364     }
365     return EXIT_SUCCESS;
366 }
367 
368 }  // namespace
369 
370 int main(int argc, char** argv) {
371     try {
372         if (argc == LCQI_GPT2_TINY_ARGC) {
373             return run_tiny_mode();
374         }
375         return run_real_model_mode(argc, argv);
376     } catch (const std::exception& error) {
377         std::cerr << "error: " << error.what() << "\n";
378         return EXIT_FAILURE;
379     }
```

```
380 }
```

第 6 章

工具、报告、Benchmark 与回归测试

6.1 工具不是附属品

LCQI 的工程证据不只在 C++ 源码里。smoke 脚本决定真实模型怎样下载、校验和运行；benchmark 程序决定性能口径；trace、ledger 和 serving probe 决定报告字段；CTest 决定哪些行为能长期回归。读者必须把这些文件当作工程合同的一部分。

6.2 Benchmark、Trace、Ledger 与 Serving Probe

bench.cpp 输出 int8 kernel benchmark。它服务性能观察，但必须和 max diff 对照一起看。

Listing 6.1 src/bench.cpp

```cpp
#include <lcqi/int8_kernels.hpp>

#include <algorithm>
#include <cstdlib>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>

namespace {

constexpr const char* LCQI_TARGET_ARCH_X86_64 = "x86_64";
constexpr const char* LCQI_TARGET_ARCH_I386 = "i386";
constexpr const char* LCQI_TARGET_ARCH_UNKNOWN = "unknown";
constexpr std::int32_t LCQI_DEFAULT_REPEAT = 200;
constexpr int LCQI_REPEAT_ARG_INDEX = 1;
constexpr std::int32_t LCQI_LARGE_CASE_REPEAT_DIVISOR = 4;
constexpr std::int32_t LCQI_MIN_LARGE_CASE_REPEAT = 1;
constexpr std::int32_t LCQI_DECODE_SHAPE_SIZE = 4096;

const char* target_arch() noexcept {
#if defined(__x86_64__) || defined(_M_X64)
    return LCQI_TARGET_ARCH_X86_64;
#elif defined(__i386__) || defined(_M_IX86)
    return LCQI_TARGET_ARCH_I386;
#else
    return LCQI_TARGET_ARCH_UNKNOWN;
#endif
}

std::int32_t parse_repeat(int argc, char** argv) {
    if (argc <= LCQI_REPEAT_ARG_INDEX) {
        return LCQI_DEFAULT_REPEAT;
```

```
    }
    const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[LCQI_REPEAT_ARG_INDEX]));
    if (repeat <= 0) {
        throw std::runtime_error("repeat must be positive");
    }
    return repeat;
}

std::vector<lcqi::KernelBenchmarkCase> make_cases(std::int32_t repeat) {
    return {
        {128, 128, 16, repeat},
        {256, 256, 16, repeat},
        {512, 512, 16, repeat},
        {513, 257, 16, repeat},
        {1024, 4096, 16, std::max(LCQI_MIN_LARGE_CASE_REPEAT,
                                  repeat / LCQI_LARGE_CASE_REPEAT_DIVISOR)},
        {LCQI_DECODE_SHAPE_SIZE, LCQI_DECODE_SHAPE_SIZE, 16,
         std::max(LCQI_MIN_LARGE_CASE_REPEAT, repeat / LCQI_LARGE_CASE_REPEAT_DIVISOR)},
    };
}

}  // namespace

int main(int argc, char** argv) {
    try {
        const std::int32_t repeat = parse_repeat(argc, argv);
        std::cout
            << "target_arch,input_size,output_size,output_block,repeat,backend,available,"
               "average_us,max_abs_diff,checksum\n";
        for (const lcqi::KernelBenchmarkCase& benchmark_case : make_cases(repeat)) {
            const lcqi::KernelBenchmarkResult result =
                lcqi::benchmark_linear_i8_case(benchmark_case);
            for (const lcqi::KernelBackendBenchmark& backend : result.backends) {
                std::cout << target_arch() << ','
                          << result.benchmark_case.input_size << ','
                          << result.benchmark_case.output_size << ','
                          << result.benchmark_case.output_block_size << ','
                          << result.benchmark_case.repeat << ','
                          << backend.name << ','
                          << (backend.available ? 1 : 0) << ','
                          << backend.average_us << ','
                          << backend.max_abs_diff << ','
                          << backend.checksum << '\n';
            }
        }
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
```

```
        std::cerr << "error: " << error.what() << '\n';
        return EXIT_FAILURE;
    }
}
```

trace.cpp 输出 reference decoder checkpoint。它用于定位错误边界，不应该被简化成只打印最终 token。

Listing 6.2 src/trace.cpp

```
#include <lcqi/reference_decoder.hpp>
#include <lcqi/tokenizer.hpp>

#include <algorithm>
#include <array>
#include <cstdlib>
#include <exception>
#include <filesystem>
#include <iostream>
#include <span>
#include <sstream>
#include <string>
#include <string_view>

namespace {

constexpr std::array<std::int32_t, 3> LCQI_TRACE_TOKEN_IDS{1, 2, 3};
constexpr std::string_view LCQI_TRACE_PROMPT = "tok1 tok2 tok3";
constexpr std::int32_t LCQI_TRACE_METADATA_LAYER = -1;
constexpr std::int32_t LCQI_TRACE_BOS_ID = 0;
constexpr std::int32_t LCQI_TRACE_EOS_ID = 4;
constexpr std::size_t LCQI_TRACE_VALUE_LIMIT = 8;

std::filesystem::path tokenizer_fixture_path() {
    return std::filesystem::path(__FILE__).parent_path().parent_path() /
           "models" / "tiny_tokenizer.txt";
}

std::vector<std::int32_t> decoder_tokens_from_prompt(const lcqi::TokenizerModel
    ↪ & tokenizer,
                                                     std::string_view prompt) {
    std::vector<std::int32_t> full_tokens = lcqi::encode_prompt(tokenizer,
    ↪ prompt);
    if (full_tokens.size() != LCQI_TRACE_TOKEN_IDS.size() + 2 ||
        full_tokens.front() != LCQI_TRACE_BOS_ID ||
        full_tokens.back() != LCQI_TRACE_EOS_ID) {
        throw std::runtime_error("unexpected tokenizer fixture ids");
    }

    std::vector<std::int32_t> decoder_tokens(
        full_tokens.begin() + 1,
        full_tokens.end() - 1);
    if (!std::equal(decoder_tokens.begin(),
```

```
                    decoder_tokens.end(),
                    LCQI_TRACE_TOKEN_IDS.begin(),
                    LCQI_TRACE_TOKEN_IDS.end())) {
        throw std::runtime_error("unexpected decoder token ids");
    }
    return decoder_tokens;
}

std::string join_ints(std::span<const std::int32_t> values) {
    std::ostringstream stream;
    for (std::size_t index = 0; index < values.size(); ++index) {
        if (index != 0) {
            stream << 'x';
        }
        stream << values[index];
    }
    return stream.str();
}

std::string join_int_values(std::span<const std::int32_t> values) {
    std::ostringstream stream;
    for (std::size_t index = 0; index < values.size(); ++index) {
        if (index != 0) {
            stream << ';';
        }
        stream << values[index];
    }
    return stream.str();
}

std::int32_t checksum_ints(std::span<const std::int32_t> values) {
    std::int32_t result = 0;
    for (const std::int32_t value : values) {
        result += value;
    }
    return result;
}

std::int32_t max_abs_ints(std::span<const std::int32_t> values) {
    std::int32_t result = 0;
    for (const std::int32_t value : values) {
        result = std::max(result, std::abs(value));
    }
    return result;
}

std::string join_floats(std::span<const float> values) {
    std::ostringstream stream;
    const std::size_t count = std::min(values.size(), LCQI_TRACE_VALUE_LIMIT);
    for (std::size_t index = 0; index < count; ++index) {
        if (index != 0) {
            stream << ';';
```

```
        }
        stream << values[index];
    }
    if (values.size() > LCQI_TRACE_VALUE_LIMIT) {
        stream << ";...";
    }
    return stream.str();
}

void print_trace_csv(const lcqi::ReferenceDecodeTraceResult& trace,
                     const lcqi::TokenizerModel& tokenizer,
                     std::span<const std::int32_t> full_tokens) {
    std::cout << "name,token_position,layer_id,shape,stride,dtype,layout,checksum,max_abs,values\n";
    std::cout << "prompt,0," << LCQI_TRACE_METADATA_LAYER
              << ",scalar,scalar,utf8,text,0,0," << LCQI_TRACE_PROMPT << '\n';
    std::cout << "tokenizer,0," << LCQI_TRACE_METADATA_LAYER
              << ',' << full_tokens.size() << ",1,i32,contiguous,"
              << checksum_ints(full_tokens) << ','
              << max_abs_ints(full_tokens) << ','
              << join_int_values(full_tokens) << '\n';
    std::cout << "tokenizer_contract,0," << LCQI_TRACE_METADATA_LAYER
              << ",scalar,scalar,u64,fnv1a,"
              << tokenizer_contract_hash(tokenizer) << ",0,"
              << tokenizer.chat_template_hash << '\n';
    for (const lcqi::ReferenceTensorCheckpoint& checkpoint : trace.checkpoints) {
        std::cout << checkpoint.name << ','
                  << checkpoint.token_position << ','
                  << checkpoint.layer_id << ','
                  << join_ints(checkpoint.shape) << ','
                  << join_ints(checkpoint.stride) << ','
                  << checkpoint.dtype << ','
                  << checkpoint.layout << ','
                  << checkpoint.checksum << ','
                  << checkpoint.max_abs << ','
                  << join_floats(checkpoint.values) << '\n';
    }
    std::cout << "sampler_argmax,"
              << LCQI_TRACE_TOKEN_IDS.size() - 1 << ','
              << LCQI_TRACE_METADATA_LAYER
              << ',' << trace.result.logits.size() << ",1,f32,argmax,"
              << "0,0,token=" << trace.result.predicted_token << '\n';
    std::cout << "generated_token,"
              << LCQI_TRACE_TOKEN_IDS.size() << ','
              << LCQI_TRACE_METADATA_LAYER
              << ",scalar,scalar,i32,generated,"
              << trace.result.predicted_token << ','
              << trace.result.predicted_token << ','
              << trace.result.predicted_token << '\n';
}

```

```
} // namespace

int main() {
    try {
        const lcqi::TokenizerModel tokenizer =
            lcqi::load_tokenizer(tokenizer_fixture_path());
        const std::vector<std::int32_t> full_tokens =
            lcqi::encode_prompt(tokenizer, LCQI_TRACE_PROMPT);
        const std::vector<std::int32_t> decoder_tokens =
            decoder_tokens_from_prompt(tokenizer, LCQI_TRACE_PROMPT);
        const lcqi::ReferenceDecoderModel model = lcqi::
    ↪ make_tiny_reference_decoder_model();
        const lcqi::ReferenceDecodeTraceResult trace =
            lcqi::run_reference_decode_with_trace(model, decoder_tokens);
        print_trace_csv(trace, tokenizer, full_tokens);
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << '\n';
        return EXIT_FAILURE;
    }
}
```

ledger.cpp 输出 7B 规模账本。它是估算，不是实测吞吐；读者要区分 FLOPs、KV 容量、权重/KV 字节和 bandwidth-only tokens/s 上限。

Listing 6.3 src/ledger.cpp

```
#include <cstdlib>
#include <cstdint>
#include <exception>
#include <iomanip>
#include <iostream>
#include <string_view>

namespace {

constexpr double LCQI_BYTES_PER_GB = 1000.0 * 1000.0 * 1000.0;
constexpr double LCQI_BYTES_PER_MIB = 1024.0 * 1024.0;
constexpr double LCQI_FLOPS_PER_MAC = 2.0;
constexpr std::int32_t LCQI_SEVENB_LAYER_COUNT = 32;
constexpr std::int32_t LCQI_SEVENB_HIDDEN_SIZE = 4096;
constexpr std::int32_t LCQI_SEVENB_QUERY_HEADS = 32;
constexpr std::int32_t LCQI_SEVENB_KV_HEADS = 8;
constexpr std::int32_t LCQI_SEVENB_HEAD_DIM = 128;
constexpr std::int32_t LCQI_SEVENB_INTERMEDIATE_SIZE = 11008;
constexpr std::int32_t LCQI_SEVENB_CONTEXT_LENGTH = 4096;

struct DTypeLedger {
    std::string_view name;
    double weight_gb_per_token = 0.0;
    double kv_element_bytes = 0.0;
};

```

```
constexpr DTypeLedger LCQI_DTYPE_LEDGERS[] = {
    {"fp16", 14.0, 2.0},
    {"int8", 7.0, 2.0},
    {"int4", 3.7, 2.0},
};

constexpr double LCQI_BANDWIDTH_GB_PER_S[] = {
    50.0,
    100.0,
    200.0,
    600.0,
    1000.0,
    1600.0,
};

double linear_macs_per_layer() noexcept {
    const double hidden = LCQI_SEVENB_HIDDEN_SIZE;
    const double kv_width = LCQI_SEVENB_KV_HEADS * LCQI_SEVENB_HEAD_DIM;
    const double qkv = hidden * (hidden + 2.0 * kv_width);
    const double output_projection = hidden * hidden;
    const double mlp = 3.0 * hidden * LCQI_SEVENB_INTERMEDIATE_SIZE;
    return qkv + output_projection + mlp;
}

double attention_macs_per_layer() noexcept {
    return 2.0 * LCQI_SEVENB_QUERY_HEADS * LCQI_SEVENB_CONTEXT_LENGTH *
           LCQI_SEVENB_HEAD_DIM;
}

double kv_bytes_per_token_all_layers(double element_bytes) noexcept {
    return static_cast<double>(LCQI_SEVENB_LAYER_COUNT) * 2.0 *
           LCQI_SEVENB_KV_HEADS * LCQI_SEVENB_HEAD_DIM * element_bytes;
}

double kv_read_gb_per_decode_token(double element_bytes) noexcept {
    const double bytes = static_cast<double>(LCQI_SEVENB_CONTEXT_LENGTH) *
                         kv_bytes_per_token_all_layers(element_bytes);
    return bytes / LCQI_BYTES_PER_GB;
}

double kv_capacity_mib(double element_bytes, std::int32_t context_length) noexcept {
    const double bytes = static_cast<double>(context_length) *
                         kv_bytes_per_token_all_layers(element_bytes);
    return bytes / LCQI_BYTES_PER_MIB;
}

void print_model_rows() {
    const double linear_macs = linear_macs_per_layer();
    const double attention_macs = attention_macs_per_layer();
    const double total_flops =
        (linear_macs + attention_macs) * LCQI_SEVENB_LAYER_COUNT *
```

```
    ↪ LCQI_FLOPS_PER_MAC;
78      std::cout << "section,item,value,unit,notes\n";
79      std::cout << "model,layers," << LCQI_SEVENB_LAYER_COUNT << ",count,7B ↩
    ↪ fixture\n";
80      std::cout << "model,hidden," << LCQI_SEVENB_HIDDEN_SIZE << ",elements,H\n";
81      std::cout << "model,query_heads," << LCQI_SEVENB_QUERY_HEADS << ",count,GQA↩
    ↪  query heads\n";
82      std::cout << "model,kv_heads," << LCQI_SEVENB_KV_HEADS << ",count,GQA KV ↩
    ↪ heads\n";
83      std::cout << "model,head_dim," << LCQI_SEVENB_HEAD_DIM << ",elements,per ↩
    ↪ head\n";
84      std::cout << "compute,linear_macs_per_layer," << linear_macs << ",macs,↩
    ↪ decode token\n";
85      std::cout << "compute,attention_macs_per_layer," << attention_macs
86                << ",macs,context=4096\n";
87      std::cout << "compute,total_decode_flops_per_token," << total_flops
88                << ",flops,linear plus attention\n";
89  }
90
91  void print_kv_rows() {
92      for (const std::int32_t context : {1024, 2048, 4096, 8192}) {
93          std::cout << "kv,fp16_capacity_context_" << context << ','
94                    << kv_capacity_mib(2.0, context)
95                    << ",MiB,single session\n";
96      }
97  }
98
99  void print_dtype_rows() {
100     for (const DTypeLedger& dtype : LCQI_DTYPE_LEDGERS) {
101         const double kv_read_gb = kv_read_gb_per_decode_token(dtype.↩
    ↪ kv_element_bytes);
102         const double total_gb = dtype.weight_gb_per_token + kv_read_gb;
103         std::cout << "decode_bytes," << dtype.name << "_weight_gb,"
104                   << dtype.weight_gb_per_token << ",GB/token,weight stream\n";
105         std::cout << "decode_bytes," << dtype.name << "_kv_read_gb,"
106                   << kv_read_gb << ",GB/token,context=4096\n";
107         std::cout << "decode_bytes," << dtype.name << "_total_gb,"
108                   << total_gb << ",GB/token,weight plus KV\n";
109         for (const double bandwidth : LCQI_BANDWIDTH_GB_PER_S) {
110             std::cout << "bandwidth_limit," << dtype.name << "_at_"
111                       << static_cast<std::int32_t>(bandwidth) << "_gbps,"
112                       << bandwidth / total_gb
113                       << ",tokens/s,bandwidth-only upper bound\n";
114         }
115     }
116 }
117
118 }  // namespace
119
120 int main() {
121     try {
122         std::cout << std::fixed << std::setprecision(3);
```

```
        print_model_rows();
        print_kv_rows();
        print_dtype_rows();
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << '\n';
        return EXIT_FAILURE;
    }
}
```

serving_probe.cpp 固定短请求、RAG 请求、取消请求和超长请求。它把 TTFT、TPOT、拒绝和取消回收变成可观察字段。

Listing 6.4 src/serving_probe.cpp

```
#include <algorithm>
#include <cstdlib>
#include <cstdint>
#include <exception>
#include <cmath>
#include <iomanip>
#include <iostream>
#include <string_view>
#include <vector>

namespace {

constexpr double LCQI_BYTES_PER_MIB = 1024.0 * 1024.0;
constexpr std::int32_t LCQI_PROBE_LAYER_COUNT = 32;
constexpr std::int32_t LCQI_PROBE_KV_HEADS = 8;
constexpr std::int32_t LCQI_PROBE_HEAD_DIM = 128;
constexpr double LCQI_PROBE_KV_ELEMENT_BYTES = 2.0;
constexpr double LCQI_PROBE_KV_BUDGET_MIB = 512.0;
constexpr double LCQI_PROBE_WORKSPACE_MIB = 64.0;
constexpr double LCQI_PROBE_ARENA_MIB = 32.0;
constexpr double LCQI_PROBE_TRACE_MIB = 4.0;
constexpr double LCQI_PROBE_QUEUE_WAIT_PER_REQUEST_MS = 8.0;
constexpr double LCQI_PROBE_PREFILL_MS_PER_TOKEN = 0.08;
constexpr double LCQI_PROBE_DECODE_STEP_MS = 14.0;
constexpr double LCQI_PROBE_CANCEL_RECLAIM_MS = 2.0;
constexpr std::size_t LCQI_PERCENTILE_MIN_INDEX = 1;

struct ProbeRequest {
    std::string_view request_id;
    std::int32_t prompt_tokens = 0;
    std::int32_t max_new_tokens = 0;
    bool cancel_after_prefill = false;
};

struct ProbeResult {
    std::string_view request_id;
    std::int32_t accepted = 0;
    std::string_view rejection_reason;
```

```
    std::int32_t prompt_tokens = 0;
    std::int32_t reserved_tokens = 0;
    double kv_reserved_mib = 0.0;
    double queue_wait_ms = 0.0;
    double prefill_ms = 0.0;
    double ttft_ms = 0.0;
    double decode_step_p95_ms = 0.0;
    double tpot_ms = 0.0;
    std::int32_t completion_tokens = 0;
    std::string_view finish_reason;
    double cancel_reclaim_ms = 0.0;
};

const std::vector<ProbeRequest> LCQI_PROBE_REQUESTS{
    {"req_short", 32, 4, false},
    {"req_rag", 512, 8, false},
    {"req_cancel", 128, 16, true},
    {"req_too_long", 4096, 512, false},
};

double kv_mib_for_tokens(std::int32_t tokens) noexcept {
    const double bytes = static_cast<double>(tokens) * LCQI_PROBE_LAYER_COUNT *↩
    ↪  2.0 *
                         LCQI_PROBE_KV_HEADS * LCQI_PROBE_HEAD_DIM *
                         LCQI_PROBE_KV_ELEMENT_BYTES;
    return bytes / LCQI_BYTES_PER_MIB;
}

double percentile(std::vector<double> values, double fraction) {
    if (values.empty()) {
        return 0.0;
    }
    std::sort(values.begin(), values.end());
    const std::size_t max_index = values.size() - LCQI_PERCENTILE_MIN_INDEX;
    const std::size_t index =
        std::min(max_index,
                 static_cast<std::size_t>(
                     std::ceil(fraction * static_cast<double>(max_index))));
    return values[index];
}

std::vector<ProbeResult> run_probe() {
    std::vector<ProbeResult> results;
    double reserved_kv_mib = 0.0;
    std::int32_t accepted_before = 0;
    for (const ProbeRequest& request : LCQI_PROBE_REQUESTS) {
        ProbeResult result;
        result.request_id = request.request_id;
        result.prompt_tokens = request.prompt_tokens;
        result.reserved_tokens = request.prompt_tokens + request.max_new_tokens↩
    ↪ ;
        result.kv_reserved_mib = kv_mib_for_tokens(result.reserved_tokens);
```

```
        const double static_session_mib =
            result.kv_reserved_mib + LCQI_PROBE_WORKSPACE_MIB +
            LCQI_PROBE_ARENA_MIB + LCQI_PROBE_TRACE_MIB;
        if (reserved_kv_mib + result.kv_reserved_mib > LCQI_PROBE_KV_BUDGET_MIB ) {
            result.accepted = 0;
            result.rejection_reason = "memory_budget_exceeded";
            result.finish_reason = "rejected";
            results.push_back(result);
            continue;
        }

        reserved_kv_mib += result.kv_reserved_mib;
        result.accepted = 1;
        result.rejection_reason = "none";
        result.queue_wait_ms = static_cast<double>(accepted_before) *
                               LCQI_PROBE_QUEUE_WAIT_PER_REQUEST_MS;
        result.prefill_ms = static_cast<double>(request.prompt_tokens) *
                            LCQI_PROBE_PREFILL_MS_PER_TOKEN;
        result.ttft_ms = result.queue_wait_ms + result.prefill_ms + LCQI_PROBE_DECODE_STEP_MS;
        result.decode_step_p95_ms = LCQI_PROBE_DECODE_STEP_MS +
                                    static_session_mib / 512.0;
        result.tpot_ms = result.decode_step_p95_ms;
        if (request.cancel_after_prefill) {
            result.completion_tokens = 0;
            result.finish_reason = "cancelled";
            result.cancel_reclaim_ms = LCQI_PROBE_CANCEL_RECLAIM_MS;
            reserved_kv_mib -= result.kv_reserved_mib;
        } else {
            result.completion_tokens = request.max_new_tokens;
            result.finish_reason = "max_tokens";
        }
        ++accepted_before;
        results.push_back(result);
    }
    return results;
}

void print_request_rows(const std::vector<ProbeResult>& results) {
    std::cout << "row_type,request_id,accepted,rejection_reason,prompt_tokens,reserved_tokens,"
                 "kv_reserved_mib,queue_wait_ms,prefill_ms,ttft_ms,decode_step_p95_ms,"
                 "tpot_ms,completion_tokens,finish_reason,cancel_reclaim_ms,metric_name,"
                 "metric_value\n";
    for (const ProbeResult& result : results) {
        std::cout << "request," << result.request_id << ','
                  << result.accepted << ','
                  << result.rejection_reason << ','
                  << result.prompt_tokens << ','
```

```
                  << result.reserved_tokens << ','
                  << result.kv_reserved_mib << ','
                  << result.queue_wait_ms << ','
                  << result.prefill_ms << ','
                  << result.ttft_ms << ','
                  << result.decode_step_p95_ms << ','
                  << result.tpot_ms << ','
                  << result.completion_tokens << ','
                  << result.finish_reason << ','
                  << result.cancel_reclaim_ms << ",,\n";
    }
}

void print_summary_row(const std::vector<ProbeResult>& results) {
    std::vector<double> ttft_values;
    std::vector<double> queue_values;
    double accepted = 0.0;
    double rejected = 0.0;
    double cancelled = 0.0;
    double reserved_peak_mib = 0.0;
    double running_reserved_mib = 0.0;
    for (const ProbeResult& result : results) {
        if (result.accepted == 0) {
            rejected += 1.0;
            continue;
        }
        accepted += 1.0;
        ttft_values.push_back(result.ttft_ms);
        queue_values.push_back(result.queue_wait_ms);
        running_reserved_mib += result.kv_reserved_mib;
        reserved_peak_mib = std::max(reserved_peak_mib, running_reserved_mib);
        if (result.finish_reason == "cancelled") {
            cancelled += 1.0;
            running_reserved_mib -= result.kv_reserved_mib;
        }
    }
    const double total = accepted + rejected;
    const double rejection_rate = total == 0.0 ? 0.0 : rejected / total;
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "accepted_count," << accepted << '\n';
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,"
              << LCQI_PROBE_CANCEL_RECLAIM_MS << ",cancel_count," << cancelled ↩
    ↪ << '\n';
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "rejected_count," << rejected << '\n';
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "rejection_rate," << rejection_rate << '\n';
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "kv_reserved_peak_mib," << reserved_peak_mib << '\n';
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "queue_wait_p95_ms," << percentile(queue_values, 0.95) << '\n'↩
    ↪ ;
```

```
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "ttft_p95_ms," << percentile(ttft_values, 0.95) << '\n';
}

}  // namespace

int main() {
    try {
        std::cout << std::fixed << std::setprecision(3);
        const std::vector<ProbeResult> results = run_probe();
        print_request_rows(results);
        print_summary_row(results);
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << '\n';
        return EXIT_FAILURE;
    }
}
```

onednn_bench.cpp 是可选外部库对标入口。它只在环境安装 oneDNN 时构建，不能把不可用平台写成已验证性能。

Listing 6.5 src/onednn_bench.cpp

```
#include <dnnl.hpp>

#include <chrono>
#include <cstdint>
#include <cstdlib>
#include <iostream>
#include <numeric>
#include <stdexcept>
#include <string>
#include <unordered_map>
#include <vector>

namespace {

constexpr std::int64_t LCQI_ONEDNN_BATCH = 1;
constexpr std::int64_t LCQI_ONEDNN_K = 4096;
constexpr std::int64_t LCQI_ONEDNN_O = 4096;
constexpr std::int32_t LCQI_ONEDNN_DEFAULT_REPEAT = 20;
constexpr std::int32_t LCQI_ONEDNN_WARMUP = 3;
constexpr int LCQI_REPEAT_ARG_INDEX = 1;
constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
constexpr float LCQI_INPUT_SCALE = 0.125F;
constexpr std::int32_t LCQI_INPUT_MODULUS = 17;
constexpr std::int32_t LCQI_INPUT_CENTER = 8;
constexpr std::int32_t LCQI_WEIGHT_MODULUS = 23;
constexpr std::int32_t LCQI_WEIGHT_CENTER = 11;
constexpr float LCQI_WEIGHT_SCALE = 0.03125F;

std::int32_t parse_repeat(int argc, char** argv) {
```

```
    if (argc <= LCQI_REPEAT_ARG_INDEX) {
        return LCQI_ONEDNN_DEFAULT_REPEAT;
    }
    const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[LCQI_REPEAT_ARG_INDEX]));
    if (repeat <= 0) {
        throw std::runtime_error("repeat must be positive");
    }
    return repeat;
}

std::vector<float> make_input() {
    std::vector<float> input(static_cast<std::size_t>(LCQI_ONEDNN_K));
    for (std::int64_t index = 0; index < LCQI_ONEDNN_K; ++index) {
        input[static_cast<std::size_t>(index)] =
            static_cast<float>((index % LCQI_INPUT_MODULUS) - LCQI_INPUT_CENTER) *
            LCQI_INPUT_SCALE;
    }
    return input;
}

std::vector<float> make_weights_kn() {
    std::vector<float> weights(static_cast<std::size_t>(LCQI_ONEDNN_K * LCQI_ONEDNN_O));
    for (std::int64_t k = 0; k < LCQI_ONEDNN_K; ++k) {
        for (std::int64_t out = 0; out < LCQI_ONEDNN_O; ++out) {
            const std::int64_t row_major_out_k = out * LCQI_ONEDNN_K + k;
            const float value =
                static_cast<float>((row_major_out_k % LCQI_WEIGHT_MODULUS) -
                                   LCQI_WEIGHT_CENTER) *
                LCQI_WEIGHT_SCALE;
            weights[static_cast<std::size_t>(k * LCQI_ONEDNN_O + out)] = value;
        }
    }
    return weights;
}

float checksum(const std::vector<float>& values) {
    return std::accumulate(values.begin(), values.end(), 0.0F);
}

}  // namespace

int main(int argc, char** argv) {
    try {
        const std::int32_t repeat = parse_repeat(argc, argv);
        std::vector<float> input = make_input();
        std::vector<float> weights = make_weights_kn();
        std::vector<float> output(static_cast<std::size_t>(LCQI_ONEDNN_BATCH * LCQI_ONEDNN_O),
                                  0.0F);
```

```
78
79          dnnl::engine engine(dnnl::engine::kind::cpu, 0);
80          dnnl::stream stream(engine);
81
82          const dnnl::memory::dims source_dims = {LCQI_ONEDNN_BATCH, ↩
    ↪ LCQI_ONEDNN_K};
83          const dnnl::memory::dims weight_dims = {LCQI_ONEDNN_K, LCQI_ONEDNN_O};
84          const dnnl::memory::dims output_dims = {LCQI_ONEDNN_BATCH, ↩
    ↪ LCQI_ONEDNN_O};
85
86          const dnnl::memory::desc source_desc(
87              source_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag↩
    ↪ ::ab);
88          const dnnl::memory::desc weight_desc(
89              weight_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag↩
    ↪ ::ab);
90          const dnnl::memory::desc output_desc(
91              output_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag↩
    ↪ ::ab);
92
93          dnnl::memory source_memory(source_desc, engine, input.data());
94          dnnl::memory weight_memory(weight_desc, engine, weights.data());
95          dnnl::memory output_memory(output_desc, engine, output.data());
96
97          dnnl::matmul::primitive_desc matmul_desc(engine, source_desc, ↩
    ↪ weight_desc, output_desc);
98          dnnl::matmul matmul(matmul_desc);
99          const std::unordered_map<int, dnnl::memory> arguments = {
100             {DNNL_ARG_SRC, source_memory},
101             {DNNL_ARG_WEIGHTS, weight_memory},
102             {DNNL_ARG_DST, output_memory},
103         };
104
105         for (std::int32_t index = 0; index < LCQI_ONEDNN_WARMUP; ++index) {
106             matmul.execute(stream, arguments);
107             stream.wait();
108         }
109
110         const auto begin = std::chrono::steady_clock::now();
111         for (std::int32_t index = 0; index < repeat; ++index) {
112             matmul.execute(stream, arguments);
113             stream.wait();
114         }
115         const auto end = std::chrono::steady_clock::now();
116         const std::chrono::duration<double> elapsed = end - begin;
117         const double average_us =
118             elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double↩
    ↪ >(repeat);
119
120         std::cout << "library,input_size,output_size,dtype,repeat,average_us,↩
    ↪ checksum\n";
121         std::cout << "onednn," << LCQI_ONEDNN_K << ',' << LCQI_ONEDNN_O
```

```
                  << ",f32," << repeat << ',' << average_us << ','
                  << checksum(output) << '\n';
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << '\n';
        return EXIT_FAILURE;
    }
}
```

6.3 外部模型 Smoke 与对比脚本

run_smollm2_small_smoke.py 下载并校验 SmolLM2-135M-Instruct 的 GGUF 量化文件，构建固定 commit 的 llama-simple-chat，在 CPU 上运行一次真实开源 small 模型生成，并输出可复查报告。它是外部真实模型 smoke，不替代 LCQI tiny golden trace。

Listing 6.6 tools/run_smollm2_small_smoke.py

```python
#!/usr/bin/env python3
from __future__ import annotations

import argparse
import datetime as dt
import hashlib
import os
import re
import shlex
import shutil
import subprocess
import sys
from dataclasses import dataclass
from pathlib import Path


MODEL_SOURCE_ID = "HuggingFaceTB/SmolLM2-135M-Instruct"
MODEL_GGUF_REPO_ID = "bartowski/SmolLM2-135M-Instruct-GGUF"
MODEL_FILE_NAME = "SmolLM2-135M-Instruct-Q4_K_M.gguf"
MODEL_URL = (
    "https://huggingface.co/bartowski/SmolLM2-135M-Instruct-GGUF"
    f"/resolve/main/{MODEL_FILE_NAME}"
)
MODEL_EXPECTED_BYTES = 105454432
MODEL_EXPECTED_SHA256 = (
    "2e8040ceae7815abe0dcb3540b9995eaa1fa0d2ca9e797d0a635ae4433c68c2d"
)
LLAMA_CPP_REPO_URL = "https://github.com/ggml-org/llama.cpp.git"
LLAMA_CPP_COMMIT = "b3fed31b99f9bd37725833674252bccb429bb183"
LLAMA_TARGET = "llama-simple-chat"
DEFAULT_PROMPT = "What is 2+3? Answer in one short sentence."
DEFAULT_CONTEXT_TOKENS = 512
DEFAULT_TIMEOUT_SECONDS = 180
DEFAULT_BUILD_JOBS = 2
```

```
35
36
37 @dataclass(frozen=True)
38 class CommandResult:
39     args: list[str]
40     returncode: int
41     stdout: str
42     stderr: str
43
44
45 def script_path() -> Path:
46     return Path(__file__).resolve()
47
48
49 def volume_dir() -> Path:
50     return script_path().parents[1]
51
52
53 def repo_dir() -> Path:
54     return script_path().parents[3]
55
56
57 def default_cache_dir() -> Path:
58     explicit_cache = os.environ.get("LCQI_SMOLLM2_CACHE_DIR")
59     if explicit_cache:
60         return Path(explicit_cache).expanduser()
61     xdg_cache = os.environ.get("XDG_CACHE_HOME")
62     cache_root = Path(xdg_cache).expanduser() if xdg_cache else Path.home() / "↩
    ↪ .cache"
63     return cache_root / "lcqi-smollm2-small-smoke"
64
65
66 def default_report_path() -> Path:
67     return volume_dir() / "results" / "lcqi-smollm2-small-model-smoke.txt"
68
69
70 def command_line(args: list[str]) -> str:
71     return " ".join(shlex.quote(arg) for arg in args)
72
73
74 def require_tool(name: str) -> str:
75     path = shutil.which(name)
76     if path is None:
77         raise RuntimeError(f"required tool not found in PATH: {name}")
78     return path
79
80
81 def run_command(
82     args: list[str],
83     *,
84     cwd: Path | None = None,
85     input_text: str | None = None,
```

```
    timeout_seconds: int | None = None,
) -> CommandResult:
    try:
        completed = subprocess.run(
            args,
            cwd=str(cwd) if cwd else None,
            input=input_text,
            text=True,
            capture_output=True,
            timeout=timeout_seconds,
            check=False,
        )
    except subprocess.TimeoutExpired as exc:
        stdout = exc.stdout if isinstance(exc.stdout, str) else ""
        stderr = exc.stderr if isinstance(exc.stderr, str) else ""
        return CommandResult(args=args, returncode=124, stdout=stdout, stderr=stderr)

    return CommandResult(
        args=args,
        returncode=completed.returncode,
        stdout=completed.stdout,
        stderr=completed.stderr,
    )


def ensure_success(result: CommandResult, action: str) -> None:
    if result.returncode == 0:
        return
    raise RuntimeError(
        f"{action} failed with exit code {result.returncode}\n"
        f"command: {command_line(result.args)}\n"
        f"stdout tail:\n{tail(result.stdout)}\n"
        f"stderr tail:\n{tail(result.stderr)}"
    )


def tail(text: str, max_chars: int = 4000) -> str:
    if len(text) <= max_chars:
        return text
    return text[-max_chars:]


def sha256_file(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for block in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(block)
    return digest.hexdigest()


def verify_model_file(path: Path) -> tuple[bool, str, int]:
```

```
    if not path.exists():
        return False, "", 0
    actual_bytes = path.stat().st_size
    actual_sha = sha256_file(path)
    return (
        actual_bytes == MODEL_EXPECTED_BYTES
        and actual_sha == MODEL_EXPECTED_SHA256,
        actual_sha,
        actual_bytes,
    )


def download_model(model_path: Path, *, force_download: bool, no_download: bool) -> None:
    model_path.parent.mkdir(parents=True, exist_ok=True)

    if force_download and model_path.exists():
        model_path.unlink()

    is_valid, actual_sha, actual_bytes = verify_model_file(model_path)
    if is_valid:
        print(f"[lcqi-smoke] model cache hit: {model_path}")
        return

    if model_path.exists():
        print(
            "[lcqi-smoke] cached model hash mismatch, redownloading: "
            f"bytes={actual_bytes}, sha256={actual_sha}"
        )
        model_path.unlink()

    if no_download:
        raise RuntimeError(
            f"model file is missing or invalid and --no-download was set: {model_path}"
        )

    require_tool("curl")
    part_path = model_path.with_suffix(model_path.suffix + ".part")
    if part_path.exists():
        part_path.unlink()

    print(f"[lcqi-smoke] downloading small GGUF model: {MODEL_GGUF_REPO_ID}/{MODEL_FILE_NAME}")
    result = run_command(
        [
            "curl",
            "-L",
            "--fail",
            "--retry",
            "3",
            "-o",
```

```
            str(part_path),
            MODEL_URL,
        ]
    )
    ensure_success(result, "download SmolLM2 GGUF")

    downloaded_ok, downloaded_sha, downloaded_bytes = verify_model_file(part_path)
    if not downloaded_ok:
        raise RuntimeError(
            "downloaded model did not match expected identity: "
            f"bytes={downloaded_bytes}, sha256={downloaded_sha}"
        )
    part_path.replace(model_path)
    print(f"[lcqi-smoke] model verified: sha256={MODEL_EXPECTED_SHA256}")


def ensure_llama_simple_chat(cache_dir: Path, *, jobs: int, skip_build: bool) -> Path:
    cached_binary = cache_dir / "llama.cpp" / "build" / "bin" / LLAMA_TARGET
    source_dir = cache_dir / "llama.cpp"
    build_dir = source_dir / "build"
    if cached_binary.exists() and os.access(cached_binary, os.X_OK) and is_pinned_llama_checkout(source_dir):
        print(f"[lcqi-smoke] llama.cpp binary cache hit: {cached_binary}")
        return cached_binary

    if skip_build:
        raise RuntimeError(f"llama.cpp binary is missing and --skip-build was set: {cached_binary}")

    require_tool("git")
    require_tool("cmake")

    cache_dir.mkdir(parents=True, exist_ok=True)

    if not (source_dir / ".git").exists():
        print(f"[lcqi-smoke] cloning llama.cpp into cache: {source_dir}")
        result = run_command(
            [
                "git",
                "clone",
                "--filter=blob:none",
                "--no-checkout",
                LLAMA_CPP_REPO_URL,
                str(source_dir),
            ]
        )
        ensure_success(result, "clone llama.cpp")

    print(f"[lcqi-smoke] checking out pinned llama.cpp commit: {LLAMA_CPP_COMMIT}")
```

```
    fetch = run_command(
        ["git", "-C", str(source_dir), "fetch", "--depth", "1", "origin", LLAMA_CPP_COMMIT]
    )
    ensure_success(fetch, "fetch pinned llama.cpp commit")
    checkout = run_command(["git", "-C", str(source_dir), "checkout", "--detach", LLAMA_CPP_COMMIT])
    ensure_success(checkout, "checkout pinned llama.cpp commit")

    print(f"[lcqi-smoke] configuring llama.cpp build: {build_dir}")
    configure = run_command(
        [
            "cmake",
            "-S",
            str(source_dir),
            "-B",
            str(build_dir),
            "-DCMAKE_BUILD_TYPE=Release",
            "-DLLAMA_BUILD_TESTS=OFF",
            "-DLLAMA_BUILD_EXAMPLES=ON",
            "-DLLAMA_BUILD_SERVER=OFF",
            "-DGGML_NATIVE=OFF",
        ]
    )
    ensure_success(configure, "configure llama.cpp")

    print(f"[lcqi-smoke] building {LLAMA_TARGET} with -j{jobs}")
    build = run_command(
        ["cmake", "--build", str(build_dir), "--target", LLAMA_TARGET, "-j", str(jobs)]
    )
    ensure_success(build, f"build {LLAMA_TARGET}")

    if not cached_binary.exists():
        raise RuntimeError(f"expected binary was not produced: {cached_binary}")
    return cached_binary


def is_pinned_llama_checkout(source_dir: Path) -> bool:
    if not (source_dir / ".git").exists():
        return False
    result = run_command(["git", "-C", str(source_dir), "rev-parse", "HEAD"])
    if result.returncode != 0:
        return False
    return result.stdout.strip() == LLAMA_CPP_COMMIT


def strip_ansi(text: str) -> str:
    return re.sub(r"\x1b\[[0-?]*[ -/]*[@-~]", "", text)


```

```python
def extract_response(stdout: str) -> str:
    cleaned = strip_ansi(stdout).replace("\r", "")
    chunks = [chunk.strip() for chunk in re.split(r"(?:^|\n)> ?", cleaned) if chunk.strip()]
    if not chunks:
        return ""
    return chunks[0]


def run_model_smoke(
    llama_binary: Path,
    model_path: Path,
    *,
    prompt: str,
    context_tokens: int,
    timeout_seconds: int,
) -> tuple[CommandResult, str]:
    command = [
        str(llama_binary),
        "-m",
        str(model_path),
        "-ngl",
        "0",
        "-c",
        str(context_tokens),
    ]
    print("[lcqi-smoke] running real small model generation")
    result = run_command(
        command,
        input_text=prompt.rstrip("\n") + "\n\n",
        timeout_seconds=timeout_seconds,
    )
    ensure_success(result, "run small model smoke")

    response = extract_response(result.stdout)
    if not response:
        raise RuntimeError(
            "small model smoke exited successfully but produced no assistant text\n"
            f"stdout:\n{result.stdout}\n"
            f"stderr:\n{result.stderr}"
        )
    return result, response


def current_repo_commit() -> str:
    result = run_command(["git", "-C", str(repo_dir()), "rev-parse", "--short", "HEAD"])
    if result.returncode != 0:
        return "unknown"
    return result.stdout.strip()

```

```
330
331 def current_uname() -> str:
332     result = run_command(["uname", "-a"])
333     if result.returncode != 0:
334         return "unknown"
335     return result.stdout.strip()
336
337
338 def write_report(
339     report_path: Path,
340     *,
341     cache_dir: Path,
342     llama_binary: Path,
343     model_path: Path,
344     prompt: str,
345     result: CommandResult,
346     response: str,
347 ) -> None:
348     report_path.parent.mkdir(parents=True, exist_ok=True)
349     clean_stdout = strip_ansi(result.stdout).strip()
350     report = "\n".join(
351         [
352             "LCQI SmolLM2 Small Model Smoke",
353             "",
354             f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
355             f"repo_commit={current_repo_commit()}",
356             f"host={current_uname()}",
357             f"python={sys.version.split()[0]}",
358             f"cache_dir={cache_dir}",
359             "",
360             f"model_source_id={MODEL_SOURCE_ID}",
361             f"model_gguf_repo_id={MODEL_GGUF_REPO_ID}",
362             f"model_file={MODEL_FILE_NAME}",
363             f"model_url={MODEL_URL}",
364             f"model_expected_bytes={MODEL_EXPECTED_BYTES}",
365             f"model_expected_sha256={MODEL_EXPECTED_SHA256}",
366             f"model_path={model_path}",
367             "",
368             f"llama_cpp_repo={LLAMA_CPP_REPO_URL}",
369             f"llama_cpp_commit={LLAMA_CPP_COMMIT}",
370             f"llama_target={LLAMA_TARGET}",
371             f"llama_binary={llama_binary}",
372             "",
373             f"prompt={prompt}",
374             f"command={command_line(result.args)}",
375             f"exit_code={result.returncode}",
376             "",
377             "validation=PASS: model hash matched, llama-simple-chat exited 0, "
378             "assistant response was non-empty",
379             "",
380             "assistant_response:",
381             response,
```

```
            "",
            "stdout_clean_tail:",
            tail(clean_stdout, 2000),
            "",
            "stderr_tail:",
            tail(result.stderr.strip(), 2000),
            "",
        ]
    )
    report_path.write_text(report, encoding="utf-8")


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description=(
            "Run a real local small open-source model smoke for CPU Volume 3. "
            "The script caches llama.cpp and the GGUF model outside the Git repository."
        )
    )
    parser.add_argument("--cache-dir", type=Path, default=default_cache_dir())
    parser.add_argument("--model-path", type=Path, default=None)
    parser.add_argument("--llama-bin", type=Path, default=None)
    parser.add_argument("--report", type=Path, default=default_report_path())
    parser.add_argument("--prompt", default=DEFAULT_PROMPT)
    parser.add_argument("--ctx", type=int, default=DEFAULT_CONTEXT_TOKENS)
    parser.add_argument("--timeout", type=int, default=DEFAULT_TIMEOUT_SECONDS)
    parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
    parser.add_argument("--force-download", action="store_true")
    parser.add_argument("--no-download", action="store_true")
    parser.add_argument("--skip-build", action="store_true")
    return parser.parse_args()


def main() -> int:
    args = parse_args()
    cache_dir = args.cache_dir.expanduser().resolve()
    model_path = (
        args.model_path.expanduser().resolve()
        if args.model_path
        else cache_dir / "models" / MODEL_FILE_NAME
    )
    report_path = args.report.expanduser().resolve()

    try:
        download_model(
            model_path,
            force_download=args.force_download,
            no_download=args.no_download,
        )
        if args.llama_bin:
            llama_binary = args.llama_bin.expanduser().resolve()
```

```
            if not llama_binary.exists() or not os.access(llama_binary, os.X_OK↩
    ↪ ):
                raise RuntimeError(f"--llama-bin is not executable: {↩
    ↪ llama_binary}")
        else:
            llama_binary = ensure_llama_simple_chat(
                cache_dir,
                jobs=args.jobs,
                skip_build=args.skip_build,
            )

        result, response = run_model_smoke(
            llama_binary,
            model_path,
            prompt=args.prompt,
            context_tokens=args.ctx,
            timeout_seconds=args.timeout,
        )
        write_report(
            report_path,
            cache_dir=cache_dir,
            llama_binary=llama_binary,
            model_path=model_path,
            prompt=args.prompt,
            result=result,
            response=response,
        )
    except RuntimeError as exc:
        print(f"[lcqi-smoke] ERROR: {exc}", file=sys.stderr)
        return 1

    print(f"report={report_path}")
    print(f"model_sha256={MODEL_EXPECTED_SHA256}")
    print(f"assistant_response={response}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

run_gpt2_smoke.py 下载标准 GPT-2 的 config.json、vocab.json、merges.txt 和 model.safetensors，再调用 LCQI 自研 lcqi_gpt2 reference path 生成 token。它证明自研 safetensors、BPE、GPT-2 forward、KV cache decode 和 greedy generation 已连成闭环。

Listing 6.7 tools/run_gpt2_smoke.py

```
#!/usr/bin/env python3
from __future__ import annotations

import argparse
import datetime as dt
import hashlib
```

```python
import os
import shlex
import subprocess
import sys
import urllib.request
from dataclasses import dataclass
from pathlib import Path


MODEL_REPO_ID = "openai-community/gpt2"
MODEL_REVISION = "main"
MODEL_FILE_IDENTITIES = {
    "config.json": (
        665,
        "0daed7749b4f02b8f76240d5444551d7b08712dab4d0adb8239c56ba823bb7b4",
    ),
    "vocab.json": (
        1042301,
        "196139668be63f3b5d6574427317ae82f612a97c5d1cdaf36ed2256dbf636783",
    ),
    "merges.txt": (
        456318,
        "1ce1664773c50f3e0cc8842619a93edc4624525b728b188a9e0be33b7726adc5",
    ),
    "model.safetensors": (
        548105171,
        "248dfc3911869ec493c76e65bf2fcf7f615828b0254c12b473182f0f81d3a707",
    ),
}
DEFAULT_PROMPT = "Hello, my name is"
DEFAULT_MAX_NEW_TOKENS = 1
DEFAULT_TIMEOUT_SECONDS = 300
DEFAULT_BUILD_JOBS = 2


@dataclass(frozen=True)
class CommandResult:
    args: list[str]
    returncode: int
    stdout: str
    stderr: str


def script_path() -> Path:
    return Path(__file__).resolve()


def volume_dir() -> Path:
    return script_path().parents[1]


def repo_dir() -> Path:
```

```
    return script_path().parents[3]


def default_cache_dir() -> Path:
    explicit_cache = os.environ.get("LCQI_GPT2_CACHE_DIR")
    if explicit_cache:
        return Path(explicit_cache).expanduser()
    xdg_cache = os.environ.get("XDG_CACHE_HOME")
    cache_root = Path(xdg_cache).expanduser() if xdg_cache else Path.home() / ".cache"
    return cache_root / "lcqi-gpt2-smoke" / MODEL_REPO_ID.replace("/", "--")


def default_build_dir() -> Path:
    return volume_dir() / "build" / "lcqi-release"


def default_report_path() -> Path:
    return volume_dir() / "results" / "lcqi-gpt2-smoke.txt"


def command_line(args: list[str]) -> str:
    return " ".join(shlex.quote(arg) for arg in args)


def run_command(
    args: list[str],
    *,
    cwd: Path | None = None,
    timeout_seconds: int | None = None,
) -> CommandResult:
    try:
        completed = subprocess.run(
            args,
            cwd=str(cwd) if cwd else None,
            text=True,
            capture_output=True,
            timeout=timeout_seconds,
            check=False,
        )
    except subprocess.TimeoutExpired as exc:
        stdout = exc.stdout if isinstance(exc.stdout, str) else ""
        stderr = exc.stderr if isinstance(exc.stderr, str) else ""
        return CommandResult(args=args, returncode=124, stdout=stdout, stderr=stderr)
    return CommandResult(
        args=args,
        returncode=completed.returncode,
        stdout=completed.stdout,
        stderr=completed.stderr,
    )

```

```python

def ensure_success(result: CommandResult, action: str) -> None:
    if result.returncode == 0:
        return
    raise RuntimeError(
        f"{action} failed with exit code {result.returncode}\n"
        f"command: {command_line(result.args)}\n"
        f"stdout tail:\n{tail(result.stdout)}\n"
        f"stderr tail:\n{tail(result.stderr)}"
    )


def tail(text: str, max_chars: int = 4000) -> str:
    if len(text) <= max_chars:
        return text
    return text[-max_chars:]


def sha256_file(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for block in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(block)
    return digest.hexdigest()


def model_url(file_name: str) -> str:
    return (
        f"https://huggingface.co/{MODEL_REPO_ID}/resolve/"
        f"{MODEL_REVISION}/{file_name}"
    )


def download_file(url: str, target: Path, timeout_seconds: int) -> None:
    part_path = target.with_suffix(target.suffix + ".part")
    if part_path.exists():
        part_path.unlink()
    request = urllib.request.Request(url, headers={"User-Agent": "lcqi-gpt2-↩
    ↪ smoke"})
    with urllib.request.urlopen(request, timeout=timeout_seconds) as response:
        with part_path.open("wb") as output:
            while True:
                block = response.read(1024 * 1024)
                if not block:
                    break
                output.write(block)
    part_path.replace(target)


def file_identity_matches(path: Path, expected_bytes: int, expected_sha256: str↩
    ↪ ) -> bool:
    return (
```

```
        path.exists()
        and path.stat().st_size == expected_bytes
        and sha256_file(path) == expected_sha256
    )


def ensure_model_files(cache_dir: Path, *, no_download: bool, timeout_seconds: int) -> None:
    cache_dir.mkdir(parents=True, exist_ok=True)
    missing = [
        name
        for name, (expected_bytes, expected_sha256) in MODEL_FILE_IDENTITIES.items()
        if not file_identity_matches(cache_dir / name, expected_bytes, expected_sha256)
    ]
    if not missing:
        print(f"[lcqi-gpt2] model cache hit: {cache_dir}")
        return
    if no_download:
        raise RuntimeError(
            f"missing GPT-2 model files and --no-download was set: {missing}"
        )
    for file_name in missing:
        target = cache_dir / file_name
        if target.exists():
            target.unlink()
        print(f"[lcqi-gpt2] downloading {MODEL_REPO_ID}/{file_name}")
        download_file(model_url(file_name), target, timeout_seconds)
        expected_bytes, expected_sha256 = MODEL_FILE_IDENTITIES[file_name]
        if not file_identity_matches(target, expected_bytes, expected_sha256):
            raise RuntimeError(
                f"downloaded GPT-2 file identity mismatch: {file_name}"
            )


def build_lcqi_gpt2(build_dir: Path, jobs: int) -> Path:
    configure = run_command(
        [
            "cmake",
            "-S",
            str(volume_dir()),
            "-B",
            str(build_dir),
            "-DCMAKE_BUILD_TYPE=Release",
        ],
        timeout_seconds=DEFAULT_TIMEOUT_SECONDS,
    )
    ensure_success(configure, "configure LCQI")
    build = run_command(
        [
            "cmake",
```

```
            "--build",
            str(build_dir),
            "--target",
            "lcqi_gpt2",
            "-j",
            str(jobs),
        ],
        timeout_seconds=DEFAULT_TIMEOUT_SECONDS,
    )
    ensure_success(build, "build lcqi_gpt2")
    binary = build_dir / "labs" / "linux_cpu_inference" / "lcqi_gpt2"
    if not binary.exists():
        raise RuntimeError(f"lcqi_gpt2 binary not found: {binary}")
    return binary


def write_report(
    report_path: Path,
    *,
    cache_dir: Path,
    binary: Path,
    prompt: str,
    max_new_tokens: int,
    engine: str,
    result: CommandResult,
) -> None:
    report_path.parent.mkdir(parents=True, exist_ok=True)
    lines = [
        f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
        f"model_repo_id={MODEL_REPO_ID}",
        f"model_revision={MODEL_REVISION}",
        f"model_cache_dir={cache_dir}",
    ]
    for file_name, (expected_bytes, expected_sha256) in MODEL_FILE_IDENTITIES.items():
        path = cache_dir / file_name
        lines.append(f"file.{file_name}.bytes={path.stat().st_size}")
        lines.append(f"file.{file_name}.sha256={sha256_file(path)}")
        lines.append(f"file.{file_name}.expected_bytes={expected_bytes}")
        lines.append(f"file.{file_name}.expected_sha256={expected_sha256}")
    lines.extend(
        [
            f"binary={binary}",
            f"prompt={prompt}",
            f"max_new_tokens={max_new_tokens}",
            f"engine={engine}",
            f"command={command_line(result.args)}",
            f"exit_code={result.returncode}",
            "stdout_begin",
            result.stdout.rstrip(),
            "stdout_end",
            "stderr_begin",
```

```
            tail(result.stderr).rstrip(),
            "stderr_end",
        ]
    )
    passed = result.returncode == 0 and "generated_text " in result.stdout
    lines.append(f"validation={'PASS' if passed else 'FAIL'}")
    report_path.write_text("\n".join(lines) + "\n", encoding="utf-8")


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Run LCQI self-owned GPT-2 smoke on a HuggingFace safetensors directory."
    )
    parser.add_argument("--cache-dir", type=Path, default=default_cache_dir())
    parser.add_argument("--build-dir", type=Path, default=default_build_dir())
    parser.add_argument("--report", type=Path, default=default_report_path())
    parser.add_argument("--prompt", default=DEFAULT_PROMPT)
    parser.add_argument("--max-new-tokens", type=int, default=DEFAULT_MAX_NEW_TOKENS)
    parser.add_argument("--engine", choices=("cached", "full"), default="cached")
    parser.add_argument("--benchmark", action="store_true")
    parser.add_argument("--timeout-seconds", type=int, default=DEFAULT_TIMEOUT_SECONDS)
    parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
    parser.add_argument("--no-download", action="store_true")
    return parser.parse_args()


def main() -> int:
    args = parse_args()
    try:
        if args.max_new_tokens < 0:
            raise RuntimeError("--max-new-tokens cannot be negative")
        ensure_model_files(
            args.cache_dir,
            no_download=args.no_download,
            timeout_seconds=args.timeout_seconds,
        )
        binary = build_lcqi_gpt2(args.build_dir, args.jobs)
        command = [
            str(binary),
            "--engine",
            args.engine,
            str(args.cache_dir),
            args.prompt,
            str(args.max_new_tokens),
        ]
        if args.benchmark:
            command.insert(1, "--benchmark")
        result = run_command(command, cwd=repo_dir(), timeout_seconds=args.
```

```
    ↪ timeout_seconds)
307         write_report(
308             args.report,
309             cache_dir=args.cache_dir,
310             binary=binary,
311             prompt=args.prompt,
312             max_new_tokens=args.max_new_tokens,
313             engine=args.engine,
314             result=result,
315         )
316         print(f"[lcqi-gpt2] report: {args.report}")
317         print(tail(result.stdout, max_chars=2000))
318         return 0 if result.returncode == 0 else result.returncode
319     except Exception as error:
320         print(f"[lcqi-gpt2] error: {error}", file=sys.stderr)
321         return 1
322
323
324 if __name__ == "__main__":
325     raise SystemExit(main())
```

run_gpt2_benchmark_compare.py 用同一个 GPT-2 F32 safetensors 模型分别运行 LCQI full-prefix 与 cached-KV 路径，并在本机存在 llama.cpp/SmolLM2 缓存时附带成熟引擎参考。它的报告说明算法差距、kernel 差距和模型口径差异。

Listing 6.8 tools/run_gpt2_benchmark_compare.py

```
1  #!/usr/bin/env python3
2  from __future__ import annotations
3
4  import argparse
5  import datetime as dt
6  import os
7  import re
8  import shlex
9  import subprocess
10 import sys
11 from dataclasses import dataclass
12 from pathlib import Path
13
14 import run_gpt2_smoke
15
16
17 DEFAULT_PROMPT = "Hello, my name is"
18 DEFAULT_MAX_NEW_TOKENS = 4
19 DEFAULT_BUILD_JOBS = 2
20 DEFAULT_TIMEOUT_SECONDS = 300
21 DEFAULT_SMOLLM2_CACHE_DIR = Path.home() / ".cache" / "lcqi-smollm2-small-smoke"
22 SMOLLM2_GGUF = "SmolLM2-135M-Instruct-Q4_K_M.gguf"
23
24
25 @dataclass(frozen=True)
26 class EngineRun:
```

```
    name: str
    command: list[str]
    returncode: int
    stdout: str
    stderr: str
    metrics: dict[str, str]


def volume_dir() -> Path:
    return Path(__file__).resolve().parents[1]


def repo_dir() -> Path:
    return Path(__file__).resolve().parents[3]


def default_report_path() -> Path:
    return volume_dir() / "results" / "lcqi-gpt2-benchmark-compare.txt"


def default_build_dir() -> Path:
    return volume_dir() / "build" / "lcqi-release"


def command_line(args: list[str]) -> str:
    return " ".join(shlex.quote(arg) for arg in args)


def tail(text: str, max_chars: int = 4000) -> str:
    return text if len(text) <= max_chars else text[-max_chars:]


def run_command(args: list[str], *, timeout_seconds: int) -> tuple[int, str, ↩
    ↪ str]:
    try:
        completed = subprocess.run(
            args,
            cwd=repo_dir(),
            text=True,
            capture_output=True,
            timeout=timeout_seconds,
            check=False,
        )
    except subprocess.TimeoutExpired as exc:
        stdout = exc.stdout if isinstance(exc.stdout, str) else ""
        stderr = exc.stderr if isinstance(exc.stderr, str) else ""
        return 124, stdout, stderr
    return completed.returncode, completed.stdout, completed.stderr


def parse_lcqi_metrics(stdout: str) -> dict[str, str]:
    metrics: dict[str, str] = {}
```

```
    for line in stdout.splitlines():
        if not line:
            continue
        key, _, value = line.partition(" ")
        if key in {
            "engine",
            "generated_text",
            "benchmark_load_ms",
            "benchmark_tokenize_ms",
            "benchmark_prefill_ms",
            "benchmark_decode_ms",
            "benchmark_generate_ms",
            "benchmark_total_ms",
            "benchmark_prompt_tokens",
            "benchmark_generated_tokens",
            "benchmark_prefill_steps",
            "benchmark_decode_steps",
            "benchmark_generate_tokens_per_second",
            "benchmark_decode_tokens_per_second",
            "benchmark_kv_cache_bytes",
        }:
            metrics[key] = value
    return metrics


def run_lcqi(
    binary: Path,
    model_dir: Path,
    *,
    engine: str,
    prompt: str,
    max_new_tokens: int,
    timeout_seconds: int,
) -> EngineRun:
    command = [
        str(binary),
        "--benchmark",
        "--engine",
        engine,
        str(model_dir),
        prompt,
        str(max_new_tokens),
    ]
    returncode, stdout, stderr = run_command(command, timeout_seconds=↩
    ↪ timeout_seconds)
    return EngineRun(
        name=f"lcqi_{engine}",
        command=command,
        returncode=returncode,
        stdout=stdout,
        stderr=stderr,
        metrics=parse_lcqi_metrics(stdout),
```

```
    )


def llama_bench_binary(cache_dir: Path) -> Path:
    return cache_dir / "llama.cpp" / "build" / "bin" / "llama-bench"


def smollm2_model_path(cache_dir: Path) -> Path:
    return cache_dir / "models" / SMOLLM2_GGUF


def parse_llama_bench_metrics(stdout: str) -> dict[str, str]:
    metrics: dict[str, str] = {}
    lines = [line.strip() for line in stdout.splitlines() if line.strip()]
    for line in lines:
        if not line.startswith("|"):
            continue
        columns = [column.strip() for column in line.strip("|").split("|")]
        if len(columns) != 7 or columns[0] == "model" or columns[0].startswith(
    "-"):
            continue
        metrics["llama_model"] = columns[0]
        metrics["llama_size"] = columns[1]
        metrics["llama_params"] = columns[2]
        metrics["llama_backend"] = columns[3]
        metrics["llama_threads"] = columns[4]
        test_name = columns[5]
        tokens_per_second = re.sub(r"\s+.*$", "", columns[6])
        if test_name.startswith("pp"):
            metrics["llama_prefill_test"] = test_name
            metrics["llama_prefill_tps"] = tokens_per_second
        elif test_name.startswith("tg"):
            metrics["llama_decode_test"] = test_name
            metrics["llama_decode_tps"] = tokens_per_second
    return metrics


def run_llama_bench(cache_dir: Path, *, timeout_seconds: int) -> EngineRun | 
    None:
    binary = llama_bench_binary(cache_dir)
    model = smollm2_model_path(cache_dir)
    if not binary.exists() or not os.access(binary, os.X_OK) or not model.
    exists():
        return None
    command = [
        str(binary),
        "-m",
        str(model),
        "-ngl",
        "0",
        "-p",
        "5",
```

```
        "-n",
        "4",
        "-r",
        "1",
    ]
    returncode, stdout, stderr = run_command(command, timeout_seconds=timeout_seconds)
    return EngineRun(
        name="llama_cpp_smollm2_q4_k_m",
        command=command,
        returncode=returncode,
        stdout=stdout,
        stderr=stderr,
        metrics=parse_llama_bench_metrics(stdout),
    )


def write_report(path: Path, runs: list[EngineRun], *, prompt: str, max_new_tokens: int) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    lines = [
        "LCQI GPT-2 Benchmark Compare",
        "",
        f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
        f"repo_commit={current_commit()}",
        f"working_tree_dirty={working_tree_dirty()}",
        f"python={sys.version.split()[0]}",
        f"prompt={prompt}",
        f"max_new_tokens={max_new_tokens}",
        "",
        "scope=LCQI full_prefix and cached_kv run the same GPT-2 F32 safetensors model. "
        "llama.cpp uses SmolLM2-135M-Instruct Q4_K_M as an external mature-engine reference, "
        "so its numbers are engineering context, not same-model quality ranking.",
        "",
    ]
    for run in runs:
        lines.extend(
            [
                f"[{run.name}]",
                f"returncode={run.returncode}",
                f"command={command_line(run.command)}",
            ]
        )
        for key in sorted(run.metrics):
            lines.append(f"{key}={run.metrics[key]}")
        lines.extend(
            [
                "stdout_tail:",
                tail(run.stdout.strip(), 2000),
```

```
                "stderr_tail:",
                tail(run.stderr.strip(), 2000),
                "",
            ]
        )
    while lines and lines[-1] == "":
        lines.pop()
    path.write_text("\n".join(lines) + "\n", encoding="utf-8")


def current_commit() -> str:
    result = subprocess.run(
        ["git", "-C", str(repo_dir()), "rev-parse", "--short", "HEAD"],
        text=True,
        capture_output=True,
        check=False,
    )
    return result.stdout.strip() if result.returncode == 0 else "unknown"


def working_tree_dirty() -> str:
    result = subprocess.run(
        ["git", "-C", str(repo_dir()), "status", "--porcelain"],
        text=True,
        capture_output=True,
        check=False,
    )
    if result.returncode != 0:
        return "unknown"
    return "true" if result.stdout.strip() else "false"


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Compare LCQI GPT-2 full-prefix and KV-cache paths, with optional llama.cpp context."
    )
    parser.add_argument("--cache-dir", type=Path, default=run_gpt2_smoke.default_cache_dir())
    parser.add_argument("--build-dir", type=Path, default=default_build_dir())
    parser.add_argument("--report", type=Path, default=default_report_path())
    parser.add_argument("--prompt", default=DEFAULT_PROMPT)
    parser.add_argument("--max-new-tokens", type=int, default=DEFAULT_MAX_NEW_TOKENS)
    parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
    parser.add_argument("--timeout-seconds", type=int, default=DEFAULT_TIMEOUT_SECONDS)
    parser.add_argument("--no-download", action="store_true")
    parser.add_argument("--skip-llama", action="store_true")
    parser.add_argument("--smollm2-cache-dir", type=Path, default=DEFAULT_SMOLLM2_CACHE_DIR)
    return parser.parse_args()
```

```
272
273
274 def main() -> int:
275     args = parse_args()
276     if args.max_new_tokens < 0:
277         print("[lcqi-compare] --max-new-tokens cannot be negative", file=sys.
    ↪ stderr)
278         return 1
279     try:
280         run_gpt2_smoke.ensure_model_files(
281             args.cache_dir,
282             no_download=args.no_download,
283             timeout_seconds=args.timeout_seconds,
284         )
285         binary = run_gpt2_smoke.build_lcqi_gpt2(args.build_dir, args.jobs)
286         runs = [
287             run_lcqi(
288                 binary,
289                 args.cache_dir,
290                 engine="full",
291                 prompt=args.prompt,
292                 max_new_tokens=args.max_new_tokens,
293                 timeout_seconds=args.timeout_seconds,
294             ),
295             run_lcqi(
296                 binary,
297                 args.cache_dir,
298                 engine="cached",
299                 prompt=args.prompt,
300                 max_new_tokens=args.max_new_tokens,
301                 timeout_seconds=args.timeout_seconds,
302             ),
303         ]
304         if not args.skip_llama:
305             llama_run = run_llama_bench(
306                 args.smollm2_cache_dir.expanduser().resolve(),
307                 timeout_seconds=args.timeout_seconds,
308             )
309             if llama_run is not None:
310                 runs.append(llama_run)
311         write_report(args.report, runs, prompt=args.prompt, max_new_tokens=args
    ↪ .max_new_tokens)
312     except Exception as error:
313         print(f"[lcqi-compare] error: {error}", file=sys.stderr)
314         return 1
315
316     for run in runs:
317         print(f"{run.name}: returncode={run.returncode}")
318         for key in sorted(run.metrics):
319             print(f"  {key}={run.metrics[key]}")
320     print(f"report={args.report}")
321     return 0 if all(run.returncode == 0 for run in runs) else 1
```

```

if __name__ == "__main__":
    raise SystemExit(main())
```

`run_lcqi_benchmark.sh` 是本地 benchmark 收集脚本。它适合在固定机器上反复运行，把同一组 shape 和 backend 的结果落到报告文件。

Listing 6.9 tools/run_lcqi_benchmark.sh

```
#!/usr/bin/env bash
set -euo pipefail

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
VOLUME_DIR="$(cd "${SCRIPT_DIR}/.." && pwd)"
REPO_DIR="$(cd "${VOLUME_DIR}/../.." && pwd)"
BUILD_DIR="${VOLUME_DIR}/build/lcqi-release"
RESULT_DIR="${VOLUME_DIR}/results"
REPEAT="${1:-200}"
STAMP="$(date -u +%Y%m%dT%H%M%SZ)"
CSV_PATH="${RESULT_DIR}/lcqi-bench-${STAMP}.csv"
META_PATH="${RESULT_DIR}/lcqi-bench-${STAMP}.meta.txt"
PERF_PATH="${RESULT_DIR}/lcqi-bench-${STAMP}.perf.txt"

mkdir -p "${RESULT_DIR}"

cmake -S "${VOLUME_DIR}" -B "${BUILD_DIR}" -DCMAKE_BUILD_TYPE=Release
cmake --build "${BUILD_DIR}"
ctest --test-dir "${BUILD_DIR}" --output-on-failure

{
  echo "timestamp_utc=${STAMP}"
  echo "repeat=${REPEAT}"
  echo "commit=$(git -C "${REPO_DIR}" rev-parse --short HEAD)"
  echo "uname=$(uname -a)"
  echo "compiler=$(${CXX:-c++} --version | sed -n '1p')"
  echo "cmake=$(cmake --version | sed -n '1p')"
  echo "bench=${BUILD_DIR}/labs/linux_cpu_inference/lcqi_bench"
} > "${META_PATH}"

"${BUILD_DIR}/labs/linux_cpu_inference/lcqi_bench" "${REPEAT}" > "${CSV_PATH}"

if command -v perf >/dev/null 2>&1; then
  perf stat \
    -e cycles,instructions,cache-misses,branches,branch-misses \
    "${BUILD_DIR}/labs/linux_cpu_inference/lcqi_bench" "${REPEAT}" \
    > /dev/null 2> "${PERF_PATH}" || true
else
  echo "perf_unavailable=1" > "${PERF_PATH}"
fi

echo "csv=${CSV_PATH}"
echo "meta=${META_PATH}"
```

```
echo "perf=${PERF_PATH}"
```

6.4 完整测试

lcqi_tests.cpp 把 safetensors、tokenizer、tiny MLP、int8 kernel、reference decoder、GPT-2 tiny path、trace、ledger 和 serving probe 串成验收网。新增优化或 shape 时，必须先扩展这里的 golden 和边界样例。

Listing 6.10 tests/lcqi_tests.cpp

```
#include <lcqi/gpt2_reference.hpp>
#include <lcqi/f32_kernels.hpp>
#include <lcqi/ggml_matvec.hpp>
#include <lcqi/ggml_tensors.hpp>
#include <lcqi/gguf.hpp>
#include <lcqi/inference.hpp>
#include <lcqi/int8_kernels.hpp>
#include <lcqi/llama_gguf.hpp>
#include <lcqi/model.hpp>
#include <lcqi/q4_k.hpp>
#include <lcqi/q5_0_packed.hpp>
#include <lcqi/reference_decoder.hpp>
#include <lcqi/safetensors.hpp>
#include <lcqi/tokenizer.hpp>

#include <array>
#include <cmath>
#include <cstring>
#include <cstdlib>
#include <filesystem>
#include <fstream>
#include <iostream>
#include <span>
#include <sstream>
#include <stdexcept>
#include <string>
#include <utility>
#include <vector>

namespace {

constexpr float LCQI_TEST_EPSILON = 1.0e-5F;
constexpr std::int32_t LCQI_TEST_INPUT_SIZE = 4;
constexpr std::int32_t LCQI_TEST_HIDDEN_SIZE = 3;
constexpr std::int32_t LCQI_TEST_OUTPUT_SIZE = 2;
constexpr std::int32_t LCQI_TEST_PREDICTED_CLASS = 1;
constexpr float LCQI_TEST_LOGIT_0 = -0.48F;
constexpr float LCQI_TEST_LOGIT_1 = 1.06F;
constexpr float LCQI_TEST_HIDDEN_0 = 0.0F;
constexpr float LCQI_TEST_HIDDEN_1 = 0.4F;
constexpr float LCQI_TEST_HIDDEN_2 = 1.4F;
constexpr float LCQI_RMS_EXPECTED_0 = 0.848528F;
```

```
constexpr float LCQI_RMS_EXPECTED_1 = 0.565685F;
constexpr float LCQI_ATTENTION_EXPECTED_0 = 2.0F;
constexpr float LCQI_ATTENTION_EXPECTED_1 = 3.0F;
constexpr float LCQI_KERNEL_MAX_DIFF = 1.0e-5F;
constexpr float LCQI_F32_KERNEL_MAX_DIFF = 1.0e-4F;
constexpr std::int32_t LCQI_SIMD_TEST_BLOCK = 8;
constexpr std::size_t LCQI_F32_DOT_TEST_SIZE = 257;
constexpr std::size_t LCQI_F32_ROW_TEST_INPUT_SIZE = 65;
constexpr std::size_t LCQI_F32_ROW_TEST_OUTPUT_SIZE = 17;
constexpr std::size_t LCQI_F32_BATCH_TEST_SIZE = 5;
constexpr std::int32_t LCQI_REFERENCE_DECODER_PREDICTED_TOKEN = 0;
constexpr std::size_t LCQI_REFERENCE_DECODER_VOCAB_SIZE = 5;
constexpr std::array<float, LCQI_REFERENCE_DECODER_VOCAB_SIZE> ↩
    ↪ LCQI_REFERENCE_DECODER_LOGITS{
    0.352516979F,
    -0.126884326F,
    0.0797837451F,
    -0.29797405F,
    0.127030417F,
};
constexpr std::int32_t LCQI_REFERENCE_TRACE_TOKEN_COUNT = 3;
constexpr std::int32_t LCQI_REFERENCE_TRACE_HIDDEN_SIZE = 4;
constexpr std::int32_t LCQI_REFERENCE_TRACE_FIRST_TOKEN = 0;
constexpr std::uint64_t LCQI_TOKENIZER_HASH_EXPECTED = 1345290284538833373 4ULL;
constexpr std::int32_t LCQI_SAFETENSORS_PREFIX_BYTES = 8;
constexpr std::int32_t LCQI_TEST_BYTE_BITS = 8;
constexpr std::uint64_t LCQI_SAFETENSORS_VALID_PAYLOAD_BYTES = 48;
constexpr std::uint64_t LCQI_SAFETENSORS_EMBED_BEGIN = 0;
constexpr std::uint64_t LCQI_SAFETENSORS_EMBED_END = 24;
constexpr std::uint64_t LCQI_SAFETENSORS_QPROJ_BEGIN = 24;
constexpr std::uint64_t LCQI_SAFETENSORS_QPROJ_END = 48;
constexpr unsigned int LCQI_BYTE_MASK = 0xFFU;
constexpr std::int32_t LCQI_GPT2_PREDICTED_TOKEN = 3;
constexpr std::size_t LCQI_GPT2_VOCAB_SIZE = 6;
constexpr std::array<float, LCQI_GPT2_VOCAB_SIZE> LCQI_GPT2_LOGITS{
    0.407358F,
    -0.504049F,
    -0.107834F,
    0.840337F,
    -0.450123F,
    -0.156763F,
};
constexpr std::int32_t LCQI_GPT2_GENERATED_LENGTH = 5;
constexpr std::int32_t LCQI_GPT2_TOKENIZER_HELLO_ID = 7;
constexpr std::uint16_t LCQI_TEST_F16_ONE = 0x3C00U;
constexpr std::uint16_t LCQI_TEST_F16_HALF = 0x3800U;
constexpr std::uint16_t LCQI_TEST_F16_NEGATIVE_TWO = 0xC000U;
constexpr std::uint16_t LCQI_TEST_BF16_ONE_POINT_FIVE = 0x3FC0U;
constexpr std::uint16_t LCQI_TEST_BF16_NEGATIVE_HALF = 0xBF00U;
constexpr std::uint32_t LCQI_GGUF_TEST_VERSION = 3;
constexpr std::uint32_t LCQI_GGUF_TEST_ALIGNMENT = 32;
constexpr std::int64_t LCQI_GGUF_TEST_TENSOR_COUNT = 1;
```

```
constexpr std::int64_t LCQI_GGUF_TEST_METADATA_COUNT = 3;
constexpr std::int32_t LCQI_GGUF_TYPE_UINT32 = 4;
constexpr std::int32_t LCQI_GGUF_TYPE_FLOAT32 = 6;
constexpr std::int32_t LCQI_GGUF_TYPE_STRING = 8;
constexpr std::int32_t LCQI_GGUF_TYPE_ARRAY = 9;
constexpr std::int32_t LCQI_GGUF_TENSOR_TYPE_Q4_K = 12;
constexpr std::uint64_t LCQI_GGUF_TEST_TENSOR_OFFSET = 0;
constexpr float LCQI_Q4K_TEST_BLOCK_SCALE = 0.5F;
constexpr float LCQI_Q4K_TEST_BLOCK_MIN = 0.25F;
constexpr float LCQI_Q4K_Q8_DIFF_LIMIT = 0.08F;
constexpr float LCQI_GGML_MATVEC_MAX_DIFF = 1.0e-4F;
constexpr float LCQI_GGML_Q8_MATVEC_MAX_DIFF = 0.2F;
constexpr std::int32_t LCQI_GGML_TEST_Q5_HALF_VALUES = 16;
constexpr std::int32_t LCQI_GGML_TEST_Q5_ZERO_POINT = 16;
constexpr std::int32_t LCQI_GGML_TEST_Q6_LANE = 5;
constexpr std::int32_t LCQI_LLAMA_TEST_HIDDEN = 4;
constexpr std::int32_t LCQI_LLAMA_TEST_HEADS = 2;
constexpr std::int32_t LCQI_LLAMA_TEST_KV_HEADS = 1;
constexpr std::int32_t LCQI_LLAMA_TEST_HEAD_DIM = 2;
constexpr std::int32_t LCQI_LLAMA_TEST_INTERMEDIATE = 4;
constexpr std::int32_t LCQI_LLAMA_TEST_VOCAB = 4;
constexpr std::int32_t LCQI_LLAMA_TEST_CONTEXT = 8;
constexpr std::int32_t LCQI_LLAMA_TEST_LAYER_COUNT = 1;
constexpr std::int32_t LCQI_LLAMA_TEST_EXPECTED_TOKEN = 2;
constexpr float LCQI_LLAMA_TEST_SMALL_ATTENTION_VALUE = 0.1F;
constexpr float LCQI_LLAMA_TEST_SMALL_OUTPUT_SCALE = 0.1F;
constexpr std::int32_t LCQI_LLAMA_Q4_TEST_HIDDEN = 256;
constexpr std::int32_t LCQI_LLAMA_Q4_TEST_HEADS = 1;
constexpr std::int32_t LCQI_LLAMA_Q4_TEST_KV_HEADS = 1;
constexpr std::int32_t LCQI_LLAMA_Q4_TEST_HEAD_DIM = 256;
constexpr std::int32_t LCQI_LLAMA_Q4_TEST_INTERMEDIATE = 256;
constexpr std::int32_t LCQI_LLAMA_Q4_TEST_VOCAB = 2;
constexpr std::int32_t LCQI_LLAMA_Q4_TEST_CONTEXT = 4;
constexpr std::int32_t LCQI_LLAMA_Q4_TEST_LAYER_COUNT = 1;
constexpr const char* LCQI_TEST_LLAMA_BATCH_PREFILL_ENV = "↩
    ↪ LCQI_LLAMA_BATCH_PREFILL";
constexpr const char* LCQI_TEST_LLAMA_GGML_DIRECT_ENV = "LCQI_LLAMA_GGML_DIRECT↩
    ↪ ";
constexpr const char* LCQI_TEST_LLAMA_GGML_SELECTIVE_DIRECT_ENV =
    "LCQI_LLAMA_GGML_SELECTIVE_DIRECT";
constexpr const char* LCQI_TEST_LLAMA_Q5_0_PACKED_ENV = "LCQI_LLAMA_Q5_0_PACKED↩
    ↪ ";
constexpr const char* LCQI_TEST_LLAMA_Q6_K_DIRECT_ENV = "LCQI_LLAMA_Q6_K_DIRECT↩
    ↪ ";
constexpr const char* LCQI_TEST_LLAMA_Q8_0_DIRECT_ENV = "LCQI_LLAMA_Q8_0_DIRECT↩
    ↪ ";
constexpr const char* LCQI_TEST_FALSE_ENV = "0";
constexpr const char* LCQI_TEST_TRUE_ENV = "1";

struct KernelShapeCase {
    std::int32_t input_size = 0;
    std::int32_t output_size = 0;
```

```cpp
    std::int32_t output_block_size = 0;
};

struct RawTensorFixture {
    std::string name;
    std::string dtype;
    std::vector<std::int64_t> shape;
    std::vector<std::uint8_t> bytes;
};

struct FloatTensorFixture {
    std::string name;
    std::vector<std::int64_t> shape;
    std::vector<float> values;
};

struct GgufTensorFixture {
    std::string name;
    std::vector<std::int64_t> shape;
    lcqi::GgmlType type = lcqi::GgmlType::f32;
    std::vector<std::uint8_t> bytes;
    std::uint64_t relative_offset = 0;
};

struct ScopedEnvVar {
    std::string name;
    std::string old_value;
    bool had_old_value = false;

    ScopedEnvVar(const char* env_name, const char* value) : name(env_name) {
        const char* old = std::getenv(env_name);
        if (old != nullptr) {
            this->old_value = old;
            this->had_old_value = true;
        }
        if (setenv(env_name, value, 1) != 0) {
            throw std::runtime_error("failed to set test environment variable");
        }
    }

    ScopedEnvVar(const ScopedEnvVar&) = delete;
    ScopedEnvVar& operator=(const ScopedEnvVar&) = delete;

    ~ScopedEnvVar() {
        if (this->had_old_value) {
            static_cast<void>(setenv(this->name.c_str(), this->old_value.c_str(), 1));
        } else {
            static_cast<void>(unsetenv(this->name.c_str()));
        }
    }
```

```
};

constexpr std::array<KernelShapeCase, 4> LCQI_KERNEL_SHAPE_CASES{{
    {17, 19, 8},
    {33, 7, 8},
    {64, 32, 16},
    {65, 31, 16},
}};

void require(bool condition, const char* message) {
    if (!condition) {
        throw std::runtime_error(message);
    }
}

void require_close(float actual, float expected, const char* message) {
    if (std::fabs(actual - expected) > LCQI_TEST_EPSILON) {
        throw std::runtime_error(message);
    }
}

std::filesystem::path test_model_path() {
    return std::filesystem::path(__FILE__).parent_path().parent_path() /
           "models" / "tiny_mlp_i8.txt";
}

std::filesystem::path test_tokenizer_path() {
    return std::filesystem::path(__FILE__).parent_path().parent_path() /
           "models" / "tiny_tokenizer.txt";
}

std::filesystem::path temp_file_path(const std::string& name) {
    return std::filesystem::temp_directory_path() / name;
}

void write_le_u64(std::ofstream& output, std::uint64_t value) {
    for (std::int32_t i = 0; i < LCQI_SAFETENSORS_PREFIX_BYTES; ++i) {
        const char byte = static_cast<char>(
            (value >> (LCQI_TEST_BYTE_BITS * i)) & LCQI_BYTE_MASK);
        output.write(&byte, 1);
    }
}

void append_le_u16(std::vector<std::uint8_t>& output, std::uint16_t value) {
    output.push_back(static_cast<std::uint8_t>(value & LCQI_BYTE_MASK));
    output.push_back(static_cast<std::uint8_t>((value >> LCQI_TEST_BYTE_BITS) &
    ↪  LCQI_BYTE_MASK));
}

void append_le_u32(std::vector<std::uint8_t>& output, std::uint32_t value) {
    for (std::int32_t i = 0; i < 4; ++i) {
        output.push_back(static_cast<std::uint8_t>(
```

```
            (value >> (LCQI_TEST_BYTE_BITS * i)) & LCQI_BYTE_MASK));
    }
}

void append_le_i32(std::vector<std::uint8_t>& output, std::int32_t value) {
    std::uint32_t bits = 0;
    std::memcpy(&bits, &value, sizeof(bits));
    append_le_u32(output, bits);
}

void append_le_u64(std::vector<std::uint8_t>& output, std::uint64_t value) {
    for (std::int32_t i = 0; i < 8; ++i) {
        output.push_back(static_cast<std::uint8_t>(
            (value >> (LCQI_TEST_BYTE_BITS * i)) & LCQI_BYTE_MASK));
    }
}

void append_le_i64(std::vector<std::uint8_t>& output, std::int64_t value) {
    std::uint64_t bits = 0;
    std::memcpy(&bits, &value, sizeof(bits));
    append_le_u64(output, bits);
}

void append_gguf_string(std::vector<std::uint8_t>& output, const std::string& text) {
    append_le_u64(output, static_cast<std::uint64_t>(text.size()));
    output.insert(output.end(), text.begin(), text.end());
}

void append_le_f32(std::vector<std::uint8_t>& output, float value) {
    std::uint32_t bits = 0;
    std::memcpy(&bits, &value, sizeof(bits));
    for (std::int32_t i = 0; i < 4; ++i) {
        output.push_back(static_cast<std::uint8_t>(
            (bits >> (LCQI_TEST_BYTE_BITS * i)) & LCQI_BYTE_MASK));
    }
}

void write_safetensors_fixture(const std::filesystem::path& path,
                               const std::string& header,
                               std::uint64_t payload_bytes) {
    std::ofstream output(path, std::ios::binary);
    if (!output) {
        throw std::runtime_error("cannot create safetensors test fixture");
    }
    write_le_u64(output, static_cast<std::uint64_t>(header.size()));
    output.write(header.data(), static_cast<std::streamsize>(header.size()));
    for (std::uint64_t i = 0; i < payload_bytes; ++i) {
        const char byte = static_cast<char>(i & 0xFFU);
        output.write(&byte, 1);
    }
}
```

```
293
294 std::string json_shape(std::span<const std::int64_t> shape) {
295     std::ostringstream stream;
296     stream << '[';
297     for (std::size_t index = 0; index < shape.size(); ++index) {
298         if (index != 0) {
299             stream << ',';
300         }
301         stream << shape[index];
302     }
303     stream << ']';
304     return stream.str();
305 }
306
307 void write_safetensors_raw_fixture(const std::filesystem::path& path,
308                                    std::span<const RawTensorFixture> tensors) {
309     std::ostringstream header;
310     header << "{\"__metadata__\":{\"format\":\"lcqi-test\"}";
311     std::uint64_t offset = 0;
312     for (const RawTensorFixture& tensor : tensors) {
313         header << ",\"" << tensor.name << "\":{\"dtype\":\"" << tensor.dtype
314                << "\",\"shape\":" << json_shape(tensor.shape)
315                << ",\"data_offsets\":[" << offset << ','
316                << offset + tensor.bytes.size() << "]}";
317         offset += tensor.bytes.size();
318     }
319     header << '}';
320
321     std::ofstream output(path, std::ios::binary);
322     if (!output) {
323         throw std::runtime_error("cannot create safetensors raw test fixture");
324     }
325     const std::string header_text = header.str();
326     write_le_u64(output, static_cast<std::uint64_t>(header_text.size()));
327     output.write(header_text.data(), static_cast<std::streamsize>(header_text.↩
    ↪ size()));
328     for (const RawTensorFixture& tensor : tensors) {
329         output.write(reinterpret_cast<const char*>(tensor.bytes.data()),
330                      static_cast<std::streamsize>(tensor.bytes.size()));
331     }
332 }
333
334 std::vector<std::uint8_t> f32_payload(std::span<const float> values) {
335     std::vector<std::uint8_t> bytes;
336     bytes.reserve(values.size() * 4);
337     for (const float value : values) {
338         append_le_f32(bytes, value);
339     }
340     return bytes;
341 }
342
343 std::vector<float> transpose_linear_to_hf_conv1d(const lcqi::Gpt2LinearF32& ↩
```

```
    ↪ linear) {
344     std::vector<float> transposed(
345         static_cast<std::size_t>(linear.input_size) *
346             static_cast<std::size_t>(linear.output_size),
347         0.0F);
348     for (std::int32_t out = 0; out < linear.output_size; ++out) {
349         for (std::int32_t in = 0; in < linear.input_size; ++in) {
350             transposed[static_cast<std::size_t>(in) *
351                            static_cast<std::size_t>(linear.output_size) +
352                        static_cast<std::size_t>(out)] =
353                 linear.weights[static_cast<std::size_t>(out) *
354                                    static_cast<std::size_t>(linear.input_size) ↩
    ↪ +
355                                static_cast<std::size_t>(in)];
356         }
357     }
358     return transposed;
359 }
360
361 void add_f32_tensor(std::vector<RawTensorFixture>& tensors,
362                     std::string name,
363                     std::vector<std::int64_t> shape,
364                     std::span<const float> values) {
365     tensors.push_back(RawTensorFixture{
366         std::move(name),
367         "F32",
368         std::move(shape),
369         f32_payload(values),
370     });
371 }
372
373 void write_text_file(const std::filesystem::path& path, const std::string& text↩
    ↪ ) {
374     std::ofstream output(path);
375     if (!output) {
376         throw std::runtime_error("cannot create text fixture");
377     }
378     output << text;
379 }
380
381 std::vector<std::uint8_t> make_q4_k_test_block() {
382     std::vector<std::uint8_t> block(
383         static_cast<std::size_t>(lcqi::LCQI_Q4_K_BLOCK_BYTES),
384         0);
385     block[0] = 0x00U;
386     block[1] = 0x38U;
387     block[2] = 0x00U;
388     block[3] = 0x34U;
389
390     std::array<std::uint8_t, static_cast<std::size_t>(lcqi::LCQI_Q4_K_SUBBLOCKS↩
    ↪ )> scales{
391         1, 2, 3, 4, 5, 6, 7, 8,
```

```
392     };
393     std::array<std::uint8_t, static_cast<std::size_t>(lcqi::LCQI_Q4_K_SUBBLOCKS↩
        ↪ )> mins{
394         1, 1, 2, 2, 3, 3, 4, 4,
395     };
396     for (std::size_t index = 0; index < 4; ++index) {
397         block[4 + index] = static_cast<std::uint8_t>(
398             (scales[index] & 0x3FU) | ((scales[index + 4] >> 4U) << 6U));
399         block[8 + index] = static_cast<std::uint8_t>(
400             (mins[index] & 0x3FU) | ((mins[index + 4] >> 4U) << 6U));
401         block[12 + index] = static_cast<std::uint8_t>(
402             (scales[index + 4] & 0x0FU) | ((mins[index + 4] & 0x0FU) << 4U));
403     }
404
405     for (std::int32_t pair = 0; pair < 4; ++pair) {
406         const std::int32_t low_subblock = pair * 2;
407         const std::int32_t high_subblock = low_subblock + 1;
408         for (std::int32_t index = 0; index < lcqi::LCQI_Q4_K_SUBBLOCK_VALUES; ↩
        ↪ ++index) {
409             const std::uint8_t low = static_cast<std::uint8_t>(
410                 (low_subblock + index) & 0x0F);
411             const std::uint8_t high = static_cast<std::uint8_t>(
412                 (high_subblock + index + 1) & 0x0F);
413             block[static_cast<std::size_t>(16 + pair * 32 + index)] =
414                 static_cast<std::uint8_t>(low | (high << 4U));
415         }
416     }
417     return block;
418 }
419
420 std::vector<float> expected_q4_k_test_values() {
421     const std::array<float, static_cast<std::size_t>(lcqi::LCQI_Q4_K_SUBBLOCKS)↩
        ↪ > scales{
422         1, 2, 3, 4, 5, 6, 7, 8,
423     };
424     const std::array<float, static_cast<std::size_t>(lcqi::LCQI_Q4_K_SUBBLOCKS)↩
        ↪ > mins{
425         1, 1, 2, 2, 3, 3, 4, 4,
426     };
427     std::vector<float> values(
428         static_cast<std::size_t>(lcqi::LCQI_QK_K_BLOCK_VALUES),
429         0.0F);
430     for (std::int32_t subblock = 0; subblock < lcqi::LCQI_Q4_K_SUBBLOCKS; ++↩
        ↪ subblock) {
431         for (std::int32_t index = 0; index < lcqi::LCQI_Q4_K_SUBBLOCK_VALUES; ↩
        ↪ ++index) {
432             const std::int32_t quantized =
433                 (subblock % 2 == 0)
434                     ? ((subblock + index) & 0x0F)
435                     : ((subblock + index + 1) & 0x0F);
436             values[static_cast<std::size_t>(
437                 subblock * lcqi::LCQI_Q4_K_SUBBLOCK_VALUES + index)] =
```

```
438                 LCQI_Q4K_TEST_BLOCK_SCALE * scales[static_cast<std::size_t>(↩
    ↪ subblock)] *
439                     static_cast<float>(quantized) -
440                 LCQI_Q4K_TEST_BLOCK_MIN * mins[static_cast<std::size_t>(↩
    ↪ subblock)];
441         }
442     }
443     return values;
444 }
445
446 std::vector<std::uint8_t> make_q8_0_test_block() {
447     std::vector<std::uint8_t> block(
448         static_cast<std::size_t>(lcqi::LCQI_Q8_0_BLOCK_BYTES),
449         0);
450     block[0] = static_cast<std::uint8_t>(LCQI_TEST_F16_HALF & LCQI_BYTE_MASK);
451     block[1] = static_cast<std::uint8_t>((LCQI_TEST_F16_HALF >> ↩
    ↪ LCQI_TEST_BYTE_BITS) &
452                                          LCQI_BYTE_MASK);
453     for (std::int32_t index = 0; index < lcqi::LCQI_Q8_0_BLOCK_VALUES; ++index)↩
    ↪  {
454         block[static_cast<std::size_t>(2 + index)] =
455             static_cast<std::uint8_t>(static_cast<std::int8_t>(index - 16));
456     }
457     return block;
458 }
459
460 std::vector<std::uint8_t> make_q5_0_test_block() {
461     std::vector<std::uint8_t> block(
462         static_cast<std::size_t>(lcqi::LCQI_Q5_0_BLOCK_BYTES),
463         0);
464     block[0] = static_cast<std::uint8_t>(LCQI_TEST_F16_HALF & LCQI_BYTE_MASK);
465     block[1] = static_cast<std::uint8_t>((LCQI_TEST_F16_HALF >> ↩
    ↪ LCQI_TEST_BYTE_BITS) &
466                                          LCQI_BYTE_MASK);
467
468     std::uint32_t qh = 0;
469     for (std::int32_t index = 0; index < lcqi::LCQI_Q5_0_BLOCK_VALUES / 2; ++↩
    ↪ index) {
470         const std::int32_t first_quantized = index - ↩
    ↪ LCQI_GGML_TEST_Q5_ZERO_POINT;
471         const std::int32_t second_quantized = index;
472         const std::uint8_t first_encoded =
473             static_cast<std::uint8_t>(first_quantized + ↩
    ↪ LCQI_GGML_TEST_Q5_ZERO_POINT);
474         const std::uint8_t second_encoded =
475             static_cast<std::uint8_t>(second_quantized + ↩
    ↪ LCQI_GGML_TEST_Q5_ZERO_POINT);
476         if ((first_encoded & 0x10U) != 0U) {
477             qh |= (1U << index);
478         }
479         if ((second_encoded & 0x10U) != 0U) {
480             qh |= (1U << (index + LCQI_GGML_TEST_Q5_HALF_VALUES));
```

```
481         }
482         const std::uint8_t low = static_cast<std::uint8_t>(first_encoded & 0x0FU);
483         const std::uint8_t high = static_cast<std::uint8_t>(second_encoded & 0x0FU);
484         block[static_cast<std::size_t>(6 + index)] =
485             static_cast<std::uint8_t>(low | (high << 4U));
486     }
487     block[2] = static_cast<std::uint8_t>(qh & LCQI_BYTE_MASK);
488     block[3] = static_cast<std::uint8_t>((qh >> LCQI_TEST_BYTE_BITS) & LCQI_BYTE_MASK);
489     block[4] = static_cast<std::uint8_t>((qh >> (LCQI_TEST_BYTE_BITS * 2)) & LCQI_BYTE_MASK);
490     block[5] = static_cast<std::uint8_t>((qh >> (LCQI_TEST_BYTE_BITS * 3)) & LCQI_BYTE_MASK);
491     return block;
492 }
493
494 std::vector<std::uint8_t> make_q6_k_test_block() {
495     std::vector<std::uint8_t> block(
496         static_cast<std::size_t>(lcqi::LCQI_Q6_K_BLOCK_BYTES),
497         0);
498     const std::size_t ql_offset = 0;
499     const std::size_t qh_offset = static_cast<std::size_t>(lcqi::LCQI_Q6_K_BLOCK_VALUES / 2);
500     const std::size_t scales_offset =
501         qh_offset + static_cast<std::size_t>(lcqi::LCQI_Q6_K_BLOCK_VALUES / 4);
502     for (std::int32_t index = 0; index < 16; ++index) {
503         block[scales_offset + static_cast<std::size_t>(index)] = 1;
504     }
505     block[static_cast<std::size_t>(lcqi::LCQI_Q6_K_BLOCK_BYTES - 2)] =
506         static_cast<std::uint8_t>(LCQI_TEST_F16_ONE & LCQI_BYTE_MASK);
507     block[static_cast<std::size_t>(lcqi::LCQI_Q6_K_BLOCK_BYTES - 1)] =
508         static_cast<std::uint8_t>((LCQI_TEST_F16_ONE >> LCQI_TEST_BYTE_BITS) &
509                                   LCQI_BYTE_MASK);
510
511     const auto store_q6 = [&block, ql_offset, qh_offset](std::int32_t position,
512                                                          std::uint8_t encoded) {
513         const std::int32_t half = position / 128;
514         const std::int32_t local = position % 128;
515         const std::size_t ql_base = ql_offset + static_cast<std::size_t>(half * 64);
516         const std::size_t qh_base = qh_offset + static_cast<std::size_t>(half * 32);
517         const std::uint8_t low = encoded & 0x0FU;
518         const std::uint8_t high = static_cast<std::uint8_t>((encoded >> 4U) & 0x03U);
519         if (local < 32) {
520             block[ql_base + static_cast<std::size_t>(local)] =
521                 static_cast<std::uint8_t>(
522                     (block[ql_base + static_cast<std::size_t>(local)] & 0xF0U) | low);
```

```
            block[qh_base + static_cast<std::size_t>(local)] =
                static_cast<std::uint8_t>(
                    (block[qh_base + static_cast<std::size_t>(local)] & 0xFCU) | high);
        } else if (local < 64) {
            const std::size_t lane = static_cast<std::size_t>(local - 32);
            block[ql_base + static_cast<std::size_t>(32) + lane] =
                static_cast<std::uint8_t>(
                    (block[ql_base + static_cast<std::size_t>(32) + lane] & 0xF0U) | low);
            block[qh_base + lane] =
                static_cast<std::uint8_t>((block[qh_base + lane] & 0xF3U) | (high << 2U));
        } else if (local < 96) {
            const std::size_t lane = static_cast<std::size_t>(local - 64);
            block[ql_base + lane] =
                static_cast<std::uint8_t>(
                    (block[ql_base + lane] & 0x0FU) | (low << 4U));
            block[qh_base + lane] =
                static_cast<std::uint8_t>((block[qh_base + lane] & 0xCFU) | (high << 4U));
        } else {
            const std::size_t lane = static_cast<std::size_t>(local - 96);
            block[ql_base + static_cast<std::size_t>(32) + lane] =
                static_cast<std::uint8_t>(
                    (block[ql_base + static_cast<std::size_t>(32) + lane] & 0x0FU) |
                    (low << 4U));
            block[qh_base + lane] =
                static_cast<std::uint8_t>((block[qh_base + lane] & 0x3FU) | (high << 6U));
        }
    };

    store_q6(LCQI_GGML_TEST_Q6_LANE, 35);
    store_q6(32 + LCQI_GGML_TEST_Q6_LANE, 31);
    store_q6(64 + LCQI_GGML_TEST_Q6_LANE, 63);
    store_q6(96 + LCQI_GGML_TEST_Q6_LANE, 0);
    store_q6(128 + LCQI_GGML_TEST_Q6_LANE, 36);
    return block;
}

void write_gguf_q4_k_fixture(const std::filesystem::path& path) {
    std::vector<std::uint8_t> bytes;
    bytes.push_back('G');
    bytes.push_back('G');
    bytes.push_back('U');
    bytes.push_back('F');
    append_le_u32(bytes, LCQI_GGUF_TEST_VERSION);
    append_le_i64(bytes, LCQI_GGUF_TEST_TENSOR_COUNT);
    append_le_i64(bytes, LCQI_GGUF_TEST_METADATA_COUNT);

```

```
    append_gguf_string(bytes, "general.alignment");
    append_le_i32(bytes, LCQI_GGUF_TYPE_UINT32);
    append_le_u32(bytes, LCQI_GGUF_TEST_ALIGNMENT);
    append_gguf_string(bytes, "general.architecture");
    append_le_i32(bytes, LCQI_GGUF_TYPE_STRING);
    append_gguf_string(bytes, "lcqi-test");
    append_gguf_string(bytes, "tokenizer.ggml.tokens");
    append_le_i32(bytes, LCQI_GGUF_TYPE_ARRAY);
    append_le_i32(bytes, LCQI_GGUF_TYPE_STRING);
    append_le_u64(bytes, 2);
    append_gguf_string(bytes, "<bos>");
    append_gguf_string(bytes, "hello");

    append_gguf_string(bytes, "blk.0.ffn_up.weight");
    append_le_u32(bytes, 2);
    append_le_i64(bytes, lcqi::LCQI_QK_K_BLOCK_VALUES);
    append_le_i64(bytes, 1);
    append_le_i32(bytes, LCQI_GGUF_TENSOR_TYPE_Q4_K);
    append_le_u64(bytes, LCQI_GGUF_TEST_TENSOR_OFFSET);
    while (bytes.size() % LCQI_GGUF_TEST_ALIGNMENT != 0) {
        bytes.push_back(0);
    }
    const std::vector<std::uint8_t> block = make_q4_k_test_block();
    bytes.insert(bytes.end(), block.begin(), block.end());

    std::ofstream output(path, std::ios::binary);
    if (!output) {
        throw std::runtime_error("cannot create GGUF fixture");
    }
    output.write(reinterpret_cast<const char*>(bytes.data()),
                 static_cast<std::streamsize>(bytes.size()));
}

void append_gguf_uint32_metadata(std::vector<std::uint8_t>& bytes,
                                 const std::string& key,
                                 std::uint32_t value) {
    append_gguf_string(bytes, key);
    append_le_i32(bytes, LCQI_GGUF_TYPE_UINT32);
    append_le_u32(bytes, value);
}

void append_gguf_float32_metadata(std::vector<std::uint8_t>& bytes,
                                  const std::string& key,
                                  float value) {
    append_gguf_string(bytes, key);
    append_le_i32(bytes, LCQI_GGUF_TYPE_FLOAT32);
    append_le_f32(bytes, value);
}

void append_gguf_string_metadata(std::vector<std::uint8_t>& bytes,
                                 const std::string& key,
                                 const std::string& value) {
```

```
    append_gguf_string(bytes, key);
    append_le_i32(bytes, LCQI_GGUF_TYPE_STRING);
    append_gguf_string(bytes, value);
}

void write_gguf_fixture(const std::filesystem::path& path,
                        std::span<GgufTensorFixture> tensors,
                        const std::vector<std::uint8_t>& metadata_bytes) {
    std::uint64_t relative_offset = 0;
    for (GgufTensorFixture& tensor : tensors) {
        tensor.relative_offset = relative_offset;
        relative_offset += tensor.bytes.size();
        while (relative_offset % LCQI_GGUF_TEST_ALIGNMENT != 0) {
            ++relative_offset;
        }
    }

    std::vector<std::uint8_t> bytes;
    bytes.push_back('G');
    bytes.push_back('G');
    bytes.push_back('U');
    bytes.push_back('F');
    append_le_u32(bytes, LCQI_GGUF_TEST_VERSION);
    append_le_u64(bytes, static_cast<std::uint64_t>(tensors.size()));
    append_le_u64(bytes, 14);
    bytes.insert(bytes.end(), metadata_bytes.begin(), metadata_bytes.end());

    for (const GgufTensorFixture& tensor : tensors) {
        append_gguf_string(bytes, tensor.name);
        append_le_u32(bytes, static_cast<std::uint32_t>(tensor.shape.size()));
        for (const std::int64_t extent : tensor.shape) {
            append_le_i64(bytes, extent);
        }
        append_le_i32(bytes, static_cast<std::int32_t>(tensor.type));
        append_le_u64(bytes, tensor.relative_offset);
    }
    while (bytes.size() % LCQI_GGUF_TEST_ALIGNMENT != 0) {
        bytes.push_back(0);
    }
    std::uint64_t written_relative = 0;
    for (const GgufTensorFixture& tensor : tensors) {
        while (written_relative < tensor.relative_offset) {
            bytes.push_back(0);
            ++written_relative;
        }
        bytes.insert(bytes.end(), tensor.bytes.begin(), tensor.bytes.end());
        written_relative += tensor.bytes.size();
    }

    std::ofstream output(path, std::ios::binary);
    if (!output) {
        throw std::runtime_error("cannot create GGUF fixture");
```

```
    }
    output.write(reinterpret_cast<const char*>(bytes.data()),
                 static_cast<std::streamsize>(bytes.size()));
}

void add_gguf_f32_tensor(std::vector<GgufTensorFixture>& tensors,
                         std::string name,
                         std::vector<std::int64_t> shape,
                         std::span<const float> values) {
    tensors.push_back(GgufTensorFixture{
        std::move(name),
        std::move(shape),
        lcqi::GgmlType::f32,
        f32_payload(values),
        0,
    });
}

void add_gguf_q4_k_tensor(std::vector<GgufTensorFixture>& tensors,
                          std::string name,
                          std::vector<std::int64_t> shape,
                          const std::vector<std::uint8_t>& bytes) {
    tensors.push_back(GgufTensorFixture{
        std::move(name),
        std::move(shape),
        lcqi::GgmlType::q4_k,
        bytes,
        0,
    });
}

void add_gguf_quantized_tensor(std::vector<GgufTensorFixture>& tensors,
                               std::string name,
                               std::vector<std::int64_t> shape,
                               lcqi::GgmlType type,
                               const std::vector<std::uint8_t>& bytes) {
    tensors.push_back(GgufTensorFixture{
        std::move(name),
        std::move(shape),
        type,
        bytes,
        0,
    });
}

std::vector<std::uint8_t> repeated_ggml_block(const std::vector<std::uint8_t>& block,
                                              std::int32_t input_size,
                                              std::int32_t output_size,
                                              std::int64_t block_values) {
    const std::size_t block_count =
        static_cast<std::size_t>((input_size / block_values) * output_size);
```

```
    std::vector<std::uint8_t> bytes;
    bytes.reserve(block.size() * block_count);
    for (std::size_t index = 0; index < block_count; ++index) {
        bytes.insert(bytes.end(), block.begin(), block.end());
    }
    return bytes;
}

void write_tiny_llama_gguf_fixture(const std::filesystem::path& path) {
    std::vector<std::uint8_t> metadata;
    append_gguf_uint32_metadata(metadata, "general.alignment", LCQI_GGUF_TEST_ALIGNMENT);
    append_gguf_string_metadata(metadata, "general.architecture", "llama");
    append_gguf_string_metadata(metadata, "general.name", "lcqi tiny llama");
    append_gguf_uint32_metadata(metadata, "llama.embedding_length", LCQI_LLAMA_TEST_HIDDEN);
    append_gguf_uint32_metadata(metadata, "llama.block_count", LCQI_LLAMA_TEST_LAYER_COUNT);
    append_gguf_uint32_metadata(metadata, "llama.feed_forward_length", LCQI_LLAMA_TEST_INTERMEDIATE);
    append_gguf_uint32_metadata(metadata, "llama.attention.head_count", LCQI_LLAMA_TEST_HEADS);
    append_gguf_uint32_metadata(metadata, "llama.attention.head_count_kv", LCQI_LLAMA_TEST_KV_HEADS);
    append_gguf_uint32_metadata(metadata, "llama.rope.dimension_count", LCQI_LLAMA_TEST_HEAD_DIM);
    append_gguf_float32_metadata(metadata, "llama.rope.freq_base", 10000.0F);
    append_gguf_uint32_metadata(metadata, "llama.context_length", LCQI_LLAMA_TEST_CONTEXT);
    append_gguf_uint32_metadata(metadata, "llama.vocab_size", LCQI_LLAMA_TEST_VOCAB);
    append_gguf_float32_metadata(metadata, "llama.attention.layer_norm_rms_epsilon", 1.0e-5F);
    append_gguf_uint32_metadata(metadata, "tokenizer.ggml.eos_token_id", 3);

    std::vector<GgufTensorFixture> tensors;
    const std::vector<float> embedding{
        1.0F, 0.0F, 0.0F, 0.0F,
        0.0F, 1.0F, 0.0F, 0.0F,
        0.0F, 2.0F, 0.0F, 0.0F,
        0.0F, 0.0F, 1.0F, 0.0F,
    };
    add_gguf_f32_tensor(tensors,
                        "token_embd.weight",
                        {LCQI_LLAMA_TEST_HIDDEN, LCQI_LLAMA_TEST_VOCAB},
                        embedding);
    add_gguf_f32_tensor(tensors,
                        "output_norm.weight",
                        {LCQI_LLAMA_TEST_HIDDEN},
                        std::vector<float>(LCQI_LLAMA_TEST_HIDDEN, 1.0F));
    add_gguf_f32_tensor(tensors,
                        "blk.0.attn_norm.weight",
```

```
                        {LCQI_LLAMA_TEST_HIDDEN},
                        std::vector<float>(LCQI_LLAMA_TEST_HIDDEN, 1.0F));
    add_gguf_f32_tensor(tensors,
                        "blk.0.ffn_norm.weight",
                        {LCQI_LLAMA_TEST_HIDDEN},
                        std::vector<float>(LCQI_LLAMA_TEST_HIDDEN, 1.0F));
    const std::vector<float> hidden_to_hidden(
        static_cast<std::size_t>(LCQI_LLAMA_TEST_HIDDEN * ↩
    ↪ LCQI_LLAMA_TEST_HIDDEN),
        0.0F);
    std::vector<float> hidden_to_kv(
        static_cast<std::size_t>(LCQI_LLAMA_TEST_HIDDEN * ↩
    ↪ LCQI_LLAMA_TEST_HEAD_DIM),
        0.0F);
    hidden_to_kv[1] = LCQI_LLAMA_TEST_SMALL_ATTENTION_VALUE;
    std::vector<float> attention_output = hidden_to_hidden;
    attention_output[0] = LCQI_LLAMA_TEST_SMALL_OUTPUT_SCALE;
    attention_output[10] = LCQI_LLAMA_TEST_SMALL_OUTPUT_SCALE;
    const std::vector<float> hidden_to_intermediate(
        static_cast<std::size_t>(LCQI_LLAMA_TEST_HIDDEN * ↩
    ↪ LCQI_LLAMA_TEST_INTERMEDIATE),
        0.0F);
    const std::vector<float> intermediate_to_hidden(
        static_cast<std::size_t>(LCQI_LLAMA_TEST_INTERMEDIATE * ↩
    ↪ LCQI_LLAMA_TEST_HIDDEN),
        0.0F);
    add_gguf_f32_tensor(tensors,
                        "blk.0.attn_q.weight",
                        {LCQI_LLAMA_TEST_HIDDEN, LCQI_LLAMA_TEST_HIDDEN},
                        hidden_to_hidden);
    add_gguf_f32_tensor(tensors,
                        "blk.0.attn_k.weight",
                        {LCQI_LLAMA_TEST_HIDDEN, LCQI_LLAMA_TEST_HEAD_DIM},
                        hidden_to_kv);
    add_gguf_f32_tensor(tensors,
                        "blk.0.attn_v.weight",
                        {LCQI_LLAMA_TEST_HIDDEN, LCQI_LLAMA_TEST_HEAD_DIM},
                        hidden_to_kv);
    add_gguf_f32_tensor(tensors,
                        "blk.0.attn_output.weight",
                        {LCQI_LLAMA_TEST_HIDDEN, LCQI_LLAMA_TEST_HIDDEN},
                        attention_output);
    add_gguf_f32_tensor(tensors,
                        "blk.0.ffn_gate.weight",
                        {LCQI_LLAMA_TEST_HIDDEN, LCQI_LLAMA_TEST_INTERMEDIATE},
                        hidden_to_intermediate);
    add_gguf_f32_tensor(tensors,
                        "blk.0.ffn_up.weight",
                        {LCQI_LLAMA_TEST_HIDDEN, LCQI_LLAMA_TEST_INTERMEDIATE},
                        hidden_to_intermediate);
    add_gguf_f32_tensor(tensors,
                        "blk.0.ffn_down.weight",
```

```
                        {LCQI_LLAMA_TEST_INTERMEDIATE, LCQI_LLAMA_TEST_HIDDEN},
                        intermediate_to_hidden);
    write_gguf_fixture(path, tensors, metadata);
}

void write_tiny_llama_ggml_direct_gguf_fixture(const std::filesystem::path& path) {
    std::vector<std::uint8_t> metadata;
    append_gguf_uint32_metadata(metadata, "general.alignment", LCQI_GGUF_TEST_ALIGNMENT);
    append_gguf_string_metadata(metadata, "general.architecture", "llama");
    append_gguf_string_metadata(metadata, "general.name", "lcqi ggml direct tiny llama");
    append_gguf_uint32_metadata(metadata, "llama.embedding_length", LCQI_LLAMA_Q4_TEST_HIDDEN);
    append_gguf_uint32_metadata(metadata, "llama.block_count", LCQI_LLAMA_Q4_TEST_LAYER_COUNT);
    append_gguf_uint32_metadata(metadata,
                                "llama.feed_forward_length",
                                LCQI_LLAMA_Q4_TEST_INTERMEDIATE);
    append_gguf_uint32_metadata(metadata, "llama.attention.head_count", LCQI_LLAMA_Q4_TEST_HEADS);
    append_gguf_uint32_metadata(metadata,
                                "llama.attention.head_count_kv",
                                LCQI_LLAMA_Q4_TEST_KV_HEADS);
    append_gguf_uint32_metadata(metadata,
                                "llama.rope.dimension_count",
                                LCQI_LLAMA_Q4_TEST_HEAD_DIM);
    append_gguf_float32_metadata(metadata, "llama.rope.freq_base", 10000.0F);
    append_gguf_uint32_metadata(metadata, "llama.context_length", LCQI_LLAMA_Q4_TEST_CONTEXT);
    append_gguf_uint32_metadata(metadata, "llama.vocab_size", LCQI_LLAMA_Q4_TEST_VOCAB);
    append_gguf_float32_metadata(metadata, "llama.attention.layer_norm_rms_epsilon", 1.0e-5F);
    append_gguf_uint32_metadata(metadata, "tokenizer.ggml.eos_token_id", 1);

    std::vector<GgufTensorFixture> tensors;
    std::vector<float> embedding(
        static_cast<std::size_t>(LCQI_LLAMA_Q4_TEST_HIDDEN * LCQI_LLAMA_Q4_TEST_VOCAB),
        0.0F);
    embedding[0] = 1.0F;
    embedding[static_cast<std::size_t>(LCQI_LLAMA_Q4_TEST_HIDDEN)] = 0.5F;
    add_gguf_f32_tensor(tensors,
                        "token_embd.weight",
                        {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_VOCAB},
                        embedding);
    add_gguf_f32_tensor(tensors,
                        "output_norm.weight",
                        {LCQI_LLAMA_Q4_TEST_HIDDEN},
                        std::vector<float>(LCQI_LLAMA_Q4_TEST_HIDDEN, 1.0F));
```

```
856     add_gguf_f32_tensor(tensors,
857                         "blk.0.attn_norm.weight",
858                         {LCQI_LLAMA_Q4_TEST_HIDDEN},
859                         std::vector<float>(LCQI_LLAMA_Q4_TEST_HIDDEN, 1.0F));
860     add_gguf_f32_tensor(tensors,
861                         "blk.0.ffn_norm.weight",
862                         {LCQI_LLAMA_Q4_TEST_HIDDEN},
863                         std::vector<float>(LCQI_LLAMA_Q4_TEST_HIDDEN, 1.0F));
864
865     const std::vector<std::uint8_t> q5_matrix = repeated_ggml_block(
866         std::vector<std::uint8_t>(static_cast<std::size_t>(lcqi::↩
    ↪ LCQI_Q5_0_BLOCK_BYTES), 0),
867         LCQI_LLAMA_Q4_TEST_HIDDEN,
868         LCQI_LLAMA_Q4_TEST_HIDDEN,
869         lcqi::LCQI_Q5_0_BLOCK_VALUES);
870     const std::vector<std::uint8_t> q8_matrix = repeated_ggml_block(
871         std::vector<std::uint8_t>(static_cast<std::size_t>(lcqi::↩
    ↪ LCQI_Q8_0_BLOCK_BYTES), 0),
872         LCQI_LLAMA_Q4_TEST_HIDDEN,
873         LCQI_LLAMA_Q4_TEST_HIDDEN,
874         lcqi::LCQI_Q8_0_BLOCK_VALUES);
875     const std::vector<std::uint8_t> q6_matrix = repeated_ggml_block(
876         std::vector<std::uint8_t>(static_cast<std::size_t>(lcqi::↩
    ↪ LCQI_Q6_K_BLOCK_BYTES), 0),
877         LCQI_LLAMA_Q4_TEST_INTERMEDIATE,
878         LCQI_LLAMA_Q4_TEST_HIDDEN,
879         lcqi::LCQI_Q6_K_BLOCK_VALUES);
880
881     add_gguf_quantized_tensor(tensors,
882                               "blk.0.attn_q.weight",
883                               {LCQI_LLAMA_Q4_TEST_HIDDEN, ↩
    ↪ LCQI_LLAMA_Q4_TEST_HIDDEN},
884                               lcqi::GgmlType::q5_0,
885                               q5_matrix);
886     add_gguf_quantized_tensor(tensors,
887                               "blk.0.attn_k.weight",
888                               {LCQI_LLAMA_Q4_TEST_HIDDEN, ↩
    ↪ LCQI_LLAMA_Q4_TEST_HEAD_DIM},
889                               lcqi::GgmlType::q5_0,
890                               q5_matrix);
891     add_gguf_quantized_tensor(tensors,
892                               "blk.0.attn_v.weight",
893                               {LCQI_LLAMA_Q4_TEST_HIDDEN, ↩
    ↪ LCQI_LLAMA_Q4_TEST_HEAD_DIM},
894                               lcqi::GgmlType::q8_0,
895                               q8_matrix);
896     add_gguf_quantized_tensor(tensors,
897                               "blk.0.attn_output.weight",
898                               {LCQI_LLAMA_Q4_TEST_HIDDEN, ↩
    ↪ LCQI_LLAMA_Q4_TEST_HIDDEN},
899                               lcqi::GgmlType::q5_0,
900                               q5_matrix);
```

```
901     add_gguf_quantized_tensor(tensors,
902                               "blk.0.ffn_gate.weight",
903                               {LCQI_LLAMA_Q4_TEST_HIDDEN, ↩
      ↪ LCQI_LLAMA_Q4_TEST_INTERMEDIATE},
904                               lcqi::GgmlType::q5_0,
905                               q5_matrix);
906     add_gguf_quantized_tensor(tensors,
907                               "blk.0.ffn_up.weight",
908                               {LCQI_LLAMA_Q4_TEST_HIDDEN, ↩
      ↪ LCQI_LLAMA_Q4_TEST_INTERMEDIATE},
909                               lcqi::GgmlType::q8_0,
910                               q8_matrix);
911     add_gguf_quantized_tensor(tensors,
912                               "blk.0.ffn_down.weight",
913                               {LCQI_LLAMA_Q4_TEST_INTERMEDIATE, ↩
      ↪ LCQI_LLAMA_Q4_TEST_HIDDEN},
914                               lcqi::GgmlType::q6_k,
915                               q6_matrix);
916     write_gguf_fixture(path, tensors, metadata);
917 }
918
919 void write_tiny_llama_q4_direct_gguf_fixture(const std::filesystem::path& path)↩
      ↪  {
920     std::vector<std::uint8_t> metadata;
921     append_gguf_uint32_metadata(metadata, "general.alignment", ↩
      ↪ LCQI_GGUF_TEST_ALIGNMENT);
922     append_gguf_string_metadata(metadata, "general.architecture", "llama");
923     append_gguf_string_metadata(metadata, "general.name", "lcqi q4 direct tiny ↩
      ↪ llama");
924     append_gguf_uint32_metadata(metadata, "llama.embedding_length", ↩
      ↪ LCQI_LLAMA_Q4_TEST_HIDDEN);
925     append_gguf_uint32_metadata(metadata, "llama.block_count", ↩
      ↪ LCQI_LLAMA_Q4_TEST_LAYER_COUNT);
926     append_gguf_uint32_metadata(metadata, "llama.feed_forward_length", ↩
      ↪ LCQI_LLAMA_Q4_TEST_INTERMEDIATE);
927     append_gguf_uint32_metadata(metadata, "llama.attention.head_count", ↩
      ↪ LCQI_LLAMA_Q4_TEST_HEADS);
928     append_gguf_uint32_metadata(metadata, "llama.attention.head_count_kv", ↩
      ↪ LCQI_LLAMA_Q4_TEST_KV_HEADS);
929     append_gguf_uint32_metadata(metadata, "llama.rope.dimension_count", ↩
      ↪ LCQI_LLAMA_Q4_TEST_HEAD_DIM);
930     append_gguf_float32_metadata(metadata, "llama.rope.freq_base", 10000.0F);
931     append_gguf_uint32_metadata(metadata, "llama.context_length", ↩
      ↪ LCQI_LLAMA_Q4_TEST_CONTEXT);
932     append_gguf_uint32_metadata(metadata, "llama.vocab_size", ↩
      ↪ LCQI_LLAMA_Q4_TEST_VOCAB);
933     append_gguf_float32_metadata(metadata, "llama.attention.↩
      ↪ layer_norm_rms_epsilon", 1.0e-5F);
934     append_gguf_uint32_metadata(metadata, "tokenizer.ggml.eos_token_id", 1);
935
936     std::vector<GgufTensorFixture> tensors;
937     std::vector<float> embedding(
```

```
        static_cast<std::size_t>(LCQI_LLAMA_Q4_TEST_HIDDEN * LCQI_LLAMA_Q4_TEST_VOCAB),
        0.0F);
    embedding[0] = 1.0F;
    embedding[static_cast<std::size_t>(LCQI_LLAMA_Q4_TEST_HIDDEN)] = 0.5F;
    add_gguf_f32_tensor(tensors,
                        "token_embd.weight",
                        {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_VOCAB},
                        embedding);
    add_gguf_f32_tensor(tensors,
                        "output_norm.weight",
                        {LCQI_LLAMA_Q4_TEST_HIDDEN},
                        std::vector<float>(LCQI_LLAMA_Q4_TEST_HIDDEN, 1.0F));
    add_gguf_f32_tensor(tensors,
                        "blk.0.attn_norm.weight",
                        {LCQI_LLAMA_Q4_TEST_HIDDEN},
                        std::vector<float>(LCQI_LLAMA_Q4_TEST_HIDDEN, 1.0F));
    add_gguf_f32_tensor(tensors,
                        "blk.0.ffn_norm.weight",
                        {LCQI_LLAMA_Q4_TEST_HIDDEN},
                        std::vector<float>(LCQI_LLAMA_Q4_TEST_HIDDEN, 1.0F));

    const std::vector<std::uint8_t> zero_q4_matrix(
        static_cast<std::size_t>(LCQI_LLAMA_Q4_TEST_HIDDEN) *
            static_cast<std::size_t>(lcqi::LCQI_Q4_K_BLOCK_BYTES),
        0);
    add_gguf_q4_k_tensor(tensors,
                         "blk.0.attn_q.weight",
                         {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_HIDDEN},
                         zero_q4_matrix);
    add_gguf_q4_k_tensor(tensors,
                         "blk.0.attn_k.weight",
                         {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_HEAD_DIM},
                         zero_q4_matrix);
    add_gguf_q4_k_tensor(tensors,
                         "blk.0.attn_v.weight",
                         {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_HEAD_DIM},
                         zero_q4_matrix);
    add_gguf_q4_k_tensor(tensors,
                         "blk.0.attn_output.weight",
                         {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_HIDDEN},
                         zero_q4_matrix);
    add_gguf_q4_k_tensor(tensors,
                         "blk.0.ffn_gate.weight",
                         {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_INTERMEDIATE},
                         zero_q4_matrix);
    add_gguf_q4_k_tensor(tensors,
```

```
                            "blk.0.ffn_up.weight",
                            {LCQI_LLAMA_Q4_TEST_HIDDEN, LCQI_LLAMA_Q4_TEST_INTERMEDIATE},
                            zero_q4_matrix);
    add_gguf_q4_k_tensor(tensors,
                            "blk.0.ffn_down.weight",
                            {LCQI_LLAMA_Q4_TEST_INTERMEDIATE, LCQI_LLAMA_Q4_TEST_HIDDEN},
                            zero_q4_matrix);
    write_gguf_fixture(path, tensors, metadata);
}

void write_tiny_gpt2_safetensors(const std::filesystem::path& path,
                                 const lcqi::Gpt2ReferenceModel& model) {
    std::vector<RawTensorFixture> tensors;
    add_f32_tensor(tensors,
                   "transformer.wte.weight",
                   {model.config.vocab_size, model.config.hidden_size},
                   model.token_embedding);
    add_f32_tensor(tensors,
                   "transformer.wpe.weight",
                   {model.config.max_positions, model.config.hidden_size},
                   model.position_embedding);
    for (std::int32_t layer_id = 0; layer_id < model.config.layer_count; ++layer_id) {
        const lcqi::Gpt2LayerWeightsF32& layer =
            model.layers[static_cast<std::size_t>(layer_id)];
        const std::string prefix = "transformer.h." + std::to_string(layer_id) + ".";
        add_f32_tensor(tensors, prefix + "ln_1.weight", {model.config.hidden_size}, layer.ln_1_weight);
        add_f32_tensor(tensors, prefix + "ln_1.bias", {model.config.hidden_size}, layer.ln_1_bias);
        const std::vector<float> c_attn_hf = transpose_linear_to_hf_conv1d(layer.c_attn);
        add_f32_tensor(tensors,
                       prefix + "attn.c_attn.weight",
                       {model.config.hidden_size,
                        model.config.hidden_size * 3},
                       c_attn_hf);
        add_f32_tensor(tensors,
                       prefix + "attn.c_attn.bias",
                       {model.config.hidden_size * 3},
                       layer.c_attn.bias);
        const std::vector<float> c_proj_hf = transpose_linear_to_hf_conv1d(layer.c_proj);
        add_f32_tensor(tensors,
                       prefix + "attn.c_proj.weight",
                       {model.config.hidden_size, model.config.hidden_size},
                       c_proj_hf);
        add_f32_tensor(tensors,
                       prefix + "attn.c_proj.bias",
```

```
                        {model.config.hidden_size},
                        layer.c_proj.bias);
        add_f32_tensor(tensors, prefix + "ln_2.weight", {model.config.hidden_size}, layer.ln_2_weight);
        add_f32_tensor(tensors, prefix + "ln_2.bias", {model.config.hidden_size}, layer.ln_2_bias);
        const std::vector<float> c_fc_hf = transpose_linear_to_hf_conv1d(layer.c_fc);
        add_f32_tensor(tensors,
                        prefix + "mlp.c_fc.weight",
                        {model.config.hidden_size, model.config.intermediate_size},
                        c_fc_hf);
        add_f32_tensor(tensors,
                        prefix + "mlp.c_fc.bias",
                        {model.config.intermediate_size},
                        layer.c_fc.bias);
        const std::vector<float> mlp_proj_hf =
            transpose_linear_to_hf_conv1d(layer.mlp_c_proj);
        add_f32_tensor(tensors,
                        prefix + "mlp.c_proj.weight",
                        {model.config.intermediate_size, model.config.hidden_size},
                        mlp_proj_hf);
        add_f32_tensor(tensors,
                        prefix + "mlp.c_proj.bias",
                        {model.config.hidden_size},
                        layer.mlp_c_proj.bias);
    }
    add_f32_tensor(tensors,
                   "transformer.ln_f.weight",
                   {model.config.hidden_size},
                   model.final_ln_weight);
    add_f32_tensor(tensors,
                   "transformer.ln_f.bias",
                   {model.config.hidden_size},
                   model.final_ln_bias);
    write_safetensors_raw_fixture(path, tensors);
}

void test_tiny_mlp() {
    const lcqi::TinyMlpModel model = lcqi::load_model(test_model_path());
    require(model.input_size == LCQI_TEST_INPUT_SIZE, "input size mismatch");
    require(model.hidden_size == LCQI_TEST_HIDDEN_SIZE, "hidden size mismatch");
    require(model.output_size == LCQI_TEST_OUTPUT_SIZE, "output size mismatch");

    const std::vector<float> input{1.0F, 2.0F, -1.0F, 0.5F};
    const lcqi::InferenceResult result = lcqi::run_inference(model, input);

    require(result.predicted_class == LCQI_TEST_PREDICTED_CLASS, "unexpected
```

```
    ↪ predicted class");
1073    require(result.logits.size() == static_cast<std::size_t>(↩
    ↪ LCQI_TEST_OUTPUT_SIZE),
1074            "unexpected logits size");
1075    require_close(result.logits[0], LCQI_TEST_LOGIT_0, "logit 0 mismatch");
1076    require_close(result.logits[1], LCQI_TEST_LOGIT_1, "logit 1 mismatch");
1077
1078    std::vector<float> hidden(static_cast<std::size_t>(LCQI_TEST_HIDDEN_SIZE), ↩
    ↪ 0.0F);
1079    lcqi::linear_i8(model.hidden, input, hidden);
1080    lcqi::relu_inplace(hidden);
1081    require_close(hidden[0], LCQI_TEST_HIDDEN_0, "hidden 0 mismatch");
1082    require_close(hidden[1], LCQI_TEST_HIDDEN_1, "hidden 1 mismatch");
1083    require_close(hidden[2], LCQI_TEST_HIDDEN_2, "hidden 2 mismatch");
1084 }
1085
1086 void test_tokenizer_contract() {
1087    const lcqi::TokenizerModel tokenizer = lcqi::load_tokenizer(↩
    ↪ test_tokenizer_path());
1088    require(tokenizer.bos_id == 0, "tokenizer BOS id mismatch");
1089    require(tokenizer.eos_id == 4, "tokenizer EOS id mismatch");
1090    require(tokenizer.unk_id == -1, "tokenizer UNK id mismatch");
1091    require(tokenizer.chat_template_hash == "tiny-chat-template-v1",
1092            "tokenizer template hash mismatch");
1093
1094    const std::vector<std::int32_t> token_ids =
1095        lcqi::encode_prompt(tokenizer, "tok1 tok2 tok3");
1096    const std::vector<std::int32_t> expected{0, 1, 2, 3, 4};
1097    require(token_ids == expected, "tokenizer prompt ids mismatch");
1098    require(lcqi::tokenizer_contract_hash(tokenizer) == ↩
    ↪ LCQI_TOKENIZER_HASH_EXPECTED,
1099            "tokenizer contract hash mismatch");
1100 }
1101
1102 void test_tokenizer_rejects_unknown_without_unk() {
1103    const lcqi::TokenizerModel tokenizer = lcqi::load_tokenizer(↩
    ↪ test_tokenizer_path());
1104    bool threw = false;
1105    try {
1106        static_cast<void>(lcqi::encode_prompt(tokenizer, "tok1 missing_token"))↩
    ↪ ;
1107    } catch (const std::runtime_error&) {
1108        threw = true;
1109    }
1110    require(threw, "tokenizer should reject unknown token when unk id is absent↩
    ↪ ");
1111 }
1112
1113 void test_safetensors_manifest_contract() {
1114    const std::filesystem::path path =
1115        temp_file_path("lcqi_safetensors_manifest_contract.safetensors");
1116    const std::string header =
```

```
        R"({"__metadata__":{"format":"lcqi-fixture"},"model.embed_tokens.weight":{"dtype":"F32","shape":[2,3],"data_offsets":[0,24]},"model.layers.0.attn.q_proj.weight":{"dtype":"F16","shape":[4,3],"data_offsets":[24,48]}})";
    write_safetensors_fixture(path, header, LCQI_SAFETENSORS_VALID_PAYLOAD_BYTES);

    const lcqi::SafeTensorManifest manifest = lcqi::load_safetensors_manifest(path);
    std::filesystem::remove(path);

    require(manifest.header_size == static_cast<std::uint64_t>(header.size()),
            "safetensors header size mismatch");
    require(manifest.data_start_offset ==
                static_cast<std::uint64_t>(LCQI_SAFETENSORS_PREFIX_BYTES) +
                    static_cast<std::uint64_t>(header.size()),
            "safetensors data start mismatch");
    require(manifest.metadata.size() == 1, "safetensors metadata count mismatch");
    require(manifest.metadata[0].first == "format" &&
                manifest.metadata[0].second == "lcqi-fixture",
            "safetensors metadata mismatch");
    require(manifest.tensors.size() == 2, "safetensors tensor count mismatch");

    const lcqi::SafeTensorEntry* embedding =
        manifest.find_tensor("model.embed_tokens.weight");
    require(embedding != nullptr, "safetensors embedding tensor missing");
    require(embedding->dtype == "F32", "safetensors embedding dtype mismatch");
    require(embedding->shape == std::vector<std::int64_t>({2, 3}),
            "safetensors embedding shape mismatch");
    require(embedding->data_begin == LCQI_SAFETENSORS_EMBED_BEGIN &&
                embedding->data_end == LCQI_SAFETENSORS_EMBED_END,
            "safetensors embedding offset mismatch");
    require(embedding->byte_size() ==
                LCQI_SAFETENSORS_EMBED_END - LCQI_SAFETENSORS_EMBED_BEGIN,
            "safetensors embedding byte size mismatch");

    const lcqi::SafeTensorEntry* q_proj =
        manifest.find_tensor("model.layers.0.attn.q_proj.weight");
    require(q_proj != nullptr, "safetensors q projection tensor missing");
    require(q_proj->dtype == "F16", "safetensors q projection dtype mismatch");
    require(q_proj->data_begin == LCQI_SAFETENSORS_QPROJ_BEGIN &&
                q_proj->data_end == LCQI_SAFETENSORS_QPROJ_END,
            "safetensors q projection offset mismatch");
}

void test_safetensors_rejects_bad_offsets() {
    const std::filesystem::path path =
        temp_file_path("lcqi_safetensors_bad_offsets.safetensors");
    const std::string header =
        R"({"tensor":{"dtype":"F32","shape":[4],"data_offsets":[0,32]}})";
    write_safetensors_fixture(path, header, 4);
```

```
1163
1164     bool threw = false;
1165     try {
1166         static_cast<void>(lcqi::load_safetensors_manifest(path));
1167     } catch (const std::runtime_error&) {
1168         threw = true;
1169     }
1170     std::filesystem::remove(path);
1171     require(threw, "safetensors parser should reject offsets beyond payload");
1172 }
1173
1174 void test_safetensors_rejects_bad_dtype_shape_size() {
1175     const std::filesystem::path path =
1176         temp_file_path("lcqi_safetensors_bad_dtype_shape_size.safetensors");
1177     const std::string header =
1178         R"({"tensor":{"dtype":"F32","shape":[4],"data_offsets":[0,8]}})";
1179     write_safetensors_fixture(path, header, 8);
1180
1181     bool threw = false;
1182     try {
1183         static_cast<void>(lcqi::load_safetensors_manifest(path));
1184     } catch (const std::runtime_error&) {
1185         threw = true;
1186     }
1187     std::filesystem::remove(path);
1188     require(threw, "safetensors parser should reject dtype/shape byte mismatch"↩
        ↪ );
1189 }
1190
1191 void test_safetensors_reads_float_tensors() {
1192     const std::filesystem::path path =
1193         temp_file_path("lcqi_safetensors_float_read.safetensors");
1194     std::vector<std::uint8_t> f16_bytes;
1195     append_le_u16(f16_bytes, LCQI_TEST_F16_ONE);
1196     append_le_u16(f16_bytes, LCQI_TEST_F16_NEGATIVE_TWO);
1197     std::vector<std::uint8_t> bf16_bytes;
1198     append_le_u16(bf16_bytes, LCQI_TEST_BF16_ONE_POINT_FIVE);
1199     append_le_u16(bf16_bytes, LCQI_TEST_BF16_NEGATIVE_HALF);
1200
1201     const std::array<RawTensorFixture, 3> tensors{{
1202         {"f32", "F32", {2}, f32_payload(std::array<float, 2>{1.25F, -2.5F})},
1203         {"f16", "F16", {2}, f16_bytes},
1204         {"bf16", "BF16", {2}, bf16_bytes},
1205     }};
1206     write_safetensors_raw_fixture(path, tensors);
1207
1208     const lcqi::SafeTensorFile file = lcqi::load_safetensors_file(path);
1209     std::filesystem::remove(path);
1210     const std::vector<float> f32 = lcqi::read_safetensor_f32_tensor(file, "f32"↩
        ↪ );
1211     const std::vector<float> f16 = lcqi::read_safetensor_f32_tensor(file, "f16"↩
        ↪ );
```

```
    const std::vector<float> bf16 = lcqi::read_safetensor_f32_tensor(file, "bf16");

    require_close(f32[0], 1.25F, "safetensors F32 value 0 mismatch");
    require_close(f32[1], -2.5F, "safetensors F32 value 1 mismatch");
    require_close(f16[0], 1.0F, "safetensors F16 value 0 mismatch");
    require_close(f16[1], -2.0F, "safetensors F16 value 1 mismatch");
    require_close(bf16[0], 1.5F, "safetensors BF16 value 0 mismatch");
    require_close(bf16[1], -0.5F, "safetensors BF16 value 1 mismatch");
}

void test_gguf_manifest_reads_q4_k_tensor() {
    const std::filesystem::path path = temp_file_path("lcqi_q4_k_fixture.gguf");
    write_gguf_q4_k_fixture(path);

    const lcqi::GgufManifest manifest = lcqi::load_gguf_manifest(path);
    const lcqi::GgufTensorInfo* tensor =
        manifest.find_tensor("blk.0.ffn_up.weight");
    require(tensor != nullptr, "GGUF Q4_K tensor missing");
    const std::vector<std::uint8_t> bytes =
        lcqi::read_gguf_tensor_bytes(path, *tensor);
    std::filesystem::remove(path);

    require(manifest.version == LCQI_GGUF_TEST_VERSION, "GGUF version mismatch");
    require(manifest.alignment == LCQI_GGUF_TEST_ALIGNMENT, "GGUF alignment mismatch");
    require(manifest.metadata.size() == LCQI_GGUF_TEST_METADATA_COUNT,
            "GGUF metadata count mismatch");
    const lcqi::GgufMetadataEntry* tokens = manifest.find_metadata("tokenizer.ggml.tokens");
    require(tokens != nullptr, "GGUF tokenizer tokens metadata missing");
    require(tokens->string_values == std::vector<std::string>({"<bos>", "hello"}),
            "GGUF tokenizer string array mismatch");
    require(manifest.tensors.size() == LCQI_GGUF_TEST_TENSOR_COUNT,
            "GGUF tensor count mismatch");
    require(tensor->type == lcqi::GgmlType::q4_k, "GGUF tensor type mismatch");
    require(tensor->shape == std::vector<std::int64_t>({lcqi::LCQI_QK_K_BLOCK_VALUES, 1}),
            "GGUF tensor shape mismatch");
    require(tensor->byte_size == lcqi::LCQI_Q4_K_BLOCK_BYTES,
            "GGUF tensor byte size mismatch");
    require(bytes == make_q4_k_test_block(), "GGUF tensor payload mismatch");
}

void test_q4_k_dequant_and_dot_paths() {
    const std::vector<std::uint8_t> block = make_q4_k_test_block();
    const std::vector<float> expected = expected_q4_k_test_values();
    const std::vector<float> decoded =
        lcqi::dequantize_q4_k(block, lcqi::LCQI_QK_K_BLOCK_VALUES);
```

```
1257
1258     require(decoded.size() == expected.size(), "Q4_K decoded size mismatch");
1259     for (std::size_t index = 0; index < expected.size(); ++index) {
1260         require_close(decoded[index], expected[index], "Q4_K decoded value ↩
         ↪ mismatch");
1261     }
1262
1263     std::vector<float> input(expected.size(), 0.0F);
1264     for (std::size_t index = 0; index < input.size(); ++index) {
1265         input[index] = static_cast<float>(static_cast<std::int32_t>(index % 11U↩
         ↪ ) - 5) *
1266                        0.125F;
1267     }
1268     float expected_dot = 0.0F;
1269     for (std::size_t index = 0; index < expected.size(); ++index) {
1270         expected_dot += expected[index] * input[index];
1271     }
1272     const float q4_f32_dot = lcqi::dot_q4_k_f32(block, input);
1273     require_close(q4_f32_dot, expected_dot, "Q4_K F32 dot mismatch");
1274
1275     const std::vector<lcqi::Q8KBlock> q8_input = lcqi::quantize_q8_k_input(↩
         ↪ input);
1276     const float q4_q8_dot = lcqi::dot_q4_k_q8(block, q8_input);
1277     require(std::fabs(q4_q8_dot - q4_f32_dot) <= LCQI_Q4K_Q8_DIFF_LIMIT,
1278             "Q4_K Q8 dot drift is too large");
1279
1280     std::vector<float> matvec_output(1, 0.0F);
1281     lcqi::matvec_q4_k_q8(block, 1, lcqi::LCQI_QK_K_BLOCK_VALUES, q8_input, ↩
         ↪ matvec_output);
1282     require_close(matvec_output[0], q4_q8_dot, "Q4_K Q8 matvec mismatch");
1283 }
1284
1285 void test_ggml_mixed_dequantizers() {
1286     std::vector<std::uint8_t> f32_bytes;
1287     append_le_f32(f32_bytes, 1.25F);
1288     append_le_f32(f32_bytes, -2.5F);
1289     const std::vector<float> f32 =
1290         lcqi::dequantize_ggml_tensor(lcqi::GgmlType::f32, f32_bytes, 2);
1291     require_close(f32[0], 1.25F, "GGML F32 decoded value 0 mismatch");
1292     require_close(f32[1], -2.5F, "GGML F32 decoded value 1 mismatch");
1293
1294     const std::vector<std::uint8_t> q8 = make_q8_0_test_block();
1295     const std::vector<float> q8_values =
1296         lcqi::dequantize_ggml_tensor(lcqi::GgmlType::q8_0,
1297                                      q8,
1298                                      lcqi::LCQI_Q8_0_BLOCK_VALUES);
1299     require_close(q8_values[0], -8.0F, "Q8_0 decoded value 0 mismatch");
1300     require_close(q8_values[16], 0.0F, "Q8_0 decoded value 16 mismatch");
1301     require_close(q8_values[31], 7.5F, "Q8_0 decoded value 31 mismatch");
1302
1303     const std::vector<std::uint8_t> q5 = make_q5_0_test_block();
1304     const std::vector<float> q5_values =
```

```
1305         lcqi::dequantize_ggml_tensor(lcqi::GgmlType::q5_0,
1306                                      q5,
1307                                      lcqi::LCQI_Q5_0_BLOCK_VALUES);
1308     require_close(q5_values[0], -8.0F, "Q5_0 decoded value 0 mismatch");
1309     require_close(q5_values[1], -7.5F, "Q5_0 decoded value 1 mismatch");
1310     require_close(q5_values[15], -0.5F, "Q5_0 decoded value 15 mismatch");
1311     require_close(q5_values[16], 0.0F, "Q5_0 decoded value 16 mismatch");
1312     require_close(q5_values[31], 7.5F, "Q5_0 decoded value 31 mismatch");
1313 
1314     const std::vector<std::uint8_t> q6 = make_q6_k_test_block();
1315     const std::vector<float> q6_values =
1316         lcqi::dequantize_ggml_tensor(lcqi::GgmlType::q6_k,
1317                                      q6,
1318                                      lcqi::LCQI_Q6_K_BLOCK_VALUES);
1319     require_close(q6_values[LCQI_GGML_TEST_Q6_LANE], 3.0F, "Q6_K q1 mismatch");
1320     require_close(q6_values[32 + LCQI_GGML_TEST_Q6_LANE], -1.0F, "Q6_K q2 ↩
        ↪ mismatch");
1321     require_close(q6_values[64 + LCQI_GGML_TEST_Q6_LANE], 31.0F, "Q6_K q3 ↩
        ↪ mismatch");
1322     require_close(q6_values[96 + LCQI_GGML_TEST_Q6_LANE], -32.0F, "Q6_K q4 ↩
        ↪ mismatch");
1323     require_close(q6_values[128 + LCQI_GGML_TEST_Q6_LANE], 4.0F, "Q6_K second ↩
        ↪ half mismatch");
1324 
1325     bool rejected_bad_size = false;
1326     try {
1327         static_cast<void>(lcqi::dequantize_ggml_tensor(
1328             lcqi::GgmlType::q8_0,
1329             std::span<const std::uint8_t>(q8.data(), q8.size() - 1U),
1330             lcqi::LCQI_Q8_0_BLOCK_VALUES));
1331     } catch (const std::runtime_error&) {
1332         rejected_bad_size = true;
1333     }
1334     require(rejected_bad_size, "GGML dequantizer accepted a truncated Q8_0 ↩
        ↪ block");
1335 }
1336 
1337 void test_ggml_quantized_f32_matvec_paths() {
1338     const std::vector<float> input32 = [] {
1339         std::vector<float> values(static_cast<std::size_t>(lcqi::↩
        ↪ LCQI_Q8_0_BLOCK_VALUES), 0.0F);
1340         for (std::size_t index = 0; index < values.size(); ++index) {
1341             values[index] =
1342                 static_cast<float>(static_cast<std::int32_t>(index % 9U) - 4) *↩
        ↪  0.125F;
1343         }
1344         return values;
1345     }();
1346     const std::vector<float> input256 = [] {
1347         std::vector<float> values(static_cast<std::size_t>(lcqi::↩
        ↪ LCQI_Q6_K_BLOCK_VALUES), 0.0F);
1348         for (std::size_t index = 0; index < values.size(); ++index) {
```

```
            values[index] =
                static_cast<float>(static_cast<std::int32_t>(index % 13U) - 6) * 0.0625F;
        }
        return values;
    }();

    const auto assert_matvec_matches_dequantized =
        [](lcqi::GgmlType type,
           const std::vector<std::uint8_t>& block,
           std::span<const float> input,
           const char* message) {
            const std::int64_t columns = static_cast<std::int64_t>(input.size());
            const std::vector<std::uint8_t> matrix = [&] {
                std::vector<std::uint8_t> bytes;
                bytes.reserve(block.size() * 2U);
                bytes.insert(bytes.end(), block.begin(), block.end());
                bytes.insert(bytes.end(), block.begin(), block.end());
                return bytes;
            }();
            std::vector<float> direct(2, 0.0F);
            lcqi::matvec_ggml_quantized_f32(type, matrix, 2, columns, input, direct);
            const std::vector<lcqi::Q8_0InputBlock> q8_0_input =
                lcqi::quantize_q8_0_input(input);
            std::vector<float> q8_0_direct(2, 0.0F);
            lcqi::matvec_ggml_quantized_q8_0(type,
                                             matrix,
                                             2,
                                             columns,
                                             q8_0_input,
                                             q8_0_direct);
            std::vector<float> q8_0_row_range_direct(2, 0.0F);
            lcqi::matvec_ggml_quantized_q8_0_rows_unchecked(type,
                                                            matrix,
                                                            2,
                                                            columns,
                                                            q8_0_input,
                                                            q8_0_row_range_direct,
                                                            0,
                                                            1);
            lcqi::matvec_ggml_quantized_q8_0_rows_unchecked(type,
                                                            matrix,
                                                            2,
                                                            columns,
                                                            q8_0_input,
                                                            q8_0_row_range_direct,
                                                            1,
                                                            2);
```

```
1396            constexpr std::size_t LCQI_GGML_DIRECT_BATCH_SIZE = 3;
1397            std::vector<lcqi::Q8_0InputBlock> batch_q8_0_input(
1398                LCQI_GGML_DIRECT_BATCH_SIZE * q8_0_input.size());
1399            std::vector<float> expected_batch(LCQI_GGML_DIRECT_BATCH_SIZE * 2U,↩
    ↪  0.0F);
1400            std::vector<float> direct_batch(expected_batch.size(), 0.0F);
1401            for (std::size_t batch = 0; batch < LCQI_GGML_DIRECT_BATCH_SIZE; ++↩
    ↪ batch) {
1402                std::vector<float> batch_input(input.begin(), input.end());
1403                for (std::size_t index = 0; index < batch_input.size(); ++index↩
    ↪ ) {
1404                    batch_input[index] +=
1405                        static_cast<float>(static_cast<std::int32_t>(batch) - ↩
    ↪ 1) *
1406                        0.015625F;
1407                }
1408                const std::vector<lcqi::Q8_0InputBlock> batch_quantized =
1409                    lcqi::quantize_q8_0_input(batch_input);
1410                std::copy(batch_quantized.begin(),
1411                          batch_quantized.end(),
1412                          batch_q8_0_input.begin() +
1413                              static_cast<std::ptrdiff_t>(batch * ↩
    ↪ batch_quantized.size()));
1414                lcqi::matvec_ggml_quantized_q8_0(
1415                    type,
1416                    matrix,
1417                    2,
1418                    columns,
1419                    batch_quantized,
1420                    std::span<float>(expected_batch).subspan(batch * 2U, 2U));
1421            }
1422            lcqi::matvec_ggml_quantized_q8_0_batch_rows_unchecked(
1423                type,
1424                matrix,
1425                2,
1426                columns,
1427                batch_q8_0_input,
1428                static_cast<std::int64_t>(LCQI_GGML_DIRECT_BATCH_SIZE),
1429                direct_batch,
1430                2,
1431                0,
1432                2);
1433            const lcqi::F32RowMax q8_0_max =
1434                lcqi::max_dot_ggml_quantized_q8_0(type,
1435                                                  matrix,
1436                                                  2,
1437                                                  columns,
1438                                                  q8_0_input);
1439            const lcqi::F32RowMax q8_0_subrange_max =
1440                lcqi::max_dot_ggml_quantized_q8_0_rows_unchecked(type,
1441                                                                 matrix,
1442                                                                 2,
```

```
                                                                columns,
                                                                q8_0_input,
                                                                1,
                                                                2);
            const std::vector<float> weights =
                lcqi::dequantize_ggml_tensor(type, matrix, columns * 2);
            std::array<float, 2> reference{0.0F, 0.0F};
            for (std::size_t row = 0; row < reference.size(); ++row) {
                for (std::size_t column = 0; column < input.size(); ++column) {
                    reference[row] +=
                        weights[row * input.size() + column] * input[column];
                }
                if (std::fabs(reference[row] - direct[row]) > LCQI_GGML_MATVEC_MAX_DIFF) {
                    throw std::runtime_error(message);
                }
                if (std::fabs(reference[row] - q8_0_direct[row]) >
                    LCQI_GGML_Q8_MATVEC_MAX_DIFF) {
                    throw std::runtime_error(message);
                }
                if (std::fabs(q8_0_direct[row] - q8_0_row_range_direct[row]) >
                    LCQI_GGML_Q8_MATVEC_MAX_DIFF) {
                    throw std::runtime_error(message);
                }
            }
            require(q8_0_max.row == 0U, "GGML Q8_0 max-dot selected the wrong row");
            require(std::fabs(q8_0_max.value - q8_0_direct[0]) <=
                        LCQI_GGML_Q8_MATVEC_MAX_DIFF,
                    "GGML Q8_0 max-dot value differs from matvec output");
            require(q8_0_subrange_max.row == 1U,
                    "GGML Q8_0 row-range max-dot selected the wrong row");
            require(std::fabs(q8_0_subrange_max.value - q8_0_direct[1]) <=
                        LCQI_GGML_Q8_MATVEC_MAX_DIFF,
                    "GGML Q8_0 row-range max-dot value differs from matvec output");
            for (std::size_t index = 0; index < direct_batch.size(); ++index) {
                if (std::fabs(expected_batch[index] - direct_batch[index]) >
                    LCQI_GGML_Q8_MATVEC_MAX_DIFF) {
                    throw std::runtime_error(message);
                }
            }
        };

    require(lcqi::ggml_type_has_f32_direct_matvec(lcqi::GgmlType::q5_0),
            "Q5_0 direct matvec support flag mismatch");
    require(lcqi::ggml_type_has_f32_direct_matvec(lcqi::GgmlType::q6_k),
            "Q6_K direct matvec support flag mismatch");
    require(lcqi::ggml_type_has_f32_direct_matvec(lcqi::GgmlType::q8_0),
            "Q8_0 direct matvec support flag mismatch");
    require(!lcqi::ggml_type_has_f32_direct_matvec(lcqi::GgmlType::f32),
            "F32 should not be reported as quantized direct matvec");
```

```
1492     require(lcqi::ggml_type_has_q8_0_direct_matvec(lcqi::GgmlType::q5_0),
1493             "Q5_0 Q8_0 direct matvec support flag mismatch");
1494     require(lcqi::ggml_type_has_q8_0_direct_matvec(lcqi::GgmlType::q6_k),
1495             "Q6_K Q8_0 direct matvec support flag mismatch");
1496     require(lcqi::ggml_type_has_q8_0_direct_matvec(lcqi::GgmlType::q8_0),
1497             "Q8_0 Q8_0 direct matvec support flag mismatch");
1498     require(!lcqi::ggml_type_has_q8_0_direct_matvec(lcqi::GgmlType::f32),
1499             "F32 should not be reported as Q8_0 quantized direct matvec");
1500
1501     assert_matvec_matches_dequantized(lcqi::GgmlType::q8_0,
1502                                       make_q8_0_test_block(),
1503                                       input32,
1504                                       "Q8_0 direct matvec mismatch");
1505     assert_matvec_matches_dequantized(lcqi::GgmlType::q5_0,
1506                                       make_q5_0_test_block(),
1507                                       input32,
1508                                       "Q5_0 direct matvec mismatch");
1509     assert_matvec_matches_dequantized(lcqi::GgmlType::q6_k,
1510                                       make_q6_k_test_block(),
1511                                       input256,
1512                                       "Q6_K direct matvec mismatch");
1513 }
1514
1515 void test_q5_0_packed_matvec_matches_ggml_q8_0_path() {
1516     std::vector<float> input(static_cast<std::size_t>(lcqi::↩
        ↪ LCQI_Q5_0_BLOCK_VALUES), 0.0F);
1517     for (std::size_t index = 0; index < input.size(); ++index) {
1518         input[index] =
1519             static_cast<float>(static_cast<std::int32_t>(index % 11U) - 5) * ↩
        ↪ 0.0625F;
1520     }
1521     const std::vector<std::uint8_t> block = make_q5_0_test_block();
1522     std::vector<std::uint8_t> matrix;
1523     matrix.reserve(block.size() * 2U);
1524     matrix.insert(matrix.end(), block.begin(), block.end());
1525     matrix.insert(matrix.end(), block.begin(), block.end());
1526
1527     const std::vector<lcqi::Q8_0InputBlock> q8_0_input = lcqi::↩
        ↪ quantize_q8_0_input(input);
1528     std::vector<float> ggml_direct(2, 0.0F);
1529     lcqi::matvec_ggml_quantized_q8_0(lcqi::GgmlType::q5_0,
1530                                      matrix,
1531                                      2,
1532                                      lcqi::LCQI_Q5_0_BLOCK_VALUES,
1533                                      q8_0_input,
1534                                      ggml_direct);
1535     const std::vector<lcqi::Q5_0PackedBlock> packed =
1536         lcqi::pack_q5_0_blocks(matrix, lcqi::LCQI_Q5_0_BLOCK_VALUES * 2);
1537     std::vector<float> packed_direct(2, 0.0F);
1538     lcqi::matvec_q5_0_packed_q8_0(packed,
1539                                   2,
1540                                   lcqi::LCQI_Q5_0_BLOCK_VALUES,
```

```
                                    q8_0_input,
                                    packed_direct);
    std::vector<float> row_range_direct(2, 0.0F);
    lcqi::matvec_q5_0_packed_q8_0_rows_unchecked(packed,
                                                 2,
                                                 lcqi::LCQI_Q5_0_BLOCK_VALUES,
                                                 q8_0_input,
                                                 row_range_direct,
                                                 0,
                                                 1);
    lcqi::matvec_q5_0_packed_q8_0_rows_unchecked(packed,
                                                 2,
                                                 lcqi::LCQI_Q5_0_BLOCK_VALUES,
                                                 q8_0_input,
                                                 row_range_direct,
                                                 1,
                                                 2);
    const auto assert_packed_batch_matches_per_token =
        [&](std::size_t batch_size, const char* message) {
            std::vector<lcqi::Q8_0InputBlock> batch_q8_0_input(
                batch_size * q8_0_input.size());
            std::vector<float> expected_batch(batch_size * ggml_direct.size(), 0.0F);
            std::vector<float> packed_batch(expected_batch.size(), 0.0F);
            for (std::size_t batch = 0; batch < batch_size; ++batch) {
                std::vector<float> batch_input = input;
                for (std::size_t index = 0; index < batch_input.size(); ++index) {
                    batch_input[index] +=
                        static_cast<float>(static_cast<std::int32_t>(batch) - 1) * 0.03125F;
                }
                const std::vector<lcqi::Q8_0InputBlock> batch_quantized =
                    lcqi::quantize_q8_0_input(batch_input);
                std::copy(batch_quantized.begin(),
                          batch_quantized.end(),
                          batch_q8_0_input.begin() +
                              static_cast<std::ptrdiff_t>(batch * batch_quantized.size()));
                lcqi::matvec_q5_0_packed_q8_0(
                    packed,
                    2,
                    lcqi::LCQI_Q5_0_BLOCK_VALUES,
                    batch_quantized,
                    std::span<float>(expected_batch)
                        .subspan(batch * ggml_direct.size(), ggml_direct.size()));
            }
            lcqi::matvec_q5_0_packed_q8_0_batch_rows_unchecked(
                packed,
                2,
                lcqi::LCQI_Q5_0_BLOCK_VALUES,
```

```
                batch_q8_0_input,
                static_cast<std::int64_t>(batch_size),
                packed_batch,
                static_cast<std::int64_t>(ggml_direct.size()),
                0,
                2);
            for (std::size_t index = 0; index < packed_batch.size(); ++index) {
                require(std::fabs(expected_batch[index] - packed_batch[index]) <=
                            LCQI_GGML_Q8_MATVEC_MAX_DIFF,
                        message);
            }
        };
    assert_packed_batch_matches_per_token(3, "Q5_0 packed batch tail-3 output mismatch");
    assert_packed_batch_matches_per_token(5, "Q5_0 packed batch 4+1 output mismatch");
    assert_packed_batch_matches_per_token(11, "Q5_0 packed batch 8+3 output mismatch");
    for (std::size_t row = 0; row < ggml_direct.size(); ++row) {
        require(std::fabs(ggml_direct[row] - packed_direct[row]) <=
                    LCQI_GGML_Q8_MATVEC_MAX_DIFF,
                "Q5_0 packed output differs from GGML Q8_0 direct path");
        require(std::fabs(packed_direct[row] - row_range_direct[row]) <=
                    LCQI_GGML_Q8_MATVEC_MAX_DIFF,
                "Q5_0 packed row range output differs from full path");
    }
}

void test_llama_gguf_loader_and_generation() {
    const std::filesystem::path path = temp_file_path("lcqi_tiny_llama_fixture.gguf");
    write_tiny_llama_gguf_fixture(path);

    const lcqi::LlamaGgufLoadedModel loaded =
        lcqi::load_llama_gguf_reference_model(path);
    const std::vector<std::int32_t> prompt{1};
    const lcqi::LlamaGgufGenerationResult result =
        lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1);
    std::filesystem::remove(path);

    require(loaded.model.architecture == "llama", "LLaMA GGUF architecture mismatch");
    require(loaded.model.decoder.config.hidden_size == LCQI_LLAMA_TEST_HIDDEN,
            "LLaMA GGUF hidden size mismatch");
    require(loaded.model.decoder.layers.size() == LCQI_LLAMA_TEST_LAYER_COUNT,
            "LLaMA GGUF layer count mismatch");
    require(loaded.model.lm_head_tied_to_embedding,
            "LLaMA GGUF tiny fixture should tie lm head to embeddings");
    require(result.generated_ids == std::vector<std::int32_t>({1, LCQI_LLAMA_TEST_EXPECTED_TOKEN}),
            "LLaMA GGUF generated ids mismatch");
```

```
1633     require(result.predicted_first_token == LCQI_LLAMA_TEST_EXPECTED_TOKEN,
1634             "LLaMA GGUF predicted token mismatch");
1635     require(loaded.report.tensors_loaded == 11, "LLaMA GGUF tensor load count ↩
         ↪ mismatch");
1636     require(loaded.report.f32_weight_bytes == loaded.report.↩
         ↪ quantized_weight_bytes,
1637             "LLaMA GGUF F32 fixture byte accounting mismatch");
1638     require(loaded.report.q4_k_direct_tensors == 0,
1639             "LLaMA GGUF F32 fixture should not use direct Q4_K tensors");
1640     require(loaded.report.ggml_direct_tensors == 0,
1641             "LLaMA GGUF F32 fixture should not use direct GGML tensors");
1642     require(result.hotspots.f32_fallback_calls == 7,
1643             "LLaMA GGUF F32 fixture linear fallback call count mismatch");
1644     require(result.hotspots.q4_k_direct_calls == 0,
1645             "LLaMA GGUF F32 fixture should not call Q4_K direct matvec");
1646     require(result.hotspots.ggml_direct_calls == 0,
1647             "LLaMA GGUF F32 fixture should not call GGML direct matvec");
1648 }
1649
1650 void test_llama_gguf_default_packs_q5_0_tensors() {
1651     const std::filesystem::path path = temp_file_path("↩
         ↪ lcqi_tiny_llama_q5_0_packed.gguf");
1652     write_tiny_llama_ggml_direct_gguf_fixture(path);
1653
1654     const ScopedEnvVar disable_ggml_direct(LCQI_TEST_LLAMA_GGML_DIRECT_ENV,
1655                                            LCQI_TEST_FALSE_ENV);
1656     const ScopedEnvVar disable_selective_ggml_direct(↩
         ↪ LCQI_TEST_LLAMA_GGML_SELECTIVE_DIRECT_ENV,
1657                                                      LCQI_TEST_FALSE_ENV);
1658     const ScopedEnvVar enable_q5_0_packed(LCQI_TEST_LLAMA_Q5_0_PACKED_ENV,
1659                                           LCQI_TEST_TRUE_ENV);
1660     const ScopedEnvVar enable_q8_0_direct(LCQI_TEST_LLAMA_Q8_0_DIRECT_ENV,
1661                                           LCQI_TEST_TRUE_ENV);
1662     const lcqi::LlamaGgufLoadedModel loaded =
1663         lcqi::load_llama_gguf_reference_model(path);
1664     const std::vector<std::int32_t> prompt{0};
1665     const lcqi::LlamaGgufGenerationResult result =
1666         lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1);
1667     std::filesystem::remove(path);
1668
1669     require(loaded.report.q5_0_packed_tensors == 4,
1670             "default LLaMA GGUF path should pack Q5_0 tensors");
1671     require(loaded.report.ggml_direct_tensors == 0,
1672             "default LLaMA GGUF path should not use experimental GGML direct ↩
         ↪ tensors");
1673     require(loaded.report.q8_0_direct_tensors == 2,
1674             "default LLaMA GGUF path should keep Q8_0 tensors direct");
1675     require(loaded.report.f32_fallback_tensors == 5,
1676             "default LLaMA GGUF path should leave F32 and Q6_K tensors in ↩
         ↪ fallback");
1677     require(result.hotspots.q5_0_packed_calls == 4,
1678             "default LLaMA GGUF path Q5_0 packed call count mismatch");
```

```
    require(result.hotspots.q8_0_direct_calls == 2,
            "default LLaMA GGUF path Q8_0 direct call count mismatch");
    require(result.hotspots.f32_fallback_calls == 1,
            "default LLaMA GGUF path fallback call count mismatch");
    require(result.hotspots.ggml_direct_calls == 0,
            "default LLaMA GGUF path should not call experimental GGML direct");
    require(result.generated_ids == std::vector<std::int32_t>({0, 0}),
            "default LLaMA GGUF Q5_0 packed generated ids mismatch");
}

void test_llama_gguf_batch_prefill_uses_q5_0_f32_sidecar_path() {
    const std::filesystem::path path =
        temp_file_path("lcqi_tiny_llama_q5_0_batch_prefill.gguf");
    write_tiny_llama_ggml_direct_gguf_fixture(path);

    const ScopedEnvVar disable_ggml_direct(LCQI_TEST_LLAMA_GGML_DIRECT_ENV,
                                           LCQI_TEST_FALSE_ENV);
    const ScopedEnvVar disable_selective_ggml_direct(
        LCQI_TEST_LLAMA_GGML_SELECTIVE_DIRECT_ENV,
                                                     LCQI_TEST_FALSE_ENV);
    const ScopedEnvVar enable_q5_0_packed(LCQI_TEST_LLAMA_Q5_0_PACKED_ENV,
                                          LCQI_TEST_TRUE_ENV);
    const ScopedEnvVar enable_q8_0_direct(LCQI_TEST_LLAMA_Q8_0_DIRECT_ENV,
                                          LCQI_TEST_TRUE_ENV);
    const lcqi::LlamaGgufLoadedModel loaded =
        lcqi::load_llama_gguf_reference_model(path);
    const std::vector<std::int32_t> prompt{0, 0, 0};
    {
        const ScopedEnvVar disable_batch_prefill(
        LCQI_TEST_LLAMA_BATCH_PREFILL_ENV,
                                                 LCQI_TEST_FALSE_ENV);
        const lcqi::LlamaGgufGenerationResult token_path =
            lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1);
        const ScopedEnvVar enable_batch_prefill(
        LCQI_TEST_LLAMA_BATCH_PREFILL_ENV,
                                                LCQI_TEST_TRUE_ENV);
        const lcqi::LlamaGgufGenerationResult batch_path =
            lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1);
        require(batch_path.generated_ids == token_path.generated_ids,
                "Q5_0 packed batch prefill generated ids differ from token path");
        require(batch_path.predicted_first_token == token_path.
        predicted_first_token,
                "Q5_0 packed batch prefill predicted token differs from token
        path");
        require(batch_path.hotspots.q5_0_packed_calls == 0,
                "Q5_0 batch prefill should not use packed decode kernels");
        require(batch_path.hotspots.q5_0_batch_f32_calls ==
                    static_cast<std::uint64_t>(loaded.report.
        q5_0_packed_tensors),
                "Q5_0 batch prefill should call each F32 sidecar tensor once");
```

```
        require(token_path.hotspots.q5_0_packed_calls >
                    batch_path.hotspots.q5_0_packed_calls,
                "Q5_0 batch prefill should leave packed matvecs to token/decode path");
    }
    std::filesystem::remove(path);
}

void test_llama_gguf_parallel_generation_matches_serial() {
    const std::filesystem::path path = temp_file_path("lcqi_tiny_llama_parallel.gguf");
    write_tiny_llama_gguf_fixture(path);

    const lcqi::LlamaGgufLoadedModel loaded =
        lcqi::load_llama_gguf_reference_model(path);
    const std::vector<std::int32_t> prompt{1};
    const lcqi::LlamaGgufGenerationResult serial =
        lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1);
    lcqi::LlamaGgufExecutionOptions parallel_options;
    parallel_options.worker_count = 2;
    parallel_options.parallel_min_rows = 1;
    const lcqi::LlamaGgufGenerationResult parallel =
        lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1, parallel_options);
    std::filesystem::remove(path);

    require(parallel.worker_count == parallel_options.worker_count,
            "parallel LLaMA worker count mismatch");
    require(parallel.generated_ids == serial.generated_ids,
            "parallel LLaMA generated ids mismatch");
    require(parallel.predicted_first_token == serial.predicted_first_token,
            "parallel LLaMA predicted token mismatch");
    require(parallel.hotspots.f32_fallback_calls == serial.hotspots.f32_fallback_calls,
            "parallel LLaMA fallback call count mismatch");
    require(parallel.hotspots.q4_k_direct_calls == serial.hotspots.q4_k_direct_calls,
            "parallel LLaMA Q4 direct call count mismatch");
    require(parallel.hotspots.ggml_direct_calls == serial.hotspots.ggml_direct_calls,
            "parallel LLaMA GGML direct call count mismatch");
}

void test_llama_gguf_q4_direct_generation() {
    const std::filesystem::path path = temp_file_path("lcqi_tiny_llama_q4_direct.gguf");
    write_tiny_llama_q4_direct_gguf_fixture(path);

    const lcqi::LlamaGgufLoadedModel loaded =
        lcqi::load_llama_gguf_reference_model(path);
    const std::vector<std::int32_t> prompt{0};
    const lcqi::LlamaGgufGenerationResult result =
```

```
1768         lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1);
1769     std::filesystem::remove(path);
1770
1771     require(loaded.model.decoder.config.hidden_size == ↩
     ↪ LCQI_LLAMA_Q4_TEST_HIDDEN,
1772             "LLaMA Q4_K fixture hidden size mismatch");
1773     require(loaded.report.tensors_loaded == 11, "LLaMA Q4_K fixture tensor load↩
     ↪  count mismatch");
1774     require(loaded.report.q4_k_direct_tensors == 7,
1775             "LLaMA Q4_K fixture direct tensor count mismatch");
1776     require(loaded.report.ggml_direct_tensors == 0,
1777             "LLaMA Q4_K fixture should not use GGML direct tensors");
1778     require(loaded.report.f32_fallback_tensors == 4,
1779             "LLaMA Q4_K fixture fallback tensor count mismatch");
1780     require(loaded.report.direct_quantized_weight_bytes >
1781                 loaded.report.fallback_dequantized_weight_bytes,
1782             "LLaMA Q4_K fixture should keep direct quantized matrix bytes");
1783     require(result.hotspots.q4_k_direct_calls == 7,
1784             "LLaMA Q4_K fixture direct matvec call count mismatch");
1785     require(result.hotspots.ggml_direct_calls == 0,
1786             "LLaMA Q4_K fixture should not call GGML direct matvec");
1787     require(result.hotspots.f32_fallback_calls == 0,
1788             "LLaMA Q4_K fixture should not call fallback linear matvec");
1789     require(result.generated_ids == std::vector<std::int32_t>({0, 0}),
1790             "LLaMA Q4_K fixture generated ids mismatch");
1791 }
1792
1793 void test_llama_gguf_ggml_direct_generation() {
1794     const ScopedEnvVar enable_ggml_direct(LCQI_TEST_LLAMA_GGML_DIRECT_ENV,
1795                                           LCQI_TEST_TRUE_ENV);
1796     const std::filesystem::path path = temp_file_path("↩
     ↪ lcqi_tiny_llama_ggml_direct.gguf");
1797     write_tiny_llama_ggml_direct_gguf_fixture(path);
1798
1799     const lcqi::LlamaGgufLoadedModel loaded =
1800         lcqi::load_llama_gguf_reference_model(path);
1801     const std::vector<std::int32_t> prompt{0};
1802     const lcqi::LlamaGgufGenerationResult result =
1803         lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1);
1804     std::filesystem::remove(path);
1805
1806     require(loaded.model.decoder.config.hidden_size == ↩
     ↪ LCQI_LLAMA_Q4_TEST_HIDDEN,
1807             "LLaMA GGML fixture hidden size mismatch");
1808     require(loaded.report.tensors_loaded == 11, "LLaMA GGML fixture tensor load↩
     ↪  count mismatch");
1809     require(loaded.report.q4_k_direct_tensors == 0,
1810             "LLaMA GGML fixture should not use Q4_K direct tensors");
1811     require(loaded.report.ggml_direct_tensors == 7,
1812             "LLaMA GGML fixture direct tensor count mismatch");
1813     require(loaded.report.f32_fallback_tensors == 4,
1814             "LLaMA GGML fixture fallback tensor count mismatch");
```

```
    require(result.hotspots.ggml_direct_calls == 7,
            "LLaMA GGML fixture direct matvec call count mismatch");
    require(result.hotspots.q4_k_direct_calls == 0,
            "LLaMA GGML fixture should not call Q4_K direct matvec");
    require(result.hotspots.f32_fallback_calls == 0,
            "LLaMA GGML fixture should not call fallback linear matvec");
    require(result.generated_ids == std::vector<std::int32_t>({0, 0}),
            "LLaMA GGML fixture generated ids mismatch");
}

void test_llama_gguf_selective_ggml_direct_keeps_q6_fallback() {
    const ScopedEnvVar enable_selective_ggml_direct(
        LCQI_TEST_LLAMA_GGML_SELECTIVE_DIRECT_ENV,
        LCQI_TEST_TRUE_ENV);
    const std::filesystem::path path = temp_file_path("lcqi_tiny_llama_selective_ggml.gguf");
    write_tiny_llama_ggml_direct_gguf_fixture(path);

    const lcqi::LlamaGgufLoadedModel loaded =
        lcqi::load_llama_gguf_reference_model(path);
    const std::vector<std::int32_t> prompt{0};
    const lcqi::LlamaGgufGenerationResult result =
        lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1);
    std::filesystem::remove(path);

    require(loaded.report.q4_k_direct_tensors == 0,
            "selective GGML fixture should not use Q4_K direct tensors");
    require(loaded.report.ggml_direct_tensors == 6,
            "selective GGML fixture should only keep Q5_0/Q8_0 tensors direct");
    require(loaded.report.f32_fallback_tensors == 5,
            "selective GGML fixture should dequantize Q6_K through the F32 fallback");
    require(result.hotspots.ggml_direct_calls == 6,
            "selective GGML fixture direct call count mismatch");
    require(result.hotspots.f32_fallback_calls == 1,
            "selective GGML fixture Q6_K fallback call count mismatch");
    require(result.hotspots.q4_k_direct_calls == 0,
            "selective GGML fixture should not call Q4_K direct matvec");
    require(result.generated_ids == std::vector<std::int32_t>({0, 0}),
            "selective GGML fixture generated ids mismatch");
}

void test_llama_gguf_q6_k_direct_experimental_path() {
    const ScopedEnvVar disable_ggml_direct(LCQI_TEST_LLAMA_GGML_DIRECT_ENV,
                                           LCQI_TEST_FALSE_ENV);
    const ScopedEnvVar disable_selective_ggml_direct(LCQI_TEST_LLAMA_GGML_SELECTIVE_DIRECT_ENV,
                                                     LCQI_TEST_FALSE_ENV);
    const ScopedEnvVar enable_q5_0_packed(LCQI_TEST_LLAMA_Q5_0_PACKED_ENV,
                                          LCQI_TEST_TRUE_ENV);
    const ScopedEnvVar enable_q8_0_direct(LCQI_TEST_LLAMA_Q8_0_DIRECT_ENV,
```

```
                                            LCQI_TEST_TRUE_ENV);
    const ScopedEnvVar enable_q6_k_direct(LCQI_TEST_LLAMA_Q6_K_DIRECT_ENV,
                                            LCQI_TEST_TRUE_ENV);
    const std::filesystem::path path =
        temp_file_path("lcqi_tiny_llama_q6_k_direct.gguf");
    write_tiny_llama_ggml_direct_gguf_fixture(path);

    const lcqi::LlamaGgufLoadedModel loaded =
        lcqi::load_llama_gguf_reference_model(path);
    const std::vector<std::int32_t> prompt{0};
    const lcqi::LlamaGgufGenerationResult result =
        lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1);
    std::filesystem::remove(path);

    require(loaded.report.q5_0_packed_tensors == 4,
            "Q6_K direct fixture should still pack Q5_0 tensors");
    require(loaded.report.q6_k_direct_tensors == 1,
            "Q6_K direct fixture should keep the Q6_K tensor direct");
    require(loaded.report.q8_0_direct_tensors == 2,
            "Q6_K direct fixture should still keep Q8_0 tensors direct");
    require(loaded.report.f32_fallback_tensors == 4,
            "Q6_K direct fixture should remove Q6_K from the F32 fallback set");
    require(result.hotspots.q6_k_direct_calls == 1,
            "Q6_K direct fixture should call the Q6_K direct path once");
    require(result.hotspots.f32_fallback_calls == 0,
            "Q6_K direct fixture should not call fallback linear matvec");
    require(result.generated_ids == std::vector<std::int32_t>({0, 0}),
            "Q6_K direct fixture generated ids mismatch");
}

void test_gpt2_tiny_forward_and_generation() {
    const lcqi::Gpt2ReferenceModel model = lcqi::make_tiny_gpt2_reference_model();
    const std::vector<std::int32_t> tokens{1, 2, 3};
    const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(model, tokens);

    require(result.logits.size() == LCQI_GPT2_VOCAB_SIZE, "GPT-2 logits size mismatch");
    require(result.predicted_token == LCQI_GPT2_PREDICTED_TOKEN,
            "GPT-2 predicted token mismatch");
    for (std::size_t index = 0; index < LCQI_GPT2_LOGITS.size(); ++index) {
        require_close(result.logits[index], LCQI_GPT2_LOGITS[index], "GPT-2 logit mismatch");
    }

    const std::vector<std::int32_t> generated =
        lcqi::gpt2_generate_greedy(model, tokens, 2);
    const std::vector<std::int32_t> expected{1, 2, 3, 3, 3};
    require(generated == expected, "GPT-2 greedy generation mismatch");
    require(static_cast<std::int32_t>(generated.size()) ==
```

```
        ↪ LCQI_GPT2_GENERATED_LENGTH,
1910            "GPT-2 generated length mismatch");
1911
1912    lcqi::Gpt2KvCache cache(model.config);
1913    lcqi::Gpt2ForwardResult cached_result;
1914    for (const std::int32_t token : tokens) {
1915        cached_result = lcqi::run_gpt2_forward_cached(model, cache, token);
1916    }
1917    require(cache.filled_tokens() == static_cast<std::int32_t>(tokens.size()),
1918            "GPT-2 KV cache filled token count mismatch");
1919    require(cache.byte_size() > 0, "GPT-2 KV cache byte size should be non-zero↩
        ↪ ");
1920    require(cached_result.predicted_token == result.predicted_token,
1921            "cached GPT-2 predicted token mismatch");
1922    require(cached_result.logits.size() == result.logits.size(),
1923            "cached GPT-2 logits size mismatch");
1924    for (std::size_t index = 0; index < result.logits.size(); ++index) {
1925        require_close(cached_result.logits[index],
1926                      result.logits[index],
1927                      "cached GPT-2 logit mismatch");
1928    }
1929
1930    lcqi::Gpt2CachedGreedyDecoder decoder_with_logits(model);
1931    lcqi::Gpt2ForwardResult decoder_result;
1932    for (const std::int32_t token : tokens) {
1933        decoder_result = decoder_with_logits.step_with_logits(token);
1934    }
1935    require(decoder_with_logits.filled_tokens() == static_cast<std::int32_t>(↩
        ↪ tokens.size()),
1936            "GPT-2 optimized decoder filled token count mismatch");
1937    require(decoder_with_logits.kv_cache_bytes() == cache.byte_size(),
1938            "GPT-2 optimized decoder KV byte size mismatch");
1939    require(decoder_result.predicted_token == result.predicted_token,
1940            "optimized cached GPT-2 predicted token mismatch");
1941    require(decoder_result.logits.size() == result.logits.size(),
1942            "optimized cached GPT-2 logits size mismatch");
1943    for (std::size_t index = 0; index < result.logits.size(); ++index) {
1944        require_close(decoder_result.logits[index],
1945                      result.logits[index],
1946                      "optimized cached GPT-2 logit mismatch");
1947    }
1948
1949    lcqi::Gpt2ExecutionOptions parallel_options;
1950    parallel_options.worker_count = 2;
1951    parallel_options.parallel_min_rows = 1;
1952    lcqi::Gpt2CachedGreedyDecoder parallel_decoder(model, nullptr, ↩
        ↪ parallel_options);
1953    lcqi::Gpt2ForwardResult parallel_result;
1954    for (const std::int32_t token : tokens) {
1955        parallel_result = parallel_decoder.step_with_logits(token);
1956    }
1957    require(parallel_decoder.worker_count() == parallel_options.worker_count,
```

```
            "parallel GPT-2 decoder worker count mismatch");
    require(parallel_result.predicted_token == result.predicted_token,
            "parallel cached GPT-2 predicted token mismatch");
    require(parallel_result.logits.size() == result.logits.size(),
            "parallel cached GPT-2 logits size mismatch");
    for (std::size_t index = 0; index < result.logits.size(); ++index) {
        require_close(parallel_result.logits[index],
                      result.logits[index],
                      "parallel cached GPT-2 logit mismatch");
    }

    lcqi::Gpt2CachedGreedyDecoder greedy_decoder(model);
    std::int32_t greedy_predicted = 0;
    for (const std::int32_t token : tokens) {
        greedy_predicted = greedy_decoder.step(token);
    }
    require(greedy_predicted == result.predicted_token,
            "optimized logits-free GPT-2 predicted token mismatch");

    lcqi::Gpt2CachedGreedyDecoder prefill_decoder(model);
    for (std::size_t index = 0; index + 1 < tokens.size(); ++index) {
        prefill_decoder.advance_without_prediction(tokens[index]);
    }
    require(prefill_decoder.filled_tokens() ==
                static_cast<std::int32_t>(tokens.size() - 1U),
            "GPT-2 prefill-only decoder filled token count mismatch");
    const std::int32_t prefill_predicted = prefill_decoder.step(tokens.back());
    require(prefill_decoder.filled_tokens() == static_cast<std::int32_t>(tokens↩
    ↪ .size()),
            "GPT-2 prefill decoder final filled token count mismatch");
    require(prefill_predicted == result.predicted_token,
            "GPT-2 prefill-skipped predicted token mismatch");

    lcqi::Gpt2HotspotProfile hotspot_profile;
    lcqi::Gpt2CachedGreedyDecoder profiled_decoder(model, &hotspot_profile);
    std::int32_t profiled_predicted = 0;
    for (const std::int32_t token : tokens) {
        profiled_predicted = profiled_decoder.step(token);
    }
    require(profiled_predicted == result.predicted_token,
            "profiled GPT-2 predicted token mismatch");
    require(hotspot_profile.decoder_steps == static_cast<std::int64_t>(tokens.↩
    ↪ size()),
            "profiled GPT-2 decoder step count mismatch");
    require(hotspot_profile.layer_steps ==
                static_cast<std::int64_t>(tokens.size()) * model.config.↩
    ↪ layer_count,
            "profiled GPT-2 layer step count mismatch");
    require(hotspot_profile.total_step_ms >= hotspot_profile.lm_head_ms,
            "profiled GPT-2 total time should include lm head time");

    const std::vector<std::int32_t> cached_generated =
```

```
2007         lcqi::gpt2_generate_greedy_cached(model, tokens, 2);
2008     require(cached_generated == expected, "cached GPT-2 greedy generation ↩
         ↪ mismatch");
2009 }
2010
2011 void test_gpt2_byte_bpe_tokenizer() {
2012     const std::filesystem::path vocab_path = temp_file_path("lcqi_gpt2_vocab.↩
         ↪ json");
2013     const std::filesystem::path merges_path = temp_file_path("lcqi_gpt2_merges.↩
         ↪ txt");
2014     const std::string space_marker = "\xC4\xA0";
2015     const std::string vocab =
2016         "{\"Hello\":7,\",\":8,\"" + space_marker + "world\":9,\"!\":10}";
2017     const std::string merges =
2018         "#version: 0.2\n"
2019         "H e\n"
2020         "He l\n"
2021         "Hel l\n"
2022         "Hell o\n" +
2023         space_marker + " w\n" +
2024         space_marker + "w o\n" +
2025         space_marker + "wo r\n" +
2026         space_marker + "wor l\n" +
2027         space_marker + "worl d\n";
2028     write_text_file(vocab_path, vocab);
2029     write_text_file(merges_path, merges);
2030
2031     const lcqi::Gpt2Tokenizer tokenizer =
2032         lcqi::load_gpt2_tokenizer(vocab_path, merges_path, -1, -1);
2033     std::filesystem::remove(vocab_path);
2034     std::filesystem::remove(merges_path);
2035
2036     const std::vector<std::int32_t> ids =
2037         lcqi::gpt2_encode(tokenizer, "Hello, world!");
2038     const std::vector<std::int32_t> expected{
2039         LCQI_GPT2_TOKENIZER_HELLO_ID,
2040         8,
2041         9,
2042         10,
2043     };
2044     require(ids == expected, "GPT-2 BPE ids mismatch");
2045     require(lcqi::gpt2_decode(tokenizer, ids) == "Hello, world!",
2046             "GPT-2 BPE decode mismatch");
2047 }
2048
2049 void test_gpt2_loads_hf_style_tiny_checkpoint() {
2050     const std::filesystem::path directory =
2051         temp_file_path("lcqi_gpt2_tiny_checkpoint_dir");
2052     std::filesystem::remove_all(directory);
2053     std::filesystem::create_directories(directory);
2054
2055     const lcqi::Gpt2ReferenceModel original = lcqi::↩
```

```
        ↪ make_tiny_gpt2_reference_model();
2056    write_text_file(
2057        directory / "config.json",
2058        R"({"model_type":"gpt2","n_embd":4,"n_head":2,"n_layer":1,"n_positions"↩
        ↪ :8,"n_ctx":8,"vocab_size":6,"n_inner":8,"layer_norm_epsilon":0.00001,"↩
        ↪ activation_function":"gelu_new","bos_token_id":0,"eos_token_id":5})");
2059    write_tiny_gpt2_safetensors(directory / "model.safetensors", original);
2060
2061    const lcqi::Gpt2ReferenceModel loaded = lcqi::load_gpt2_from_directory(↩
        ↪ directory);
2062    const std::vector<std::int32_t> tokens{1, 2, 3};
2063    const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(loaded, ↩
        ↪ tokens);
2064    std::filesystem::remove_all(directory);
2065
2066    require(result.predicted_token == LCQI_GPT2_PREDICTED_TOKEN,
2067            "loaded GPT-2 predicted token mismatch");
2068    for (std::size_t index = 0; index < LCQI_GPT2_LOGITS.size(); ++index) {
2069        require_close(result.logits[index],
2070                      LCQI_GPT2_LOGITS[index],
2071                      "loaded GPT-2 logit mismatch");
2072    }
2073 }
2074
2075 void test_gpt2_loader_rejects_bad_shape() {
2076    const std::filesystem::path directory =
2077        temp_file_path("lcqi_gpt2_bad_shape_dir");
2078    std::filesystem::remove_all(directory);
2079    std::filesystem::create_directories(directory);
2080
2081    const lcqi::Gpt2ReferenceModel original = lcqi::↩
        ↪ make_tiny_gpt2_reference_model();
2082    write_text_file(
2083        directory / "config.json",
2084        R"({"model_type":"gpt2","n_embd":5,"n_head":1,"n_layer":1,"n_positions"↩
        ↪ :8,"n_ctx":8,"vocab_size":6,"n_inner":8,"layer_norm_epsilon":0.00001,"↩
        ↪ activation_function":"gelu_new","bos_token_id":0,"eos_token_id":5})");
2085    write_tiny_gpt2_safetensors(directory / "model.safetensors", original);
2086
2087    bool threw = false;
2088    try {
2089        static_cast<void>(lcqi::load_gpt2_from_directory(directory));
2090    } catch (const std::runtime_error&) {
2091        threw = true;
2092    }
2093    std::filesystem::remove_all(directory);
2094    require(threw, "GPT-2 loader should reject bad tensor shape");
2095 }
2096
2097 void test_reference_rms_norm() {
2098    const std::vector<float> input{3.0F, 4.0F};
2099    const std::vector<float> weight{1.0F, 0.5F};
```

```
2100     std::vector<float> output(2, 0.0F);
2101     lcqi::rms_norm(input, weight, 0.0F, output);
2102     require_close(output[0], LCQI_RMS_EXPECTED_0, "RMSNorm output 0 mismatch");
2103     require_close(output[1], LCQI_RMS_EXPECTED_1, "RMSNorm output 1 mismatch");
2104 }
2105
2106 void test_reference_rope() {
2107     std::vector<float> heads{1.0F, 0.0F, 0.0F, 1.0F};
2108     lcqi::apply_rope(heads, 1, 4, 1, 10000.0F);
2109     require_close(heads[0], std::cos(1.0F), "RoPE pair 0 cosine mismatch");
2110     require_close(heads[1], std::sin(1.0F), "RoPE pair 0 sine mismatch");
2111     require_close(heads[2], -std::sin(0.01F), "RoPE pair 1 negative sine ↩
        ↪ mismatch");
2112     require_close(heads[3], std::cos(0.01F), "RoPE pair 1 cosine mismatch");
2113 }
2114
2115 void test_reference_kv_attention() {
2116     lcqi::DecoderConfig config;
2117     config.hidden_size = 4;
2118     config.query_heads = 2;
2119     config.kv_heads = 1;
2120     config.head_dim = 2;
2121     config.intermediate_size = 4;
2122     config.vocab_size = 4;
2123     config.max_context = 4;
2124
2125     lcqi::ReferenceKVCache cache(config, 1);
2126     const std::vector<float> key{1.0F, 0.0F};
2127     const std::vector<float> value{LCQI_ATTENTION_EXPECTED_0, ↩
        ↪ LCQI_ATTENTION_EXPECTED_1};
2128     cache.append(0, 0, key, value);
2129     require(cache.filled_tokens() == 1, "KV filled token count mismatch");
2130     require_close(cache.key(0, 0, 0)[0], 1.0F, "KV key read mismatch");
2131
2132     const std::vector<float> query{1.0F, 0.0F, 0.0F, 1.0F};
2133     std::vector<float> output(4, 0.0F);
2134     lcqi::attention_decode(config, cache, 0, 0, query, output);
2135     require_close(output[0], LCQI_ATTENTION_EXPECTED_0, "attention head 0 dim 0↩
        ↪  mismatch");
2136     require_close(output[1], LCQI_ATTENTION_EXPECTED_1, "attention head 0 dim 1↩
        ↪  mismatch");
2137     require_close(output[2], LCQI_ATTENTION_EXPECTED_0, "attention head 1 dim 0↩
        ↪  mismatch");
2138     require_close(output[3], LCQI_ATTENTION_EXPECTED_1, "attention head 1 dim 1↩
        ↪  mismatch");
2139 }
2140
2141 void test_reference_decoder_end_to_end() {
2142     const lcqi::ReferenceDecoderModel model = lcqi::↩
        ↪ make_tiny_reference_decoder_model();
2143     const std::vector<std::int32_t> tokens{1, 2, 3};
2144     const lcqi::ReferenceDecodeResult result = lcqi::run_reference_decode(model↩
```

```
    , tokens);
    require(result.logits.size() == LCQI_REFERENCE_DECODER_VOCAB_SIZE,
            "reference decoder logits size mismatch");
    require(result.predicted_token == LCQI_REFERENCE_DECODER_PREDICTED_TOKEN,
            "reference decoder predicted token mismatch");
    for (std::size_t index = 0; index < LCQI_REFERENCE_DECODER_LOGITS.size(); ++index) {
        require_close(result.logits[index],
                      LCQI_REFERENCE_DECODER_LOGITS[index],
                      "reference decoder golden logit mismatch");
    }
}

void test_reference_decoder_trace_contract() {
    const lcqi::ReferenceDecoderModel model = lcqi::make_tiny_reference_decoder_model();
    const std::vector<std::int32_t> tokens{1, 2, 3};
    const lcqi::ReferenceDecodeTraceResult trace =
        lcqi::run_reference_decode_with_trace(model, tokens);

    require(trace.result.predicted_token == LCQI_REFERENCE_DECODER_PREDICTED_TOKEN,
            "reference trace predicted token mismatch");
    require(!trace.checkpoints.empty(), "reference trace checkpoints missing");

    const lcqi::ReferenceTensorCheckpoint& first = trace.checkpoints.front();
    require(first.name == "embedding", "first checkpoint should be embedding");
    require(first.token_position == LCQI_REFERENCE_TRACE_FIRST_TOKEN,
            "first checkpoint token position mismatch");
    require(first.dtype == "f32", "first checkpoint dtype mismatch");
    require(first.layout == "row_major", "first checkpoint layout mismatch");
    require(first.shape.size() == 1 &&
                first.shape[0] == LCQI_REFERENCE_TRACE_HIDDEN_SIZE,
            "first checkpoint shape mismatch");
    require(first.stride.size() == 1 && first.stride[0] == 1,
            "first checkpoint stride mismatch");

    bool saw_logits = false;
    bool saw_kv_slot = false;
    for (const lcqi::ReferenceTensorCheckpoint& checkpoint : trace.checkpoints) {
        if (checkpoint.name == "kv_cache_key_slot") {
            saw_kv_slot = true;
            require(checkpoint.shape.size() == 4, "KV checkpoint rank mismatch");
            require(checkpoint.layout == "kv_contiguous", "KV checkpoint layout mismatch");
        }
        if (checkpoint.name == "logits") {
            saw_logits = true;
            require(checkpoint.token_position == LCQI_REFERENCE_TRACE_TOKEN_COUNT - 1,
```

```
2189                     "logits checkpoint token position mismatch");
2190             require(checkpoint.shape.size() == 1 &&
2191                         checkpoint.shape[0] ==
2192                             static_cast<std::int32_t>(↩
    ↪ LCQI_REFERENCE_DECODER_VOCAB_SIZE),
2193                     "logits checkpoint shape mismatch");
2194             require(checkpoint.values.size() == LCQI_REFERENCE_DECODER_LOGITS.↩
    ↪ size(),
2195                     "logits checkpoint value size mismatch");
2196             for (std::size_t index = 0; index < LCQI_REFERENCE_DECODER_LOGITS.↩
    ↪ size(); ++index) {
2197                 require_close(checkpoint.values[index],
2198                               LCQI_REFERENCE_DECODER_LOGITS[index],
2199                               "trace logits checkpoint mismatch");
2200             }
2201         }
2202     }
2203     require(saw_kv_slot, "KV checkpoint missing");
2204     require(saw_logits, "logits checkpoint missing");
2205 }
2206
2207 void test_packed_i8_kernel_matches_scalar() {
2208     for (const KernelShapeCase& shape_case : LCQI_KERNEL_SHAPE_CASES) {
2209         const lcqi::QuantizedLinearLayer layer =
2210             lcqi::make_deterministic_i8_layer(shape_case.input_size,
2211                                               shape_case.output_size,
2212                                               0.03125F);
2213         const lcqi::PackedLinearI8 packed =
2214             lcqi::pack_linear_i8(layer, shape_case.output_block_size);
2215         const std::vector<float> input = lcqi::make_deterministic_input(layer.↩
    ↪ input_size);
2216         std::vector<float> scalar_output(static_cast<std::size_t>(layer.↩
    ↪ output_size), 0.0F);
2217         std::vector<float> packed_output(static_cast<std::size_t>(layer.↩
    ↪ output_size), 0.0F);
2218         lcqi::linear_i8(layer, input, scalar_output);
2219         lcqi::linear_i8_packed(packed, input, packed_output);
2220         require(lcqi::max_abs_diff(scalar_output, packed_output) <= ↩
    ↪ LCQI_KERNEL_MAX_DIFF,
2221                 "packed int8 kernel output differs from scalar");
2222
2223         const lcqi::PackedLinearI8 simd_packed =
2224             lcqi::pack_linear_i8(layer, LCQI_SIMD_TEST_BLOCK);
2225         if (lcqi::linear_i8_packed_avx2_available()) {
2226             std::vector<float> avx2_output(static_cast<std::size_t>(layer.↩
    ↪ output_size), 0.0F);
2227             lcqi::linear_i8_packed_avx2(simd_packed, input, avx2_output);
2228             require(lcqi::max_abs_diff(scalar_output, avx2_output) <= ↩
    ↪ LCQI_KERNEL_MAX_DIFF,
2229                     "AVX2 int8 kernel output differs from scalar");
2230         }
2231     }
```

```
}

void test_f32_dot_kernel_matches_scalar() {
    std::vector<float> lhs(LCQI_F32_DOT_TEST_SIZE, 0.0F);
    std::vector<float> rhs(LCQI_F32_DOT_TEST_SIZE, 0.0F);
    for (std::size_t index = 0; index < LCQI_F32_DOT_TEST_SIZE; ++index) {
        lhs[index] = static_cast<float>(static_cast<std::int32_t>(index % 19U) + 1) *
                     0.03125F;
        rhs[index] = static_cast<float>(static_cast<std::int32_t>(index % 23U) - 11) *
                     0.0625F;
    }

    const float scalar =
        lcqi::dot_f32_scalar_unchecked(lhs.data(), rhs.data(), lhs.size());
    const float dispatched = lcqi::dot_f32(lhs, rhs);
    require(std::fabs(scalar - dispatched) <= LCQI_F32_KERNEL_MAX_DIFF,
            "F32 dot kernel output differs from scalar");

    if (lcqi::dot_f32_avx2_available()) {
        const float avx2 =
            lcqi::dot_f32_avx2_unchecked(lhs.data(), rhs.data(), lhs.size());
        require(std::fabs(scalar - avx2) <= LCQI_F32_KERNEL_MAX_DIFF,
                "AVX2 F32 dot kernel output differs from scalar");
    }

    bool rejected_mismatch = false;
    try {
        (void)lcqi::dot_f32(std::span<const float>(lhs.data(), lhs.size() - 1U), rhs);
    } catch (const std::runtime_error&) {
        rejected_mismatch = true;
    }
    require(rejected_mismatch, "F32 dot kernel accepted mismatched sizes");
}

void test_f32_row_kernels_match_scalar() {
    std::vector<float> weights(LCQI_F32_ROW_TEST_INPUT_SIZE *
                                   LCQI_F32_ROW_TEST_OUTPUT_SIZE,
                               0.0F);
    std::vector<float> input(LCQI_F32_ROW_TEST_INPUT_SIZE, 0.0F);
    std::vector<float> bias(LCQI_F32_ROW_TEST_OUTPUT_SIZE, 0.0F);
    for (std::size_t index = 0; index < weights.size(); ++index) {
        weights[index] = static_cast<float>(static_cast<std::int32_t>(index % 29U) - 14) *
                         0.015625F;
    }
    for (std::size_t index = 0; index < input.size(); ++index) {
        input[index] = static_cast<float>(static_cast<std::int32_t>(index % 13U) - 6) *
                       0.03125F;
```

```
    }
    for (std::size_t index = 0; index < bias.size(); ++index) {
        bias[index] = static_cast<float>(static_cast<std::int32_t>(index % 7U) - 3) *
                      0.125F;
    }

    std::vector<float> scalar_output(LCQI_F32_ROW_TEST_OUTPUT_SIZE, 0.0F);
    std::vector<float> dispatched_output(LCQI_F32_ROW_TEST_OUTPUT_SIZE, 0.0F);
    lcqi::linear_f32_rows_scalar_unchecked(weights.data(),
                                           input.data(),
                                           bias.data(),
                                           input.size(),
                                           0,
                                           LCQI_F32_ROW_TEST_OUTPUT_SIZE,
                                           scalar_output.data());
    lcqi::linear_f32_rows_unchecked(weights.data(),
                                    input.data(),
                                    bias.data(),
                                    input.size(),
                                    0,
                                    LCQI_F32_ROW_TEST_OUTPUT_SIZE,
                                    dispatched_output.data());
    for (std::size_t index = 0; index < scalar_output.size(); ++index) {
        require(std::fabs(scalar_output[index] - dispatched_output[index]) <=
                    LCQI_F32_KERNEL_MAX_DIFF,
                "F32 linear row kernel output differs from scalar");
    }

    const lcqi::F32RowMax scalar_max =
        lcqi::max_dot_f32_rows_scalar_unchecked(weights.data(),
                                                input.data(),
                                                input.size(),
                                                0,
                                                LCQI_F32_ROW_TEST_OUTPUT_SIZE);
    const lcqi::F32RowMax dispatched_max =
        lcqi::max_dot_f32_rows_unchecked(weights.data(),
                                         input.data(),
                                         input.size(),
                                         0,
                                         LCQI_F32_ROW_TEST_OUTPUT_SIZE);
    require(scalar_max.row == dispatched_max.row,
            "F32 row max kernel selected a different row");
    require(std::fabs(scalar_max.value - dispatched_max.value) <= LCQI_F32_KERNEL_MAX_DIFF,
            "F32 row max kernel value differs from scalar");

    if (lcqi::dot_f32_avx2_available()) {
        std::vector<float> avx2_output(LCQI_F32_ROW_TEST_OUTPUT_SIZE, 0.0F);
        lcqi::linear_f32_rows_avx2_unchecked(weights.data(),
                                             input.data(),
                                             bias.data(),
```

```
                                            input.size(),
                                            0,
                                            LCQI_F32_ROW_TEST_OUTPUT_SIZE,
                                            avx2_output.data());
        for (std::size_t index = 0; index < scalar_output.size(); ++index) {
            require(std::fabs(scalar_output[index] - avx2_output[index]) <=
                        LCQI_F32_KERNEL_MAX_DIFF,
                    "AVX2 F32 linear row kernel output differs from scalar");
        }

        std::vector<float> scalar_subrange(LCQI_F32_ROW_TEST_OUTPUT_SIZE, 0.0F);
        std::vector<float> avx2_subrange(LCQI_F32_ROW_TEST_OUTPUT_SIZE, 0.0F);
        constexpr std::size_t LCQI_F32_ROW_TEST_SUBRANGE_BEGIN = 1;
        constexpr std::size_t LCQI_F32_ROW_TEST_SUBRANGE_END = 7;
        lcqi::linear_f32_rows_scalar_unchecked(weights.data(),
                                               input.data(),
                                               bias.data(),
                                               input.size(),
                                               LCQI_F32_ROW_TEST_SUBRANGE_BEGIN,
                                               LCQI_F32_ROW_TEST_SUBRANGE_END,
                                               scalar_subrange.data());
        lcqi::linear_f32_rows_avx2_unchecked(weights.data(),
                                             input.data(),
                                             bias.data(),
                                             input.size(),
                                             LCQI_F32_ROW_TEST_SUBRANGE_BEGIN,
                                             LCQI_F32_ROW_TEST_SUBRANGE_END,
                                             avx2_subrange.data());
        for (std::size_t index = LCQI_F32_ROW_TEST_SUBRANGE_BEGIN;
             index < LCQI_F32_ROW_TEST_SUBRANGE_END;
             ++index) {
            require(std::fabs(scalar_subrange[index] - avx2_subrange[index]) <=
                        LCQI_F32_KERNEL_MAX_DIFF,
                    "AVX2 F32 subrange row kernel output differs from scalar");
        }

        const lcqi::F32RowMax avx2_max =
            lcqi::max_dot_f32_rows_avx2_unchecked(weights.data(),
                                                  input.data(),
                                                  input.size(),
                                                  0,
                                                  LCQI_F32_ROW_TEST_OUTPUT_SIZE);
        require(scalar_max.row == avx2_max.row,
                "AVX2 F32 row max kernel selected a different row");
        require(std::fabs(scalar_max.value - avx2_max.value) <= LCQI_F32_KERNEL_MAX_DIFF,
                "AVX2 F32 row max kernel value differs from scalar");
    }
}
```

```
2377
2378 void test_f32_batch_row_kernels_match_scalar() {
2379     std::vector<float> weights(LCQI_F32_ROW_TEST_INPUT_SIZE *
2380                                    LCQI_F32_ROW_TEST_OUTPUT_SIZE,
2381                                0.0F);
2382     std::vector<float> input(LCQI_F32_BATCH_TEST_SIZE * ↩
        ↪ LCQI_F32_ROW_TEST_INPUT_SIZE, 0.0F);
2383     std::vector<float> bias(LCQI_F32_ROW_TEST_OUTPUT_SIZE, 0.0F);
2384     for (std::size_t index = 0; index < weights.size(); ++index) {
2385         weights[index] =
2386             static_cast<float>(static_cast<std::int32_t>(index % 19U) - 9) * ↩
        ↪ 0.03125F;
2387     }
2388     for (std::size_t index = 0; index < input.size(); ++index) {
2389         input[index] =
2390             static_cast<float>(static_cast<std::int32_t>(index % 23U) - 11) * ↩
        ↪ 0.046875F;
2391     }
2392     for (std::size_t index = 0; index < bias.size(); ++index) {
2393         bias[index] =
2394             static_cast<float>(static_cast<std::int32_t>(index % 5U) - 2) * ↩
        ↪ 0.125F;
2395     }
2396
2397     constexpr std::size_t LCQI_F32_BATCH_ROW_BEGIN = 1;
2398     constexpr std::size_t LCQI_F32_BATCH_ROW_END = 16;
2399     std::vector<float> scalar(LCQI_F32_BATCH_TEST_SIZE * ↩
        ↪ LCQI_F32_ROW_TEST_OUTPUT_SIZE, 0.0F);
2400     std::vector<float> dispatched(scalar.size(), 0.0F);
2401     for (std::size_t batch = 0; batch < LCQI_F32_BATCH_TEST_SIZE; ++batch) {
2402         lcqi::linear_f32_rows_scalar_unchecked(
2403             weights.data(),
2404             input.data() + batch * LCQI_F32_ROW_TEST_INPUT_SIZE,
2405             bias.data(),
2406             LCQI_F32_ROW_TEST_INPUT_SIZE,
2407             LCQI_F32_BATCH_ROW_BEGIN,
2408             LCQI_F32_BATCH_ROW_END,
2409             scalar.data() + batch * LCQI_F32_ROW_TEST_OUTPUT_SIZE);
2410     }
2411     lcqi::linear_f32_batch_rows_unchecked(weights.data(),
2412                                           input.data(),
2413                                           bias.data(),
2414                                           LCQI_F32_ROW_TEST_INPUT_SIZE,
2415                                           LCQI_F32_BATCH_TEST_SIZE,
2416                                           LCQI_F32_BATCH_ROW_BEGIN,
2417                                           LCQI_F32_BATCH_ROW_END,
2418                                           LCQI_F32_ROW_TEST_OUTPUT_SIZE,
2419                                           dispatched.data());
2420     for (std::size_t batch = 0; batch < LCQI_F32_BATCH_TEST_SIZE; ++batch) {
2421         for (std::size_t row = LCQI_F32_BATCH_ROW_BEGIN; row < ↩
        ↪ LCQI_F32_BATCH_ROW_END; ++row) {
2422             const std::size_t index = batch * LCQI_F32_ROW_TEST_OUTPUT_SIZE + ↩
```

```
        ↪ row;
2423                require(std::fabs(scalar[index] - dispatched[index]) <= ↩
        ↪ LCQI_F32_KERNEL_MAX_DIFF,
2424                        "F32 batch row kernel output differs from scalar");
2425            }
2426        }
2427    }
2428
2429    void test_llama_gguf_batch_prefill_matches_token_path() {
2430        const std::filesystem::path path = temp_file_path("↩
        ↪ lcqi_tiny_llama_batch_prefill.gguf");
2431        write_tiny_llama_gguf_fixture(path);
2432
2433        const lcqi::LlamaGgufLoadedModel loaded =
2434            lcqi::load_llama_gguf_reference_model(path);
2435        const std::vector<std::int32_t> prompt{1, 2, 3};
2436        {
2437            const ScopedEnvVar disable_batch_prefill(↩
        ↪ LCQI_TEST_LLAMA_BATCH_PREFILL_ENV,
2438                                                     LCQI_TEST_FALSE_ENV);
2439            const lcqi::LlamaGgufGenerationResult token_path =
2440                lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1);
2441            const ScopedEnvVar enable_batch_prefill(↩
        ↪ LCQI_TEST_LLAMA_BATCH_PREFILL_ENV,
2442                                                    LCQI_TEST_TRUE_ENV);
2443            const lcqi::LlamaGgufGenerationResult batch_path =
2444                lcqi::llama_gguf_generate_greedy(loaded.model, prompt, 1);
2445            require(batch_path.generated_ids == token_path.generated_ids,
2446                    "batch prefill generated ids differ from token path");
2447            require(batch_path.predicted_first_token == token_path.↩
        ↪ predicted_first_token,
2448                    "batch prefill predicted token differs from token path");
2449            require(batch_path.prefill_steps == token_path.prefill_steps,
2450                    "batch prefill step count differs from token path");
2451            require(batch_path.hotspots.f32_fallback_calls < token_path.hotspots.↩
        ↪ f32_fallback_calls,
2452                    "batch prefill should reduce fallback linear call count");
2453        }
2454        std::filesystem::remove(path);
2455    }
2456
2457    }  // namespace
2458
2459    int main() {
2460        try {
2461            test_tiny_mlp();
2462            test_tokenizer_contract();
2463            test_tokenizer_rejects_unknown_without_unk();
2464            test_safetensors_manifest_contract();
2465            test_safetensors_rejects_bad_offsets();
2466            test_safetensors_rejects_bad_dtype_shape_size();
2467            test_safetensors_reads_float_tensors();
```

```
        test_gguf_manifest_reads_q4_k_tensor();
        test_q4_k_dequant_and_dot_paths();
        test_ggml_mixed_dequantizers();
        test_ggml_quantized_f32_matvec_paths();
        test_q5_0_packed_matvec_matches_ggml_q8_0_path();
        test_llama_gguf_loader_and_generation();
        test_llama_gguf_default_packs_q5_0_tensors();
        test_llama_gguf_batch_prefill_uses_q5_0_f32_sidecar_path();
        test_llama_gguf_parallel_generation_matches_serial();
        test_llama_gguf_q4_direct_generation();
        test_llama_gguf_ggml_direct_generation();
        test_llama_gguf_selective_ggml_direct_keeps_q6_fallback();
        test_llama_gguf_q6_k_direct_experimental_path();
        test_gpt2_tiny_forward_and_generation();
        test_gpt2_byte_bpe_tokenizer();
        test_gpt2_loads_hf_style_tiny_checkpoint();
        test_gpt2_loader_rejects_bad_shape();
        test_reference_rms_norm();
        test_reference_rope();
        test_reference_kv_attention();
        test_reference_decoder_end_to_end();
        test_reference_decoder_trace_contract();
        test_packed_i8_kernel_matches_scalar();
        test_f32_dot_kernel_matches_scalar();
        test_f32_row_kernels_match_scalar();
        test_f32_batch_row_kernels_match_scalar();
        test_llama_gguf_batch_prefill_matches_token_path();

        std::cout << "lcqi tests passed\n";
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
        std::cerr << "lcqi test failure: " << error.what() << "\n";
        return EXIT_FAILURE;
    }
}
```

读码检查清单

构建入口 是否能从 CMake 文件看出每个可执行目标依赖哪些源码，哪些目标属于默认 CTest，哪些目标依赖外部环境。

资产边界 模型文件、tokenizer、safetensors header 和外部下载模型是否都有校验或拒绝路径。

公共合同 头文件是否把 shape、dtype、token、trace checkpoint、KV cache 地址和返回字段说清楚。

正确性路径 tiny MLP、reference decoder 和 GPT-2 tiny fixture 是否有可复查 golden。

性能口径 benchmark 是否同时报告 backend、available、耗时、误差和 checksum，避免只有速度没有正确性。

真实模型证据 GPT-2 smoke 和 SmolLM2 smoke 是否记录模型文件 hash、命令、输出和验证结论。

回归门禁 CTest 是否覆盖坏资产、坏 shape、tokenizer 合同、trace 关键字段、ledger 固定公式和 serving probe 拒绝路径。

如果某段新代码不能回答上面的检查项，它就还不能进入第三册主线工程。源码卷的作用是把这条标准显式化：代码不只是能编译，还要能解释、能测量、能拒绝坏输入、能被长期回归。